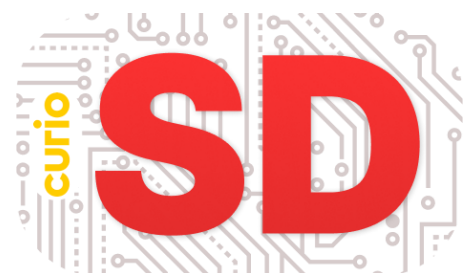


CSD-IV

Native Systemintegratie:
Entity Framework en API's



Over dit document

Opleiding: Software Developer
Blokken: E
Vak(ken): S_NATIVE

Versiebeheer

Versie	Datum	Auteur	Aanpassingen
1.1	12-9-2023	tl10	Enkele kleine verbeteringen in hoofdstuk 2 OOP
1.2	19-9-2023	tl10	Verbeteringen aan hoofdstuk 2 OOP inleveropdracht Enkele kleine verbeteringen in hoofdstuk 2 OOP
1.3-1.8	10-11-2023	tl10	Kleine verbeteringen in hoofdstuk 3 en 4. Enkele toevoegingen: samenwerken in Git (app.config), ThenInclude uitleg.
1.9	13-11-2023	tl10	Uitbreiding hoofdstuk 5: ContentDialog, Afbeeldingen, StorageFile
1.10	24-11-2023	tl10	Toevoeging koppeltabel seeden eind hoofdstuk 4
1.11	27-2-2024	tl10	Toevoeging 4.16 NuGet
1.12	23-8-2024	tl10	Aanpassing studiepunten + .NET 7.0 → .NET 8.0
1.13 - 1.14	3-9-2024	tl10	Migrations naar nieuw Hoofdstuk 6 verplaatst (wip). Stappenplan versimpelt + ListView naar eigen paragraaf verplaatst.
1.15.1	17-9-2024	tl10	Theorie en opdracht toegevoegd over 'Frame' en 'Page' in H3
1.16	31-10-2024	tl10	Onnodige voorbeeldcode bij RPG Classes weggehaald + Opdracht 5.4 toegevoegd + eerste versie H6
1.17	1-11-2024	tl10	Uitleg over kleuren en stijlen toegevoegd aan het eind van H5 + Session token uitleg H7 verduidelijkt
1.18	6-12-2024	tl10	H5: Voorbeeld toegevoegd ContentDialog in C# + verduidelijken opdracht 7.14 en theorie over enumeration H7
2.0	8-5-2025	tl10	Samenvoegen CSD V + VI in IV. Expliciete instructies validatie toegevoegd.
2.1	2-9-2025	tl10	Fix versie en packages in stappenplan
2.2	11-9-2025	tl10	Missende paragrafen 3.5 en 3.6 gevuld
2.3	23-9-2025	tl10	Corrigeren feedbackmoment codes
2.4	10-10-2025	tl10	App.config uitleg verbeteren + in H3 Frame en Pages reguliere uitleg maken + Uitbreiden uitleg relaties H4
2.5	14-11-2025	tl10	Uitbreiding uitleg PDFSharp

Theorieblok

Opdracht

? Een term: die je misschien nog niet kent

Inhoudsopgave

I. Over deze module	4
II. Studiewijzer	6
III. Studiepunten.....	7
1. C#: Opwarmen.....	8
2. C#: OOP	11
3. Entity Framework: Introductie	33
4. Entity Framework: Relaties.....	58
5. Entity Framework: CRUD	87
6. Applicatie Implementeren	118
7. Authenticatie & Autorisatie	128
8. Een API consumeren.....	145
9. Eigen webserver en API bouwen	154
10. WinUI app verbinden met eigen API	170
11. Een API beveiligen	184

I. Over deze module

Doelstelling en relevantie

In deze lessenreeks zul je leren hoe je systemen kunt integreren in C#. In deze module gaan we ons richten op het werken met data uit databases. Later in de module gaan we die data ophalen en opslaan via een API. Dit zijn essentiële vaardigheden omdat steeds meer systemen bestaan uit afzonderlijke onderdelen die met elkaar moeten communiceren.

Tijdens deze lessen zul je niet alleen leren coderen, maar ook kennismaken met andere belangrijke concepten. We zullen werken met object georiënteerd programmeren (OOP) om structuur aan te brengen in je code.

Daarnaast gaan we aan de slag met het lezen van UML diagrammen. Je leert hoe je een Entity Relationship Diagram (ERD) kunt lezen. In de PRO lessen leer je hoe je deze diagrammen zelf gaat bedenken en tekenen. Het ERD helpt je om de relaties tussen verschillende entiteiten in je systeem te visualiseren.

Waar de hoofdstukken over C#, OOP en Entity Framework uiterst belangrijk zijn ter voorbereiding van jouw proeve van bekwaamheid (eindexamen). Zijn de hoofdstukken over het werken met API's belangrijk voor jou als Software Ontwikkelaar. Je hoeft op je examen dus niet per se met API's te werken, maar het is een mooie extra vaardigheid waar je bij toekomstige werkgevers mee gaat werken.

Werkwijze

Deze module is opgedeeld in meerdere hoofdstukken, met daarin verschillende sub-secties. Je zal doorgaans meer dan een week doen over het afronden van een hoofdstuk. Daarbij wordt van je verwacht dat je inleveropdrachten maakt en inlevert. Het is van belang dat je deze opdrachten maakt, zodat je optimaal voorbereid bent voor je proeve van bekwaamheid. Vraag de docent om feedback wanneer je daar behoefte aan hebt.

Aan het einde van dit blok krijg je bij de Project-lessen oefenexamens waarmee je kunt meten in hoeverre je op niveau bent voor het examen.

Samenhang met andere onderdelen

Het is van belang dat je de voorgaande CSD modules succesvol hebt afgerond. In het bijzonder is CSD-IV belangrijk, waar je hebt leren werken met Entity Framework en WinUI3 (of UWP, of WPF).

Ook belangrijk zijn de modules over ontwerpen (ONT), testen (TST) en samenwerken (SAM). Die laatste twee krijgen in deze module minder aandacht, daarom is het belangrijk dat je er zelf aandacht aan geeft tijdens de projecten in dit blok.

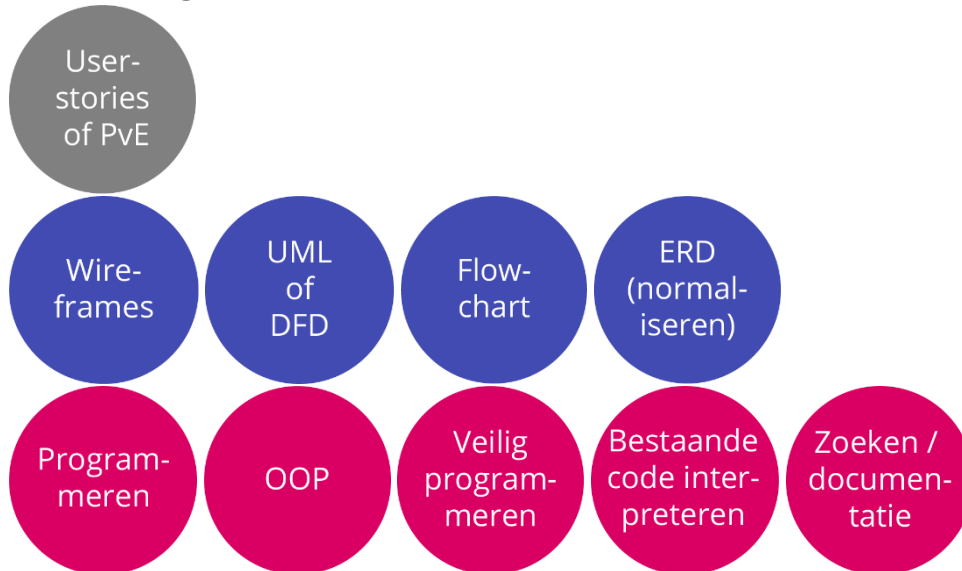
Materiaal- en middelenlijst

- Visual Studio 2022
 - Zorg dat je Visual Studio geüpdatet hebt naar de laatste beschikbare versie.
 - De versie van .NET die nodig is voor WinUI ontwikkeling
(op het moment van schrijven is dat .NET 8.0)
 - *Workload:* Windows Application Development
 - *Workload:* .NET desktop development
- Toegang tot Itslearning

Feedback-momenten

Gedurende dit blok meten we je voortgang met feedback-momenten. Je krijgt studiepunten voor ieder succesvol afgerond feedbackmoment. In deze module tellen enkele van de Inleveropdrachten als feedbackmoment. Je herkent die inleveropdrachten aan dit icoon: 📁

Samenhang met de ICO-V



II. Studiewijzer

Week	Hoofdstuk			Onderwerpen	Inleveropdracht
1	C#	1	Opwarmen	Herhaling algemeen	Coding Race
2		2	OOP	Herhaling Klassen, Eigenschappen	Multiplayer Classes
3				Methodes, Inheritance, Static, Abstract	 RPG Classes
4	Entity Framework	3	Introductie	Gegevens seeden en ophalen	 ChatMeister proof-of-concept
5				EF gegevens tonen in een WinUI3 app.	
6				Gegevens tussen schermen uitwisselen	
7		4	Relaties	Relaties tussen entiteiten	DevCitySim (deel 3)
8				Gegevens filteren	
9		5	CRUD	Gegevens invoeren	 GameBib CRUD
10				Gegevens verwijderen	
11				Een complete CRUD applicatie bouwen	
12		6	Applicatie Implementeren	Onze applicatie in productie implementeren	
13	Simpel Authenticatie & Autorisatie	7	Authenticatie	Authenticatie & Hashing	GameBib Compleet
14			Autorisatie	Autorisatie	
15	API's	8	Een API consumeren	API's in moderne systemen	Een API-Client (waarin de gebruiker kan zoeken)
16				Gegevens ophalen en tonen	
17		9	Eigen webserver en API bouwen	Een eigen webserver en API bouwen	
18					
19		10	WinUI app verbinden met eigen API	Een WinUI app verbinden met eigen API	Games Leaderboard
20					
21		11	Beveiligen	Een API beveiligen met een sleutel	 API Bouwen en Consumeren
22					
23					
24	1 lesweek extra ruimte om een hoofdstuk langer te behandelen waar noodzakelijk				

Inleveropdrachten met het -icoon betreffen een feedbackmoment. Je ontvangt studiepunten voor het succesvol afronden ervan.

III. Studiepunten

Code	Blokweek	Onderwerp	Cesuur	Checks
FMWX	3	RPG Classes Inleveropdracht	70%	• Je levert de inleveropdracht 'RPG Classes' in op Itslearning. • De opdracht en inleverinstructies zijn te vinden aan het einde van Hoofdstuk 2 in dit CSD-IV moduleboek.
FZQU	6	ChatMeister Proof-of- Concept	70%	• Je levert de inleveropdracht 'ChatMeister: Proof-of-Concept' in op Itslearning. • De opdracht en inleverinstructies zijn te vinden aan het einde van Hoofdstuk 3 in dit CSD-IV moduleboek.
FQC4	11	GameBib CRUD	70%	• Je levert de inleveropdracht 'GameBib CRUD' in op Itslearning. • De opdracht en inleverinstructies zijn te vinden aan het einde van Hoofdstuk 5 in dit CSD-IV moduleboek.
F4KC	22	API bouwen en consumeren	70%	• Je levert de inleveropdracht 'API Bouwen en Consumeren' in op Itslearning. • De opdracht en inleverinstructies zijn te vinden aan het einde van Hoofdstuk 11 in dit CSD-IV moduleboek.

Een cesuur van 70 procent betekent dat je bij een opdracht van 10 punten minimaal 7 punten moet halen voor een 5,5. Gebruik eventueel deze site om de mogelijke cijfers te berekenen: <https://cijferberekenen.nl/>.

1. C#: Opwarmen

Misschien is het al even geleden dat je met C# aan de slag bent geweest. Daarom gaan we eerst aan de slag met het “opwarmen” van onze basis C# vaardigheden. Dat doen we in de vorm van een race: **wie in de klas lukt het om als eerst alle onderstaande opdrachten af te ronden?**

1.1. Spelregels

Je mag online kleine stukken informatie opzoeken over C#, maar **zoek geen complete oplossingen!**

✓ **Juist:** Een specifieke zoekopdracht: “csharp user input”.

✗ **Onjuist:** Zoek niet hele opdrachten: “csharp fizzbuzz”

Gebruik ook **geen** ChatGPT, GitHub Copilot, of ander hulpmiddel om de opdrachten voor je te laten oplossen. Met deze oefening warmen we JOU op, niet een Large Language Model (LLM).

Als het niet lukt om een opdracht af te ronden, ga dan door naar de volgende. Je hoeft je niet te schamen als geen van de opdrachten lukt, dan weten we in ieder geval dat extra aandacht nodig is om je kennis en vaardigheden op te halen.

Ontwikkel iedere opdracht als een **Console Applicatie**. Bewaar alle opdrachten, zodat je ze achteraf allemaal kunt inleveren.



Klaar voor de start?

Start aan de onderstaande opdrachten wanneer de docent het startsignaal geeft.

1.2. Opdracht: Inhoud van een bak

Maak een programma dat als input vraagt: de lengte, hoogte en breedte van een bak.

Als output krijg je: de inhoud van de bak, in liters.

1.3. Opdracht: Inhoud van een maatbeker

Maak een programma dat als input vraagt: de hoogte en straal van een cilinder (maatbeker).

Als output krijg je: de inhoud van de beker, in liters.

1.4. Opdracht: BTW berekenen

Als je iets koopt, betaal je daarover altijd belasting: de BTW. Voor vrijwel alle dingen geldt: je betaalt 21% BTW. Er is ook een lager tarief, met name voor eten en drinken betaal je maar 9% BTW. Tenslotte zijn er dingen waarover je géén BTW betaalt, bijvoorbeeld over onderwijs (je lesgeld voor deze opleiding dus).

Maak een programma dat het eindbedrag kan berekenen. Het programma vraagt als eerste onder welk tarief het ding valt (met een keuzemenu). Daarna voer je de prijs in. Als output krijg je te zien: de prijs inclusief én exclusief BTW, en de precieze hoeveelheid BTW die je betaalt (zoals het ook altijd op een kassabonnetje staat vermeld).

1.5. Opdracht: Fizzbuzz

Het programma toont een reeks getallen van 1 t/m 30. Echter voor getallen die deelbaar zijn door 3 wordt niet het getal getoond, maar het woord “fizz”. Voor getallen die deelbaar zijn door 5 zie je “buzz”. Voor getallen die deelbaar zijn door 3 én 5 zie je “fizzbuzz”. Tip: gebruik de modulo-operator.

Voorbeeld-output:

```
1
2
fizz
4
buzz
fizz
7
8
fizz
buzz
11
fizz
13
14
fizzbuzz
```

1.6. Opdracht: Achtbanen

Sla de gegevens van deze achtbanen op in een lijst, en toon ze daarna allemaal op het scherm in de vorm “In **1981** werd de **Python** geopend in de **Efteling**”.

Naam	Park	Bouwjaar
Python	Efteling	1981
Fenix	Toverland	2018
Troy	Toverland	2007
Baron 1898	Efteling	2015
Condor	Walibi	1994
Xpress	Walibi	2000

1.7. Opdracht: Achtbaan toevoegen

Breidt het programma van 1.6 uit, zodat je vóór het tonen van de lijst via de Console nog een achtbaan kunt toevoegen aan de lijst. Bonuspunt als je meerdere achtbanen kunt toevoegen.

1.8. Opdracht: Achtbaan voorwaarden

Breidt het programma van 1.6 en 1.7 uit, zodat er gevraagd wordt om 2 jaartallen in te voeren. Het programma toont daarna alleen nog maar achtbanen die gebouwd zijn tussen die twee jaartallen.

1.9. Opdracht: Omwisselen

Nu volgt een code-technische uitdaging. Gegeven deze variabelen:

```
string fizz = "buzz";
string foo = "bar";
```

Schrijf een programma dat de inhoud van deze twee variabelen omwisselt. Zodat je een print(foo) als resultaat “buzz” zal geven. Gebruik zo min mogelijk regels.

1.10. Opdracht: Fibonacci

Schrijf een programma dat de rij van Fibonacci als output geeft (minstens de eerste 20 getallen). Zie ook <https://www.fibonacci.com/nl/rij-van-fibonacci/>.

1.11. Letters sorteren

Je programma vraagt als input één woord. Als output krijg je de letters van dat woord in alfabetische volgorde. Bijvoorbeeld:

Input: "software"

Output: "aeforstw"



Dat waren alle opdrachten van deze race!

Inleveropdracht 1.12: Coding Race

Lever in op Itslearning:

- Een .zip-bestand in met daarin alle projecten van de bovenstaande opdrachten.
- Een tekstdocument met antwoord op de volgende vragen:
 - Hoe snel was je klaar met deze race?
 - Hoe ging het maken van de opdrachten?
 - Welke C# onderwerpen moet jij nog eens bestuderen?
 - In welke lessen heb je over die C# onderwerpen geleerd, die nu moeizaam gingen?
 - Hoe ga je jezelf voorbereiden op de volgende lessen, waar je deze C# vaardigheden moet beheersen?

Ga thuis aan de slag om jezelf voor te bereiden op de volgende les. **Let op dat je alle bovenstaande opdrachten zónder hulp van een Kunstmatige Intelligentie moeten kunnen maken. Benader jouw docent als je vragen/zorgen hebt.**

Succes!

2. C#: OOP

2.1. Klassen en Eigenschappen in C#

In C# worden klassen gebruikt om objecten te definiëren. Een klasse is als een blauwdruk of een sjabloon waarmee je objecten kunt maken. Een object is een specifieke instantie van een klasse.

Een klasse bevat gegevens en functies die met die gegevens kunnen werken. De gegevens worden ook wel "eigenschappen" genoemd. Eigenschappen definiëren de kenmerken van een object. Bijvoorbeeld, als we een klasse "Auto" hebben, kunnen de eigenschappen zijn: merk, model, kleur en aantal deuren.

Hier is een voorbeeld van een eenvoudige klasse in C# die als data record dient:



```
1 public class Persoon
2 {
3     // Eigenschappen
4     public string Naam { get; set; }
5     public int Leeftijd { get; set; }
6 }
```

In dit voorbeeld hebben we de klasse "Persoon" gedefinieerd met twee eigenschappen: "Naam" en "Leeftijd". De eigenschappen zijn gedefinieerd met behulp van de "get" en "set" *accessors*, waardoor we waarden kunnen ophalen en toewijzen aan deze eigenschappen.

2.2. Klassen schrijven

Zorg ervoor dat je de deelopdrachten begrijpt en dat je kunt uitleggen wat er gebeurt in elke stap. Probeer ook zelf variaties te maken op de gegeven deelopdrachten om je begrip verder te ontwikkelen.

Deelopdracht 1:

1. Neem de 'Persoon' klasse over uit het voorbeeld voorafgaand aan deze opdracht.
2. Maak een nieuwe klasse genaamd "Product" met de volgende eigenschappen:
 - Naam (van het product)
 - Prijs (in euro's)
 - Voorraad (aantal beschikbare exemplaren)

Deelopdracht 2:

1. Maak een object van de klasse "Persoon" met de naam "persoon1".
2. Wijs de waarde "John" toe aan de eigenschap "Naam" van het object.
3. Wijs de waarde 25 toe aan de eigenschap "Leeftijd" van het object.
4. Schrijf code om de waarden van de eigenschappen van "persoon1" af te drukken naar de console.

Deelopdracht 3:

1. Maak een object van de klasse "Product" met de naam "product1".
2. Wijs een productnaam, prijs en voorraadwaarde toe aan de eigenschappen van het object.
3. Schrijf code om de waarden van de eigenschappen van "product1" af te drukken naar de console.

2.3. Instanties van klassen

In de vorige paragraaf hebben we geleerd over klassen en eigenschappen in C#. Nu gaan we verder met het concept van instanties van klassen.

Een instantie van een klasse wordt ook wel een object genoemd. Een object is een specifiek exemplaar van een klasse. Het is als een fysiek object dat je kunt aanraken, terwijl de klasse zelf meer een abstract concept is.

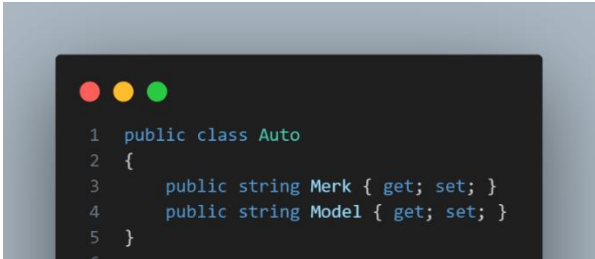
Wanneer we een klasse definiëren, maken we eigenlijk een blauwdruk voor objecten. We specificeren de eigenschappen en functies die objecten van die klasse zullen hebben. Om daadwerkelijk objecten te maken, moeten we instanties van die klasse creëren.

Verskil tussen Instanties en Klassen:

Het belangrijkste verschil tussen instanties en klassen is dat een klasse een abstracte definitie is, terwijl een instantie een specifiek object is dat is gemaakt op basis van die klasse. Klassen bevatten de algemene structuur en kenmerken, terwijl instanties de concrete gegevens en specifieke toestanden vertegenwoordigen.

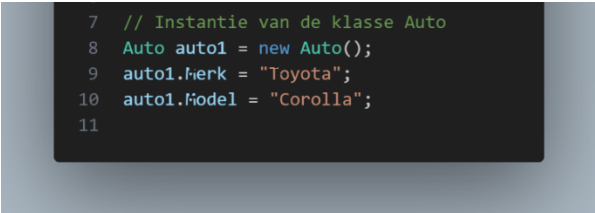
Laten we eens kijken naar een voorbeeld om het verschil te verduidelijken:

Een klasse in een eigen bestand:



```
1 public class Auto
2 {
3     public string Merk { get; set; }
4     public string Model { get; set; }
5 }
```

Ergens in de Program.Main:



```
7 // Instantie van de klasse Auto
8 Auto auto1 = new Auto();
9 auto1.Merk = "Toyota";
10 auto1.Model = "Corolla";
11
```

In dit voorbeeld hebben we de klasse "Auto" gedefinieerd met de eigenschappen "Merk" en "Model". Vervolgens hebben we een instantie van de klasse gemaakt met de naam "auto1". Deze instantie bevat de specifieke gegevens voor een auto van het merk "Toyota" en het model "Corolla".

2.4. Klassenrelaties

Klassen kunnen ook relaties hebben tot andere klassen. Deze relaties kunnen verschillende vormen aannemen, zoals associaties, aggregaties of overerving (ook wel bekend als erfelijkheid).

- Een **associatie** wordt gebruikt om te beschrijven dat twee klassen met elkaar verbonden zijn op een bepaalde manier, bijvoorbeeld dat een klas een ander object bevat of ermee samenwerkt.
- Een **aggregatie** wordt gebruikt wanneer een klasse een verzameling objecten bevat. Het is als een deel-geheel relatie, waarbij het geheel (de ene klasse) bestaat uit delen (de andere klasse).
- **Overerving** wordt gebruikt wanneer een klasse eigenschappen en functies erft van een andere klasse. Dit helpt om codehergebruik te bevorderen en de structuur van klassen te organiseren.

Bij **aggregatie** vindt per definitie ook **associatie** plaats. Aggregatie is een specifiek type associatie waarbij een klasse een verzameling van andere objecten bevat (de delen). Het geheel en de delen zijn dus met elkaar geassocieerd.

Associatie is een breder begrip dat verwijst naar de relatie tussen objecten van verschillende klassen, waarbij ze op een bepaalde manier met elkaar verbonden zijn. Aggregatie is een specifieke vorm van associatie die de deel-geheel relatie tussen klassen weergeeft.

2.5. Klassenrelatie: associatie

Een associatie is een vorm van klassenrelatie waarbij twee klassen met elkaar verbonden zijn op een bepaalde manier. Het beschrijft de interactie of relatie tussen objecten van verschillende klassen.

Laten we een voorbeeld bekijken van een associatie tussen de klassen "Klant" en "Bestelling":

```
1 public class Klant
2 {
3     public string Naam { get; set; }
4     public List<Bestelling> Bestellingen { get; set; }
5 }
6
7 public class Bestelling
8 {
9     public int Bestelnummer { get; set; }
10    public DateTime Besteldatum { get; set; }
11 }
12
```

In dit voorbeeld heeft de klasse "Klant" een associatie met de klasse "Bestelling". De klasse "Klant" heeft een eigenschap genaamd "Bestellingen" van het type `List<Bestelling>`. Dit betekent dat elke klant meerdere bestellingen kan hebben, en de associatie tussen de klant en zijn bestellingen wordt tot uitdrukking gebracht door de lijst van "Bestelling"-objecten.

Om dit in actie te zien, kunnen we de volgende code schrijven:

```

1  Klant klant1 = new Klant();
2  klant1.Naam = "Jan";
3  klant1.Bestellingen = new List<Bestelling>();
4
5  Bestelling bestelling1 = new Bestelling();
6  bestelling1.Bestelnummer = 1;
7  bestelling1.Besteldatum = DateTime.Now;
8
9  klant1.Bestellingen.Add(bestelling1);
10
11 // Toegang tot de bestellingen van klant1
12 foreach (Bestelling bestelling in klant1.Bestellingen)
13 {
14     Console.WriteLine("Bestelnummer: " + bestelling.Bestelnummer);
15     Console.WriteLine("Besteldatum: " + bestelling.Besteldatum);
16 }
17

```

In dit voorbeeld maken we een instantie van de klasse "Klant" genaamd "klant1" en een instantie van de klasse "Bestelling" genaamd "bestelling1". We voegen vervolgens "bestelling1" toe aan de lijst van bestellingen van "klant1". Door de associatie tussen de klassen kunnen we toegang krijgen tot de bestellingen van een klant en de eigenschappen van elke bestelling afzonderlijk afdrukken.

Dit voorbeeld illustreert hoe associaties kunnen worden gebruikt om de relaties tussen objecten van verschillende klassen weer te geven en hoe we deze relaties kunnen benutten in ons programma. Ook zien we hier een voorbeeld van aggregatie, omdat een klant een verzameling van "Bestelling" bevat.

2.7. Opdracht: Instanties van klassen en relaties

Denk goed na over de relatie tussen klassen en welke eigenschappen mogelijk een ander klasse-object als datatype moeten hebben. Experimenteer met verschillende instanties en controleer of de waarden correct worden toegewezen en afgedrukt. Dit helpt je om de concepten van klassenrelaties en het gebruik van andere klassen als datatypes beter te begrijpen.

Deelopdracht 1:

1. Maak een nieuwe klasse genaamd "Student" met de eigenschappen "Naam" (van het type string) en "Leeftijd" (van het type int).
2. Maak een instantie van de klasse "Student" met de naam "student1".
3. Wijs waarden toe aan de eigenschappen van "student1".
4. Schrijf code om de waarden van de eigenschappen van "student1" af te drukken naar de console.

Deelopdracht 2:

1. Definieer een klasse genaamd "Leraar" met de eigenschappen "Naam" (van het type string) en "Vakgebied" (van het type Vak).
2. Maak een nieuwe klasse genaamd "Vak" met de eigenschappen "Naam" (van het type string) en "Omschrijving" (van het type string).

3. Maak een instantie van de klasse "Vak" met de naam "vak1".
4. Wijs waarden toe aan de eigenschappen van "vak1".
5. Maak een instantie van de klasse "Leraar" met de naam "leraar1".
6. Wijs waarden toe aan de eigenschappen van "leraar1", inclusief de instantie van het "Vak" object.
7. Schrijf code om de waarden van de eigenschappen van "leraar1" en "vak1" af te drukken naar de console.

Deelopdracht 3:

- Bedenk op basis van de onderstaande data zelf de eigenschappen en datatypes. Denk goed na of je een of meerdere extra klasse(n) moet maken.
- Maak instanties van de aanwezige klasse(n) en vul ze met de onderstaande data.
- Toon alle informatie van de instanties in de console.

Boek Titel	Naam Auteur	Geboortedatum Auteur	Jaar uitgave boek
De Hobbit	J.R.R. Tolkien	3 januari 1892	1937
Pride and Prejudice	Jane Austen	16 december 1775	1813
1984	George Orwell	25 juni 1903	1949
To Kill a Mockingbird	Harper Lee	28 april 1926	1960
Harry Potter and the Philosopher's Stone	J.K. Rowling	31 juli 1965	1997

Let op: Dit zijn fictieve gegevens en niet de werkelijke gegevens van de auteurs en boeken. Gebruik dit als testdata voor oefeningen en demonstraties.

Inleveropdracht 2.8: Multiplayer Classes

In deze inleveropdracht gaan we werken met klassen, eigenschappen, klassenrelaties en instanties. We zullen een aantal klassen definiëren die gerelateerd zijn aan multiplayer games en hun eigenschappen specificeren. Vervolgens zullen we instanties van deze klassen maken om verschillende spelers en hun statistieken weer te geven. Laten we aan de slag gaan!

Gebruik deze tabel als referentie bij het implementeren van de klassen en het toevoegen van de eigenschappen in je C#-programma:

Klasse	Eigenschappen
Speler	Naam (string): de naam van de speler
	Score (int): de score van de speler
	Level (int): het niveau van de speler
	Levens (int): het aantal levens van de speler
Team	Naam (string): de naam van het team
	Spelers (List<Speler>): een lijst met spelers van het team
Spel	Naam (string): de naam van het spel
	Teams (List<Team>): een lijst met teams van het spel

Instructies:

1. Definieer de bovengenoemde klassen (Speler, Team, Spel) in C# en voeg de vereiste eigenschappen toe aan elke klasse.
2. Implementeer een associatieve relatie tussen de klassen Speler en Team. Elk object van de klasse Speler moet aan een object van de klasse Team worden toegewezen.
3. Maak een instantie van de klasse Speler met de naam "speler1". Stel de eigenschappen (Naam, Score, Level, Levens) van "speler1" in met voorbeeldwaarden.
4. Maak een instantie van de klasse Team met de naam "team1". Voeg "speler1" toe aan het team.
5. Maak een instantie van de klasse Spel met de naam "spel1". Voeg "team1" toe aan het spel.
6. Schrijf code om de waarden van de eigenschappen van "speler1", "team1" en "spel1" af te drukken naar de console.
7. Vul de code aan zodat er nu:
 - 2 teams zijn
 - ieder team 3 spelers heeft
8. Toon alle namen van de spelers in ieder team op een slimme manier. Voorkom herhalende code.

Lever in op Itslearning:

- Een .zip-bestand in met daarin alle projecten van de bovenstaande opdrachten.

2.9. Klasse met een statische eigenschap

In C# is het mogelijk om een eigenschap of methode in een klasse als "static" te markeren. Het begrip "static" geeft aan dat de eigenschap of methode wordt gedeeld door alle instanties van die klasse, in plaats van specifiek te zijn voor elke individuele instantie.

Wanneer een eigenschap of methode als static wordt gemarkeerd, betekent dit dat deze eigenschap of methode kan worden benaderd zonder dat er een instantie van de klasse wordt gemaakt. Je kunt er rechtstreeks toegang toe krijgen via de klasse zelf.

Hier is een voorbeeld van een klasse met een enkele statische eigenschap:

```
1 public class Voorraad
2 {
3     public static int AantalProducten { get; set; }
4 }
5
```

In dit voorbeeld hebben we de klasse "Voorraad" gedefinieerd met een enkele statische eigenschap genaamd "AantalProducten". Deze eigenschap houdt het totale aantal producten bij in de voorraad. Omdat het static is, kan het worden benaderd zonder dat er een instantie van de klasse "Voorraad" nodig is.

Het meest bekende voorbeeld van een statische klasse is de 'Program'-klasse bij een console applicatie. Het is binnen onze code logisch dat er maar één versie van ons Program is. Daarom heeft Microsoft die klasse 'static' gemaakt.

Om het concept van 'static' te illustreren kunnen we een analogie bedenken. Een analogie is een overeenkomst tussen twee zaken.

Analogie: Statische Game Console vs. Individuele Spelcomputers

Stel je een gamingcentrum voor met een reeks gameconsoles. Elke console kan verschillende spellen spelen en biedt individuele gameplay-ervaringen.

- **Statische** Game Console: De statische gameconsole is een centrale console die wordt gedeeld door alle spelers in het gamingcentrum. In dit geval is de gameconsole als **static** gemarkeerd. Het betekent dat de console onafhankelijk is van individuele spelers en door iedereen kan worden gebruikt zonder dat er extra consoles nodig zijn.

Stel je voor dat de statische gameconsole een menu heeft waarin spelers verschillende games kunnen selecteren. Wanneer een speler een specifiek spel start, heeft dit invloed op de gameplay en de spelomgeving voor elke volgende speler. De keuze van één speler heeft een direct effect op de ervaring van andere spelers, omdat ze allemaal dezelfde gedeelde gameconsole gebruiken.

- **Individuele** Spelcomputers: In tegenstelling tot de statische gameconsole, hebben individuele spelcomputers een meer persoonlijke benadering. Elke speler heeft zijn eigen individuele

spelcomputer waarop hij zijn gewenste games kan spelen en een unieke gameplay-ervaring kan hebben. Deze individuele spelcomputers zijn vergelijkbaar met instanties van een klasse.

Elke speler kan zijn eigen spelcomputer aanpassen met verschillende instellingen, grafische voorkeuren, en controllerconfiguraties. Als een speler bijvoorbeeld de taal of het moeilijkheidsniveau van een spel wil wijzigen, heeft dit geen directe invloed op de gameplay van andere spelers. Elke individuele spelcomputer behoudt zijn eigen instellingen en heeft geen impact op de spelervaring van andere spelcomputers.

De analogie van de statische gameconsole versus individuele spelcomputers illustreert het concept van "static" waarbij een enkele gedeelde bron wordt gebruikt door alle instanties (spelers) en het concept van "instanties" waarbij elke instantie onafhankelijk is en zijn eigen spelervaring heeft.

2.10. Opdracht: Static om te tellen

Zorg ervoor dat je de concepten van "static" begrijpt en hoe statische eigenschappen kunnen worden gebruikt. Voer de opdrachten uit en controleer of je de waarden van de statische eigenschappen correct kunt wijzigen en afdrukken. Experimenteer ook met het maken van extra klassen met statische eigenschappen om je begrip verder te ontwikkelen.

Deelopdracht 1:

1. Neem de 'Voorraad'-klasse over uit het screenshot voorafgaand aan deze opdracht.
2. Maak een nieuwe klasse genaamd "Winkel" met een statische eigenschap genaamd "AantalKlanten". Deze eigenschap moet bijhouden hoeveel klanten er momenteel in de winkel zijn.

Deelopdracht 2:

1. Schrijf code om de waarde van de statische eigenschap "AantalProducten" van de klasse "Voorraad" te wijzigen.
2. Schrijf code om de waarde van de statische eigenschap "AantalKlanten" van de klasse "Winkel" te wijzigen.

Deelopdracht 3:

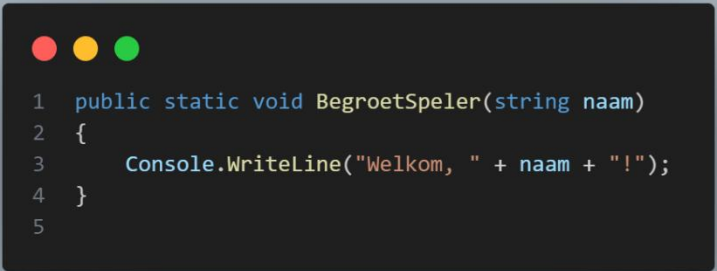
1. Schrijf code om de waarden van de statische eigenschappen "AantalProducten" van de klasse "Voorraad" en "AantalKlanten" van de klasse "Winkel" af te drukken naar de console.

2.11. Methodes in C#: Code Leesbaarheid en Functie-Aanroep

Methodes in C# zijn blokken code die een specifieke taak uitvoeren. Ze worden gebruikt om code te organiseren en functionaliteit te hergebruiken. Methodes kunnen inputparameters accepteren, bewerkingen uitvoeren en optioneel een resultaat teruggeven.

Om een methode uit te voeren, moet je deze aanroepen. Je geeft de methode de benodigde inputparameters en je kunt het resultaat (indien van toepassing) toewijzen aan een variabele of gebruiken in je code.

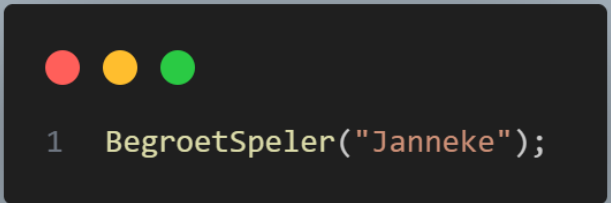
Hier is een voorbeeld van een methode genaamd "BegroetSpeler" die een begroeting afdruckt op basis van een gegeven naam:



```
1 public static void BegroetSpeler(string naam)
2 {
3     Console.WriteLine("Welkom, " + naam + "!");
4 }
5
```

In dit voorbeeld accepteert de methode "BegroetSpeler" een parameter genaamd "naam" van het type string. De methode drukt een begroeting af naar de console met behulp van de gegeven naam.

Om deze methode aan te roepen en een speler te begroeten, kun je de volgende code gebruiken:



```
1 BegroetSpeler("Janneke");
```

Dit zal de methode "BegroetSpeler" aanroepen met de waarde "John" als de naam van de speler. Als resultaat zal het de begroeting "Welkom, John!" afdrukken naar de console.

Methodes inzetten om code leesbaarheid te verbeteren:

Methodes kunnen ook worden gebruikt om de leesbaarheid van je code te verbeteren en duplicatie te verminderen. Door de logica van herhaalde code in een methode te plaatsen, kun je de methode meerdere keren aanroepen in plaats van dezelfde code herhaaldelijk te typen.

Bijvoorbeeld, als je meerdere keren de oppervlakte van een rechthoek wilt berekenen en afdrukken:

```

1  int lengte1 = 5;
2  int breedte1 = 3;
3  int oppervlakte1 = lengte1 * breedte1;
4  Console.WriteLine("De oppervlakte is: " + oppervlakte1);
5
6  int lengte2 = 7;
7  int breedte2 = 4;
8  int oppervlakte2 = lengte2 * breedte2;
9  Console.WriteLine("De oppervlakte is: " + oppervlakte2);
10

```

Je kunt dit stukje code in een methode plaatsen genaamd "BerekenEnToonOppervlakte" voor hergebruik:

```

1  public static void BerekenEnToonOppervlakte(int lengte, int breedte)
2  {
3      int oppervlakte = lengte * breedte;
4      Console.WriteLine("De oppervlakte is: " + oppervlakte);
5  }
6

```

Nu kun je de methode "BerekenEnToonOppervlakte" aanroepen telkens wanneer je de oppervlakte van een rechthoek wilt berekenen en afdrukken:

```

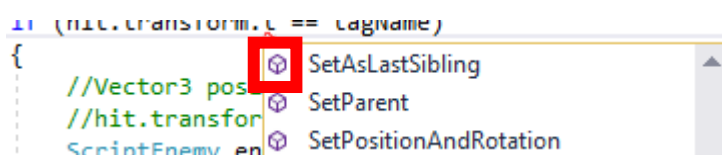
1  BerekenEnToonOppervlakte(5, 3);
2  BerekenEnToonOppervlakte(7, 4);

```

Dit maakt de code veel netter en voorkomt duplicatie van dezelfde berekeningslogica en afdrukcode.

Op deze manier illustreert het voorbeeld duidelijker dat het aanroepen van een methode de code leesbaarder maakt en herhaling van dezelfde code vermijdt.

We herkennen methodes in IntelliSense aan het roze kubus icoontje:



2.12. Opdracht: Methodes

Voer deze deelopdrachten uit en controleer of je de methodes correct hebt gedefinieerd en aangeroepen. Experimenteer ook met het maken van extra methodes om je begrip verder te ontwikkelen.

Deelopdracht 1:

1. Definieer een methode genaamd "VerdubbelGetal" die een geheel getal accepteert als parameter en het verdubbelt.
2. Roep de methode "VerdubbelGetal" aan met een geheel getal als argument en wijs het resultaat toe aan een variabele.
3. Druk het verdubbelde getal af naar de console.

Deelopdracht 2:

1. Definieer een methode genaamd "BegroetSpelers" die een array van namen accepteert als parameter en elke speler begroet.
2. Roep de methode "BegroetSpelers" aan met een array van namen als argument.

Deelopdracht 3:

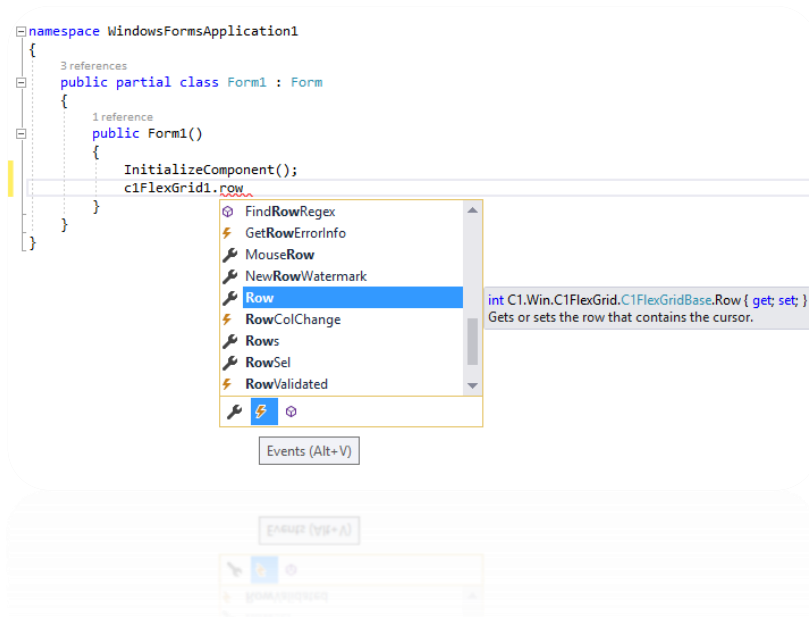
1. Identificeer een stukje code in je eigen programma dat herhaaldelijk voorkomt en geschikt is om in een methode te plaatsen.
2. Schrijf een methode om die herhaalde code te vervangen.
3. Pas je hoofdprogramma aan om de methode op de juiste plaatsen aan te roepen.

2.13. Access modifiers

Nu we relaties hebben tussen klassen, moeten we ervoor zorgen dat methoden en eigenschappen al dan niet bereikbaar zijn van buitenaf. Dit stelt ons in staat om de controle te behouden over welke aspecten van een klasse toegankelijk zijn voor andere delen van het programma. Door het gebruik van access modifiers kunnen we bewust kiezen welke eigenschappen we beschikbaar maken en welke we liever beperken tot interne gebruik binnen de klasse zelf.

Het doel van het beperken van de toegankelijkheid is om onze collega-programmeurs niet onbeperkte toegang te geven tot iedere variabele in onze klassen. We willen precies aangeven welke eigenschappen extern beschikbaar moeten zijn (met behulp van public of internal) en welke we liever binnen de klasse houden (private). Op deze manier kunnen we de implementatiedetails van een klasse verbergen en een duidelijke interface bieden voor externe interactie.

Wanneer we variabelen binnen de klasse gebruiken en niet zeker zijn of de code correct blijft werken als we toegang van buitenaf zouden toestaan, kiezen we ervoor om deze variabelen als private te markeren. Dit beperkt de toegang tot alleen de klasse zelf, waardoor we meer controle hebben over hoe deze variabelen worden gebruikt en gewijzigd. Dit zorgt voor een veiligere en meer gecontroleerde omgeving binnen de klasse.



Alleen de eigenschappen, methodes en events die door Microsoft public gemaakt zijn zien we hier. Zo voorkomen ze dat we een intern veld aanpassen binnen elementen.

Met publieke eigenschappen en methodes zeggen we dus eigenlijk: “het is de bedoeling dat je deze kunt gebruiken, we hebben ze goed getest.”

De belangrijkste access modifiers:

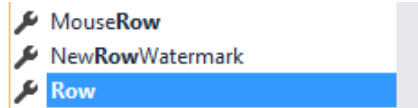
Access Modifier	Beschrijving
Public	Het public access modifier maakt een lid van een klasse toegankelijk voor alle andere delen van het programma, zowel binnen de klasse zelf als buiten de klasse. Zelfs buiten de ‘assembly’ (ofwel het project) kan het gebruikt worden. <i>Bijvoorbeeld: een hoop Windows.Forms klassen, methodes en eigenschappen zijn public, zodat ontwikkelaars deze kunnen gebruiken in hun eigen projecten.</i>
Private	Het private access modifier beperkt de toegankelijkheid van een lid tot de klasse waarin het is gedefinieerd. Een private lid kan alleen worden geopend en gebruikt binnen dezelfde klasse en is niet zichtbaar voor andere klassen of objecten.
Protected	Het protected access modifier maakt een lid toegankelijk binnen dezelfde klasse en binnen afgeleide klassen. Dit betekent dat een protected lid kan worden geopend en gebruikt door de klasse zelf en door elke klasse die van die klasse erft. Het is niet zichtbaar voor andere klassen buiten de hiërarchie van afgeleide klassen.
Internal	Het internal access modifier beperkt de toegankelijkheid van een lid tot de huidige assembly (ofwel project). Een internal lid kan worden geopend en gebruikt door andere klassen en objecten binnen dezelfde assembly, maar is niet zichtbaar voor externe assemblies.

We denken vaak niet na over hoe andere ontwikkelaars onze klassen en methodes buiten ons project kunnen gebruiken. Daarom zou ‘internal’ in de meeste gevallen beter geschikt zijn dan ‘public’. Toch zien we op internet veel voorbeelden (en gebruiken we onbewust) toch vaak ‘public’.

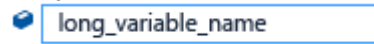
Gebruik binnen je klasse zoveel mogelijk ‘private’ voor methodes en velden, totdat je denkt dat het wel vanaf buiten de klasse moet kunnen worden benaderd. Dan maak je dat tot een public lid.

Binnen C# is de conventie:

- Willen we een veld buiten de klasse beschikbaar maken? Maak deze dan public en voeg { get; set; } toe. Dit maakt het een eigenschap. *Eigenschappen herkennen we in IntelliSense aan het moersleuteltje:*



- Willen we een veld binnen de klasse houden? Maak het veld dan private. Dit maakt het een veld. *Velden herkennen we in IntelliSense aan dit blokje:*



Wist je dat...

Het woord "lid" verwijst in de context van klassen naar een onderdeel of element van een klasse. Het kan verwijzen naar verschillende entiteiten binnen een klasse, zoals:

- Eigenschappen / **Properties**
- Methoden / **Methods**
- Velden / **Fields**
- Gebeurtenissen / **Events**

2.14. Bouwen op Bestaande Klassen

In C# en het bredere concept van object georiënteerd programmeren (OOP) is "inheritance" (erfenis) een fundamenteel concept. Inheritance stelt ons in staat om nieuwe klassen te bouwen op basis van bestaande klassen, waardoor we code kunnen hergebruiken en de structuur van ons programma kunnen organiseren.

Object georiënteerd programmeren (OOP) is een programmeerparadigma dat zich richt op het modelleren van problemen met behulp van objecten die eigenschappen en gedrag hebben. Inheritance is een belangrijk aspect van OOP en stelt ons in staat om klassen hiërarchisch te organiseren en relaties tussen klassen tot uitdrukking te brengen.

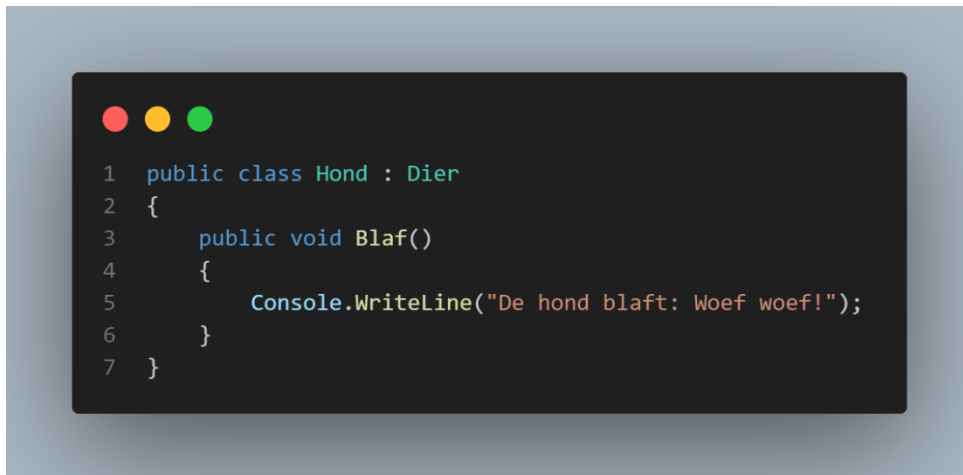
In C# kunnen we inheritance toepassen door een nieuwe klasse te definiëren op basis van een bestaande klasse. De nieuwe klasse erft dan de eigenschappen en methoden van de bestaande klasse, waardoor we deze kunnen uitbreiden of aanpassen.

Hier is een voorbeeld van een eenvoudige klasse genaamd "Dier":

```

1  public class Dier
2  {
3      public string Naam { get; set; }
4      public int Leeftijd { get; set; }
5
6      public void MaakGeluid()
7      {
8          Console.WriteLine("Het dier maakt een geluid.");
9      }
10 }
```

Nu kunnen we een nieuwe klasse "Hond" maken die erft van de klasse "Dier":



```
1 public class Hond : Dier
2 {
3     public void Blaf()
4     {
5         Console.WriteLine("De hond blaft: Woef woef!");
6     }
7 }
```

De klasse "Hond" erft de eigenschappen en de methode "MaakGeluid" van de klasse "Dier". Daarnaast heeft de klasse "Hond" ook zijn eigen specifieke methode "Blaf".

We herkennen overerving aan de 'dubbele punt' achter de klasse, waarop de klasse volgt waaruit wordt overgeërfd.

Dubbele punt in het Engels is 'colon'

2.15. Opdracht: Overerving

Voer deze deelopdrachten uit en controleer of je de nieuwe klassen hebt gedefinieerd, instanties hebt gemaakt en de eigenschappen en methoden correct hebt aangeroepen.

Deelopdracht 1:

1. Neem de Dier en Hond klassen over uit de bovenstaande uitleg.
2. Definieer een nieuwe klasse genaamd "Kat" die erft van de klasse "Dier".
3. Voeg een specifieke methode toe aan de klasse "Kat" genaamd "Miauw" die een miauwgeluid afdruckt.

Deelopdracht 2:

1. Maak een instantie van de klasse "Hond" genaamd "hond1". Stel de eigenschappen van de hond in en roep de methoden "MaakGeluid" en "Blaf" aan.
2. Maak een instantie van de klasse "Kat" genaamd "kat1". Stel de eigenschappen van de kat in en roep de methoden "MaakGeluid" en "Miauw" aan.

Deelopdracht 3:

1. Schrijf code om de waarden van de eigenschappen van "hond1" en "kat1" af te drukken naar de console.

2.16. Praktischer voorbeeld overerving

Laten we naar een praktischer voorbeeld van overerving kijken. Hieronder zien we zo code van een 'fleet management system'. Dit systeem kan worden gebruikt om de voertuigen van een transportbedrijf te beheren en te volgen. Hier is hoe de klassen in dit scenario kunnen worden toegepast:

```

1  public class Voertuig
2  {
3      public string Merk { get; set; }
4      public string Model { get; set; }
5      public int Bouwjaar { get; set; }
6
7      public virtual void StartMotor()
8      {
9          Console.WriteLine("De motor van het voertuig is gestart.");
10     }
11 }
12
13 public class Auto : Voertuig
14 {
15     public int AantalDeuren { get; set; }
16
17     public override void StartMotor()
18     {
19         Console.WriteLine("De auto start met een sleutel.");
20     }
21
22     public void SchakelVerlichtingIn()
23     {
24         Console.WriteLine("De auto verlichting is ingeschakeld.");
25     }
26 }
27
28 public class Vrachtwagen : Voertuig
29 {
30     public int MaximaalLaadgewicht { get; set; }
31
32     public override void StartMotor()
33     {
34         Console.WriteLine("De vrachtwagen start met een startknop.");
35     }
36
37     public void LaadGoederen()
38     {
39         Console.WriteLine("De vrachtwagen laadt goederen.");
40     }
41 }

```

In dit voorbeeld hebben we de basisklasse **Voertuig** gedefinieerd, die eigenschappen zoals **Merk**, **Model** en **Bouwjaar** bevat. De klasse heeft ook een virtuele methode **StartMotor**, die kan worden overschreven door de afgeleide klassen.

We hebben vervolgens de klassen **Auto** en **Vrachtwagen** gemaakt, die beiden erven van de klasse **Voertuig**. Deze afgeleide klassen voegen hun eigen specifieke eigenschappen en methoden toe, zoals **AantalDeuren** en **SchakelVerlichtingIn** voor de auto, en **MaximaalLaadgewicht** en **LaadGoederen** voor de vrachtwagen. Ze overschrijven ook de **StartMotor**-methode om hun eigen opstartgedrag te definiëren.

Met deze hiërarchie van klassen kunnen we nu instanties maken van zowel auto's als vrachtwagens en hun specifieke eigenschappen en methoden gebruiken, terwijl we ook de gemeenschappelijke kenmerken en gedragingen van voertuigen behouden.

In dit fleet management-systeem kunnen we bijvoorbeeld een lijst van voertuigen bijhouden, inclusief auto's en vrachtwagens. We kunnen de specifieke eigenschappen van elk voertuig vastleggen, zoals het merk, model, bouwjaar en aanvullende kenmerken zoals het aantal deuren voor auto's en het maximale laadgewicht voor vrachtwagens.

We kunnen ook functionaliteit toevoegen om de voertuigen te beheren. Bijvoorbeeld:

```

1  List<Voertuig> voertuigen = new List<Voertuig>();
2
3  // Toevoegen van een auto
4  Auto auto = new Auto();
5  auto.Merk = "Volkswagen";
6  auto.Model = "Golf";
7  auto.Bouwjaar = 2022;
8  auto.AantalDeuren = 5;
9  voertuigen.Add(auto);
10
11 // Toevoegen van een vrachtwagen
12 Vrachtwagen vrachtwagen = new Vrachtwagen();
13 vrachtwagen.Merk = "Scania";
14 vrachtwagen.Model = "R500";
15 vrachtwagen.Bouwjaar = 2019;
16 vrachtwagen.MaximaalLaadgewicht = 25000;
17 voertuigen.Add(vrachtwagen);
18
19 // Starten van de motor van elk voertuig
20 foreach (Voertuig voertuig in voertuigen)
21 {
22     voertuig.StartMotor();
23 }
24
25 // Specifieke acties uitvoeren voor elk type voertuig
26 if (auto is Auto autoVoertuig)
27 {
28     autoVoertuig.SchakelVerlichtingIn();
29 }
30 else if (vrachtwagen is Vrachtwagen vrachtwagenVoertuig)
31 {
32     vrachtwagenVoertuig.LaadGoederen();
33 }
34

```

Dit voorbeeld illustreert hoe we de klassen **Auto** en **Vrachtwagen** kunnen gebruiken in een fleet managementsysteem. We kunnen voertuigen van beide klassen instantiëren, hun eigenschappen instellen en hun specifieke methoden aanroepen. Door de gemeenschappelijke basis van de Voertuig-klasse kunnen we de voertuigen beheren en relevante functionaliteit toepassen op basis van hun specifieke kenmerken en gedrag.

2.17. De 'is' operator om type te bepalen

We zagen in het voorbeeld hierboven een operator die mogelijk nieuw voor je is. Je kent al operators als 'if', 'for' en comparison operators als '==' en '>'. Maar bij het voorbeeld van de Auto en Vrachtwagen die overerven van Auto zien we het gebruik van de 'is' operator.

In C# wordt de 'is'-operator gebruikt om te controleren of een object van een bepaald type is. Het retourneert een boolean-waarde (true of false) op basis van de vraag of het object kan worden toegewezen aan het opgegeven type.

De syntaxis voor het gebruik van de 'is'-operator is als volgt:

expressie is type

Hierbij is 'expressie' het object of de waarde die je wilt controleren en 'type' is het type waarmee je wilt vergelijken.

In de context van video games kunnen we de 'is'-operator gebruiken om te controleren op welk specifiek type game-object we te maken hebben. Dit is handig wanneer we verschillende soorten game-objecten hebben met unieke eigenschappen en gedragingen. Hier is een voorbeeld om de 'is'-operator in het thema van video games uit te leggen:

```
1 public abstract class GameObject { }
2
3 public class Player : GameObject { }
4
5 public class Enemy : GameObject { }
6
7 public static void Main()
8 {
9     GameObject gameObj = new Player();
10
11     if (gameObj is Player)
12     {
13         Console.WriteLine("Het object is een spelerkarakter.");
14     }
15     else if (gameObj is Enemy)
16     {
17         Console.WriteLine("Het object is een vijandkarakter.");
18     }
19     else
20     {
21         Console.WriteLine("Het object is van een ander type.");
22     }
23 }
```

In dit voorbeeld hebben we een hiërarchie van game-objectklassen: **GameObject**, **Player** en **Enemy**. We maken een instantie van **Player** en wijzen deze toe aan een variabele van het type **GameObject**.

Met behulp van de 'is'-operator controleren we of het object in de variabele van het type **Player** is. Als dat het geval is, wordt de bijbehorende boodschap afgedrukt die aangeeft dat het object een spelerkarakter is. Als het object van het type **Enemy** zou zijn, zou de daarbij horende boodschap worden afgedrukt. Als het object van een ander type zou zijn, zou de fallback-boodschap worden afgedrukt.

Door de 'is'-operator te gebruiken, kunnen we dynamisch het specifieke type van game-objecten controleren en specifieke acties uitvoeren op basis van hun type. Hiermee kunnen we game-objecten op basis van hun type, ieder ander gedrag geven en andere functionaliteiten implementeren.

2.18. Constructors

Constructors zijn speciale methoden die worden gebruikt om objecten van een klasse te initialiseren. Ze worden aangeroepen wanneer een nieuw object wordt gemaakt met het **new-sleutelwoord**. Constructors hebben dezelfde naam als de klasse waarin ze zich bevinden.

Constructors stellen ons in staat om de initiële staat van een object te configureren en de nodige initialisaties uit te voeren voordat het object wordt gebruikt. Ze bieden een mechanisme om waarden toe te wijzen aan de eigenschappen en velden van een object, waardoor het object bruikbaar wordt in onze code.

Er zijn twee soorten constructors:

- **Default Constructor:**
Een default constructor is een constructor zonder parameters. Als een klasse geen expliciete constructor heeft, biedt C# automatisch een default constructor. De default constructor initialiseert de

eigenschappen en velden van een object met de standaardwaarden die bij het gegevenstype horen (bijvoorbeeld 0 voor numerieke waarden, null voor referentietypen).

- **Custom Constructor:**

Een custom constructor is een constructor met parameters. We kunnen een custom constructor definiëren in een klasse om specifieke initialisaties uit te voeren op basis van de gegeven argumenten. Met een custom constructor kunnen we waarden doorgeven aan de constructor en deze toewijzen aan de eigenschappen en velden van het object.

Hier is een voorbeeld van een klasse genaamd "Person" met zowel een default constructor als een custom constructor:

```
1 public class Person
2 {
3     public string Name { get; set; }
4     public int Age { get; set; }
5
6     // Default constructor
7     public Person()
8     {
9         Name = "Unknown";
10        Age = 0;
11    }
12
13    // Custom constructor
14    public Person(string name, int age)
15    {
16        Name = name;
17        Age = age;
18    }
19 }
20
```

Default Constructor
(geen parameters)

Custom Constructor
(met parameters)

We kunnen nu objecten van de klasse "Person" maken en de constructors gebruiken:

```
1 Person person1 = new Person(); // Gebruikt de default constructor
2 Person person2 = new Person("John Doe", 30); // Gebruikt de custom constructor
3
4 Console.WriteLine(person1.Name + " - " + person1.Age); // Output: Unknown - 0
5 Console.WriteLine(person2.Name + " - " + person2.Age); // Output: John Doe - 30
```

In dit voorbeeld maken we een object "person1" met behulp van de default constructor en een object "person2" met behulp van de custom constructor. We kunnen vervolgens de eigenschappen van deze objecten afdrukken om te controleren of de constructors correct hebben gewerkt.

Soms willen we dat code in de default constructor ook wordt aangeroepen in custom constructors. Dat is bijvoorbeeld handig bij Windows (WinUI3) of Forms (Windows Forms), waar de 'InitializeComponent' aangeroepen moet worden om de XAML of Form te tekenen. In dat geval kunnen we het **'this'** keyword op deze manier gebruiken:

```
1 public ChatWindow()
2 {
3     this.InitializeComponent();
4 }
5
6 public ChatWindow(Chat chat)
7     :this() // roept ook default constructor aan.
8 {
9     CurrentChat = chat;
10 }
```

Dankzij die aanroep van **'this'** werkt de custom constructor hetzelfde als wanneer dit de code zou zijn:

```
1 public ChatWindow(Chat chat)
2 {
3     this.InitializeComponent();
4     CurrentChat = chat;
5 }
```

2.19. Abstracte klasse

We kunnen een abstracte klassen definiëren om gemeenschappelijke kenmerken en gedragingen van afgeleide klassen te delen. Een abstracte klasse kan niet rechtstreeks worden geïntanceerd, maar kan als basis dienen voor andere klassen.

Hier is een voorbeeld van een abstracte klasse genaamd "Shape" met een abstracte methode genaamd "CalculateArea":

```
1 public abstract class Shape
2 {
3     public abstract double CalculateArea();
4 }
```

In dit voorbeeld is "Shape" een abstracte klasse die een abstracte methode "CalculateArea" bevat. Merk op dat de abstracte methode geen implementatie heeft tussen de accolades. Dit dwingt af dat elke afgeleide klasse die van "Shape" erft, de abstracte methode moet overschrijven en een implementatie moet bieden.

Laten we een afgeleide klasse "Rectangle" maken die erft van de abstracte klasse "Shape":

```
1 public class Rectangle : Shape
2 {
3     public double Width { get; set; }
4     public double Height { get; set; }
5
6     public override double CalculateArea()
7     {
8         return Width * Height;
9     }
10 }
```

In deze afgeleide klasse "Rectangle" hebben we de abstracte methode "CalculateArea" overschreven en een implementatie gegeven op basis van de breedte en hoogte van het rechthoekige object.

Nu kunnen we instanties maken van de afgeleide klasse "Rectangle" en de methode "CalculateArea" aanroepen:

```
1 Rectangle rectangle = new Rectangle();
2 rectangle.Width = 5;
3 rectangle.Height = 3;
4 double area = rectangle.CalculateArea();
5 Console.WriteLine("De oppervlakte van het rechthoekige object is: " + area);
```

Dit zal de oppervlakte van het rechthoekige object berekenen en afdrukken naar de console.

Abstracte klassen bieden een nuttige manier om een gemeenschappelijke set eigenschappen en methoden te definiëren en ervoor te zorgen dat deze consistent worden geïmplementeerd in afgeleide klassen. Ze stellen ons ook in staat om polymorfisme toe te passen, waarbij we dezelfde abstracte klasse kunnen gebruiken om verschillende afgeleide klassen te behandelen.

Inleveropdracht 2.20: Role-Playing Game (RPG) Classes

Stel je voor dat je werkt aan de ontwikkeling van een eenvoudig RPG-spel. Je hebt de taak om enkele klassen te implementeren en functionaliteit toe te voegen met behulp van static-eigenschappen, methodes en inheritance.

1. (1 punt) Pak de code op de volgende pagina als startpunt, verdeel die netjes over meerdere klasse-bestanden.
2. (2 punten) Definieer een abstracte klasse genaamd "Character" met de volgende eigenschappen:
 - Naam (string): de naam van het personage.
 - Level (int): het niveau van het personage.
3. (1 punt) Definieer een afgeleide klasse genaamd "Player" die erft van de klasse "Character". Voeg de volgende eigenschap toe:
 - ExperiencePoints (int): de ervaringspunten van de speler.
4. (1 punt) Definieer een afgeleide klasse genaamd "Enemy" die erft van de klasse "Character". Voeg de volgende eigenschap toe:
 - AttackPower (int): de aanvalskracht van de vijand.
5. (1 punt) Implementeer een static-methode genaamd "Battle" in de klasse "Game" die een speler en een vijand als parameters accepteert. Deze methode simuleert een gevecht tussen de speler en de vijand en drukt het resultaat af op basis van hun eigenschappen.
 - (1 punt) Als de aanvalskracht van de vijand groter is dan het Level van de speler, wordt de vijand als winnaar afgedrukt.
 - (1 punt) Als de aanvalskracht van de vijand kleiner is aan het Level van de speler, wordt de speler als winnaar afgedrukt en krijgt deze 10 ExperiencePoints.
 - (1 punt) Als de aanvalskracht overeenkomt met het level, wordt het gelijkspel afgedrukt.
6. (1 punt) Maak een instantie van de klasse "Player" genaamd "player" en stel de eigenschappen in met voorbeeldwaarden. Maak ook een instantie van de klasse "Enemy" genaamd "enemy" en stel de eigenschappen in met voorbeeldwaarden.
7. (1 punt) Roep de static-methode "Battle" aan en geef de "player" en "enemy" als argumenten door.
8. (2 punten) Werkt het bovenstaande? Maak dan een lijst van Enemy objecten en vul die met instanties van steeds sterkere enemies.
9. (2 punten) Gebruik een loop om de speler en de Enemy via Game.Battle te laten vechten. Als de speler verliest stopt het spel.

Bonus:

Snel klaar? Voeg dan extra toe:

- (2 punten) Laat de speler een specifieke battle via een keuzemenu opnieuw kiezen. Zo kunnen ze trainen om beter te worden.
- (1 punt) Bij 100 ExperiencePoints gaat de speler een level omhoog.

Lever de uitwerking via Itslearning in bij de docent.

 **Let op:** Het inleveren van deze opdracht geldt als feedback-moment. Je ontvangt een studiepunt wanneer je deze succesvol afrond. Houdt na het inleveren Itslearning in de gaten en verwerk eventuele feedback die je van de docent krijgt.

*Er zijn in totaal **15 punten** te behalen bij deze opdracht. Met een cesuur van 70% moet je ten minste 10,5 punten scoren voor een voldoende (5,5).*



```
1  using System;
2
3  public abstract class Character
4  {
5      // Eigenschappen: Naam (string) en Level (int)
6
7      // Constructor
8
9
10 }
11
12 public class Player : Character
13 {
14     // Eigenschap: ExperiencePoints (int)
15
16
17 }
18
19 public class Enemy : Character
20 {
21     // Eigenschap: AttackPower (int)
22
23
24 }
25
26 public static class Game
27 {
28     // Static-methode: Battle()
29 }
30
31 public class Program
32 {
33     public static void Main()
34     {
35         // Instantieer de speler en de vijand
36
37         // Roep de Battle-methode aan
38     }
39 }
```

3. Entity Framework: Introductie

In 'Blok C' hebben jullie al kennis opgedaan over Entity Framework, een handig hulpmiddel om met databases te werken. In dit hoofdstuk gaan we deze kennis opfrissen en komen enkele nieuwe onderdelen aan bod.

Na dit hoofdstuk:

- Weet je welke stappen je moet ondernemen om Entity Framework i.c.m. MySQL te gebruiken in een WinUI 3 project.
- Weet je hoe je de database kan 'seeden' met testdata.
- Weet je hoe je gegevens kunt uitwisselen tussen twee WinUI 3 schermen

3.1. Opdracht: Opfriscursus

In deze opdracht ga je eerst samen nadenken over wat je nog weet over Entity Framework. Daarna ga je dat individueel verwerken tot een kort verslag. Hiermee bereid je een gesprek in de klas voor waarin je kennis ophaalt en deelt met klasgenoten.

- **Deelopdracht 1 (groep):** Bespreek in groepjes van twee of drie de volgende vraag: "Wat is Entity Framework en waarom is het handig bij het werken met databases?"
Neem enkele minuten de tijd om ideeën uit te wisselen en maak vervolgens individueel een lijst van de belangrijkste punten.
- **Deelopdracht 2 (individueel):** Werk individueel aan de volgende opdracht: "Noem ten minste drie voordelen van het gebruik van Entity Framework ten opzichte van het handmatig schrijven van SQL-queries."
Schrijf je antwoord op en bedenk voorbeelden ter ondersteuning.
- **Deelopdracht 3 (individueel):** Beschrijf in je eigen woorden wat een model is in Entity Framework. Wat zijn de belangrijkste kenmerken en hoe worden modellen gebruikt bij het communiceren met een database?
Schrijf een korte samenvatting van je antwoord.

AI-gebruik bij deze opdracht

Je mag AI gebruiken bij deze opdracht, maar alleen om je tekst te controleren en te verbeteren op taal. Laat de opdracht niet volledig uitvoeren door een AI zoals ChatGPT. Als je dat wel doet:

- Leer je weinig
- Onthoud je de informatie minder goed
- blijf je afhankelijk van AI
- kom je ongemotiveerd over

Gebruik AI slim en verantwoord. Zorg dat jij de kennis en vaardigheden ontwikkelt. Word geen doorgeefluik voor AI.

De docent leidt na afloop een klassikale bespreking over wat we nog weten.

3.2. Entity Framework met MySQL

Entity Framework is een **object-relationale mapper (ORM)** die het werken met databases vereenvoudigt. Een van de belangrijke kenmerken van Entity Framework is dat het "**database agnostisch**" is. Dit betekent

dat het niet afhankelijk is van een specifieke databaseprovider en het mogelijk maakt om met verschillende databases te werken zonder dat de applicatiecode hoeft te worden gewijzigd.

Wist je dat...

Wist je dat het woord "agnostisch" zowel in religieuze context als in de context van software ontwikkeling wordt gebruikt? In religie verwijst het naar de overtuiging dat het bestaan van een hogere macht (bijv.: God) **niet kan worden bewezen of ontkend**.

Bij software ontwikkeling betekent "X agnostisch" dat een softwareframework onafhankelijk is van specifieke implementaties van X en met verschillende X kan werken zonder codeaanpassingen. Op de plek van X kom je de term tegen bij:

- **Database agnostisch:** onafhankelijk is van specifieke databaseproviders, werkt met zowel MySQL, Microsoft SQL (MSSQL), tekstbestanden, etc.
- **Language agnostisch:** onafhankelijk van een specifieke programmeer taal, werkt voor C#, PHP, JavaScript, etc.

Net zoals een agnosticus in religie openstaat voor verschillende religieuze mogelijkheden, stelt een database-agnostisch framework ontwikkelaars in staat om flexibel met verschillende databases te werken zonder zich te binden aan één specifieke optie.

Het Engelse woord voor 'agnostisch' is 'agnostic'

Omdat wij al bekend zijn met **MySQL** gaan we Entity Framework daarmee combineren. Door de '**Pomelo.EntityFrameworkCore.MySql**' package te gebruiken, kunnen we de MySQL-databaseprovider toevoegen aan ons Entity Framework-project. Dit stelt ons in staat om de MySQL-database te benaderen, tabellen aan te maken, gegevens te lezen, te wijzigen en te verwijderen, en alle andere CRUD (Create, Read, Update, Delete) bewerkingen uit te voeren met behulp van de Entity Framework API.

3.3. Stappenplan Entity Framework

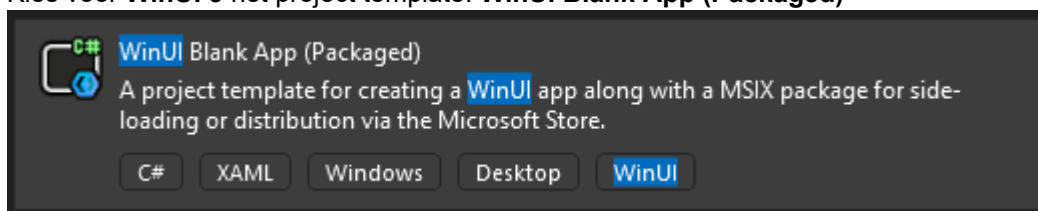
Na het stappenplan volgt een opdracht waar je dit stappenplan a.d.h.v. gedetailleerdere instructies uitvoert. Zo raak je bekend met dit proces.

De volgende stappen voer je vrijwel altijd uit in elk project waarin je met Entity Framework werkt. Na het doorlopen van de stappen heb je een project waarin je met een MySQL database kunt verbinden en gegevens kunt ophalen.

Stappenplan:

1. Open een bestaand project in Visual Studio, of begin een nieuw project en let daarbij op:

- Kies voor **WinUI 3** het project template: **WinUI Blank App (Packaged)**



Let op: in oudere versies van Visual Studio heeft deze template de naam 'Blank app, Packaged (WinUI 3 in Desktop)', als je dat ziet moet je Visual Studio updaten via de Visual Studio Installer.

- Zorg bij andere type projecten dat je ten minste **.NET 8.0** als framework versie kiest.

2. Installeer de gewenste NuGet-pakketten:

- **Pomelo.EntityFrameworkCore.MySql**

Extra Informatie:

Wij gebruiken nu MySQL omdat we daarmee bekend zijn. Als je een **Microsoft SQL** database gebruikt: installeer je de NuGet package **Microsoft.EntityFrameworkCore**.

3. Maak voor elke entiteit een Model-klasse.

Geef elke Model-klasse ten minste een 'Id'-eigenschap. Dit wordt automatisch de primaire sleutel.

4. Definieer een Database Context klasse (bijvoorbeeld genaamd AppDbContext). Zorg dat die klasse is afgeleid (overeft / 'inherit') van **Microsoft.EntityFrameworkCore.DbContext** en:

- Overschrijf de **OnConfiguring**-methode in die klasse om aan te geven hoe je een verbinding met de database wilt maken.
- Voeg in de AppDbContext "**DbSet<T>**"-eigenschappen toe voor elk model.
Dit is een verzameling (een soort lijst) van het type dat je op positie T opgeeft (bijvoorbeeld: DbSet<User> voor de User-klasse).
- Waarschijnlijk wil je ook wat testgegevens toevoegen aan de database. Dit noemen we 'database seeding' en doen we door de OnModelCreating-methode in AppDbContext te overschrijven.

Het project is nu ingesteld. Nu kun je via een instantie van de de AppDbContext de database benaderen om gegevens op te halen:

5. Maak een instantie van de AppDbContext-klasse om de database te benaderen. Je kunt de tabellen en kolommen in de database op twee manieren aanmaken:

- Aanbevolen tijdens ontwikkelen: Roep de EnsureDeleted- en EnsureCreated-methodes aan op de AppDbContext instantie.
Met EnsureDeleted verwijderen we de database. Met de EnsureCreated wordt daarna de database opgebouwd en vervolgens de 'OnModelCreating' aangeroepen.



```
1 public sealed partial class MainWindow : Window
2 {
3     public MainWindow()
4     {
5         this.InitializeComponent();
6
7         using (var db = new AppDbContext())
8         {
9             db.Database.EnsureDeleted();
10            db.Database.EnsureCreated();
11        }
12    }
13
14    // etc...
```

- In 'Live' omgevingen: werk met Migrations: <https://learn.microsoft.com/en-us/ef/core/managing-schemas/migrations/?tabs=vs>.

Met Migrations leggen we in code vast welke structuren en data er in de database aanwezig is. Dit gebruiken we in productieomgevingen, zodat we stapsgewijs databases kunnen uitbreiden en updaten met nieuwe tabellen en kolommen, zonder alle data te moeten verwijderen.

Wij kiezen voor ons gemak de EnsureDeleted en EnsureCreated methodes. Dit maakt ontwikkelen makkelijker. In productie werken we wel met Migrations. Daar kijken we in hoofdstuk 6 'Applicatie Implementeren' naar.

6. Gebruik een AppDbContext instantie om de database te benaderen.
Hier zie je een voorbeeld uit de CSD-III module:



```
1 public sealed partial class MainWindow : Window
2 {
3     private List<Car> CarList;
4
5     public MainWindow()
6     {
7         this.InitializeComponent();
8         CarList = new List<Car>();
9     }
10
11     private void loadFromDb_click(object sender, RoutedEventArgs e)
12     {
13         using (var db = new CarContext())
14         {
15             CarList = db.Cars.ToList();
16         }
17         myList.ItemsSource = CarList;
18     }
19
20     //...
```

De Database Context is hier 'CarContext' genoemd. Met de code worden alle auto's worden geladen uit de Cars eigenschap (DbSet<Car>) van de CarContext en in een List opgeslagen. Die list wordt ingesteld als ItemsSource van een lijst genaamd 'myList'. Het resultaat: alle auto's worden in de lijst getoond, bij het tonen van de MainWindow.

3.4. Opdracht: DevCitySim (Stappenplan Entity Framework)

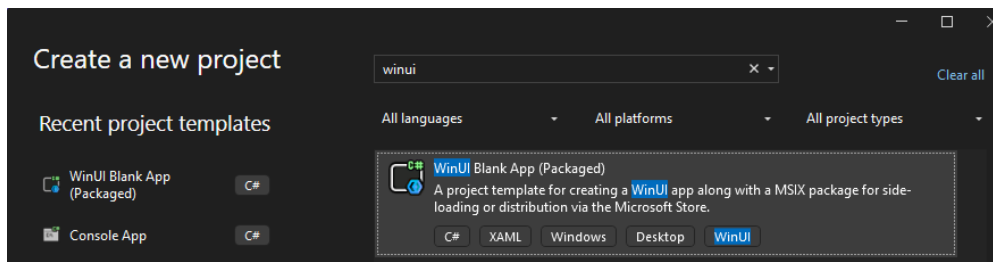
Om het stappenplan te oefenen gaan we ons richten op een game genaamd "DevCitySim: Stad Simulator". Deze game simuleert het bouwen en beheren van een virtuele stad genaamd DevCity.

In deze eerste opdracht houden we het simpel, en gaan we alleen even herhalen hoe de basis van Entity Framework werkt. Dit is hoe de applicatie eruit gaat zien, en wat we qua techniek gaan maken:

1. We gaan werken met **WinUI 3**, Entity Framework en als database een **MySQL** database.
2. We 'seeden' de database met testdata voor bewoners en gebouwen.
3. Er komen modellen voor bewoners en gebouwen.
4. Er komt een Database Context.
5. Er komt een scherm dat de bewoners gaat tonen.

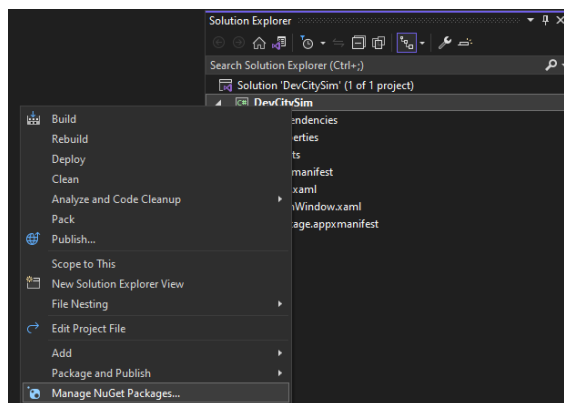
Instructies:

- Start een nieuw WinUI 3-project genaamd **DevCitySim** in Visual Studio. Gebruik het **WinUI Blank App (Packaged)** template



Let op: in oudere versies van Visual Studio heeft deze template de naam 'Blank app, Packaged (WinUI 3 in Desktop)'

- Installeer de Entity Framework via NuGet-packages in het project via je project > Manage NuGet Packages > **Browse**:

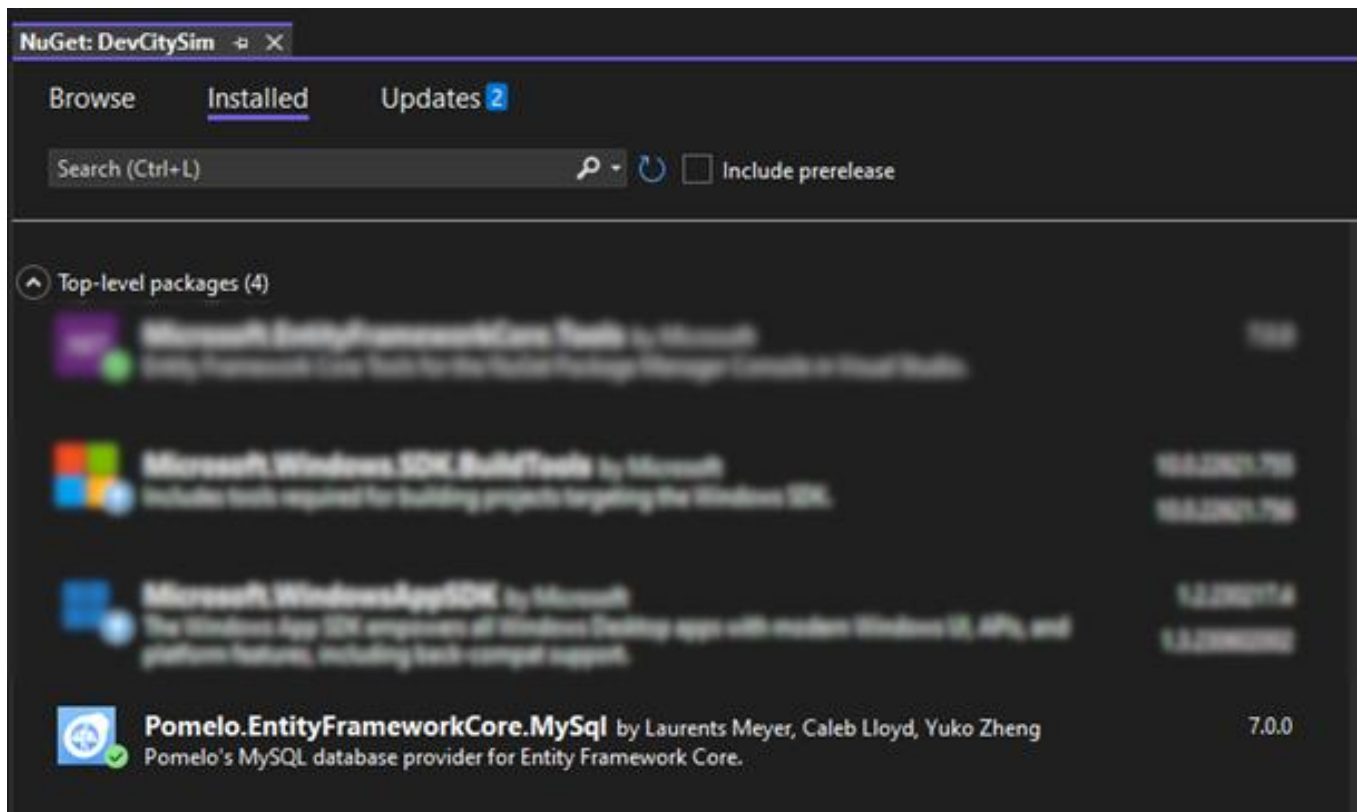


Installeer deze package:

- Pomelo.EntityFrameworkCore.MySql**

Kies als versie dezelfde 'major' versie als de .NET versie van jouw project. Als je dus .NET 8 hebt kies je de nieuwste versie van 8.x van Pomelo

Als je ze goed hebt geïnstalleerd staan de packages na afloop onder het 'Installed' tabblad:



- Maak in een nieuwe map 'Data' voor elke entiteit een Model-klasse met de juiste eigenschappen: Geef elke Model-klasse ten minste een 'Id'-eigenschap. Dit wordt automatisch de primaire sleutel.

Entiteit	Attribuut	Uitleg
Citizen	Name	Naam van bewoner, bijv.: Jane Doe
	DateOfBirth	Geboortedatum van de bewoner, bijv.: 24-10-2001
	Job	Beroep, bijv.: Software Developer
Building	Name	Naam van het gebouw of bedrijf in het gebouw, bijv.: DevCityTech
	Type	Beschrijving soort gebouw, bijv.: Kantoor, winkel, woonhuis
	Location	Adres, bijv.: Nullstraat 1

```

Citizen.cs AppDbContext.cs* MainWindow.xaml.cs MainWindow.xaml
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace DevCitySim.Data
{
    0 references
    internal class Citizen
    {
        0 references
        public int Id { get; set; }
        0 references
        public string Name { get; set; }
        0 references
        public DateTime DateOfBirth { get; set; }
        0 references
        public string Job { get; set; }
    }
}

```

- Maak een Database Context klasse genaamd AppDbContext. Zorg dat die klasse is afgeleid (overerft / 'inherit') van **Microsoft.EntityFrameworkCore.DbContext**:

```

namespace DevCitySim.Data
{
    0 references
    internal class AppDbContext : DbContext
    {
    }
}

```

Voeg een DbSet toe voor iedere model, zodat daar tabellen van gemaakt worden door Entity Framework. Stel de OnConfiguring ook zo in, dat we verbinden met MySQL:

```

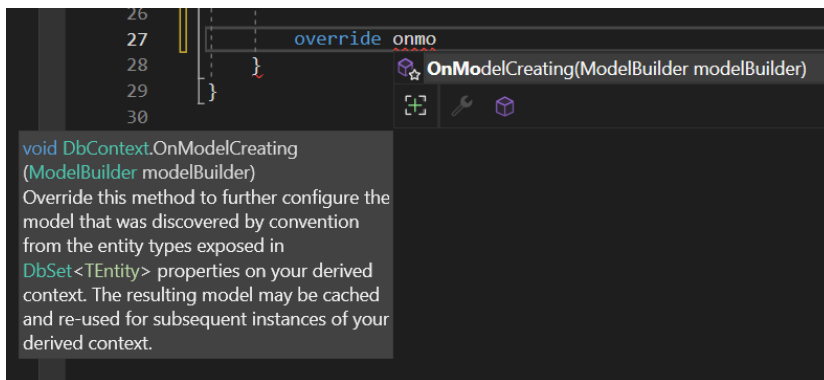
namespace DevCitySim.Data
{
    0 references
    internal class AppDbContext : DbContext
    {
        0 references
        public DbSet<Citizen> Citizens { get; set; }
        0 references
        public DbSet<Building> Buildings { get; set; }

        0 references
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            optionsBuilder.UseMySQL(
                "server=localhost;port=3306;user=root;password=;database=csd_devcitysim",
                ServerVersion.Parse("8.0.30")
            );
        }
    }
}

```

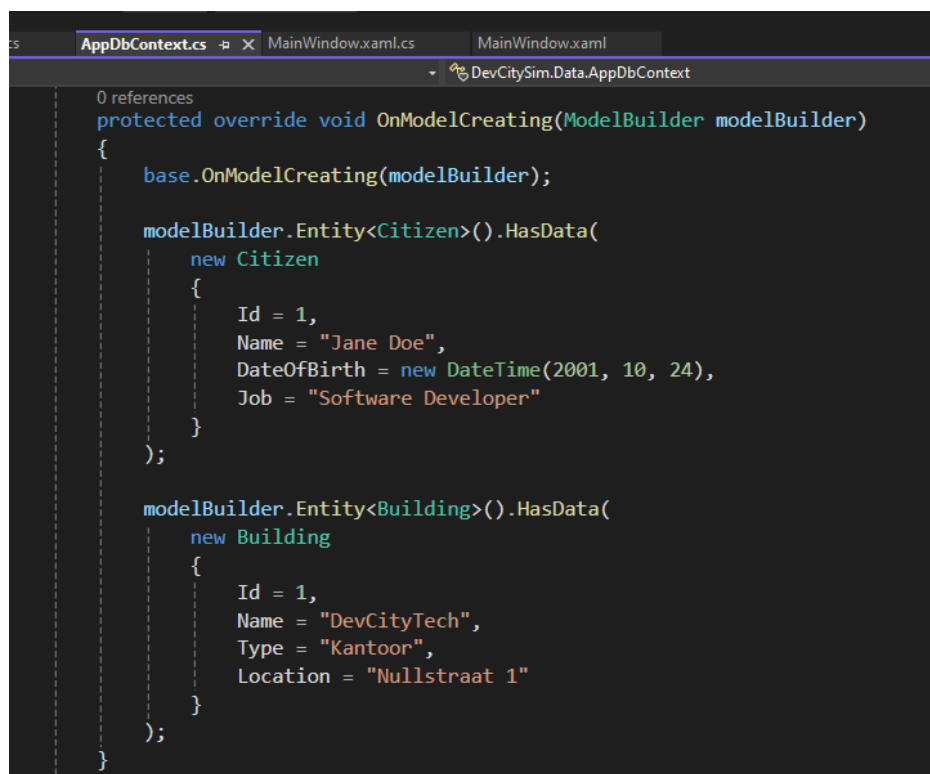
We 'overriden' / overschrijven hierboven een methode in de klasse van waaruit we overerven.

De return-type, parameters en naam van een methode die we 'overriden' hoeft u niet zelf te typen. Laten we het toevoegen van de 'OnModelCreating'-methode slim aanpakken: begin binnen de accolades van de klasse met typen van "override onmo". Wanneer de onderstaande suggestie verschijnt druk je op de TAB-toets:



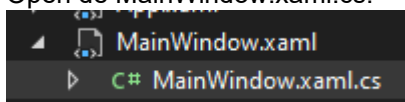
Zo krijgen we makkelijk precies de goede methode om te overschrijven in de DbContext klasse waarvan we overerven.

‘Seed’ (vul) de database met testdata door de volgende OnModelCreating (in de AppDbContext) te bouwen:



- Gebruik een instantie van de AppDbContext-klasse om de database te benaderen. We gaan de ‘EnsureCreated’ en ‘EnsureDeleted’ gebruiken om een lege testdatabase op te bouwen, iedere keer wanneer de applicatie start.

- Open de MainWindow.xaml.cs:



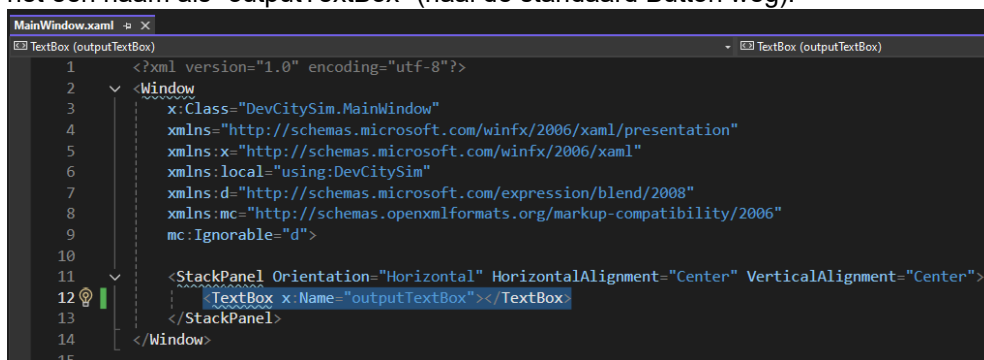
- Voeg in de **constructor van de MainWindow** de volgende code toe:



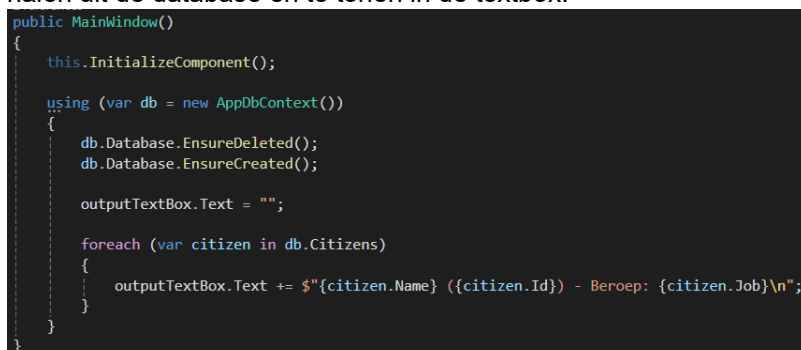
Start de applicatie op en kijk daarna in de database (via HeidiSQL of phpMyAdmin). Je ziet dat de database en twee tabellen aangemaakt zijn en met data zijn gevuld.

We gebruiken hier het “using” keyword om te zorgen dat binnen de accolades een databaseverbinding beschikbaar is. Daarbuiten wordt die netjes afgesloten.

- Gebruik de AppDbContext instantie om de database te benaderen en gegevens van bewoners op te halen en te tonen. We gaan het tonen in een TextBox:
 - a. Voeg in de MainWindow.xaml een TextBox element toe in de al bestaande StackPanel en geef het een naam als “outputTextBox” (haal de standaard Button weg):



- b. Open nu de MainWindow.xaml.cs
- c. Voeg in de constructor, na het aanmaken van de database, logica toe om de gegevens op te halen uit de database en te tonen in de textbox:



Test nu de applicatie. Je ziet de naam, id en beroep van de bewoner op het scherm.

- Voeg zelf nog 3 extra bewoners toe aan de seeder, op deze manier:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);

    modelBuilder.Entity<Citizen>().HasData(
        new Citizen
        {
            Id = 1,
            Name = "Jane Doe",
            DateOfBirth = new DateTime(2001, 10, 24),
            Job = "Software Developer"
        },
        new Citizen
        {
            Id = 2,
            Name = "John Doe",
            DateOfBirth = new DateTime(2004, 2, 1),
            Job = "Software Developer"
        }
    );
}
```

- Test opnieuw, zie je alle bewoners onder elkaar?
- De “\n” zou een nieuwe regel moeten geven na iedere bewoner. Om dit te laten werken moet de TextBox nog wel de ‘AcceptsReturn’ (accepteert ‘enters’) attribuut krijgen in de XAML:

```
<TextBox x:Name="outputTextBox"
        AcceptsReturn="True"></TextBox>
```

De instructies hierboven zijn zeer gedetailleerd gegeven, zodat je goed kunt beginnen. Maar **vanaf nu gaan we steeds meer vertrouwen op jouw vermogen om zelfstandig te werken**. Als je ergens niet uitkomt, raden we je aan om eerdere moduleboeken te raadplegen en online te zoeken. We gaan je vanaf nu vooral naar bronnen verwijzen of termen geven die je zelf kunt opzoeken. Op bepaalde momenten zullen we nog steeds ondersteuning bieden, maar we verwachten dat je steeds zelfstandiger kunt werken.

3.5. LINQ, Filteren en Sorteren

LINQ staat voor Language Integrated Query. Het helpt bij het schrijven van query's in C#. LINQ heeft methodes die werken op lijsten, arrays en databases.

Het simpelste voorbeeld van LINQ is het **ophalen van alle gegevens** uit een tabel:

```
var cars = context.Cars.ToList();
```

(context is hier een instantie van de DbContext in jouw applicatie)

Je kunt ook een **selectie met voorwaarden** hebben:

```
var redCars = context.Cars
    .Where(c => c.Color == "Red")
    .ToList();
```

Om **te sorteren** gebruiken we de OrderBy methode van LINQ:

```
var orderedCitizens = context.Citizens
    .OrderBy(c => c.Name)
    .ToList();
```

Bij al de bovenstaande voorbeelden zal de **Pomelo.EntityFrameworkCore.MySql** package de juiste MySQL queries opbouwen om te filteren en sorteren. Die wordt dan naar de Database gestuurd.

Nieuw: arrow/lambda function expressions

De OrderBy en Where-methodes van Entity Framework, die we gebruiken om gegevens te sorteren of filteren, werken op een best bijzondere manier. Het vereist een lambda-functie in de vorm van een pijlnotatie. Bij de Where-methode gebruiken we de pijlnotatie om aan te geven op welke eigenschap gesorteerd moet worden:

```
var redCars = context.Cars
    .Where(c => c.Color == "Red")
    .ToList();
```

Zo'n lambda functie heeft de volgende anatomie:

(parameters) => (returnwaarde)

Net als bij een methode kunnen we zelf de namen van de parameters kiezen. Een veelgebruikte conventie is om de eerste letters van elk woord te nemen en deze allemaal in kleine letters te schrijven. Daarom gebruiken wij hier 'c' voor Car.

Where is niet de enige methode die pijlnotaties gebruikt. Je komt bij andere code mogelijk ook deze voorbeelden tegen:

- f => true
- (f) => true
- (a, b) => a == b

3.6. Opdracht: Queries die LINQ uitvoert

Het is natuurlijk geweldig dat LINQ MySQL queries voor ons bedenkt en uitvoert. Ookal zien we de queries niet, ze gebeuren wel op de achtergrond. Daarnaast blijft het handig om MySQL queries te begrijpen en kunnen verzinnen, voor het geval je ooit handmatig in de database aan de slag moet. **Weet jij nog hoe MySQL queries opgebouwd worden?** Dan weet je wat Entity Framework op de achtergrond doet.

Vul de queries in:

1. Schrijft een query om alle kolommen van alle gegevens op te halen uit een tabel met de naam cars.
2. Schrijf een query om de name en year kolommen op te halen uit een tabel met de naam cars, maar alleen daar waar het jaar hoger is dan 2000.

We gaan in de komende theorie en opdrachten ook gegevens invoeren, aanpassen en verwijderen in de database. Weet jij nog welke queries je moet gebruiken?

3. Voor het invoeren in de database gebruik je:
..... cars (name, year, color, brand) VALUES ("CX-4", 2014, "Red", "Mazda")
4. Voor het aanpassen van regels gebruik je:
..... cars ... name = "CX-5" WHERE id = 1337
5. Voor het verwijderen van regels gebruik je:
..... cars WHERE id = 1337

3.7. Opdracht: Gegevens ophalen, filteren en sorteren

In de module CSD-III heb je geleerd hoe je kunt filteren met 'Where' en sorteren met 'OrderBy'. Zoek in dat moduleboek (of online) de uitleg erbij en voer de volgende deelopdrachten uit. Zo verfris je jouw kennis over het ophalen, filteren en sorteren van gegevens uit een database met Entity Framework.

Deelopdracht 1:

'Seed' de database met aanvullende testdata voor Bewoners. Zorg dat er ten minste 20 bewoners in de database worden toegevoegd. Zorg dat er 3 bewoners zijn die als 'Software Developer' als beroep hebben.

Deelopdracht 2:

Pas de code in de MainWindow aan zodat alleen bewoners getoond worden die 'Software Developer' als beroep hebben.

Deelopdracht 3:

Pas de code aan zodat alle bewoners gesorteerd worden op basis van hun naam.

Deelopdracht 4:

Voeg nu nog een **TextBox** toe waarmee gebruikers kunnen zoeken op naam van de bewoner. Pas de code aan zodat de inhoud van het tekstveld gebruikt wordt om de bewoners te filteren. Verwijder de eerder toegevoegde filter van het beroep.

3.8. Gegevens in een ListView tonen

In de DevCitySim opdracht eerder in dit hoofdstuk hebben we gegevens uit onze database getoond in een TextBox. Dat is niet heel netjes en beperkt ons in hoe we onze data tonen.

We willen eigenlijk een mooie lijst, waar we ieder item in de lijst een mooie stijl kunnen geven.

1. Pak de code van DevCitySim er opnieuw bij.
2. Verwijder in de MainWindow.xaml de TextBox én de StackPanel.
3. Voeg deze ListView toe:

```
<ListView x:Name="citizenListView">
  <ListView.ItemTemplate>
    <DataTemplate x:DataType="localData:Citizen">
      <StackPanel>
        <TextBlock Text="{x:Bind Name}" />
      </StackPanel>
    </DataTemplate>
  </ListView.ItemTemplate>
</ListView>
```

Let op: om als datatype de klasse 'Bewoner' te gebruiken, moeten we bovenin de MainWindow de **DevCitySim.Data namespace** toevoegen, want dat is de namespace waar de Bewoner klasse in staat:

```
1 <Window
2   xmlns:localData="using:DevCitySim.Data"
```

We geven in het bovenstaande voorbeeld de naam 'localData', maar je mag die namespaceverwijzing noemen wat je wilt zolang je maar dezelfde naam gebruikt bij de DataTemplate x:DataType.

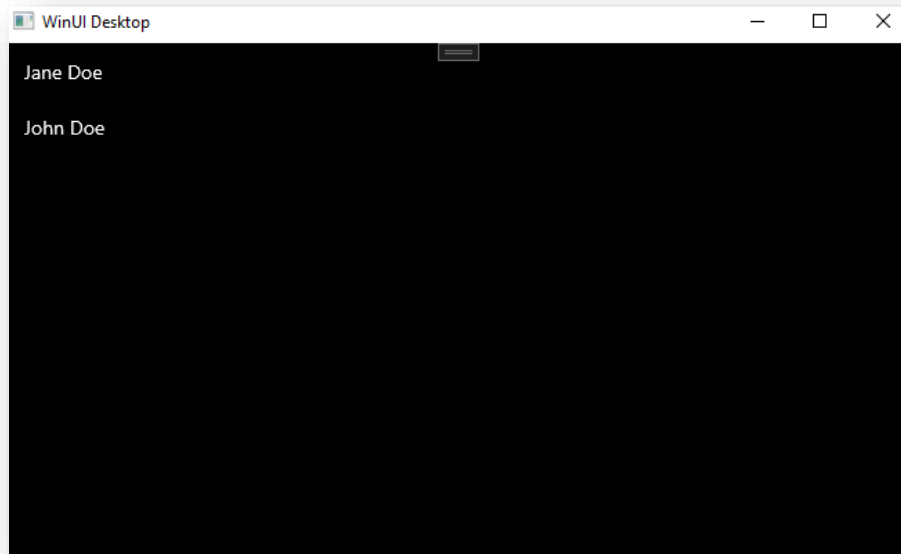
4. In de code-behind (MainWindow.xaml.cs) verwijder je nu het gedeelte wat te maken heeft met de TextBox.
5. Spreek in plaats daarvan de aangemaakte ListView aan en stel de ItemsSource eigenschap in op de DbSet met alle bewoners:

```
public sealed partial class MainWindow : Window
{
    1 reference
    public MainWindow()
    {
        this.InitializeComponent();

        using (var db = new AppDbContext())
        {
            db.Database.EnsureDeleted();
            db.Database.EnsureCreated();

            citizenListView.ItemsSource = db.Citizens;
        }
    }
}
```

6. Na opstarten is dit dan het resultaat:



7. Probeer de XAML zelf eens uit te breiden, zodat je ook een TextBlock hebt om de 'Job' eigenschap in te tonen.

3.9. Opdracht: ListView in DevCitySim implementeren

Pas jouw eerdere DevCitySim project aan, zodat nu een ListView wordt gebruikt in plaats van een TextBox.

Stappenplan:

1. Verwijder de TextBox uit de XAML
2. Verwijder de code die hoort bij de TextBox in de C#
3. Voeg in de XAML een ListView toe, let op dat je:
 - a. Geef de ListView een x:Name, zodat we die vanuit de C# kunnen instellen, kies: **citizensListView**
 - b. In de ListView een ListView.ItemTemplate maakt
 - c. In de ListView.ItemTemplate een DataTemplate maakt
 - d. Bij de DataTemplate het x:DataType attribuut specificeert
 - e. Als jouw Citizen in een Data-map staat, moet je bovenin volgens de uitleg xmlns toevoegen
4. Verwijder de foreach loop uit de C# - De ListView gaat dit vanzelf voor ons regelen, wij hoeven enkel de verzameling met alle citizens in te stellen:
5. Stel de ItemsSource in van de ListView. Als je die citizensListView hebt genoemd schrijf je:
`citizensListView.ItemsSource = db.Citizens.ToList();`

3.10. Opdracht: WinUI3 zoals het hoort: één Window, meerdere Pages

In veel moderne apps zie je dat er één scherm is en dat de inhoud daarbinnen verwisseld wordt. Kijk maar eens naar WhatsApp:

- Bij het openen van WhatsApp zie je de OverviewPage met al jouw chats
- Bij het klikken op een chat opent de ChatPage met daarin alle berichten binnen die chat

Die structuur kunnen we in WinUI3 ook maken, en is de manier die Microsoft voor ogen had toen ze WinUI3 ontworpen. In deze paragraaf bekijken we de theorie hiervan.

We bekijken aan de hand van een 'DetailDemoApp' hoe Pages werken, en hoe ze verschillen van 'Window' in WinUI3 in het doorgeven van gegevens.

Stappenplan:

1. Start een nieuw WinUI3 project (volgens het bekende template) en noem het 'DetailDemoApp'
2. Haal standaard aangemaakte knop en bijbehorende C# code weg uit de MainWindow
3. Voor het toevoegen van een pagina voeg je aan je project een 'New item...' toe en kies je:

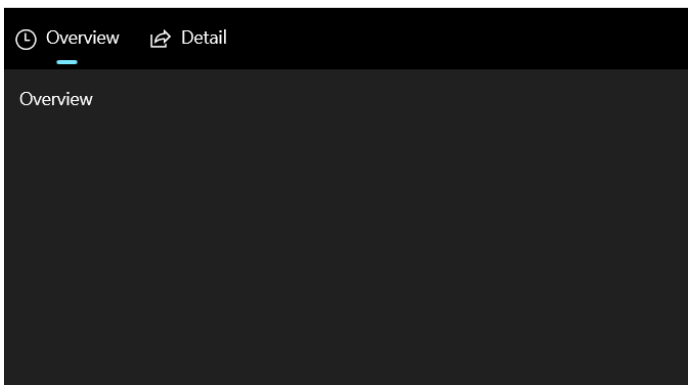


4. Voor deze demonstratie maken we twee Pages:
 - a. We geven we de eerste Page de naam: OverviewPage.
Wij spreken de conventie af om de namen van pagina's altijd met 'Page' te laten eindigen. En omdat het een Class is UpperCamelCase/PascalCase
 - b. De tweede Page krijgt de naam: DetailPage
5. We geven iedere pagina een TextBlock als inhoud, zodat we zometeen zien op welke pagina we zijn. Dit is hoe de OverviewPage eruit ziet in onze demonstratie:

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <Page
3      x:Class="DetailDemoApp.OverviewPage"
4      xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
5      xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
6      xmlns:local="using:DetailDemoApp"
7      xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
8      xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
9      mc:Ignorable="d"
10     Background="{ThemeResource ApplicationPageBackgroundThemeBrush}">
11
12     <Grid Padding="15">
13         <TextBlock>Overview</TextBlock>
14     </Grid>
15 </Page>
```

6. Na het aanmaken van de pagina's moeten we in de 'MainWindow' een plek maken waar deze pagina's een tegelijk kunnen tonen. Dit is het resultaat van wat we gaan maken:

WinUI Desktop



SelectorBar om te navigeren

Frame om de Pages in te tonen

7. We bouwen deze layout met een 'SelectorBar' en een 'Frame' element. De SelectorBar gaat ons helpen om bovenin knoppen te tonen, die de inhoud in het Frame element veranderen. Zie hieronder de XAML van onze MainWindow.xaml:

```
<?xml version="1.0" encoding="utf-8"?>
<Window
    x:Class="DetailDemoApp.MainWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:local="using:DetailDemoApp"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
    xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
    mc:Ignorable="d">

    <Grid>
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto" />
            <RowDefinition Height="*" />
        </Grid.RowDefinitions>

        <SelectorBar x:Name="contentSelectorBar"
                    Grid.Row="0"
                    SelectionChanged="contentSelectorBar_SelectionChanged">
            <SelectorBarItem x:Name="overviewSelectorBarItem"
                            Text="Overview"
                            Icon="Clock" />
            <SelectorBarItem x:Name="detailSelectorBarItem"
                            Text="Detail"
                            Icon="Share" />
        </SelectorBar>

        <Frame x:Name="contentFrame"
                Grid.Row="1"/>
    </Grid>
</Window>
```

Het element op de Row met **index 1** van de grid neemt alle (*) **overgebleven** ruimte

Het element op de Row met **index 0** van de grid krijgt **automatisch (Auto)** de ruimte die de inhoud van de regel nodig heeft

8. Bekijk het bovenstaande screenshot van de XAML en merk op:

- We gebruiken een Grid element, om een layout te maken, waarvan één regel automatisch zo groot wordt als de inhoud van die regel. De andere regel neemt de overgebleven ruimte in.
- We plaatsen de SelectorBar op de eerste regel (Grid.Row="0") en de Frame op de tweede regel.
- We geven namen aan de SelectorBar, SelectorBarItems en Frame
- We koppelen het SelectionChanged evenement op de SelectorBar. *Deze wordt aangeroepen wanneer de gebruiker op een van de knoppen klikt.*

We hebben deze koppeling automatisch laten genereren door op TAB te drukken bij deze suggestie:

```
<SelectorBar x:Name="contentSelectorBar"
             Grid.Row="0"
             SelectionChanged="">
  <SelectorBarItem x:Name="ov" <New Event Handler>
```

(Alleen dan wordt de juiste methode aangemaakt in de MainWindow.xaml.cs code-behind)

9. De laatste stap is om de gekoppelde event 'SelectionChanged' in C# te implementeren. We willen ervoor zorgen dat als de gebruiker op de knoppen drukt, het juiste scherm wordt geopend. Hieronder zie je een van de manieren waarop we kunnen checken: "als op knop ... is gedrukt, open dan pagina ... in de contentFrame":

```
private void contentSelectorBar_SelectionChanged(SelectorBar sender, SelectorBarSelectionChangedEventArgs args)
{
    SelectorBarItem selectedItem = sender.SelectedItem;

    if (selectedItem == overviewSelectorBarItem)
    {
        contentFrame.Navigate(typeof(OverviewPage));
    }
    else if (selectedItem == detailSelectorBarItem)
    {
        contentFrame.Navigate(typeof(DetailPage));
    }
}
```

10. Bekijk het bovenstaande screenshot van de C# en merk op:

- We vragen in de event handler de aangeklikte SelectorBarItem via 'sender.SelectedItem'. Omdat we de SelectorBar een naam hebben gegeven, hadden we ook 'contentSelectorBar.SelectedItem' kunnen gebruiken (doet precies hetzelfde in ons geval).
- We kunnen achterhalen op welke SelectorBarItem is geklikt door te vergelijken. Dit is waarom we alle SelectorBarItem elementen een naam hebben gegeven.
- We navigeren in het Frame naar een andere pagina via de 'Navigate'-methode. Deze methode verwacht het type van de Page waar we naartoe willen navigeren. Hiervoor gebruiken we het C# keyword 'typeof', dat ons dat type geeft als wij typeof de klassenaam geven.
- De 'Navigate'-methode kan ook nog een tweede argument krijgen om gegevens mee te sturen. Meer hierover in het volgende paragraaf.
- We roepen de Navigate-methode hier direct op de 'contentFrame' aan. **Maar je kunt vanuit een Page ook bij de Frame waarin die Page zich bevindt: Als je vanuit de OverviewPage wilt navigeren kun je het als volgt aanpakken:**

```
public sealed partial class OverviewPage : Page
{
    0 references
    public OverviewPage()
    {
        this.InitializeComponent();
    }

    0 references
    private void Button_Click(object sender, RoutedEventArgs e)
    {
        this.Frame.Navigate(typeof(DetailPage));
    }
}
```

11. Open de applicatie en klik op de knoppen. Je kunt wisselen tussen de verschillende Page klassen.

🚩 Na de bovenstaande stappen hebben we een applicatie met slechts één scherm, waarin we kunnen wisselen tussen meerdere pagina's.

🚩 3.11. Op een ListView item kunnen klikken

Je hebt mogelijk al eerder (in Blok C) met ListViews gewerkt. Toen leerde we het volgende over hoe we op een item kunnen klikken.

Om de items in een ListView 'klikbaar' te maken moeten we er in de XAML deze attributen aan toevoegen:

```
<ListView x:Name="kazenLijst"
    IsItemClickEnabled="True"
    ItemClick="kazenLijst_ItemClick">
    <ListView.ItemTemplate>
        <DataTemplate x:DataType="localData:Kaas">
            <StackPanel>
                <TextBlock Text="{x:Bind Name}" />
                <TextBlock Text="{x:Bind Id}" />
            </StackPanel>
        </DataTemplate>
    </ListView.ItemTemplate>
</ListView>
```

In dit voorbeeld wordt ieder item van de klasse 'Kaas'. In jouw code kan dit een andere klasse zijn.

Het 'ItemClick' attribuut hierboven is een Event. Tijdens het typen van de code krijg je automatisch de suggestie om de bijbehorende Event Handler methode in de C# aan te maken.

Om in die Event Handler vervolgens op te halen op welke Item is geklikt doen we het volgende:

```
0 references
private void kazenLijst_ItemClick(object sender, ItemClickEventArgs e)
{
    var selectedItem = (Kaas)e.ClickedItem;
    selectedItem.
}

★ Name
★ Equals
★ GetHashCode
★ ToString
★ Date
★ Date

string Kaas.Name { get; set; }
★ IntelliCode suggestion based on this context
```

Wanneer we eerder in onze code een verzameling met objecten van een type als Kaas in onze ListView zetten, weten we bij het klikken op een item zeker dat het dat type heeft (Kaas hierboven). We kunnen dan de e.ClickedItem 'casten' naar het type van de data. Daarna kunnen we bij alle eigenschappen van het object.

🚩 3.12. Gegevens naar een Page sturen via Navigate

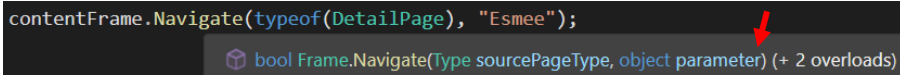
We gebruiken Navigate om te navigeren naar een andere pagina. Om bijvoorbeeld naar een pagina genaamd OverviewPage te navigeren in een Frame genaamd 'contentFrame' schrijf je:

```
contentFrame.Navigate(typeof(OverviewPage));
```

Om gegevens naar een Page te sturen heeft Microsoft het volgende bedacht:

- I. Geef gegevens mee als tweede argument aan de Navigate methode, bijvoorbeeld:

```
contentFrame.Navigate(typeof(DetailPage), "Esmee");
```



bool Frame.Navigate(Type sourcePageType, object parameter) (+ 2 overloads)

- II. Je kunt dan bij die gegevens door de 'OnNavigatedTo'-methode te 'override'-en in de pagina waar naartoe is genavigeerd.

Dit is hoe we de demonstratie uit 'DetailDemoApp' hierboven kunnen uitbreiden om dat te demonstreren:

1. Pak de 'DetailDemoApp' er weer bij.
2. Pas in de MainWindow.xaml.cs de event handler aan. We gaan bij het openen van de DetailPage jouw naam als string meegeven als tweede argument. Bijvoorbeeld:

```
contentFrame.Navigate(typeof(DetailPage), "Esmee");
```

3. Geef de TextBlock in DetailPage.xaml het volgende x:Name attribuut, zodat we er zo vanuit C# tekst in kunnen zetten:

```
<TextBlock x:Name="greetingTextBlock">...</TextBlock>
```

4. Nu gaan we zorgen dat de naam (die via Navigate wordt meegegeven) ontvangen wordt in de DetailPage.xaml.cs C# code-behind.
Maak de 'OnNavigatedTo'-override aan door te beginnen met typen van 'override onnavigatedt', en dan op TAB te drukken om deze suggestie te accepteren:

```
override onnavigatedt
```

OnNavigatedTo(NavigationEventArgs e)	void Page.OnNavigatedTo(NavigationEventArgs e)
	Invoked when the Page is loaded and becomes the current source of a parent Frame.

5. Voeg daarna deze twee regels toe in de aangemaakte methode:

```
protected override void OnNavigatedTo(NavigationEventArgs e)
{
    base.OnNavigatedTo(e);

    string name = e.Parameter as string;

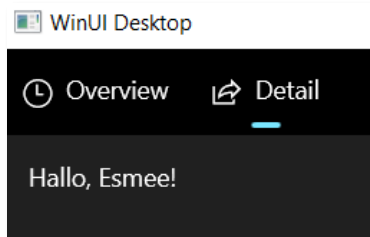
    greetingTextBlock.Text = $"Hallo, {name}!";
}
```

6. Bekijk nogmaals het bovenstaande screenshot en merk op:
 - a. Via het 'NavigationEventArgs e' komt informatie mee over hoe deze Page is geopend. Hierin bevat de 'Parameter'-eigenschap wat is meegegeven via de Navigate-methode in MainWindow.xaml.cs
 - b. Omdat Microsoft wil dat iedereen van alles kan meesturen is het datatype van 'Parameter' het type 'object'. Alle andere typen in C# erven (onzichtbaar) over van 'object'. **Met andere woorden: een 'object' datatype kan van alles bevatten.**
 - c. Wanneer we met 'object' types werken moet we het altijd weer omzetten naar het juiste type. **Omdat wij in de MainWindow.xaml.cs een string hebben meegegeven, kunnen we de Parameter eigenschap van het event casten naar een string.**

- d. We hadden het ook op deze manier kunnen casten (wat in dit geval hetzelfde effect heeft):

```
string name = (string) e.Parameter;
```

7. Start de applicatie en klik op de Detail-knop. Je ziet iets als dit:



In de bovenstaande demonstratie geven we simpelweg een string mee. Maar dankzij het feit dat Microsoft ons 'object' laat meegeven, kunnen we écht alles meegeven. Zo ook bijvoorbeeld meer complexere data, zoals een instantie van een eigen klasse:

- a. In de Navigate geven we die instantie dan mee:

```
var personInstance = new Person("Esmee", 21);  
contentFrame.Navigate(typeof(DetailPage), personInstance);
```

- b. In de OnNavigatedTo 'casten' we dan naar dat type:

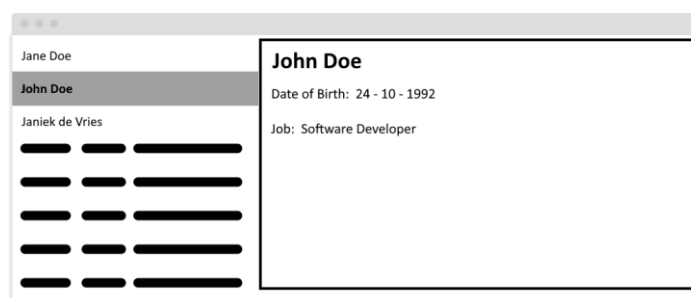
```
var person = (Person)e.Parameter;  
greetingTextBlock.Text = $"Hallo, {person.Name}!\nJe bent {person.Age} jaar oud? Cool";
```

 Je kunt de uitwerking van deze DetailDemoApp downloaden van [Itslearning](#).

Extra Tip: je kunt in een Page een Button toevoegen om terug te navigeren naar de vorige pagina. Omdat de 'contentFrame' (naam van de frame) in de MainWindow staat, kun je er niet zomaar in andere klassen bij. Microsoft heeft daarom een eigenschap gemaakt die iedere Page heeft: 'Frame'. Door simpelweg: `this.Frame.GoBack()` te gebruiken kun je de frame terug laten navigeren naar de voorliggende Page. Je kunt ook `this.Frame.Navigate` gebruiken, maar technisch gezien is `GoBack` beter. `GoBack` geeft ook een andere animatie.

3.13. Opdracht: Een detailpagina openen bij het klikken op een Citizen

Laten we de theorie uit de vorige paragraaf toepassen om dit te maken: wanneer geklikt wordt op een Citizen, opent een `CitizenDetailPage`. We gaan in de `MainWindow` zorgen dat links de lijst met Citizens te zien is, en rechts het detailscherm geopend kan worden:



Stappenplan:

1. Maak een `CitizenDetailPage` aan via (Project > Add > New Item... > WinUI3 > Page

2. Maak in de XAML van de CitizenDetailPage voor nu even een TextBlock met daarin de tekst "Hallo Citizen!". We gaan hier later informatie over de citizen tonen.
3. Pas de MainWindow XAML aan zodat je een Grid element gebruikt met 2 kolommen:
 - a. Maak de linker kolom 1/4^e groot en plaats er de **ListView** in
 - b. Maak de rechter kolom 3/4^e groot en plaats er de **Frame** in, **geef het de naam detailFrame**. *In de Frame gaat de CitizenDetailPage geladen worden.*

```
<Grid>
  <Grid.ColumnDefinitions>
    <ColumnDefinition Width="1*" />
    <ColumnDefinition Width="3*" />
  </Grid.ColumnDefinitions>

  <ListView x:Name="citizensListView"
    Grid.Column="0">
    <ListView.ItemTemplate>
      <DataTemplate x:DataType="localData:Citizen">
        <StackPanel>
          <TextBlock Text="{x:Bind Name}"
            FontWeight="Bold" />
        </StackPanel>
      </DataTemplate>
    </ListView.ItemTemplate>
  </ListView>

  <Frame x:Name="detailFrame"
    Grid.Column="1" />
</Grid>
```

4. Volg de instructies in de eerdere opdracht, zodat de gebruiker kan klikken op een citizen in de ListView.
5. Wanneer de gebruiker klikt op een citizen in de ListView, open dan de CitizenDetailPage in de detailFrame:


```
detailFrame.Navigate(typeof(CitizenDetailPage));
```
6. Test de applicatie, je ziet dat bij het klikken op een citizen, nu de 'Hallo Citizen!' tekst wordt getoond.

Nu hoeven we alleen nog te zorgen dat we de gegevens tonen van de aangeklikte citizen.

7. Pas de CitizenDetailPage XAML aan zodat er 2 TextBlock elementen zijn:
 - a. citizenNameTextBlock – Hier komt de Name van een citizen
 - b. citizenJobTextBlock – Hier komt de Job van een citizen
8. Pas nu de theorie toe uit de vorige paragrafen, zodat in de MainWindow C#, bij de Navigate, de Id van de aangeklikte citizen wordt meegestuurd. Bekijk hiervoor opnieuw deze theorie:
 - a. "Op een ListView item kunnen klikken"
 - b. "Gegevens naar een Page sturen via Navigate"

9. Pas nu de theorie toe uit de vorige paragraaf “Gegevens naar een Page sturen via Navigate” en override de `OnNavigateTo` methode in `CitizenDetailPage`. Zorg ervoor dat de `Id` die is meegestuurd wordt gebruikt om de `Citizen` weer op te halen uit de database. Toon vervolgens de gegevens van die `citizen` in de `TextBlock` elementen:

```
protected override void OnNavigatedTo(NavigationEventArgs e)
{
    var citizenId = (int)e.Parameter;

    using var db = new AppDbContext();

    var citizen = db.Citizens.FirstOrDefault(c => c.Id == citizenId);

    citizenNameTextBlock.Text = citizen.Name;
    citizenJobTextBlock.Text = citizen.Job;
}
```

Inleveropdracht 3.14: ChatMeister: Proof-of-Concept (tijdsduur: 30-60 minuten)

Je taak is om de proof-of-concept te ontwikkelen voor een chat-applicatie, waarbij Entity Framework wordt gebruikt voor gegevensopslag en -beheer met behulp van MySQL als de databaseprovider.

Met 'proof-of-concept' bedoelen we dat deze applicatie de basisfunctionaliteit van een chat-applicatie simuleert door berichten op te halen vanuit de lokale database. Er zal dus geen echte chat over het internet plaatsvinden tussen gebruikers.

Instructies:

1. (2 punten) Zet een nieuw WinUI 3 project op genaamd 'ChatMeister' en integreer Entity Framework met MySQL.
Gebruik het 'Stappenplan Entity Framework' wat te vinden is in dit hoofdstuk.
2. (3 punten) Maak de benodigde model-classes op basis van deze tabellen met testgegevens:

User		
Id	Username	Email
1	Alice	alice@example.com
2	Bob	bob@example.com

Chat	
Id	Name
1	Chatroom 1
2	Chatroom 2

Message				
Id	ChatId	UserId	Content	SentAt
1	1	1	Hello, everyone!	2023-06-27 09:15:00
2	1	2	Hey, Alice!	2023-06-27 09:17:00
3	2	1	We're doing great!	2023-06-27 10:32:00

3. (1 punt) 'Seed' de database met de bovenstaande testgegevens.
4. (2 punten) Maak een overzichtsscherm met een ListView waarin de namen van alle chats te zien zijn.
Zie het stappenplan om te zien hoe je via databinding gegevens kunt tonen in een ListView.
5. (2 punten) Maak een chatscherm waarin alle chatberichten getoond worden van een bepaalde ChatId.
6. (2 punten) Zorg dat bij het klikken op een ListView item in het overzichtsscherm, de 'Id' van die Chat wordt meegegeven bij het openen van het chatscherm. Binnen dat chatscherm wordt die Id gebruikt om alleen berichten te tonen waarvan de ChatId overeenkomt.

Zie volgende pagina voor eventuele bonusopdrachten.

Inleveren:

Je levert de opdracht als volgt in op Itslearning:

- Maak een .zip van de gehele solution
- Maak screenshots van de twee schermen in jouw applicatie
- Maak een screenshot van de C# code die het chatscherm opent en de aangeklikte Chat Id meegeeft.

Veel succes met de opdracht!

 **Let op:** Het inleveren van deze opdracht geldt als feedback-moment. Je ontvangt studiepunten wanneer je deze succesvol afrond. Houdt na het inleveren Itslearning in de gaten en verwerk eventuele feedback die je van de docent krijgt.

*Er zijn in totaal **12 punten** te behalen bij deze opdracht. Met een cesuur van 70% moet je ten minste 8,5 punten scoren voor een voldoende (5,6).*

Bonus: ChatMeister: Proof-of-Concept

Snel klaar? Zoek je extra uitdaging? Bij deze enkele aanvullende (niet verplichte) opdrachten voor de vorige inleveropdracht:

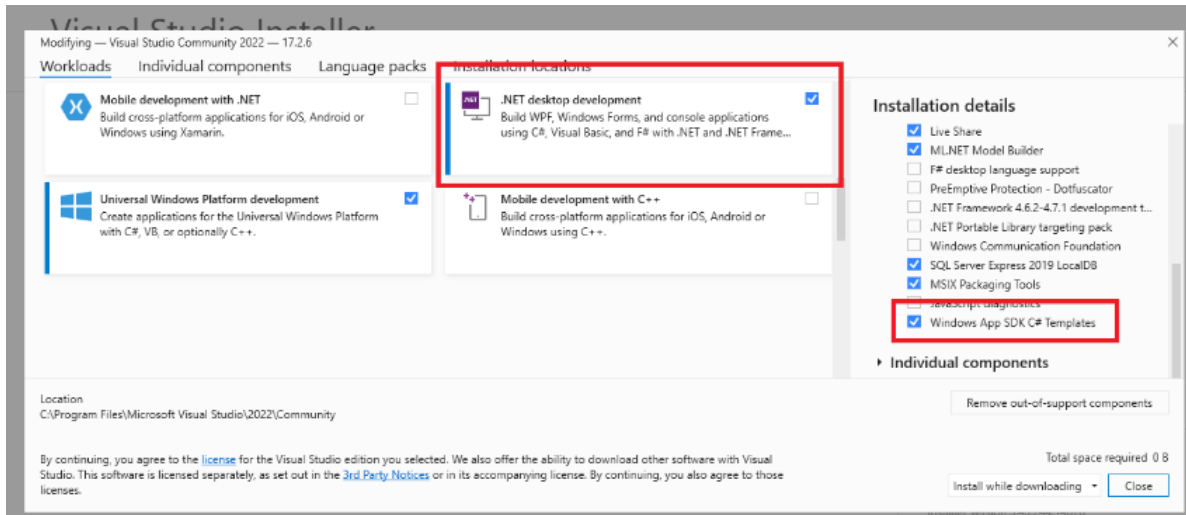
- Voeg de mogelijkheid toe om berichten (Message) in het chatscherm van de app aan te maken.
- Laat in het overzichtsscherm...
 - ...onder de naam van de Chat, ook de inhoud van het laatste chatbericht (Message) zien.
 - ...achter de naam de datum van het laatste chatbericht zien
- Laat in het chatscherm ook de datum van het bericht zien.
- Laat via een extra knop in het chatscherm een nieuw scherm openen waarin alle users staan die deelnemen aan de chat.
- Verbeter het uiterlijk van de applicatie door kleur, padding en tekstgrootte aan te passen.

3.15. Veel voorkomende problemen

In deze paragraaf kun je enkele veel voorkomende problemen vinden en de oplossingen ervoor. Zoek dit erbij wanneer je tegen problemen aanloopt.

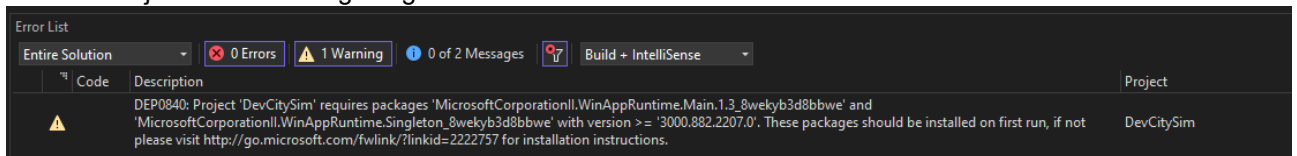
- Ik kan bij het starten van een project niet kiezen voor de optie 'WinUI Blank App (Packaged)' of 'Blank app, Packaged (WinUI 3 in Desktop)'

In nieuwere versies van Visual Studio heeft de template de naam 'WinUI Blank App (Packaged)', in oudere versies 'Blank App, Packaged (WinUI 3 in Desktop)'. Als je geen van beide opties kunt kiezen, zorg dan dat je het individuele onderdeel 'Windows App SDK C# Templates' geselecteerd hebt in de Visual Studio Installer.



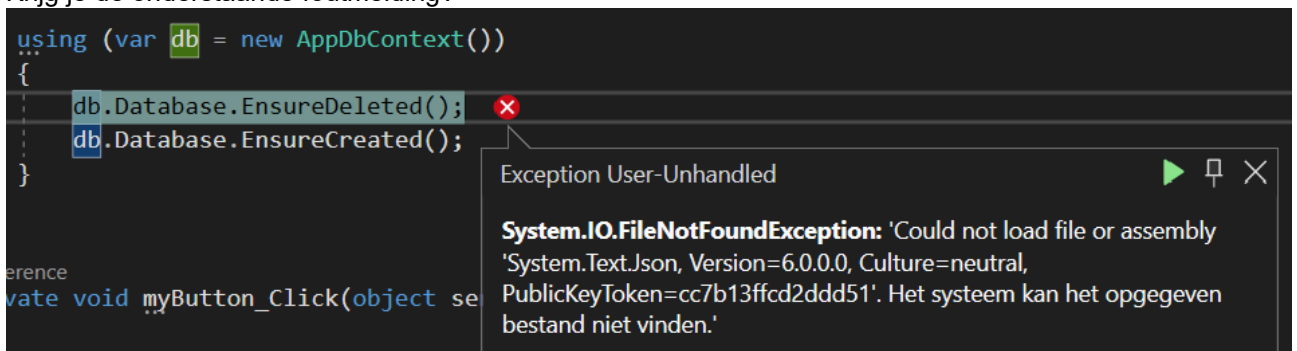
- Ik blijf een waarschuwing krijgen over 'MicrosoftCorporationII.WinAppRuntime.Main...'

Volg de link in de foutmelding: <http://go.microsoft.com/fwlink?linkid=2222757> en installeer de SDK die daar staat. Het schermje opent en sluit zonder direct iets te tonen. Door het in Command Prompt uit te voeren kun je wel de berichtgeving zien.

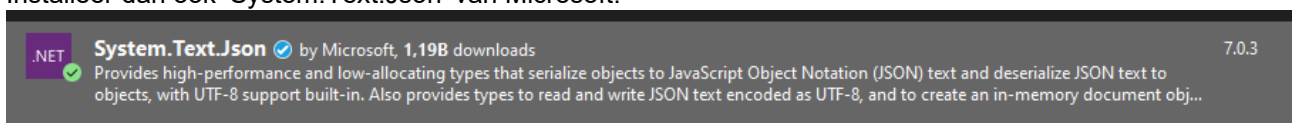


- Ik krijg een foutmelding over System.Text.Json

Krijg je de onderstaande foutmelding?



Installeer dan ook 'System.Text.Json' van Microsoft:



Een package waar Pomelo van afhankelijk is maakt gebruik van System.Text.Json. Eigenlijk zou die package dan System.Text.Json moeten verplichten, maar door een bug(?) gebeurt dat niet. Daarom moeten we deze NuGet package handmatig installeren.

4. Entity Framework: Relaties

4.1. Terugblik

Docent en klas blikken terug op het vorige hoofdstuk. Voordat we verder kunnen is het belangrijk dat we de basis van Entity Framework begrijpen. We moeten aan de hand van het stappenplan een WinUI 3 project kunnen straten waarin we Entity Framework gebruiken met MySQL. Met de klas bekijken we hoe het maken van de weekcheck '**ChatMeister: Proof-of-Concept**' inleveropdracht is verlopen.



Verliep het maken van de 'ChatMeister' inleveropdracht moeizaam? Of wil je om een andere reden extra informatie over die opdracht?

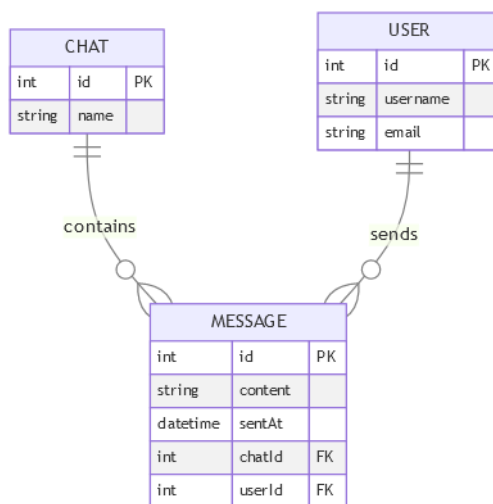
De docent kan je een videodemonstratie overhandigen waarin wordt gedemonstreerd hoe de applicatie wordt opgebouwd. Je ziet dan hoe de stappen uit het stappenplan worden uitgevoerd én hoe dit in WinUI 3 met een ListView wordt toegepast.

In dit hoofdstuk gaan we verder met Entity Framework en kijken we naar 'relaties' tussen entiteiten. Hoe entiteiten aan elkaar gerelateerd kunnen zijn en hoe we efficiënt bij elkaar horende entiteiten kunnen ophalen uit de database.

4.2. Relaties tussen entiteiten

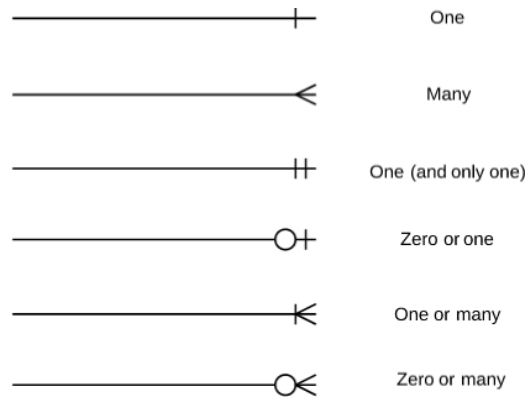
Entiteiten zijn aan elkaar gerelateerd door middel van associaties. Een associatie is een logische connectie tussen twee of meer entiteiten die aangeeft hoe ze met elkaar in verband staan. Dit kan een eenvoudige één-op-één-relatie zijn, een één-op-veel-relatie of een veel-op-veel-relatie.

Een Entity-Relationship Diagram (ERD) wordt gebruikt om de relaties tussen entiteiten schematisch weer te geven. Het is een visueel hulpmiddel dat de structuur van een database illustreert door middel van entiteiten, hun eigenschappen en de relaties tussen hen. Hier een voorbeeld van een ERD dat de relaties tussen entiteiten in de eerdere 'ChatMeister' opdracht toont:



In een ERD worden entiteiten weergegeven als rechthoeken met de naam van de entiteit erin. Eigenschappen van de entiteiten worden als attributen weergegeven, daarbij staat of het een Primary Key (PK) of Foreign Key (FK) is. Ook het datatype kan in een ERD genoteerd worden. Relaties tussen entiteiten worden aangegeven door lijnen tussen de betrokken entiteiten.

De uiteinden van een lijn tussen entiteiten maakt duidelijk welk type relatie bestaat tussen de entiteiten:



In het ERD zien we aan de lijntypes:

- Een **Chat** kan nul of meer **Messages** bevatten.
(een chat kan bestaan zonder messages, maar kan er ook meerdere hebben.)
- Een **User** kan nul of meer **Messages** hebben.
(een user kan bestaan zonder ooit een message te hebben geschreven, maar kan er ook meerdere hebben)

Diezelfde relaties hebben ook een ander uiteinde, aan die lijntypes zien we:

- Een **Message** heeft een (en altijd een) **Chat**
(een message kan niet bestaan zonder chat, er hoort er altijd precies 1 bij)
- Een **Message** heeft een (en altijd een) **User**
(een message kan niet bestaan zonder user, er hoort altijd precies 1 user bij het bericht)

Maar van alle type relaties zijn dit de **one** en de **many** zeker de belangrijkste om te begrijpen.

Bij de PRO lessen ga je in meer detail kijken naar ERD's en leer je hoe je ze zelf kunt ontwerpen en tekenen.

4.3. Relaties in Entity Framework

In de ChatMeister van vorige week bestaan de beginselen van relaties, maar maken we er nog niet optimaal gebruik van. Zo ziet je Message klasse er mogelijk zo uit:

```

1 public class Message
2 {
3     public int Id { get; set; }
4     public int ChatId { get; set; }
5     public int UserId { get; set; }
6     public string Content { get; set; }
7     public DateTime SentAt { get; set; }
8 }

```

Dankzij de 'ChatId' en 'UserId' eigenschappen bestaan er relaties tussen Message, User en Chat. Omdat deze eigenschappen verwijzen naar andere entiteiten (op afstand) noemen we ze 'Foreign Keys'.

Herhaling: Een Foreign Key verwijst naar de Primary Key van een entiteit in een andere tabel.

Voorbeeld van Primary en Foreign Keys met twee tabellen

De Primary Key's zijn geel, de Foreign Key is groen.

De Foreign Key KlasId verwijst naar Id in de klassen tabel.

studenten

Id	Naam	KlasId
12345	Janiek	2
12346	Noah	1

klassen

Id	Naam
1	TTR4-SSD3A
2	TTR4-SSD3B

Als we kijken naar onze ChatMeister code die in de ChatPage de berichten ophaalt, ziet die er mogelijk zo uit:

```
using var db = new AppDbContext();
var chat = db.Chats.First(c => c.Id == chatId);
var messages = db.Messages.Where(m => m.ChatId == chatId);

chatMessagesListView.ItemsSource = messages;
```

Maar er zijn in Entity Framework effectievere manieren om de berichten op te halen die bij een chat horen. Door **Navigation Properties** aan te maken in modellen die gerelateerd zijn aan elkaar, kunnen we deze op drie manier ophalen:

- **Eager loading:** de gerelateerde gegevens worden tegelijk opgehaald met de originele query. Je kunt dit zien als een JOIN. Bijvoorbeeld: haal alle chats en bijbehorende messages op.
- **Explicit loading:** de gerelateerde gegevens worden opgehaald op een later moment. De ontwikkelaar geeft zelf het moment aan dat dit moet gebeuren.
- **Lazy loading:** de gerelateerde gegevens worden automatisch wanneer deze nodig zijn. Dit gebeurt wanneer de gegevens benaderd worden.

Wij gaan alleen **Eager** en **Explicit loading** behandelen. Je kunt zelf meer vinden over Lazy loading via deze documentatie: <https://learn.microsoft.com/en-us/ef/core/querying/related-data/>

Laten we kijken wat **Navigation Properties** precies zijn en hoe we ze gebruiken. Neem onze eerdere Message klasse en Chat klasse. Wanneer we daar Navigation Properties aan toevoegen zien die er zo uit:

```
1 public class Message
2 {
3     public int Id { get; set; }
4     public int ChatId { get; set; }
5     public Chat Chat { get; set; } // Navigation property
6     public int UserId { get; set; }
7     public User User { get; set; } // Navigation property
8     public string Content { get; set; }
9     public DateTime SentAt { get; set; }
10 }
```



```

1 public class Chat
2 {
3     public int Id { get; set; }
4     public string Name { get; set; }
5
6     public ICollection<Message> Messages { get; set; } // Navigation property
7 }

```

Vervolgens kunnen we op deze manier alle chats and bijbehorende berichten ophalen doormiddel van **Eager loading**:

```

5     using (var db = new AppDbContext())
6     {
7         db.Database.EnsureCreated();
8
9         chatsListView.ItemsSource = db.Chats.Include(c => c.Messages);
10    }

```

De 'Include' methode is beschikbaar dankzij 'using Microsoft.EntityFrameworkCore;'. Het geeft aan dat voor alle chats ook alle bijbehorende messages worden opgehaald. Entity Framework kiest de meest effectieve methode. In het geval van MySQL zal een JOIN query uitgevoerd worden.

Om dan in het overzichtsscherm alle berichten te laten zien hoeven we daar niet nog eens de database context aan te spreken, maar kunnen we direct in de XAML via databinding verbinden aan de **Messages Navigation Property** van de chat:

```

1 <ListView x:Name="chatMessagesListView"
2     ItemsSource="{x:Bind CurrentChat.Messages}"
3     Grid.Row="1">

```

STANDAARD STAPPENPLAN RELATIES TOEVOEGEN

Om de bovenstaande theorie wat toegankelijker te maken hebben we de volgende standaard stappen bedacht:

Bij het toevoegen van een one-to-many relatie tussen twee modellen doe je het volgende:

1. Voeg een **Foreign Key Property** toe aan een van de modellen
2. Voeg de **Navigation Properties** toe aan beide modellen
3. Zorg dat er gegevens in de database staan (bijv. met **seeder**) die de modellen aan elkaar koppelen
4. Haal gerelateerde gegevens op met **.Include(...)**
5. Toon de data via de **Navigation Property** (bijv. in de XAML)

Voor de HotelApp (zie **[Extra] HotelApp Startpunt.zip** op Itslearning) ziet dit er als volgt uit.

Dit zijn de modellen in die app:

```
internal class Guest
{
    public int Id { get; set; }
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public string Nationality { get; set; }
}
```

```
internal class Room
{
    public int Id { get; set; }
    public string Type { get; set; } // e.g: Regular, Deluxe,
    Penthouse
    public int Floor { get; set; } // 0 is the ground floor
}
```

Laten we zeggen dat een Room one-to-many Guests kan hebben. Dus een Guest heeft 1 Room, terwijl een Room meerdere Guests kan hebben.

1. Voeg een Foreign Key Property toe aan een van de modellen:

```
internal class Guest
{
    public int Id { get; set; }
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public string Nationality { get; set; }
    public int RoomId { get; set; }
}
```

2. Voeg de Navigation Properties toe aan beide modellen:

```
internal class Guest
{
    public int Id { get; set; }
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public string Nationality { get; set; }

    public int RoomId { get; set; }
    public Room Room { get; set; }
}
```

```
internal class Room
{
    public int Id { get; set; }
    public string Type { get; set; } // e.g: Regular, Deluxe, Penthouse
    public int Floor { get; set; } // 0 is the ground floor

    public List<Guest> Guests { get; set; }
}
```

3. Zorg dat er gegevens in de database staan (bijv. met seeder) die de modellen aan elkaar koppelen:

```

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Room>().HasData(
        new Room
        {
            Id = 1,
            Floor = 1,
            Type = "Single Bed",
        }
    );

    modelBuilder.Entity<Guest>().HasData(
        new Guest
        {
            Id = 1,
            FirstName = "Anne",
            LastName = "De Vries",
            Nationality = "Nederlandse",
            RoomId = 1,
        }
    );
}

```

4. Haal gerelateerde gegevens op met .Include(...):

```

using var db = new AppDbContext();

// Haal alle guests op en toon ze in de ListView
var guests = db.Guests
    .Include(g => g.Room)
    .ToList();

guestsListView.ItemsSource = guests;

```

5. Toon de data via de Navigation Property (bijv. in de XAML):

```

<ListView x:Name="guestsListView"
    IsItemClickEnabled="True"
    ItemClick="guestsListView_ItemClick">
    <ListView.ItemTemplate>
        <DataTemplate x:DataType="localData:Guest">
            <StackPanel Orientation="Horizontal"
                Spacing="15">
                <TextBlock FontStyle="Italic"
                    Text="{x:Bind FirstName}" />

                <TextBlock FontWeight="Bold"
                    Text="{x:Bind LastName}" />

                <TextBlock>
                    <Run></Run>
                    <Run Text="{x:Bind Room.Type}" />
                    <Run></Run>
                    <Run Text="{x:Bind Room.Floor}" />
                    <Run></Run>
                </TextBlock>
            </StackPanel>
        </DataTemplate>
    </ListView.ItemTemplate>
</ListView>

```

4.4. Navigation properties

Entity Framework herkent Navigation Properties automatisch als die aan de regels voldoen die staan beschreven in de documentatie: <https://learn.microsoft.com/en-us/ef/core/modeling/relationships/navigation>

Bij een one-to-many relatie tussen twee entiteiten bestaat er in een van de entiteiten een verzameling, want die kan 'many' van de andere entiteit hebben. In het eerdere voorbeeld van Chat zagen we dat die een verzameling van berichten kan hebben:

```

1 public class Chat
2 {
3     public int Id { get; set; }
4     public string Name { get; set; }
5
6     public ICollection<Message> Messages { get; set; } // Navigation property
7 }

```

In dat voorbeeld gebruikte we het algemene type 'ICollection'. Onder andere List erft over van ICollection. Het kan ook zijn dat je in de documentatie, en voorbeeld online ook deze variant ziet:

```
1 public List<Message> Messages { get; set; }
```

In beide van de bovenstaande voorbeelden is de waarde van Messages **null** als de berichten nog niet geladen zijn. Om te zorgen dat de eigenschap initialiseert wordt met een lege lijst kun je dit gebruiken:

```
1 public List<Message> Messages { get; } = new();
```

Dit is de kortere versie van deze code:

```
1 public List<Message> Messages { get; } = new List<Message>();
```

4.5. Entity Framework vertrouwt op conventies

Entity Framework gebruikt conventies om automatisch de primaire sleutel (PK: Primary Key) en vreemde sleutels (FK: Foreign Key) te bepalen. Bij het zoeken naar de PK kijkt het naar eigenschappen met de namen "Id" of "<klasse>Id". **Op school volgen we de conventie om de primary key simpelweg 'Id' te noemen.**

Voor de FK zoekt Entity Framework naar overeenkomende eigenschappen in gerelateerde entiteiten. Met behulp van conventies bepaalt Entity Framework ook welke kolommen of attributen worden gebruikt voor "Navigation Properties". Daarbij wordt aan de hand van een van deze naamgevingsconventies de bijbehorende entiteit gezocht:

- <navigatie-eigenschapsnaam><PK-eigenschapsnaam>
- <navigatie-eigenschapsnaam>Id
- <primaire entiteitstype-naam><PK-eigenschapsnaam>
- <primaire entiteitstype-naam>Id

Bron: <https://learn.microsoft.com/en-us/ef/core/modeling/relationships/conventions>

Voorbeeld:

Laten we kijken hoe Entity Framework achterhaalt welke **Messages** bij een bepaalde **Chat** horen:

```

1 using (var db = new AppDbContext())
2 {
3     chatsListView.ItemsSource = db.Chats.Include(c => c.Messages);
4 }

```

Hier vragen we Entity Framework via de Navigation Property 'Messages' om alle bijbehorende berichten van alle chats op te halen.

Entity Framework gaat vervolgens (via de bovengenoemde naamgevingsconventies op zoek in de klassen):

```

1 public class Chat
2 {
3     public int Id { get; set; }
4     public string Name { get; set; }
5
6     public ICollection<Message> Messages { get; set; } // Navigation property
7 }

```

Omdat de Navigation Property 'Messages' een lijst van '**Message**'-objecten is, gaat Entity Framework op zoek in de Message klasse. **Chat** is de primaire entiteit.

Ook onthoudt Entity Framework dat de **PK** van Chat de naam '**Id**' heeft

```

1 public class Message
2 {
3     public int Id { get; set; }
4     public int ChatId { get; set; }
5     public Chat Chat { get; set; } //
6     public int UserId { get; set; }
7     public User User { get; set; } //
8     public string Content { get; set; }
9     public DateTime SentAt { get; set; }
10 }

```

Entity Framework ziet dat *naamgevingsconventie c* bestaat: **<primaire entiteitstype-naam><PK-eigenschapsnaam>**

Namelijk: **ChatId**

Nu het beeld compleet is weet Entity Framework ongeveer deze query op te bouwen:

```

SELECT * FROM Chat
JOIN Message ON Message.ChatId = Chat.Id

```

Het mooie aan Entity Framework is dat we enkel eigenschappen met de juiste namen hoeven aan te maken. Daarna kan Entity Framework zelf (complexe) queries voor ons bedenken.

Frameworks die werken met conventies (in plaats van expliciete configuraties) volgen het design principe genaamd:

Convention over configuration

Lees hier meer: https://en.wikipedia.org/wiki/Convention_over_configuration

Extra Opdracht: ChatMeister Verbeteren

- Voeg in jouw ChatMeister project de *Navigation Properties* toe uit **paragraaf 4.5**.
- Zorg dat bij het laden van chats, ook de Messages worden geladen d.m.v. *Eager Loading*. Zie daarvoor ook **paragraaf 4.5**.
- Verander jouw code zodat de berichten pas geladen worden bij het openen van een individuele chat (Explicit Loading). Lees hiervoor verder in **paragraaf 4.6 en 4.7**.

Extra challenge:

Laat bij iedere Message ook de naam zien van de User die het bericht geplaatst heeft. Je moet hiervoor Explicit Loading gebruiken om de Users op te halen, die bij ieder bericht horen, die bij iedere Chat horen.

Hint: een behulpzame zoekopdracht is "EF explicit load collection child"

Hint: Alleen de laatste van een gerelateerd model 'eager loaden'

Om in ChatMeister alle chats te eager loaden inclusief *alleen* de laatste Message, kun je deze code gebruiken:

```
var chats = db.Chats
    .Include(c => c.Messages
        .OrderByDescending(m => m.Id).Take(1)
    )
    .ToList();
```

Let op: deze code produceert een Query die alleen werkt vanaf MySQL 8 of hoger.

Om óók de bij ieder bericht horende gebruiker op te halen werkt de 'OrderByDescending en Take'-manier helaas niet. Als we simpelweg 'ThenInclude(m => m.User)' na de .Include en voor de .ToList zetten krijgen we een vreemde error. Het lukt Entity Framework niet om het verzoek te vertalen. Mogelijk is dit een beperking of bug in Entity Framework of de Pomelo package.

Om het toch voor elkaar te krijgen moeten we geen OrderByDescending en Take gebruiken, maar een Where query. Die ziet er zo uit:

```
var chatsQuery = db.Chats
    .Include(c => c.Messages
        .Where(m => m.Id == m.Chat.Messages.Max(m => m.Id))
    )
    .ThenInclude(m => m.User);

var query = chatsQuery.ToQueryString();
var chats = chatsQuery.ToList();
```

De where hierboven vraagt om "de messages waarvan de ID gelijk is aan de hoogste (Max) id, in de messages van de chat waar de query nu is".

Extra: Om te debuggen kunnen we met 'ToQueryString' laten zien welke MySQL Query door Entity Framework op de achtergrond wordt uitgevoerd. In het screenshot hierboven zetten we die in de query variabele. Gebruik eens een breakpoint om te zien wat voor "monster" van een join-query gebruikt wordt.

4.6. Explicit loading

We zagen eerder al dat we gebruik kunnen maken van 'eager loading' om bij voorbaat alle chats met bijbehorende messages op te halen:

```
5      using (var db = new AppDbContext())
6      {
7          db.Database.EnsureCreated();
8
9          chatsListView.ItemsSource = db.Chats.Include(c => c.Messages);
10     }
```

Daarnaast kunnen we ook gebruik maken van 'Explicit loading'. Daarmee vragen we Entity Framework om de gegevens op net voordat we ze nodig hebben.

Explicit Loading is de gewenste situatie wanneer er veel gerelateerde gegevens zijn. Stel je voor dat er binnen iedere Chat wel 10,000 berichten zijn. Als we dan gebruik maken van 'Eager loading' zouden al die berichten in het geheugen van de computer moeten komen. Dan loopt ons RAM geheugen enorm snel vol en wordt de computer traag.

Bij veel gerelateerde gegevens laden die dus liever 'expliciet'. Dat ziet er als volgt uit:

Wanneer de gebruiker een Chat scherm opent, halen we binnen dat scherm de berichten op met behulp van 'Explicit loading':

```
protected override void OnNavigatedTo(NavigationEventArgs e)
{
    var clickedChatId = (int)e.Parameter;

    using var db = new AppDbContext();

    var chat = db.Chats.FirstOrDefault(c => c.Id == clickedChatId);

    // Explicit Load:
    db.Entry(chat)
        .Collection(c => c.Messages)
        .Load();

    messagesListView.ItemsSource = chat.Messages;
}
```

Als je de drie regels na de 'Explicit Load:'-comment hierboven in het commentaar zou zetten, dan zou de Messages eigenschap leeg zijn en zou je geen berichten zien. *Houdt hier rekening mee tijdens het ontwikkelen: lijken gerelateerde gegevens niet geladen te worden? Controleer of je ze wel geladen hebt met eager, explicit of lazy loading.*

Je kunt hier zelf meer vinden over **Explicit Loading**: <https://learn.microsoft.com/en-us/ef/core/querying/related-data/explicit>

4.7. Change Tracking

In de code hiervoor geven we de chat ID mee aan het chat scherm via de e.Parameter. Dat is de gewenste manier om gegevens mee te geven. Maar je ziet ook wel eens dat een hele instantie wordt meegegeven:


```

private void ChatsListView_ItemClick(object sender, ItemClickEventArgs e)
{
    var chat = (Chat)e.ClickedItem;

    Frame.Navigate(typeof(DetailPage), chat);
}

```

Waarna die instantie in het geopende scherm kan worden gebruikt:

```

protected override void OnNavigatedTo(NavigationEventArgs e)
{
    var clickedChat = (Chat)e.Parameter;

    using var db = new AppDbContext();

    db.Attach(clickedChat);

    db.Entry(clickedChat)
        .Collection(c => c.Messages)
        .Load();

    messagesListView.ItemsSource = clickedChat.Messages;
}

```

Dit kan, maar is niet aan te raden, omdat er dan extra werk nodig is om het werkend te krijgen. Ook kan het zijn dat de chat instantie oude data bevat. Als we niet opnieuw de chat uit de database halen kan dat voor problemen zorgen. Dus we geven liever de Id mee, en halen dan opnieuw de data op.

Om toch een begrip te vormen over hoe Entity Framework werkt leggen we het toch uit. Het toont namelijk wel hoe EF met 'Change Tracking' bijhoudt welke wijzigingen gemaakt zijn aan objecten.

In de code hierboven staat deze regel:

```
db.Attach(clickedChat);
```

Omdat onze chatberichten door een andere AppDbContext-instantie zijn geladen (*in het overzicht scherm*), moeten we de chat 'verbinden' met de AppDbContext in de OnNavigatedTo. Dit doen we met de Attach-methode op de relevante DbSet. Wanneer je dit niet doet krijg je deze foutmelding:

```
using(var db = new AppDbContext())
{
    //db.Chats.Attach(chat); // Nodig omdat de chat uit een andere context instantie komt

    db.Entry(chat)
        .Collection(c => c.Messages)
        .Load();
}
```

Exception User-Unhandled

System.InvalidOperationException: 'The navigation 'Chat.Messages' cannot be loaded because the entity is not being tracked. Navigations can only be loaded for tracked entities.'

Zoals in de foutmelding te lezen is komt dit omdat Entity Framework 'entities tracked'. Dit wordt gedaan door het '**Change Tracking**' (wijzigingsregistratie) systeem.

Change Tracking is een mechanisme dat wordt gebruikt door Entity Framework om wijzigingen in entiteitsobjecten bij te houden voordat deze naar de database worden geschreven. Het stelt Entity Framework in staat om te detecteren welke eigenschappen zijn gewijzigd, toegevoegd of verwijderd, zodat de juiste databasebewerkingen kunnen worden uitgevoerd.

Wanneer een entiteitsobject wordt opgehaald uit de database, worden de oorspronkelijke waarden van de eigenschappen vastgelegd. Wanneer er wijzigingen worden aangebracht in de eigenschappen van het object, worden deze wijzigingen bijgehouden door de huidige waarden te vergelijken met de oorspronkelijke waarden.

Wanneer we dan later in de context aangeven "sla de wijzigingen in deze chat op". Dan weet Entity Framework of er iets verandert is en kan aan de hand daarvan een update query opbouwen.

Je kunt hier meer lezen over wat **Change Tracking** is: <https://learn.microsoft.com/en-us/ef/core/change-tracking/explicit-tracking>

Toch is de beste manier om met deze objecten om te gaan als volgt: geef de 'Id' van objecten door aan methodes en constructors (niet het hele object). Gebruik die Id dan altijd om een "verse" instantie uit de DbContext op te halen.

4.8. Opdracht: DevCitySim (deel 2)

Deze opdracht bouwt verder op de opdracht die je eerder hebt geschreven in Hoofdstuk 3, bij de opdracht 'DevCitySim (Stappenplan Entity Framework)'. Bij die opdracht hebben we enkele modellen aangemaakt, hebben we verbonden met een database en hebben we testgegevens ge-'seed' in de database. De gebruiker kan in een ListView de namen van alle bewoners zien.

We gaan de code opknappen en uitbreiden met extra functionaliteiten. Zo gaan we zorgen dat wanneer we op de naam van een bewoner klikken een nieuw scherm opent. Alle gerelateerde gebouwen worden in dat nieuwe scherm getoond.

Instructies:

1. Zoek de code van DevCitySim op, die je in hoofdstuk 3 hebt geschreven. Als je die niet meer kunt vinden: voer dan die opdracht eerst uit.
2. Voeg de benodigde eigenschappen toe aan de modellen, zodat een bewoner meerdere gebouwen kan hebben en een gebouw bij een bewoner kan horen (one-to-many):

Entiteit	Attribuut	Uitleg
Bewoner	Naam	Naam van bewoner, bijv.: Jane Doe
	Leeftijd	Leeftijd bewoner, bijv.: 19

Gebouw	Beroep	Beroep, bijv.: Software Developer
	Gebouwen	Verzameling/lijst van gebouwen die bezocht worden door de bewoner. Zie ook 'Gebouw'-entiteit
	Naam	Naam van het gebouw of bedrijf in het gebouw, bijv.: DevCityTech
	Type	Beschrijving soort gebouw, bijv.: Kantoor, winkel, woonhuis
	Locatie	Adres, bijv.: Nullstraat 1
	Bewoner	De bewoner waarbij dit gebouw hoort.

- Voeg een extra 'bewoner gebouwen'-scherm toe in de applicatie, waarin de gebouwen van een specifieke bewoner getoond kunnen worden.
- Maak gebruik van **Eager Loading** om bij het laden van alle bewoners, ook alle bijbehorende gebouwen te laden.
- Laat bij het klikken op een bewoner het gebouwen scherm openen. Toon in een ListView alle namen van gebouwen die bij die bewoner horen.
- Voeg in de seeder (OnModelCreating) **5 extra bewoners toe** en **5 extra gebouwen**.
- Zorg dat je de applicatie hebt getest en dat deze werkt.
- Zet de code die alle bewoners en gebouwen via 'Eager Loading' ophaalt in het commentaar. *Verwijder dit niet, maar bewaar dit project om later terug te kunnen zoeken hoe eager loading werkt.*
- Pas de code zo aan, dat de gebouwen nu enkel worden geladen als het gebouwen scherm opent (via **Explicit Loading**)

4.9. Many-to-many relaties

We hebben het tot nu toe over one-to-many relaties gehad. Door simpelweg een Navigation Property en de juiste Foreign Key eigenschappen toe te voegen kunnen we die implementeren. Bij een many-to-many relatie komt ook nog een 'koppeltabel' kijken. Met alles wat we tot nu toe gezien hebben, zouden we die al kunnen implementeren. De koppeltabel heeft namelijk twee one-to-many relaties, een naar iedere entiteit die gekoppeld wordt.

Onderzoek zelf in de documentatie van Microsoft hoe many-to-many relaties in Entity Framework werken: <https://learn.microsoft.com/en-us/ef/core/modeling/relationships/many-to-many>. Wij raden aan dat je voor de 'Basic many-to-many' aanpak gaat:

Basic many-to-many

In the most basic case for a many-to-many, the entity types on each end of the relationship both have a collection navigation. For example:

```
C# Copy  
  
public class Post  
{  
    public int Id { get; set; }  
    public List<Tag> Tags { get; } = new();  
}  
  
public class Tag  
{  
    public int Id { get; set; }  
    public List<Post> Posts { get; } = new();  
}
```

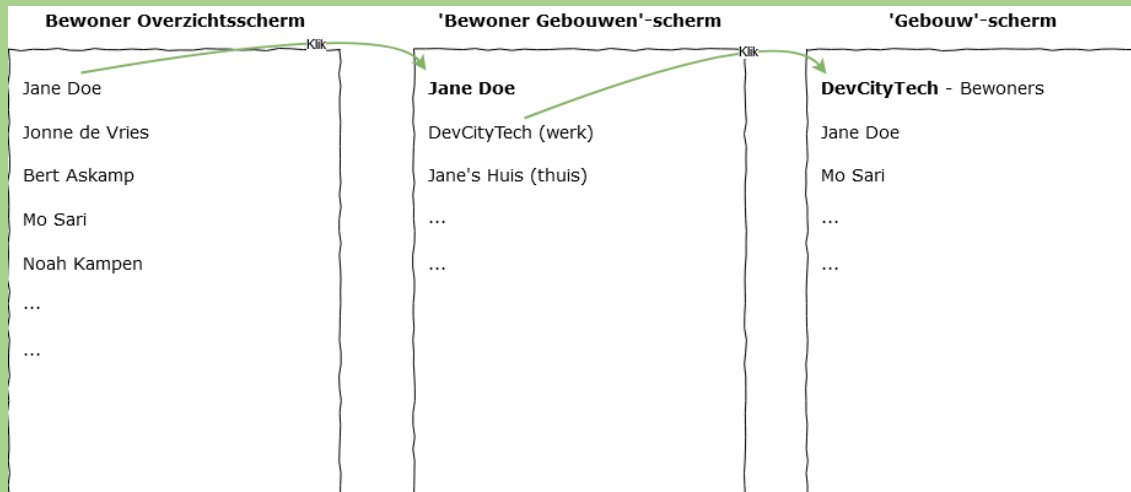
This relationship is **mapped by convention**. Even though it is not needed, an equivalent explicit

Het lijkt alsof er geen klasse is voor de koppeltabel, maar bij een many-to-many relatie is die er altijd. Entity Framework handelt dit onzichtbaar voor ons af. Ondanks dat zie je, wanneer je in de database kijkt, de koppeltabel.

Inleveropdracht 4.10: DevCitySim (deel 3)

Deze opdracht bouwt verder op de opdracht 'DevCitySim (deel 2)'. We gaan die code nu wat verbeteren en logischer inrichten.

In de vorige opdracht gingen we ervanuit dat ieder gebouw bij maar één bewoner hoort. Dat is eigenlijk niet logisch. Meerdere bewoners kunnen een verband hebben met hetzelfde gebouw. In plaats van de one-to-many relatie die we bij het vorige deel van deze opdracht maakte, gaan we nu een many-to-many relatie implementeren.



Instructies:

1. Zoek de code van DevCitySim deel 2 op, die je eerder in dit hoofdstuk hebt geschreven. Als je die niet meer kunt vinden: voer dan die opdracht eerst uit.
2. Pas de modellen aan zodat er niet langer een one-to-many relatie bestaat tussen Bewoner en Gebouw, maar een many-to-many relatie.
Onderzoek zelf met behulp van de Microsoft documentatie hoe je een many-to-many relatie kunt bewerkstelligen: <https://learn.microsoft.com/en-us/ef/core/modeling/relationships/many-to-many>
3. Pas de seeder aan, zodat meerdere bewoners relaties hebben met meerdere gebouwen. Zorg ervoor dat er ten minste 3 bewoners zijn die een relatie hebben met gebouwen, waar andere bewoners ook een relatie mee hebben.
4. Voeg een gebouw-scherm toe waarin de naam van het gebouw getoond wordt. Laat dat scherm openen bij het klikken op een gebouw in het 'bewoner gebouwen'-scherm.
5. Laat in het gebouw-scherm ook een ListView zien met daarin de namen van alle bewoners die gerelateerd zijn aan het gebouw.

Inleveren:

Je levert de opdracht als volgt in op Itslearning:

- Maak een .zip van de gehele solution
- Maak screenshots van het 'gebouw' en 'bewoner gebouwen'-scherm schermen in jouw applicatie

Veel succes met de opdracht!

4.11. Diepere relaties ophalen met 'Eager Loading'

Soms wil je ook gegevens ophalen die zijn gerelateerd aan gegevens, gerelateerd aan weer andere gegevens. Dat was een lastige zijn om te lezen (én schrijven), dus nemen we als voorbeeld de ChatMeister applicatie en ERD.

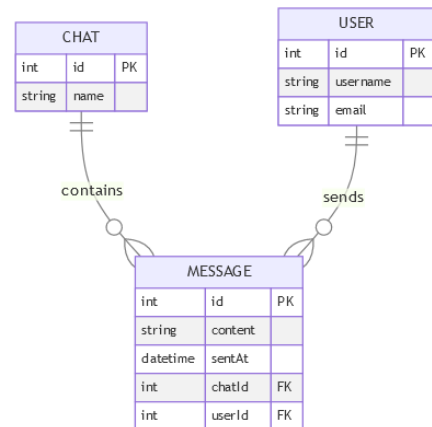
Stel dat we in ChatMeister via de Chat alle gegevens willen ophalen die bij die Chat horen. Dat zijn dus alle berichten (Message), maar ook de gegevens van gebruikers (User) die bij die berichten horen.

We zagen eerder al dat we de berichten redelijk makkelijk kunnen ophalen met Include:

```
chatsListView.ItemsSource = db.Chats.Include(c => c.Messages);
```

Maar als we van de berichten, ook de username van iedere gebruiker willen laten zien, dan willen we van ieder bericht ook de bijbehorende User ophalen. Dat kan met 'ThenInclude':

```
chatsListView.ItemsSource = db.Chats
    .Include(c => c.Messages)
    .ThenInclude(m => m.User);
```



Met de 'ThenInclude'-methode bouw je de query verder op het resultaat van de voorgaande Include. Omdat de Include met 'c.Messages' een verzameling van Message teruggeeft, ziet ThenInclude dat het type van 'm' dan Message is. We kunnen dan aangeven welke Navigation Property binnen Message ook opgehaald moet worden.

4.12. Opdracht: ChatMeister (deel 2)

Deze opdracht bouwt verder op de opdracht die je eerder hebt ontwikkeld in Hoofdstuk 3, bij de opdracht 'ChatMeister: Proof-of-concept'. In die applicatie zien we chats, waarna we bij het klikken op een chat berichten zien. We zien de UserId's en inhoud van het berichten in een chat. Dat gaan we nu uitbreiden.

Instructies:

1. Zoek de code van ChatMeister op, die je in hoofdstuk 3 hebt geschreven. Als je die niet meer kunt vinden: voer dan die opdracht eerst uit.
2. Pas de code in het overzichtsscherm aan, zodat gebruik gemaakt wordt van Eager Loading om:
 - a. Alle Chat's te laden
 - b. Ook alle Messages te laden
 - c. Ook alle Users bij iedere Message te laden
3. Pas de chat detail scherm xaml aan om voor ieder bericht in plaats van de 'UserId' de 'Username' van de gebruiker te tonen.

Bij databinding kun je ook eigenschappen binnen Navigation properties aanspreken, gewoon zoals je gewend bent in C# met een punt (.) – Bijv.: om bij de Name van Chat te komen: Chat.Name

4.13. Gegevens filteren bij relaties

Met de 'Where'-methode kunnen we gegevens filteren. Om bijvoorbeeld op een DbSet 'Companies' alle bedrijven op te halen met de naam Curio schrijven we:

```
var companies = dbContext.Companies
    .Where(c => c.Name == "Curio");
```

De 'Where'-methode verwacht een simpele arrow/lambda function expression die met een boolean aangeeft of de databaseregels meegenomen moet worden in de resultaten. Hierboven worden dus alleen databaseregels van bedrijven meegenomen waar de Name gelijk is aan "Curio".

Om te filteren bij gerelateerde data kunnen we, na een Include, ook nog een Where methode aanroepen. In het onderstaande voorbeeld werken we met bedrijven die meerdere brandblussers kunnen hebben.

Zoals we tot nu toe gezien hebben kunnen we op deze manier alle gegevens (en gerelateerde gegevens) ophalen:

```
var companies = dbContext.Companies
    .Include(c => c.FireExtinguishers);
```

Als we nu niet alle bedrijven willen, maar alleen bedrijven met de naam 'Curio', dan kunnen we de Where methode gebruiken:

```
var companies = dbContext.Companies
    .Include(c => c.FireExtinguishers)
    .Where(c => c.Name == "Curio");
```

Net als bij de Include gebruiken we hier weer een **arrow/lambda function expression**. Deze lambda functie krijgt ook een Company als parameter (omdat we met de .Companies DbSet bezig zijn). Het verschil met de Include is dat bij de Where methode wordt verwacht dat je een **bool** teruggeeft. Daarmee geef je aan of een resultaat wel of niet in de resultaten moet worden opgenomen.

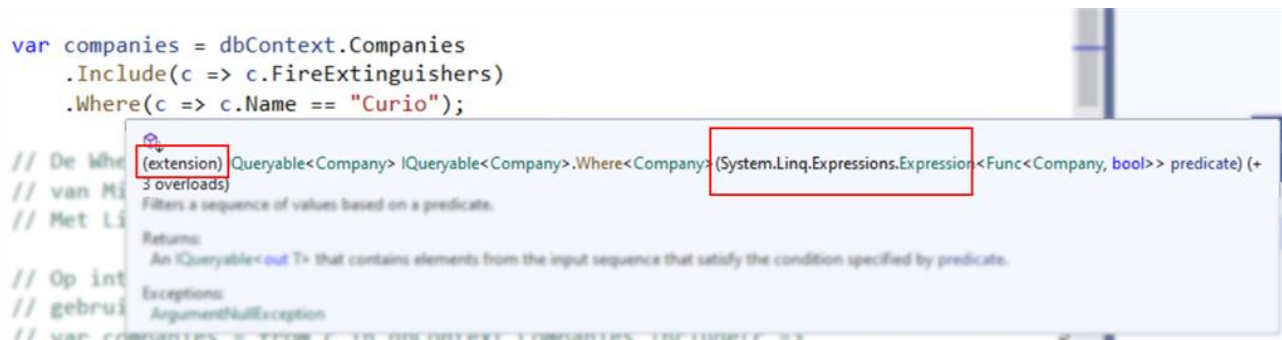
Hier nog een voorbeeld, waar we dit keer zoeken we naar alle bedrijven waarvan de naam begint met de letter "c":

```
var companies = dbContext.Companies
    .Include(c => c.FireExtinguishers)
    .Where(c => c.Name.ToLower().StartsWith("c"));
```

4.14. Microsoft LINQ

Dat we met Include kunnen werken en met Where kunnen filteren is mede dankzij LINQ van Microsoft. LINQ is als extension gemaakt en werkt op objecten van types waar je doorheen kunt 'lopen' (IEnumerable) zoals Lists. Ook werkt het op types waar je queries op kunt uitvoeren (Queryable) zoals DbSet, of het resultaat van de Include.

Je herkent dat een methode vanuit LINQ komt omdat je in IntelliSense het woord LINQ terugziet:



Achtergrond

Microsoft heeft LINQ (Language Integrated Query) ontwikkeld om het werken met datasets eenduidiger te maken. Zonder LINQ moet je zelf SQL queries schrijven (o.i.d.). De manier waarop je dat schrijft kan per taal/systeem verschillen. Met LINQ schrijf je in C# code, die op de achtergrond kan worden vertaald naar de juiste taal.

Zo bestaan er vertalingspakketten als LINQ to SQL, LINQ to XML, LINQ to Objects én (via de Pomelo package) bestaat er een soort LINQ to MySQL.

Bron: <https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/concepts/linq/>

LINQ kun je op twee manieren schrijven

Je kunt de code die je hierboven al zag ook anders schrijven, LINQ heeft namelijk een aantal unieke 'keywords' in C#. Daarmee wordt deze speciale syntax mogelijk:

```
var companies = from c in dbContext.Companies.Include(c => c.FireExtinguishers)
                where c.Name == "Curio"
                select c;
```

Maar als je "dicht bij" de gewoonlijke C# syntax wilt blijven, heeft het volgende gebruik van de LINQ methodes voorkeur over die speciale syntax:

```
var companies = dbContext.Companies
    .Include(c => c.FireExtinguishers)
    .Where(c => c.Name == "Curio");
```

4.15. Beperkingen in LINQ

Zoals we bij de Where en Include methodes zagen, wordt gebruik gemaakt van LINQ Arrow/Lambda function Expressions. In beide van die gevallen worden die functies vertaald naar een andere taal. Bij onze applicatie (die MySQL gebruikt) wordt het vertaald naar MySQL-code. **Om die reden kun je geen super-complexe logica in een arrow/lambda function expression zetten.** Het moet namelijk nog te vertalen zijn naar SQL.

Hier een voorbeeld in een Console App van een technische onmogelijkheid: We roepen hier vanuit een function expression een methode aan:


```

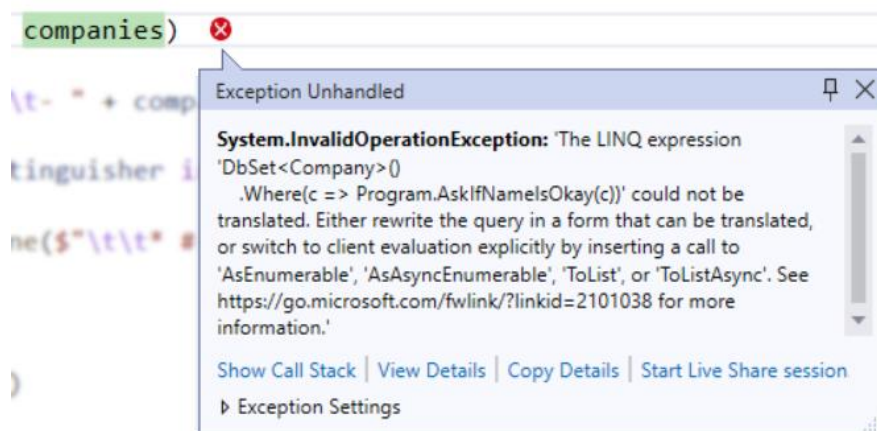
1 reference
static bool AskIfNameIsOkay(Company company)
{
    return company.Name == "Curio";
}

0 references
static void Main(string[] args)
{
    var dbContext = new AppDbContext();

    var companies = dbContext.Companies
        .Include(c => c.FireExtinguishers)
        .Where(c => AskIfNameIsOkay(c));
}

```

Wanneer we de applicatie uitvoeren krijgen we deze Exception:



Het lukt niet om die LINQ expression om te zetten naar SQL. Want de database kan nooit bij die methode in onze Program.

De suggestie die in de error wordt geboden is niet altijd een werkbare oplossing, kijk maar:

```

1 reference
static bool AskIfNameIsOkay(Company company)
{
    return company.Name == "Curio";
}

0 references
static void Main(string[] args)
{
    var dbContext = new AppDbContext();

    var companies = dbContext.Companies
        .Include(c => c.FireExtinguishers)
        .ToList()
        .Where(c => AskIfNameIsOkay(c));
}

```

Deze code werkt wel, maar alleen omdat alle companies (incl. fire extinguishers) worden opgehaald uit de database. Dan wordt op de resulterende List een Where uitgevoerd. Dit betekent dat dat allemaal in het

geheugen van de computer komt. Dat is prima bij 10 resultaten, maar wat gebeurt er bij 100? Of 1000? Of 10000? Dan is dit geen oplossing meer.

In zo'n geval is je Query eigenlijk te complex en kun je beter een simpelere function expression schrijven.

4.16. Samenwerken in een project: verschillen in onze DbContext

Stel je voor dat je bij een 'Barroc Intens' samenwerkt in een groep van 3. Op jullie apparaten zijn de database gegevens allemaal anders:

- Jij kiest als database naam 'barrocit' en hebt voor de 'root' user geen wachtwoord.
- Jouw collega kiest de naam 'curio_barroc_it' en heeft voor de 'root' user het wachtwoord 'iu21j3me'
- Jullie andere collega kiest 'curio_barroc_intens' en heeft een gebruiker 'barroc_intens' met het wachtwoord 'asdakn42js'

We vragen ons af: Hoe kun je nu een DbContext instellen, zonder steeds Merge Conflicts te krijgen? We moeten onze database gegevens ergens opslaan, waar ze niet steeds met de gegevens van collega's botsen.

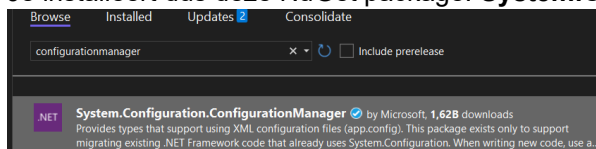
We hebben hiervoor twee opties:

(We gaan uit van een klasse AppDbContext die overerft van DbContext)

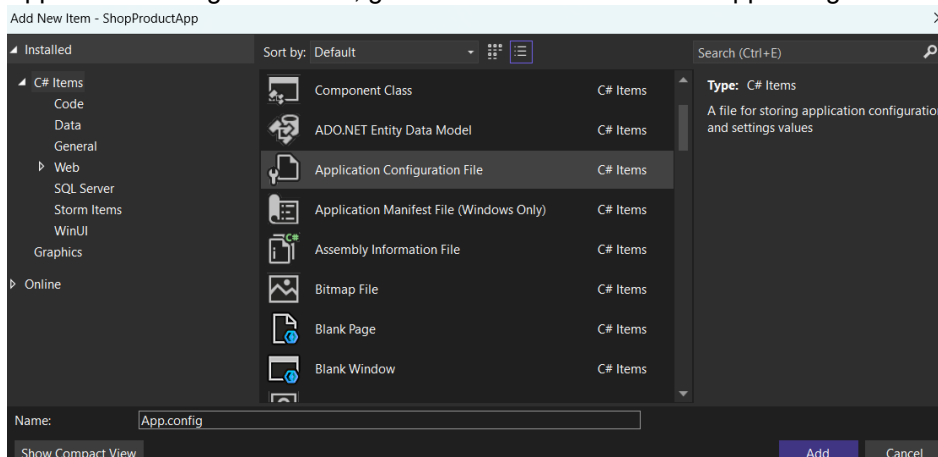
- a) **(slecht idee)** Je plaatst de hele AppDbContext.cs in de .gitignore
Dit heeft het grote nadeel dat al onze seeders en andere code in OnModelCreating niet wordt opgenomen in versiebeheer.
- b) **(aanbevolen)** We verplaatsen de connection string naar een los bestand. Dat bestand komt in .gitignore en we laden de connection string uit dat bestand in onze AppDbContext.
Dit is de gewenste manier, het maakt gebruik van een los 'App.config' bestand, waarin je de connection string plaatst.

Zie hier uitleg daarover: <https://learn.microsoft.com/en-us/ef/core/miscellaneous/connection-strings#winform--wpf-applications>

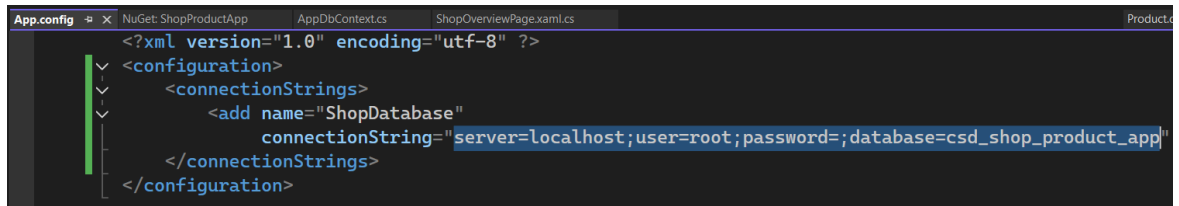
- Je installeert dus deze NuGet package: **System.Configuration.ConfigurationManager**



- Maak nu in je project een App.config aan via: (rechtsklik op je project) > Add > New Item... > Application Configuration File, geef die de standaard naam: App.config



- Verander de inhoud van dat bestand naar iets als:



```
<?xml version="1.0" encoding="utf-8" ?>
<configuration>
  <connectionStrings>
    <add name="ShopDatabase"
          connectionString="server=localhost;user=root;password=;database=csd_shop_product_app"
    />
  </connectionStrings>
</configuration>
```

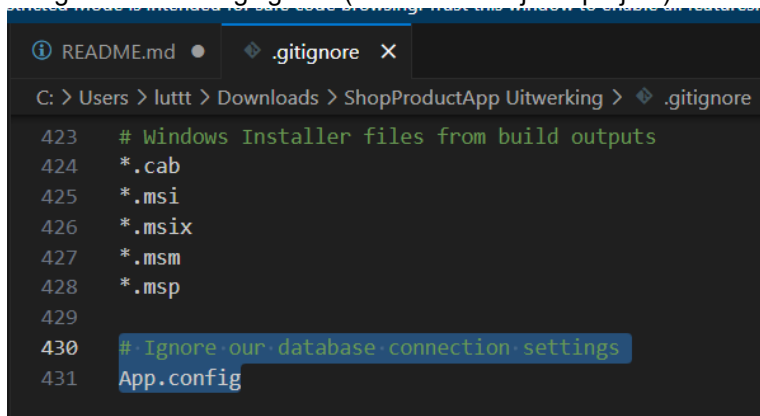
Let er op dat jouw connectionstring in dit bestand, precies hetzelfde is als nu in je AppDbContext staat. Voor MySQL dus:
 server=localhost;port=3306;user=root;password=;database=barroc_it

- In de AppDbContext wordt je OnConfiguring dan ongeveer:

```
optionsBuilder.UseMySQL(
    ConfigurationManager.ConnectionStrings["ShopDatabase"].ConnectionString,
    ServerVersion.Parse("8.0.30")
);
```

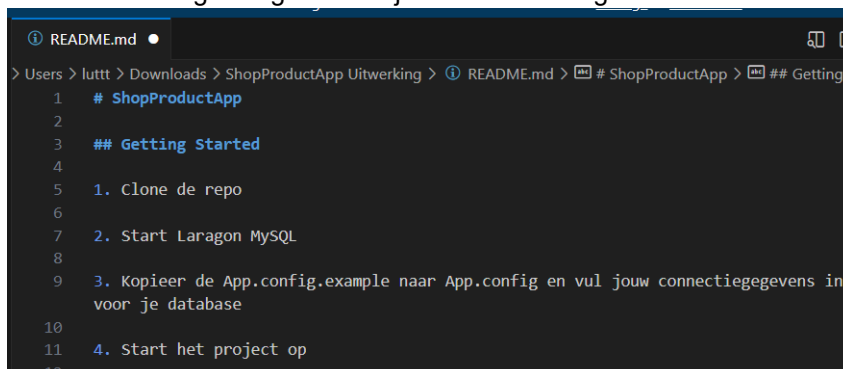
Verander **ShopDatabase** naar iets wat voor jouw project passend is. Bijvoorbeeld 'BarrocIntens', of 'DatabaseDetails'.

- Zorg dat deze App.config niet in git wordt gevolgd, door het toe te voegen aan de .gitignore. Voeg onderaan de gitignore (in de root van jullie project) toe:

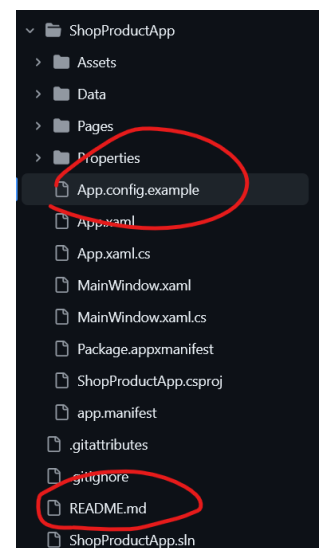


```
423 # Windows Installer files from build outputs
424 *.cab
425 *.msi
426 *.msix
427 *.msm
428 *.msp
429
430 # Ignore our database connection settings
431 App.config
```

- Maak voor het gemak van je collega's ook een App.config.example (die niet in de .gitignore staat), met daarin een voorbeeld van de in te stellen connection string. Voeg ook aan je README uitleg toe:



```
1 # ShopProductApp
2
3 ## Getting Started
4
5 1. Clone de repo
6
7 2. Start Laragon MySQL
8
9 3. Kopieer de App.config.example naar App.config en vul jouw connectiegegevens in
   voor je database
10
11 4. Start het project op
12
```



Let op: stond je App.config al in de repository, voordat je het toevoegde in de .gitignore? Dan blijft git die volgen. In dat geval moet je het bestand verwijderen, de verwijdering comitten. Pas daarna kun je het terug in het project zetten en blijft het 'ge-ignored'.

4.17. Opdracht: Meerdere relaties tussen dezelfde modellen

Deze uitleg bouwt verder op het startpunt 'H4 Opdracht 4.17 - MonkeyWhiskyClub-ManyToMany.zip' in de map Uitwerkingen op Itslearning. Download het startpunt, pak het uit en open de solution.

We gaan met dit project een probleem bekijken en de oplossing ervoor. We beginnen vanuit een werkende situatie, passen de code aan om het probleem te veroorzaken, en gaan het vervolgens oplossen.

1. Bekijk de volgende twee modellen die een many-to-many relatie tot elkaar hebben:

```
internal class Monkey
{
    11 references
    public int Id { get; set; }
    13 references
    public string Name { get; set; }
    11 references
    public string Species { get; set; }

    // ...

    2 references
    public ICollection<Club> Clubs { get; set; }
    0 references
    public ICollection<ClubMonkey> ClubMonkeys { get; set; }
}

internal class Club
{
    11 references
    public int Id { get; set; }
    11 references
    public string Name { get; set; }

    // ...

    0 references
    public ICollection<Monkey> Monkeys { get; set; }
    0 references
    public ICollection<ClubMonkey> ClubMonkeys { get; set; }
}
```

2. Zie ook onze (App)DbContext waarin we voor deze relatie vastleggen dat we het 'ClubMonkey' pivot model willen gebruiken als koppeling tussen de twee entiteiten:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Monkey>()
        .HasMany(m => m.Clubs)
        .WithMany(c => c.Monkeys)
        .UsingEntity<ClubMonkey>();
}
```

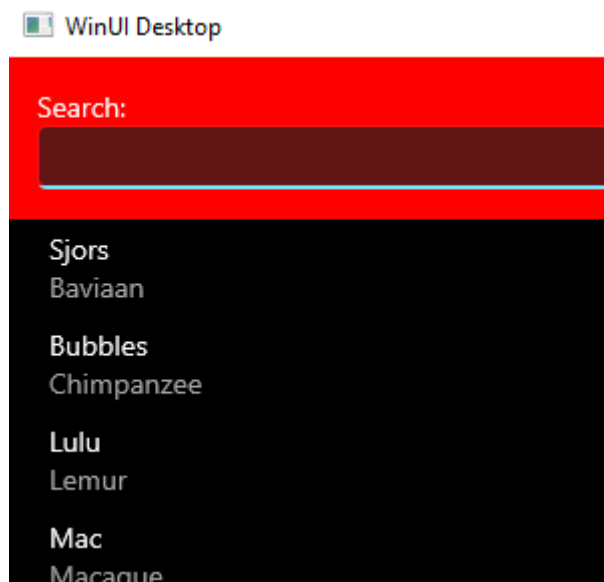
3. Observeer hoe dit pivot model eruit ziet:

```
internal class ClubMonkey
{
    1 reference
    public int ClubId { get; set; }
    1 reference
    public int MonkeyId { get; set; }

    1 reference
    public int WhiskeyInMilliliters { get; set; }
    1 reference
    public DateOnly PartyDate { get; set; }
}
```

Alle bovenstaande code werkt prima en maakt bij het opstarten van de applicatie 3 tabellen aan (monkeys, clubmonkeys, clubs).

Ook worden de tabellen succesvol gevuld met testdata en toont de applicatie een ListView met alle monkeys en hun soort:



Laten we nu eens kijken naar een uitbreiding die een probleem gaat veroorzaken:

- We willen dat iedere Monkey ook een favoriete Club heeft.
- Iedere monkey heeft precies 1 favoriete Club
- In de ListView willen we de favoriete Club van iedere Monkey laten zien.

We passen de code stap-voor-stap aan:

4. Voeg aan de Monkey-klasse twee eigenschappen toe:
 - a. De foreign key FavoriteClubId
 - b. De bijbehorende Navigation property FavoriteClub

```
internal class Monkey
{
    11 references
    public int Id { get; set; }
    13 references
    public string Name { get; set; }
    12 references
    public string Species { get; set; }

    0 references
    public int FavoriteClubId { get; set; }
    0 references
    public Club FavoriteClub { get; set; }

    2 references
    public ICollection<Club> Clubs { get; set; }
    0 references
    public ICollection<ClubMonkey> ClubMonkeys { get; set; }
}
```

5. Start de applicatie en je zult merken dat we deze foutmelding krijgen op de eerste plek waar Entity Framework moet nadenken:

```

public MainWindow()
{
    this.InitializeComponent();

    using var db = new AppDbContext();
    db.Database.EnsureDeleted();
    db.Database.EnsureCreated();

    var monkeys = db.Monkeys
        .Include(m => m.Clubs)
        // .Include(m => m.FavoriteClub)
        .ToList();
    monkeysListView.ItemsSource = monkeys;
}

```

Exception User-Unhandled

System.InvalidOperationException: 'Unable to determine the relationship represented by navigation 'Monkey.FavoriteClub' of type 'Club'. Either manually configure the relationship, or ignore this property using the '[NotMapped]' attribute or by using 'EntityTypeBuilder.Ignore' in 'OnModelCreating'.'

De foutmelding verteld ons dat de relatie niet kan worden bepaald aan de hand van de navigation property die we net hebben toegevoegd (FavoriteClub van het type Club). Iets brengt Entity Framework in de war.

6. Stop de applicatie.

7. Kijk nog eens naar de nieuwe situatie tussen de Monkey en het Club model:

```

internal class Monkey
{
    11 references
    public int Id { get; set; }
    13 references
    public string Name { get; set; }
    12 references
    public string Species { get; set; }

    0 references
    public int FavoriteClubId { get; set; }
    0 references
    public Club FavoriteClub { get; set; }

    2 references
    public ICollection<Club> Clubs { get; set; }
    0 references
    public ICollection<ClubMonkey> ClubMonkeys { get; set; }
}

```

```

internal class Club
{
    11 references
    public int Id { get; set; }
    11 references
    public string Name { get; set; }

    // ...

    1 reference
    public ICollection<Monkey> Monkeys { get; set; }
    0 references
    public ICollection<ClubMonkey> ClubMonkeys { get; set; }
}

```

We begonnen hier met twee navigation properties die naar elkaar wezen om een many-to-many te maken: Club.Monkeys en Monkey.Clubs (**hierboven aangegeven met de groene pijlen**).

Doordat de navigation properties Club.Monkeys en Monkey.Clubs beide verzamelingen zijn weet Entity Framework te bedenken dat dit een many-to-many relatie moet worden. Daarbij helpt de AppDbContext configuratie mee:

```

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Monkey>()
        .HasMany(m => m.Clubs)
        .WithMany(c => c.Monkeys)
        .UsingEntity<ClubMonkey>();
}

```

Toch raakt Entity Framework in de war, want die Club.Monkeys zou ook prima werken bij een one-to-many relatie. Dan zou die navigation property alle apen kunnen geven, waarvan de club de favoriet is (**hierboven aangegeven met de rode pijl**).

Om deze verwarring te voorkomen moeten we in Club ook nog de tegenovergestelde relatie van Monkey.FavoriteClub vastleggen. Daarbij geven we met een speciaal attribuut aan waarvan het de tegengestelde (*Engels: inverse*) relatie is.

8. Voeg aan de Club model de tegengestelde relatie toe en markeer die met het InverseProperty attribuut:

```

internal class Club
{
    11 references
    public int Id { get; set; }
    11 references
    public string Name { get; set; }

    [InverseProperty(nameof(Monkey.FavoriteClub))]
    // Is hetzelfde als [InverseProperty("FavoriteClub")]
    0 references
    public ICollection<Monkey> FavoriteClubMonkeys { get; set; }

    1 reference
    public ICollection<Monkey> Monkeys { get; set; }
    0 references
    public ICollection<ClubMonkey> ClubMonkeys { get; set; }
}

```

Na deze code is Entity Framework niet langer in de war. Het is nu geheel duidelijk welke navigation properties met elkaar te maken hebben.

Om de code verder werkend te krijgen moeten we nog wel de seeders aanpassen (zodat iedere aap een favoriete club heeft) en willen we ook nog de favoriete club tonen in de xaml.

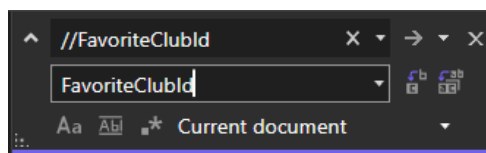
9. Pas de seeder in OnModelCreating aan, zodat iedere monkey een favoriete club id krijgt:

```

// Monkeys
modelBuilder.Entity<Monkey>().HasData(
    new Monkey
    {
        Id = 1,
        Name = "Sjors",
        Species = "Baviaan",
        FavoriteClubId = 1
    },
    new Monkey
    {
        Id = 2,
        Name = "Bubbles",
        Species = "Chimpanzee",
        FavoriteClubId = 1
    },
);

```

Tip: gebruik Ctrl + H om snel al het commentaar in het startpunt te verwijderen:



10. Pas de ListView aan in MainWindow.xaml zodat ook de favoriete club wordt getoond:


```

<ListView x:Name="monkeysListView" Grid.Row="1">
    <ListView.ItemTemplate>
        <DataTemplate x:DataType="localData:Monkey">
            <StackPanel Orientation="Vertical" Padding="5">
                <TextBlock Text="{x:Bind Name}" />
                <TextBlock Text="{x:Bind Species}"
                    Foreground="DarkGray" />
                <TextBlock Foreground="Gold">
                    <Run>Favoriete club:</Run>
                    <Run Text="{x:Bind FavoriteClub.Name}" />
                </TextBlock>
            </StackPanel>
        </DataTemplate>
    </ListView.ItemTemplate>
</ListView>

```

11. Pas de MainWindow.xaml.cs constructor aan, dat ook alle favoriete clubs van de monkeys worden geladen met Eager Loading:

```

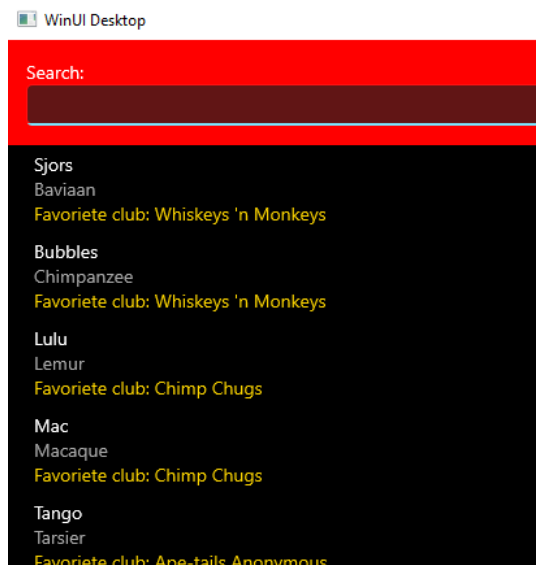
public MainWindow()
{
    this.InitializeComponent();

    using var db = new AppDbContext();
    db.Database.EnsureDeleted();
    db.Database.EnsureCreated();

    var monkeys = db.Monkeys
        .Include(m => m.Clubs)
        .Include(m => m.FavoriteClub)
        .ToList();
    monkeysListView.ItemsSource = monkeys;
}

```

12. Test de applicatie opnieuw, je ziet van alle apen de favoriete club:



4.18. Many-to-many koppeltabel seeden

In de opdracht hiervoor stond in de Data/AppDbContext.cs seeder logica in de OnModelCreating. Helemaal onderaan werd ook de koppeltabel ge-seed. **Er zijn twee manieren om een koppeltabel op te bouwen en te seeden:**

- a. **Wanneer je EF zelf een koppeltabel laat maken** seed je als volgt:

```
modelBuilder.Entity<Monkey>()
    .HasMany(m => m.Clubs)
    .WithMany(c => c.Monkeys)
    .UsingEntity(
        p => p.HasData(
            new { MonkeysId = 1, ClubsId = 1 },
            new { MonkeysId = 1, ClubsId = 2 }
        )
    );
```

Er gebeurt hier best iets bijzonders, er worden 2 anonieme objecten gemaakt. Dat zijn een soort klassen die je ter plekke declareert en invult. De notatie om een anoniem object te maken is:

```
new { EigenschapNaam = waarde }
```

We moeten dit doen, omdat er geen klasse is die de koppeltabel vertegenwoordigt.

- b. **Wanneer we klasse hebben die de koppeltabel vertegenwoordigt** ziet dat er als volgt uit:

Eerst moeten we het model hebben vastgelegd dat de koppeltabel vertegenwoordigt (ClubMonkey hieronder):

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Monkey>()
        .HasMany(m => m.Clubs)
        .WithMany(c => c.Monkeys)
        .UsingEntity<ClubMonkey>();
}
```

Je kunt dit zien als het 'pivot model' ofwel 'koppelmodel'.

Daarna kunnen we seeden zoals we gewend zijn:

```
modelBuilder.Entity<ClubMonkey>().HasData(
    new ClubMonkey
    {
        ClubId = 1,
        MonkeyId = 1,
        PartyDate = new DateOnly(2022, 10, 24),
        WhiskeyInMilliliters = 213
    }
);
```

We raden aan dat je altijd voor methode B gaat hierboven en een eigen 'pivot model' vastlegt. Op die manier kun je ook extra eigenschappen toevoegen aan de koppeltabel, wat vaak het geval is.

5. Entity Framework: CRUD

Met de 'Read' in 'CRUD' zijn we inmiddels goed bekend. We weten dat we in Entity Framework gegevens kunnen ophalen via een DbSet in een Database Context. Wanneer we ook gerelateerde gegevens willen ophalen kunnen we Include gebruiken, of voor gegevens, gerelateerd aan gegevens 'ThenInclude'.

Het lijkt dat we nog veel moeten leren als we nog maar 1 van de 4 letters in de afkorting CRUD hebben behandeld, maar er is hoop: Het invoeren (**C**reate), aanpassen (**U**psert) en verwijderen (**D**ele) van gegevens is in Entity Framework enorm makkelijk gemaakt! We zijn al ver over de helft van de lesstof van Entity Framework.

5.1. Gegevens invoeren

Entity Framework maakt het invoeren van gegevens makkelijk voor ons. Om bijvoorbeeld gegevens van een 'bedrijf' in te voeren doen we grofweg het volgende:

1. Open een verbinding met de database zoals je gewend bent:

```
using (var dbContext = new AppDbContext())
{
    // ...
}
```
2. Maak een instantie van een model wat je in de database wilt invoeren
bijvoorbeeld voor een model Company:

```
var myCompany = new Company{ Name = "Curio" };
```
3. Voeg die instantie toe aan de relevante DbSet van de Database Context
bijvoorbeeld:

```
dbContext.Companies.Add(myCompany);
```
4. Sla de Database Context op met de 'SaveChanges'-methode:

```
dbContext.SaveChanges();
```

Dat kan er bijvoorbeeld zo uitzien:

```
Console.WriteLine("Wat is de naam van het bedrijf?");
string companyNameInput = Console.ReadLine();

dbContext.Companies.Add(new Company
{
    Name = companyNameInput
});
dbContext.SaveChanges();

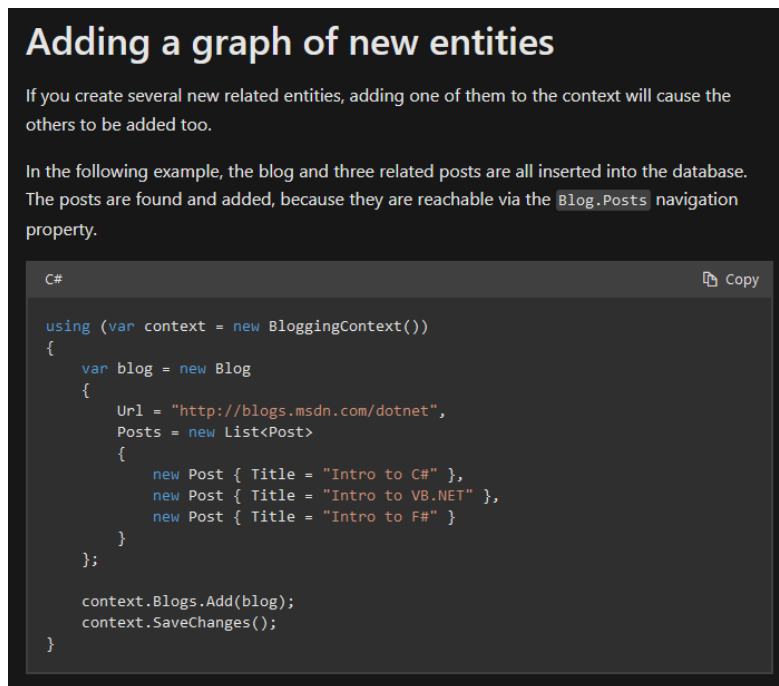
Console.WriteLine();
Console.WriteLine($"Het bedrijf {companyNameInput} is toegevoegd!");
```

dbContext is zoals gewoonlijk beschikbaar dankzij 'using (var dbContext = new AppDbContext())'

We zien dat Database Context niet automatisch opslaat wanneer je een object aan de DbSet toevoegt. **Je moet met de 'SaveChanges' methode expliciet aangeven wijzigingen aan de DbSet op te slaan.**

Hier kun je meer lezen over het invoeren van data: <https://learn.microsoft.com/en-us/ef/core/saving/basic>

Wanneer je in een enkele handeling gegevens *inclusief* daarbij gerelateerde gegevens wilt invoeren, dan kan dat als volgt:



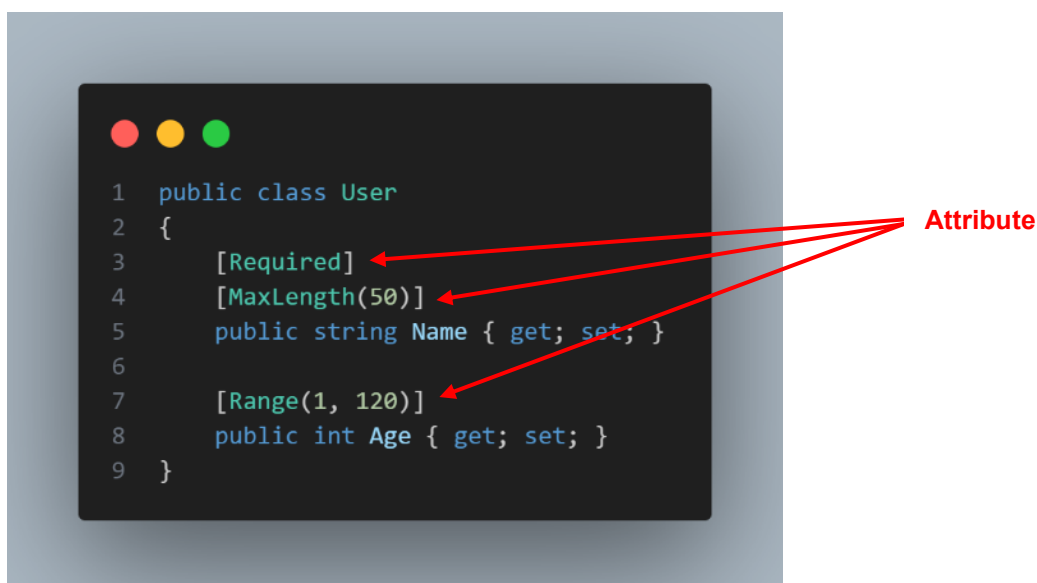
Bron: <https://learn.microsoft.com/en-us/ef/core/saving/related-data>

5.2. Invoer valideren

Om een degelijke applicatie te maken, moeten we nog wel de invoer van de gebruiker controleren. Dat noemen we **validatie** (Engels: **validation**).

Er zijn meerdere manieren om invoer te valideren, in dit moduleboek bespreken we er twee:

- If-statements:** Controleer met simpele if-statements of aan de voorwaarden wordt voldaan.
- Data Annotations:** Met 'attributen' uit de ingebouwde Package 'System.ComponentModel.DataAnnotations' leggen we voorwaarden vast:

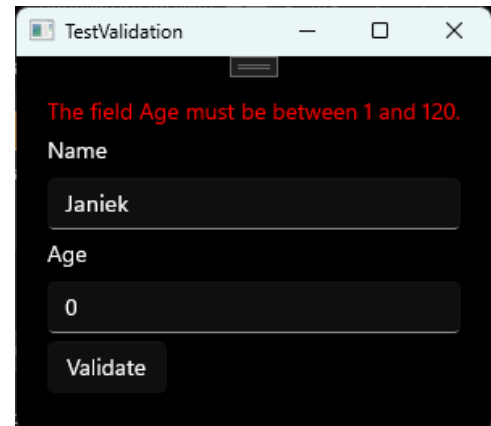


Hier betekent het Engelse woord "Annotations": annotaties. Ookwel: aanmerking, aantekening of

kanttekening. Met C# attributes kunnen we extra informatie bij onder andere classes, methodes en eigenschappen plaatsen.

We demonstreren hier de hierboven benoemde manieren van valideren a.d.h.v. de simpele applicatie hier rechts.

Bij het klikken op de 'Validate'-knop wordt 'validateButton_Click' aangeroepen. De naam en leeftijd invoeren hebben de x:Name nameTextBox en ageTextBox. Er is een standaard leeg TextBlock genaamd validationResultTextBlock waarmee in het rood een bericht getoond wordt bovenaan het formulier.



VALIDEREN MET IF-STATEMENTS

```
1 private void validateButton_Click(object sender, RoutedEventArgs e)
2 {
3     var user = new User
4     {
5         Name = nameTextBox.Text,
6         Age = int.TryParse(ageTextBox.Text, out var age) ? age : 0
7     };
8
9     var errors = new List<string>();
10
11     if (string.IsNullOrEmpty(user.Name))
12     {
13         errors.Add("The Name field is required.");
14     }
15     else if (user.Name.Length > 50)
16     {
17         errors.Add("The field Name must be a string with a maximum length of 50.");
18     }
19
20     if (user.Age < 1 || user.Age > 120)
21     {
22         errors.Add("The field Age must be between 1 and 120.");
23     }
24
25     if (errors.Count > 0)
26     {
27         validationResultTextBlock.Text = string.Join(Environment.NewLine, errors);
28     }
29     else
30     {
31         validationResultTextBlock.Text = "Validation succeeded!";
32     }
33 }
```

VALIDEREN MET DATA ANNOTATIONS

Allereerst staat bovenin het script van de User-model en de Window:

```
using System.ComponentModel.DataAnnotations;
```

De User-model krijgt deze attributen die de data 'annoteren':

```
1 public class User
2 {
3     [Required]
4     [MaxLength(50)]
5     public string Name { get; set; }
6
7     [Range(1, 120)]
8     public int Age { get; set; }
9 }
```

Bij het klikken op de knop **instantiëren** we een User met de ingevoerde gegevens. Vervolgens maken we een ValidationContext en roepen we de Validator.TryValidateObject aan die a.d.h.v. de attributen in de User-model validatie gaat uitvoeren:

```
1 private void validateButton_Click(object sender, RoutedEventArgs e)
2 {
3     var user = new User
4     {
5         Name = nameTextBox.Text,
6         Age = int.TryParse(ageTextBox.Text, out var age) ? age : 0
7     };
8
9     var context = new ValidationContext(user);
10    var results = new List<ValidationResult>();
11
12    if (!Validator.TryValidateObject(user, context, results, true))
13    {
14        var errors = new List<string>();
15
16        foreach (var validationResult in results)
17        {
18            errors.Add(validationResult.ErrorMessage);
19        }
20
21        validationResultTextBlock.Text = string.Join(Environment.NewLine, errors);
22    }
23    else
24    {
25        validationResultTextBlock.Text = "Validation succeeded!";
26    }
27 }
```

In de documentatie van Microsoft is een lijst te vinden met alle attributen die je bij een eigenschap kunt plaatsen: <https://learn.microsoft.com/en-us/dotnet/api/system.componentmodel.dataannotations>

MaxLengthAttribute	Specifies the maximum length of array or string data allowed in a property.
MetadataTypeAttribute	Specifies the metadata class to associate with a data model class.
MinLengthAttribute	Specifies the minimum length of array or string data allowed in a property.
PhoneAttribute	Specifies that a data field value is a well-formed phone number.
RangeAttribute	Specifies the numeric range constraints for the value of a data field.
RegularExpressionAttribute	Specifies that a data field value in ASP.NET Dynamic Data must match the specified regular expression.
RequiredAttribute	Specifies that a data field value is required.
ScaffoldColumnAttribute	Specifies whether a class or data column uses

```
1 [Required]
2 [MaxLength(50)]
3 public string Name { get; set; }
4
5 [Range(1, 120)]
6 public int Age { get; set; }
```

⚠ **C# Attribute Classes zijn bijzonder: je mag bij het gebruiken van de klasse de 'Attribute'-suffix weglaten.** Dus je mag zowel [RequiredAttribute] als [Required] schrijven, beide zijn valide.

🔧 5.3. Opdracht: Validatie oefenen met een aanmeldformulier

Je gaat een aanmeldformulier bouwen voor een workshop die mensen kunnen volgen. Het formulier bevat meerdere velden die gebruikers moeten invullen. Jij bent verantwoordelijk voor het bouwen van de interface en het valideren van de gegevens voordat ze opgeslagen worden.

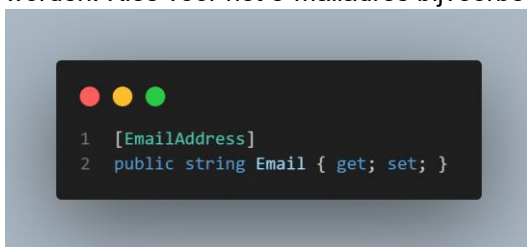
Je gaat beide manieren van valideren oefenen. Begin met validatie via if-statements:

1. Maak een project op de manier die je gewend bent, met MySQL via de Pomelo Entity Framework NuGet Package.
2. Maak een formulier waar de gebruiker deze gegevens invult:
 - i. Voornaam (tekst)
 - ii. Achternaam (tekst)
 - iii. Leeftijd (getal)
 - iv. E-mailadres (tekst)
 - v. Telefoonnummer (tekst)
 - vi. Postcode (tekst)
 - vii. Beschrijving van ervaring (meerdere regels tekst)
 - viii. Akkoord met voorwaarden (checkbox)
3. Maak een bijbehorend model aan en DbContext, zodat je deze gegevens in een MySQL database kunt opslaan.
4. Voeg onderaan het formulier een knop toe, waarmee de invoer kan worden gecontroleerd en vervolgens opgeslagen.
5. Implementeer deze controles (met behulp van if-statements):
 - i. Leeftijd moet een getal zijn van minimaal 16.
 - ii. E-mailadres moet geldig zijn, controleer: of het e-mailadres een @ en . bevat.
 - iii. Checkbox met 'akkoord voorwaarden' moet aangevinkt zijn.
6. Bedenk nu zelf aanvullende validaties die logisch zijn bij de andere invoervelden. Denk aan:
 - i. Zijn alle verplichte velden ingevuld?
 - ii. Is de leeftijd realistisch?
 - iii. Is de telefoonnummerstructuur correct?

- iv. Zijn namen niet te kort of te lang? *Zoek online eens naar de langste en kortste namen die bekend zijn.*
 - v. Heeft de postcode een geldig formaat (bijvoorbeeld 1234 AB)?
7. Laat de foutmeldingen duidelijk zien in de UI. Bijvoorbeeld met rode tekst in een TextBlock bovenaan het formulier – of – met een TextBlock onder ieder veld, zodat je per invoerveld kunt aangeven wat er mis is.
 8. Sla de gegevens enkel op in de database, wanneer er geen fouten zijn. Toon dan een nette succesmelding. *Lees evt. eerder in dit hoofdstuk de uitleg over het invoeren van gegevens in Entity Framework.*
 9. Test de applicatie.
 10. Maak een back-up van je project, zodat je deze validatie via if-statements later nog kunt raadplegen.
 11. Verwijder (nadat je een back-up van het project hebt gemaakt) de validatie via if-statements.

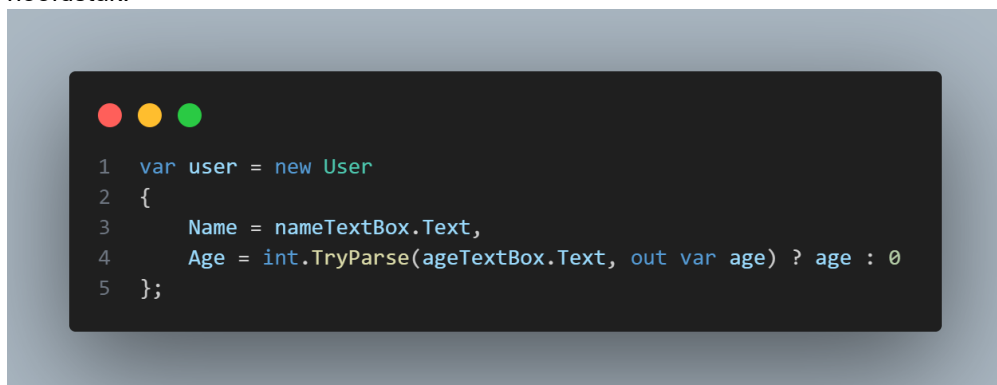
Nu gaan we validatie met Data Annotations implementeren:

12. Zoek in online en in de documentatie van Microsoft welke Data Annotations attributen je kunt gebruiken om validatie toe te passen.
13. Voeg de nodige attributen toe aan het model, die passen bij de controles die uitgevoerd moeten worden. Kies voor het e-mailadres bijvoorbeeld:



```
1 [EmailAddress]
2 public string Email { get; set; }
```

14. Controleer bij het klikken op de knop onderaan het formulier automatisch al deze attributen door:
 - i. Eerst de ingevoerde gegevens in het model in te voeren. Leer uit dit voorbeeld eerder in het hoofdstuk:



```
1 var user = new User
2 {
3     Name = nameTextBox.Text,
4     Age = int.TryParse(ageTextBox.Text, out var age) ? age : 0
5 };
```


- ii. Instantieer een `ValidationContext` en plek waar de foutmeldingen in kunnen komen:

```
1 var context = new ValidationContext(user);
2 var results = new List<ValidationResult>();
```

- iii. Controleer vervolgens de invoer met de context, en geef de lijst mee waar de foutmeldingen in gezet kunnen worden. Raadpleeg weer het voorbeeld eerder in dit hoofdstuk:

```
1 if (!Validator.TryValidateObject(user, context, results, true))
2 {
3     var errors = new List<string>();
4     foreach (var validationResult in results)
5     {
6         errors.Add(validationResult.ErrorMessage);
7     }
8     validationResult.TextBlock.Text = string.Join(Environment.NewLine, errors);
9 }
10 else
11 {
12     validationResult.TextBlock.Text = "Validation succeeded!";
13 }
14 }
```

15. Test de applicatie opnieuw.

5.4. Validatie met [RegularExpression]

De `[RegularExpression]`-attribuut in C# (meestal gebruikt in combinatie met data-annotaties) valideert of een string overeenkomt met een bepaald **patroon**. Dit patroon wordt geschreven in een **Regular Expression** (regex).

Gebruik:

```
1 [RegularExpression(@"^[0-9]{4}[A-Z]{2}$", ErrorMessage = "Postcode moet 4 cijfers gevolgd door 2 hoofdletters zijn.")]
2 public string Postcode { get; set; }
```

In dit voorbeeld valideert het attribuut of de postcode voldoet aan het formaat 1234AB.

Wat is Regular Expression?

Een regex is een **tekstpatroon** waarmee je kunt controleren of een string aan bepaalde eisen voldoet. Regex is niet alleen onderdeel van C#, maar bestaat ook in veel andere talen zoals: JavaScript, Python, Java, PHP, Ruby, Perl en zelfs in text editors zoals VS Code en Notepad++.

Voorbeelden met uitleg per teken

- Nederlandse Postcode: `^[0-9]{4}[A-Z]{2}$`
 - `^` — begin van de string
 - `[0-9]{4}` — precies 4 cijfers:
 - `[` — start set
 - `0-9` — De cijfers 0 t/m 9
 - `]` — eind set
 - `{` — start herhalingsinformatie, herhaalt wat hiervoor stond (de set) zo veel keer
 - `4` — precies 4 keer
 - `}` — eind herhalingsinformatie
 - `[A-Z]{2}` — precies 2 hoofdletters
 - `A-Z` — De letters A t/m Z (in hoofdletters)
 - `$` — einde van de string

Voorbeeld geldig: 1234AB

Ongeldig: 123AB, 1234ab, 12 34AB

Nu schrijven sommige mensen hun postcode wel eens met een spatie tussen de cijfers en letters. We zouden de regex als volgt kunnen aanpassen om ook een spatie toe te staan: `^[0-9]{4}\s?[A-Z]{2}$`

In deze aangepaste regex betekent “`\s?`” dat op die plek een spatie of ander witruimte teken (`\s`) optioneel (?) is.

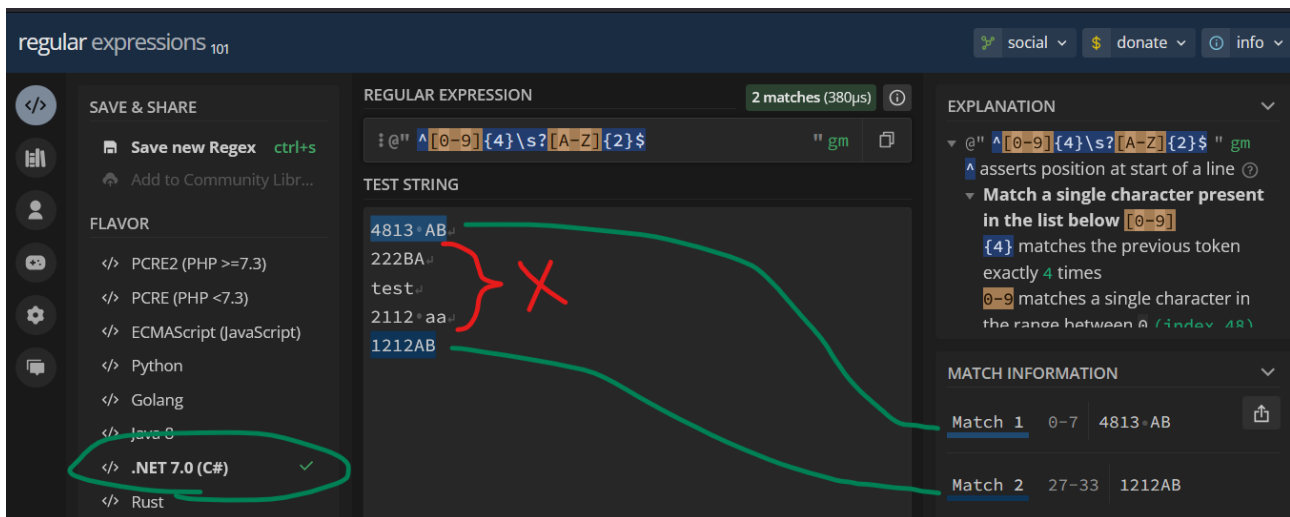
Nu ook geldig: 1234 AB

- **Telefoonnummer (NL mobiel):** `^06\d{8}$`
 - `^` — begin van de string
 - `06` — moet beginnen met 06
 - `\d{8}` — precies 8 cijfers, `\d` = cijfer
 - `$` — einde van de string

Voorbeeld geldig: 0612345678

Ongeldig: 0712345678, 06-12345678

Je kunt Regular Expression dus gebruiken om complexere validatie regels te schrijven. Via deze website kun je jouw regex testen en uitleg vinden over de overige tekens: <https://regex101.com/>



5.5. Gegevens updaten

Het bewerken van gegevens gaat volgens de volgende stappen:

1. Haal gegevens op uit een Database Context
Je krijgt instanties van modellen.
2. Wijzig de gegevens door de instanties aan te passen
3. Roep 'SaveChanges' aan op dezelfde Database Context als waar de gegevens zijn uitgehaald.
Dankzij Change Tracking weet Entity Framework hoe de Update query opgebouwd moet worden.

Je kunt voorbeeld code vinden in de documentatie van Microsoft: <https://learn.microsoft.com/en-us/ef/core/performance/efficient-updating?tabs=ef7>

Daar kun je ook vinden hoe je efficiëntere updates kunt uitvoeren, bijvoorbeeld door Batching. Met Batching EF het aantal 'roundtrips' te minimaliseren door automatisch alle updates samen te voegen in één roundtrip.

Roundtrip,

Verwijst naar het proces waarbij gegevens van een client naar een server worden verzonden en vervolgens het antwoord van de server terug naar de client wordt ontvangen. Het omvat het heen en weer gaan van gegevens tussen de client en de server.

Neem het volgende in overweging:

```
1 var blog = context.Blogs.Single(b => b.Url == "http://someblog.microsoft.com");
2 blog.Url = "http://someotherblog.microsoft.com";
3 context.Add(new Blog { Url = "http://newblog1.microsoft.com" });
4 context.Add(new Blog { Url = "http://newblog2.microsoft.com" });
5 context.SaveChanges();
```

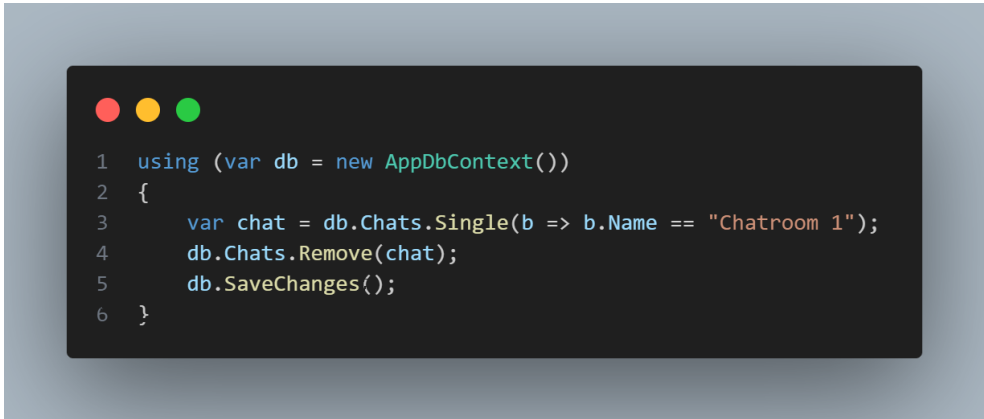
Bovenstaande code laadt een blog uit de database, wijzigt de URL en voegt vervolgens twee nieuwe blogs toe. Om dit toe te passen, worden er **twee** SQL INSERT-opdrachten en **één** UPDATE-opdracht naar de database gestuurd. In plaats van ze één voor één te verzenden wanneer de Blog-instanties worden toegevoegd, **houdt EF deze wijzigingen intern bij en voert ze uit in één roundtrip wanneer SaveChanges wordt aangeroepen.**

Het aantal opdrachten dat EF bundelt in één roundtrip hangt af van de gebruikte databaseprovider. Bijvoorbeeld, uit prestatieanalyse is gebleken dat batching over het algemeen minder efficiënt is voor (Microsoft) SQL Server wanneer er minder dan 4 opdrachten zijn. Op vergelijkbare wijze verminderen de voordelen van batching na ongeveer 40 opdrachten voor SQL Server. Daarom zal EF standaard slechts maximaal 42 opdrachten in één batch uitvoeren en aanvullende opdrachten in afzonderlijke roundtrips uitvoeren.

Bron: <https://learn.microsoft.com/en-us/ef/core/performance/efficient-updating?tabs=ef7>

5.6. Gegevens verwijderen

Het verwijderen van gegevens ziet er bijvoorbeeld zo uit:



```
1 using (var db = new AppDbContext())
2 {
3     var chat = db.Chats.Single(b => b.Name == "Chatroom 1");
4     db.Chats.Remove(chat);
5     db.SaveChanges();
6 }
```

Zorg (net als bij Update) dat je SaveChanges aanroept op dezelfde database context als waar de verwijderde gegevens worden opgehaald. Alleen dan kan de Change Tracker van Entity Framework bijhouden welke gegevens zijn verwijderd.

Wanneer je gegevens verwijdert die een relatie hebben met een andere entiteit, moet je ook die gegevens verwijderen, of de relatie verbreken. Anders krijg je (op andere plekken) foutmeldingen, omdat er nog afhankelijke gegevens bestaan, maar die dan niet meer bij de verwijderde data kunnen.

Om op een snelle manier bijbehorende gegevens te verwijderen of ontkoppelen maak je gebruik van 'Cascade Delete': <https://learn.microsoft.com/en-us/ef/core/saving/cascade-delete>

5.7. Opdracht: High Scores bijhouden - CRUD

In deze opdracht gaan we eerst herhalen wat we al kennen (Read). Daarna oefenen we de nieuwe kennis over Create, Update en Delete. Dit doen we aan de hand van een 'Leaderboard' app, waarin we scores uit games kunnen bijhouden.

Rondom veel games zijn 'Speedrunning'-gemeenschappen te vinden. Dat zijn mensen die zo snel mogelijk de game proberen uit te spelen. In deze oefening gaan we een kleine applicatie maken waarin we speedrunning scores kunnen bijhouden.

Stappenplan:

1. Start een nieuw WinUI 3 project
2. Maak eerst een overzicht van scores in een ListView. Pas hierbij toe wat je nu al vaker hebt geoefend:
 - a. Een 'HighScore' model met:
 1. Id
 2. Username: de naam van de speler die de score heeft behaald
 3. Score
 4. DurationInSeconds: hoe lang het de speler duurde om de score te halen
 5. UploadedAt: wanneer de score is ingevoerd
 - b. Maak de bijbehorende ApplicationDbContext
 - c. Voeg een Seeder toe met wat test-data scores
 - d. Toon de high scores in een ListView
3. Test de applicatie tussendoor. Zorg dat je zeker weet dat de Read werkt.
4. Maak nu een aanmaakformulier (mag in hetzelfde scherm/page) waar de gebruiker een nieuwe HighScore kan invullen. *Zorg dat er velden zijn voor alles wat de speler moet invullen (alles behalve Id)*
5. Voeg bij het aanmaakformulier ook een knop toe, en zorg dat bij het klikken op die knop de gegevens in de database worden ingevoerd. Let daarbij op de theorie hierboven:
 - a. Maak verbinding met de database:
`using (var db = new ApplicationDbContext()) // etc...`
 - b. Maak een nieuwe HighScore instantie:
`var score = new HighScore { Username = usernameTextBox.Text, // etc...`
 - c. Voeg die instantie toe aan de DbSet van HighScores en roep 'SaveChanges' aan (zie de uitleg bij 5.1)
6. Test en controleer dat het invoeren van een nieuwe score werkt.
7. Herhaal weer iets wat je al geleerd hebt, namelijk sorteren (zie Hoofdstuk 4): sorteer alle high scores in de ListView op de 'Score'.
8. Nu weer iets nieuws: het bewerken van een score:
 - a. Voeg toe dat je kunt klikken op een score in de ListView
 - b. Bij het klikken op de score worden de gegevens ervan in de TextBoxes en andere invoer elementen geladen.

- c. Sla de 'Id' op (bijvoorbeeld in een private eigenschap in de Page/Window), zodat je zometeen kunt controleren of je wilt toevoegen/opslaan.
- d. Pas de knop event handler aan, zodat als er een Id is, de ingevoerde gegevens gebruikt worden om de score aan te passen.
- e. Bekijk nogmaals de theorie over aanpassen (paragraaf 5.2). Daarin staat samengevat:
 - 1. Open opnieuw verbinding opent met de database:
`using var db = new AppDbContext // ... etc...`
 - 2. Haal het object op wat je wilt aanpassen, bijv.:
`var score = db.HighScores.FirstOrDefault(hs => hs.Id == // etc...)`
 - 3. Pas nu de score aan
 - 4. Roep SaveChanges aan op de databaseverbinding
- 9. Test en controleer dat het aanpassen van scores werkt.
- 10. Bedenk zelf een nette manier om een 'verwijder'-functie toe te voegen, voeg die toe aan de hand van de theorie hierboven (paragraaf 5.3)

Inleveropdracht 5.8: GameBib CRUD – De Game Bibliotheek (tijdsduur: 2 á 3 uur)

Welkom bij de uitdaging "GameBib"! Je bent een ontwikkelaar bij een snelgroeiend gamebedrijf en je nieuwste taak is om een beheersysteem voor de gamesbibliotheek van het bedrijf te ontwikkelen.

Functionaliteiten:

1. (4 punten) **Creëren:** Maak een systeem waarin nieuwe games kunnen worden toegevoegd. Elk spel moet een naam, genre, uitgiftedatum en rating hebben.
 - a) Naast het aanmaken van games, moeten ook Genres aangemaakt kunnen worden.
 - b) Games kunnen meerdere genres hebben. Genres kunnen bij meerdere Games horen.
2. (2 punten) **Lezen:** Het systeem moet in staat zijn om de details van elke game weer te geven. Zorg ervoor dat de gebruikers kunnen filteren op basis van genre, rating, of uitgiftedatum.
 - a) Er dient een Genre scherm te zijn, waar alle games met dat worden Genre getoond.
3. (4 punten) **Bijwerken:** Als games worden bijgewerkt met nieuwe versies of patches, moet het systeem in staat zijn om deze wijzigingen weer te geven. Ook Genres moeten kunnen worden bewerkt.
4. (2 punten) **Verwijderen:** Mocht een game niet langer beschikbaar zijn, moet het systeem in staat zijn om games te verwijderen uit de bibliotheek. Ook Genres moeten kunnen worden verwijderd.

Eisen:

- Uiteraard gebruik je C#, Entity Framework en MySQL
- Maak het systeem zo gebruiksvriendelijk mogelijk, aangezien dit zal worden gebruikt door zowel de gameontwikkelaars als de klanten.
- Hoewel het project binnen een korte tijdsspanne moet worden voltooid, moet je ervoor zorgen dat je de basisprincipes van goede softwareontwikkeling niet vergeet: schrijf leesbare code, voeg waar nodig comments toe om je gedachtegang uit te leggen.

Zie volgende pagina voor eventuele bonusopdrachten.

Het project hoeft niet een volledig ontwikkeld product te zijn. Zorg dat je demonstreert dat je alle CRUD-functionaliteiten kunt implementeren.

Inleveren:

- Lever de gehele broncode in als .zip op Itslearning.
- Maak screenshots van alle schermen in de applicatie en lever naast de .zip in op Itslearning.
- Maak screenshots van de C# code van alle schermen en lever naast de .zip in op Itslearning

Veel succes met de opdracht. Game on!

 **Let op:** Het inleveren van deze opdracht geldt als feedback-moment. Je ontvangt studiepunten wanneer je deze succesvol afrond. Houdt na het inleveren Itslearning in de gaten en verwerk eventuele feedback die je van de docent krijgt.

*Er zijn in totaal **12 punten** te behalen bij deze opdracht. Met een cesuur van 70% moet je ten minste 8,5 punten scoren voor een voldoende (5,6).*

Bonus: GameBib CRUD – De Game Bibliotheek

Snel klaar? Zoek je extra uitdaging? Bij deze enkele aanvullende (niet verplichte) opdrachten voor de vorige inleveropdracht:

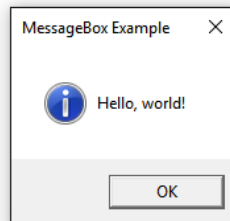
- **Zoekfunctionaliteit:** Implementeer een zoekfunctionaliteit waarmee gebruikers specifieke games kunnen vinden op basis van trefwoorden, ontwikkelaar, platform of andere relevante kenmerken. Dit verbetert de gebruikerservaring en maakt het gemakkelijker om games te ontdekken.
- **Beoordelingen en recensies:** Voeg de mogelijkheid toe voor gebruikers om beoordelingen en recensies achter te laten voor games. Hierdoor kunnen andere gebruikers de kwaliteit en populariteit van een spel beoordelen voordat ze het toevoegen aan hun eigen bibliotheek.
- **Verlanglijst:** Implementeer een functie waarmee gebruikers games kunnen toevoegen aan hun verlanglijst. Dit stelt gebruikers in staat om games te markeren die ze interessant vinden, maar nog niet hebben gekocht of toegevoegd aan hun bibliotheek.
- **Integratie van externe API's:** Integreer externe API's zoals metacritic.com of giantbomb.com om extra informatie over games op te halen, zoals gemiddelde beoordelingen, platforms waarop het spel beschikbaar is, en meer. Dit verrijkt de gamegegevens in de bibliotheek.
- **Multiplatform-ondersteuning:** Breid de functionaliteit uit zodat gebruikers kunnen aangeven op welke platforms ze games bezitten of willen bezitten, zoals pc, console (PlayStation, Xbox, Nintendo), mobiel en meer. Dit helpt gebruikers bij het organiseren van hun bibliotheek op basis van beschikbare platforms.
- **Gametijdregistratie:** Implementeer een systeem dat de speeltijd van gebruikers bijhoudt voor elk spel in hun bibliotheek. Hierdoor kunnen gebruikers hun speeltijd per spel volgen en zien hoeveel tijd ze aan elke game hebben besteed.
- **Gametrailers en screenshots:** Voeg de mogelijkheid toe om gametrailers en screenshots aan elke game toe te voegen. Gebruikers kunnen dan een beter beeld krijgen van het spel voordat ze het toevoegen aan hun bibliotheek.
- **Gebruikersbeheer:** Voeg een gebruikerssysteem toe aan de GameBib, waarbij gebruikers accounts kunnen aanmaken, inloggen en hun eigen gameverzameling kunnen beheren. Hierdoor kunnen gebruikers hun favoriete games markeren, beoordelingen achterlaten en opmerkingen plaatsen.

HANDIG OM TE WETEN

De rest van de paragrafen in dit hoofdstuk bevatten handige tips & trucs die je helpen een volledige CRUD-applicatie te maken. Je vindt informatie en opdrachten over van alles, van het tonen van meldingen tot het genereren van een PDF.

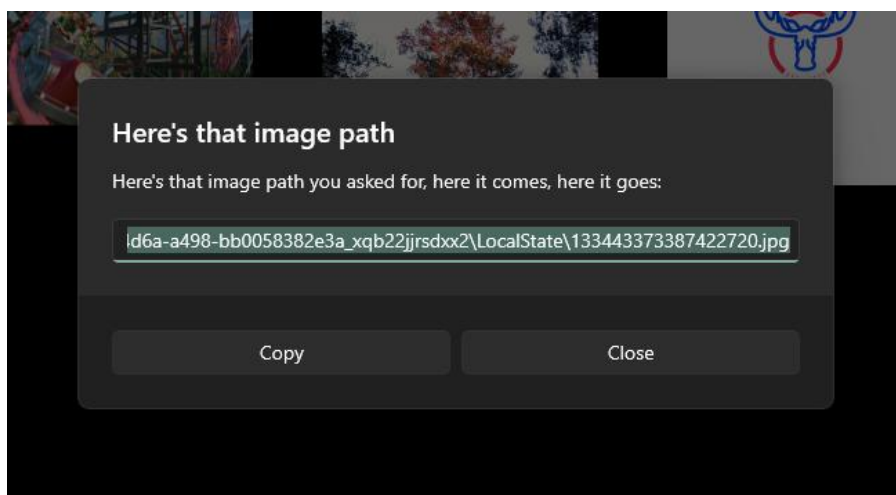
5.9. Een melding tonen met ContentDialog

In eerdere versies van het .NET Framework was het mogelijk om met 'MessageBox.Show' een bericht als dit te tonen:



Zo'n MessageBox ziet er tegenwoordig oud uit. Daarnaast kunnen we er enkel de tekst, knoppen en icoontjes van aanpassen. Tegenwoordig willen we meer flexibiliteit.

Sinds UWP en later WinUI3 kunnen we zo'n soortgelijke melding tonen met behulp van 'ContentDialog'. Daarin kunnen we niet alleen tekst tonen met verschillende knoppen, maar dankzij XAML ook andere elementen plaatsen zoals afbeeldingen of een TextBox. Dat kan er dan bijvoorbeeld zo uitzien:



Hier kun je de documentatie vinden van de ContentDialog: <https://learn.microsoft.com/en-us/windows/windows-app-sdk/api/winrt/microsoft.ui.xaml.controls.contentdialog?view=windows-app-sdk-1.4>

De bovenstaande ContentDialog is in de XAML opgebouwd met deze code:

```
<ContentDialog x:Name="imagePathDialog"
    PrimaryButtonText="Copy"
    CloseButtonText="Close"
    PrimaryButtonClick="imagePathDialog_PrimaryButtonClick">
    <ContentDialog.TitleTemplate>
        <DataTemplate>
            <StackPanel Orientation="Horizontal">
                <TextBlock Text="Here's that image path" />
            </StackPanel>
        </DataTemplate>
    </ContentDialog.TitleTemplate>
    <StackPanel>
        <TextBlock TextWrapping="WrapWholeWords">
            <Run>Here's that image path you asked for, here it comes, here it goes:</Run><LineBreak />
        </TextBlock>
        <TextBox x:Name="imagePathTextBox"
            IsReadOnly="True" />
    </StackPanel>
</ContentDialog>
```

Om de ContentDialog, die hierboven de naam 'imagePathDialog' krijgt, te openen gebruiken we:

```
await imagePathDialog.ShowAsync();
```

Als je meerdere verschillende ContentDialog's wilt kunnen tonen, kan het onhandig zijn om die allemaal in de XAML aan te moeten maken. Je kunt dan vanuit de C# een dialog maken. Hier zie je hoe een methode is geschreven, die een ContentDialog met een variabel bericht kan tonen:

```
private async Task ShowErrorPopup(string message)
{
    var errorDialog = new ContentDialog
    {
        Title = "Fout!",
        Content = message,
        CloseButtonText = "OK",
        DefaultButton = ContentDialogButton.Close,
        XamlRoot = this.XamlRoot
    };
    await errorDialog.ShowAsync();
}
```

Let op dat de 'XamlRoot'-eigenschap van 'this' niet beschikbaar in een class afgeleid van Window. Bij Page-klassen bestaat het wel. Als je de code toch in een Window wilt gebruiken, moet je een naam geven aan de Grid/StackPanel die in je Window staat. Dan kun je 'naamVanDieStackPanelOfGrid.XamlRoot' gebruiken in plaats van 'this.XamlRoot'.

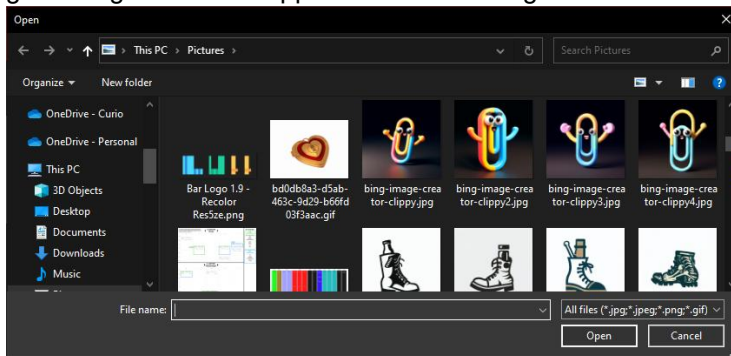
5.10. Bestanden (en afbeeldingen) verwerken

Het uploaden van een afbeelding gebeurt op het WEB en bij NATIVE applicaties vaak op dezelfde manier. **We uploaden geen afbeeldingen direct in de database, maar slaan de afbeelding op ergens op het systeem. Vervolgens slaan we de plek waar het staat op in de database.** We doen dit omdat databases geschikt zijn voor data waardoor gezocht moet worden. Een afbeelding bestaat uit 'binaire' data waarin we niet hoeven te zoeken.

Om te werken met bestanden gaan we gebruik maken van de volgende klassen van Microsoft:

2. Windows.Storage.Pickers.FileOpenPicker

De FileOpenPicker toont een scherm waarin de gebruiker een bestand kan kiezen. Met deze keuze geeft de gebruiker de applicatie toestemming om het bestand te kopiëren. Zo ziet dit scherm eruit:



3. Windows.Storage.StorageFile

Wanneer de gebruiker een bestand heeft gekozen komt die keuze terug als StorageFile instantie. Via deze instantie krijgt de applicatie toestemming om het bestand te kopiëren, verwijderen, verplaatsen, hernoemen, etc.

Waar applicaties vroeger toestemming kregen om alle bestanden in het systeem te verwijderen, moet de gebruiker sinds UWP **expliciet toestemming geven door het bestand te selecteren**. Dankzij deze beveiliging kan een applicatie niet zomaar alle bestanden op het systeem verwijderen of versleutelen. Hierdoor kan een gebruiker een UWP of WinUI applicatie makkelijker vertrouwen dan een applicatie die niet die beperkingen heeft.

Dit is hoe we de gebruiker een bestandskeuze laten maken:

1. We voegen een Button toe in de XAML met een Click event
2. In het Click event openen we de FileOpenPicker en stellen we in wat voor soort bestand we willen laten selecteren:

```
private async void FileButton_Click(object sender, RoutedEventArgs e)
{
    var fileOpenPicker = new FileOpenPicker()
    {
        ViewMode = PickerViewMode.Thumbnail,
        FileTypeFilter = { ".jpg", ".jpeg", ".png", ".gif" },
    };

    var windowHandle = WindowNative.GetWindowHandle(this);
    InitializeWithWindow.Initialize(fileOpenPicker, windowHandle);

    var file = await fileOpenPicker.PickSingleFileAsync();

    if (file == null)
    {
        return;
    }

    //...
```

3. We moeten de FileOpenPicker een 'WindowHandle' geven, zodat deze weet bij welk scherm het hoort. Als dat scherm gesloten wordt, sluit de FileOpenPicker ook.
4. Met deze code openen we daadwerkelijk het bestandskeuzeschermb:

```
var file = await fileOpenPicker.PickSingleFileAsync();
```

Omdat het lang kan duren voordat de gebruiker iets kiest, wachten we op het resultaat met 'await'. De code pauzeert hier als het ware, totdat de gebruiker een bestand kiest of het keuzeschermb sluit.

Je moet jouw event handler 'async' maken om te zorgen dat je await kunt gebruiken:

```
private async void FileButton_Click(object sender, RoutedEventArgs e)
```

5. Tot slot controleren we of de gebruiker geen bestand heeft gekozen. In dat geval stoppen we de event handler en zijn we klaar. Wanneer er wel een bestand is gekozen gaan we verder.

Nadat de gebruiker een bestand heeft gekozen bevat de variabele 'file' hierboven een StorageFile. Daarmee hebben we toestemming om te werken. **Zie hier bijvoorbeeld hoe het gekozen bestand wordt gekopieerd naar de applicatie map:**

```

var file = await fileOpenPicker.PickSingleFileAsync();

if (file == null)
{
    return (local variable) StorageFile file;
}

// Get the app's local folder to use as the destination
var localFolder = ApplicationData.Current.LocalFolder;

// Get the current file extension
var fileExtension = file.FileType;

// Specify a new name for the copied file, using the current time
var currentTime = DateTime.Now;
var renamedFileName = $"{currentTime.ToFileTime()} {fileExtension}";

await file.CopyAsync(localFolder, renamedFileName);

```

Als je de muis over 'file' houdt zie je dat het een instantie van de klasse 'StorageFile' is.

Alle instanties van klassen kunnen ook 'null' zijn. In dit geval als de gebruiker op 'Annuleren' klikt in het keuzescherf.

Weer een beveiliging van Microsoft: applicaties kunnen alleen bestanden opslaan in hun 'eigen map'. Op deze manier krijgen we toegang tot die map.

Met de bovenstaande stappen heeft de gebruiker een bestand gekozen en hebben we dat naar de applicatiemap gekopieerd. **Wanneer we dit bestand willen koppelen aan een Model met EntityFramework** **hoef je enkel de bestandsnaam op te slaan in de database.** Dat is gewoon een string die je opslaat. Later kun je die string gebruiken om in de applicatiemap (ApplicationData.Current.LocalFolder) de afbeelding weer te vinden:

```

var file = localFolder.GetFilesAsync(photoModel.Path);

```

5.11. Afbeeldingen tonen

Nu we paden van afbeeldingen hebben, willen we die gaan gebruiken om een afbeelding te tonen op het scherm. Hieronder bespreken we twee manieren waarop dat kan:

4. De afbeelding vanuit C# instellen op een Image element (via de **Source** eigenschap)
5. Meerdere afbeeldingen laten zien in een GridView, met behulp van DataBinding (**Source="{x:Bind Path}"**)

Allereerst het instellen van een enkele afbeelding vanuit C#:

Het kan best wel even zoeken zijn hoe je nu een StorageFile omzet naar een afbeelding voor in de Image.Source eigenschap. Er zijn in de WindowsAppSDK-Samples repository een hoop voorbeelden te vinden, zoals deze: <https://github.com/microsoft/WindowsAppSDK-Samples/blob/85ff2374b6184e0ea323a943e5646ff3f2432632/Samples/PhotoEditor/cs-winui/ImageFileInfo.cs#L44>

Om een StorageFile in de variabele 'file' dan te laten zien in deze XAML:

```
<Image x:Name="imagePathImage"
      Width="200"
      Height="200" />
```

Doen we dit:

```
var thumbnail = await file.GetThumbnailAsync(Windows.Storage.FileProperties.ThumbnailMode.SingleItem);
var bitmapImage = new BitmapImage();
bitmapImage.SetSource(thumbnail);
thumbnail.Dispose();

imagePathImage.Source = bitmapImage;
```

Meerdere afbeeldingen instellen in een GridView:

Een GridView is een layout-type zoals de ListView, maar het toont een raster (grid) i.p.v. een lijst (list). Voor de rest werkt het precies hetzelfde.

Voor het tonen van een verzameling StorageFiles in een GridView kunnen we net als bij de ListView databinding gebruiken om eigenschappen van elementen in te stellen. Dat ziet er bijvoorbeeld zo uit:

```
<GridView Grid.Row="1"
          x:Name="imagesGridView">
  <GridView.ItemTemplate>
    <DataTemplate x:DataType="storage:StorageFile">
      <Image Source="{x:Bind Path}"
            Width="200"
            Height="200" />
    </DataTemplate>
  </GridView.ItemTemplate>
</GridView>
```

Het DataType laat het al zien: we gaan deze GridView vullen met StorageFile objecten. Om uit te leggen in welke namespace StorageFile staat, hebben we die namespace opgezocht in de Microsoft documentatie en bovenin toegevoegd als xmlns:storage:

```
<?xml version="1.0" encoding="utf-8"?>
<Window x:Class="ImageSaver.PhotoWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:local="using:ImageSaver"
        xmlns:storage="using:Windows.Storage"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        mc:Ignorable="d">
```

Tot slot moeten we (net als bij de ListView) instellen wat de objecten zijn die in de GridView komen. Dat doen we ook via de ItemsSource eigenschap. Zie hier hoe we een methode hebben gemaakt om alle bestanden in de applicatiemap te laden en in te stellen op de GridView.ItemsSource:

```
private async void RefreshAllImages()
{
    var localFolder = ApplicationData.Current.LocalFolder;
    var allStorageFiles = await localFolder.GetFilesAsync();

    imagesGridView.ItemsSource = allStorageFiles;
}
```

Vervolgens roepen we deze methode aan wanneer we de GridView willen verversen. Bijvoorbeeld bij het opstarten van de applicatie (in de constructor):

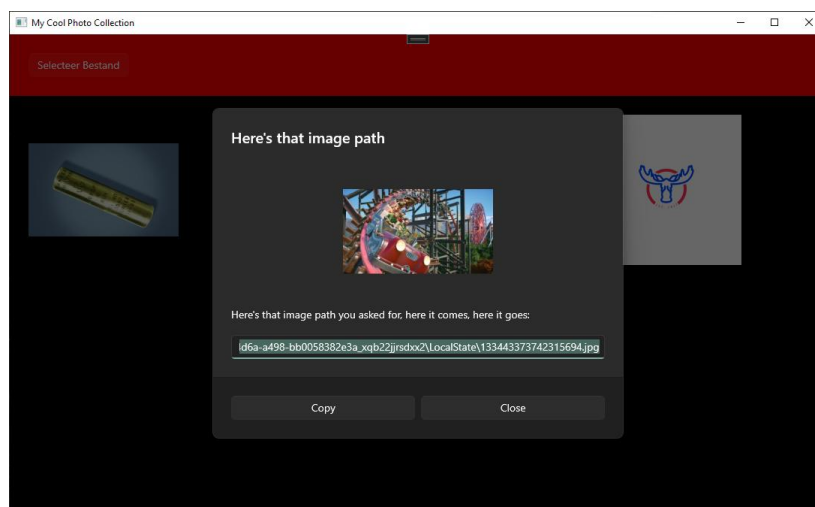
```
public sealed partial class PhotoWindow : Window
{
    2 references
    public PhotoWindow()
    {
        this.InitializeComponent();
        this.RefreshAllImages();

        Title = "My Cool Photo Collection";
    }
}
```

Of nadat de gebruiker een nieuw bestand heeft geüpload.

5.12. Opdracht: Demonstratie analyseren

De voorgaande stappen zijn samengebracht in de uitwerking 'H5 Opdracht 5.9 - ImageSaver.zip' in de Uitwerkingen map op Itslearning.



Naast de bovenstaande uitleg staan er ook wat bonusvoorbeelden in, zoals:

6. Hoe je tekst in het klembord van de gebruiker kunt zetten (zodat ze het ergens kunnen plakken)
7. Hoe je een Window een mooie titel geeft
8. Hoe je Items in de GridView wat margin geeft
9. Hoe je een ContentDialog toont met daarin een afbeelding en tekstveld

Onderzoek voor deze opdracht de applicatie. Download deze en bepaal voor jezelf welke vragen je hebt, en hoe je die kunt beantwoorden.

Kies hierna ten minste 2 stukken code die nieuw zijn (bijv.: het selecteren en opslaan van afbeeldingen en ze in een GridView tonen). **Implementeer die in een van je eerdere projecten** (bijv.: DevCitySim of ChatMeister).

Je hoeft deze opdracht niet in te leveren.

5.13. Styling van elementen in WinUI3

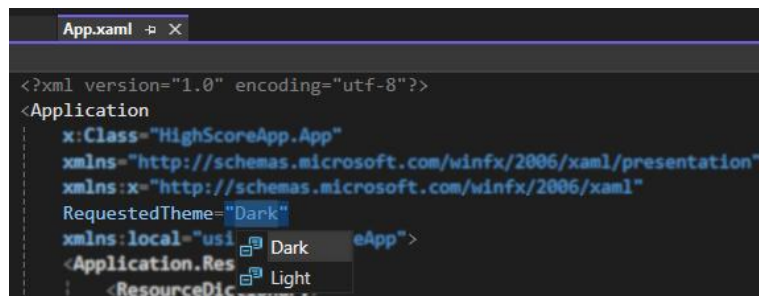
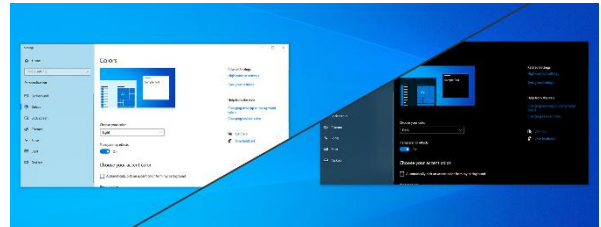
In de volgende paragrafen bekijken we enkele manieren om styling toe te passen op elementen in WinUI3. Je kunt kleuren aanpassen van teksten, randen en achtergronden.

In de documentatie van Microsoft (<https://learn.microsoft.com/en-us/windows/apps/design/style/xaml-styles>) zijn meerdere manieren te vinden waarop stijlen aangepast kunnen worden. Wij kiezen hieronder voor de (volgens ons) meest toegankelijke manieren.

5.14. Automatische of handmatige Dark/Light-mode

Wanneer je een applicatie eigen kleuren gaat geven, is het handig om één thema te kiezen. Standaard neemt een WinUI applicatie de instelling over van Windows. Door dat uit te schakelen hoef je maar één versie van de app te testen, in plaats van zowel light- als dark-mode.

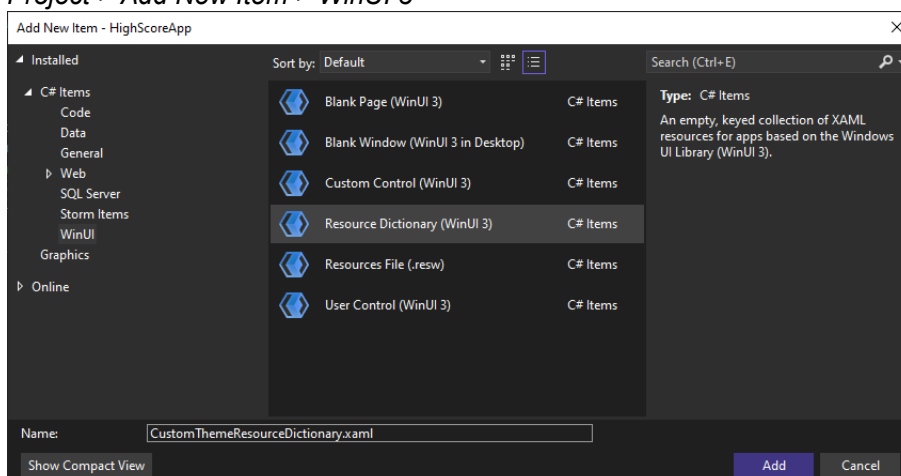
Je kunt het thema kiezen in App.xaml. Voeg dan in de Application open-tag het attribuut 'RequestedTheme' toe en kies **Dark** of **Light**:



5.15. Standaard stijlen aanpassen, zoals de kleuren van elementen

De elementen in WinUI3 gebruiken intern allerlei 'Style Keys'. Zo wordt intern de achtergrond van een knop ingesteld op de 'SolidColorBrush' stijl 'ButtonBackground'. Dat is een 'brush' (kwast) die een kleur heeft afhankelijk van het gekozen thema. We kunnen die stijlen op de volgende manier overschrijven:

1. Voeg een **Resource Dictionary (WinUI 3)** item toe aan je project via *Project > Add New Item > WinUI 3*



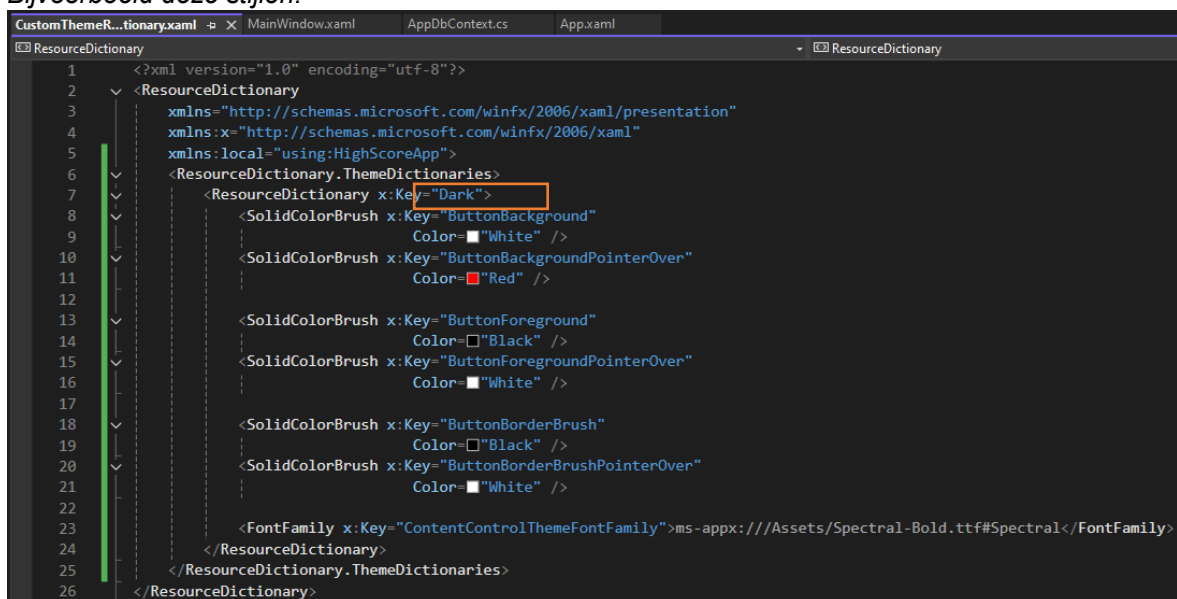
2. Geef het een naam als CustomThemeResourceDictionary.xaml
3. In dit bestand gaan we de standaard stijlen aanpassen. We moeten wel eerst zorgen dat dit bestand door onze applicatie wordt ingeladen: Open daarvoor de App.xaml van je project.

4. Voeg in het 'ResourceDictionary.MergedDictionaries' element een ResourceDictionary element toe op deze manier:

```
<?xml version="1.0" encoding="utf-8"?>
<Application
  x:Class="HighScoreApp.App"
  xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
  xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
  RequestedTheme="Dark"
  xmlns:local="using:HighScoreApp">
  <Application.Resources>
    <ResourceDictionary>
      <ResourceDictionary.MergedDictionaries>
        <XamlControlsResources xmlns="using:Microsoft.UI.Xaml.Controls" />
        <!-- Other merged dictionaries here -->
        <ResourceDictionary Source="CustomThemeResourceDictionary.xaml" />
      </ResourceDictionary.MergedDictionaries>

      <!-- Other app resources here -->
    </ResourceDictionary>
  </Application.Resources>
</Application>
```

5. Nu zijn we klaar om in de CustomThemeResourceDictionary.xaml onze stijlen toe te voegen.
Bijvoorbeeld deze stijlen:



```
1 <?xml version="1.0" encoding="utf-8"?>
2 <ResourceDictionary
3   xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
4   xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
5   xmlns:local="using:HighScoreApp">
6   <ResourceDictionary.ThemeDictionaries>
7     <ResourceDictionary x:Key="Dark">
8       <SolidColorBrush x:Key="ButtonBackground"
9         Color="White" />
10      <SolidColorBrush x:Key="ButtonBackgroundPointerOver"
11        Color="Red" />
12
13      <SolidColorBrush x:Key="ButtonForeground"
14        Color="Black" />
15      <SolidColorBrush x:Key="ButtonForegroundPointerOver"
16        Color="White" />
17
18      <SolidColorBrush x:Key="ButtonBorderBrush"
19        Color="Black" />
20      <SolidColorBrush x:Key="ButtonBorderBrushPointerOver"
21        Color="White" />
22
23      <FontFamily x:Key="ContentControlThemeFontFamily">ms-appx:///Assets/Spectral-Bold.ttf#Spectral</FontFamily>
24    </ResourceDictionary>
25  </ResourceDictionary.ThemeDictionaries>
26 </ResourceDictionary>
```

6. Met de bovenstaande stijlen ziet de knop er als volgt uit:



7. De **x:Key** waarde van de ResourceDictionary is afhankelijk van het thema. Kies hier bijvoorbeeld hetzelfde als wat je bij RequestedTheme hebt ingevuld (Light of Dark).
8. Binnen de ResourceDictionary zien we verschillende stijlen waar kleuren voor gekozen worden:

- a. ButtonBackground: De achtergrond van een knop
- b. ButtonForeground: Het lettertype van een knop
- c. ButtonBorderBrush: De rand van een knop

9. Achter de stijlnamen komt eventueel een 'state', zoals:

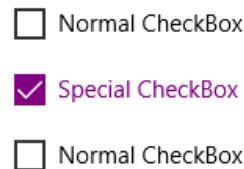
- a. Bij elementen waar je met de muis (of vinger bij touch screen) interactie mee kan hebben:
 - PointerOver: muis of vinger over het element
 - PointerPressed: muis of vinger drukt het element in
 - PointerDisabled: het element is uitschakeld voor interacties

Bijvoorbeeld: ButtonBackgroundPointerOver

- b. Bij CheckBox elementen:
 - Unchecked: wanneer het element niet is aangevinkt
 - Checked: wanneer het element wel is aangevinkt
 - UncheckedPointerOver: muis of vinger over het element, wat niet is aangevinkt

Bijvoorbeeld:

```
<SolidColorBrush x:Key="CheckBoxForegroundUnchecked"
  Color="Purple"/>
<SolidColorBrush x:Key="CheckBoxForegroundChecked"
  Color="Purple"/>
<SolidColorBrush x:Key="CheckBoxCheckGlyphForegroundChecked"
  Color="White"/>
<SolidColorBrush x:Key="CheckBoxCheckBackgroundStrokeChecked"
  Color="Purple"/>
<SolidColorBrush x:Key="CheckBoxCheckBackgroundFillChecked"
  Color="Purple"/>
```



10. Al die stijlnamen kun je vinden in het bestand 'generic.xaml' in jouw WinUI installatie. Je vindt het in: C:\Users\<jouwusername>\.nuget\packages\microsoft.windowsappsdk\1.4.231008000\lib\net6.0-windows10.0.18362.0\Microsoft.WinUI\Themes

(Afhankelijk van jouw systeem en versies kunnen de onderstreepte delen in het pad verschillen.)

```
401 <StaticResource x:Key="RadioButtonCheckGlyphStrokePointerOver" ResourceKey="TextFi
402 <StaticResource x:Key="RadioButtonCheckGlyphStrokePressed" ResourceKey="TextFi
403 <StaticResource x:Key="RadioButtonCheckGlyphStrokeDisabled" ResourceKey="TextFi
404 <!-- Resources for Windows.UI.Xaml.Controls.CheckBox -->
405 <StaticResource x:Key="CheckBoxForegroundUnchecked" ResourceKey="TextFi
406 <StaticResource x:Key="CheckBoxForegroundUncheckedPointerOver" ResourceKey="TextFi
407 <StaticResource x:Key="CheckBoxForegroundUncheckedPressed" ResourceKey="TextFi
408 <StaticResource x:Key="CheckBoxForegroundUncheckedDisabled" ResourceKey="TextFi
409 <StaticResource x:Key="CheckBoxForegroundChecked" ResourceKey="TextFi
410 <StaticResource x:Key="CheckBoxForegroundCheckedPointerOver" ResourceKey="TextFi
411 <StaticResource x:Key="CheckBoxForegroundCheckedPressed" ResourceKey="TextFi
412 <StaticResource x:Key="CheckBoxForegroundCheckedDisabled" ResourceKey="TextFi
413 <StaticResource x:Key="CheckBoxForegroundIndeterminate" ResourceKey="TextFi
414 <StaticResource x:Key="CheckBoxForegroundIndeterminatePointerOver" ResourceKey="TextFi
415 <StaticResource x:Key="CheckBoxForegroundIndeterminatePressed" ResourceKey="TextFi
416 <StaticResource x:Key="CheckBoxForegroundIndeterminateDisabled" ResourceKey="TextFi
417 <StaticResource x:Key="CheckBoxBackgroundUnchecked" ResourceKey="TextFi
418 <StaticResource x:Key="CheckBoxBackgroundUncheckedPointerOver" ResourceKey="TextFi
419 <StaticResource x:Key="CheckBoxBackgroundUncheckedPressed" ResourceKey="TextFi
420 <StaticResource x:Key="CheckBoxBackgroundUncheckedDisabled" ResourceKey="TextFi
421 <StaticResource x:Key="CheckBoxBackgroundChecked" ResourceKey="TextFi
422 <StaticResource x:Key="CheckBoxBackgroundCheckedPointerOver" ResourceKey="TextFi
423 <StaticResource x:Key="CheckBoxBackgroundCheckedPressed" ResourceKey="TextFi
```

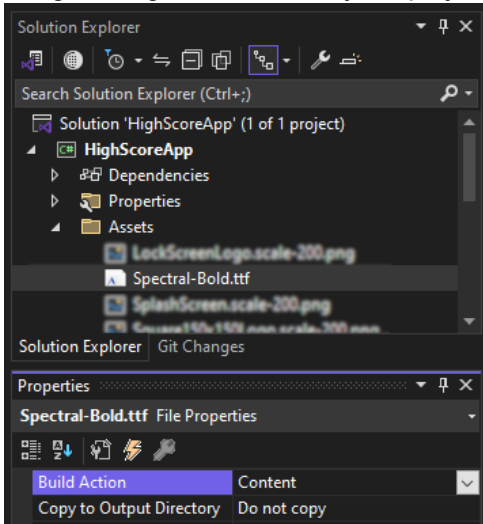
Dit is een intern bestand wat je verder niet moet aanpassen. Gebruik het alleen om de stijlnamen te vinden.

5.16. Andere stijlen dan kleur aanpassen

Voor het instellen van kleur wordt SolidColorBrush gebruikt, maar er bestaan ook andere elementen:

<FontFamily>jouw_font_path_hier</FontFamily> voor het kiezen van Fonts voor een control die tekst bevat.

- Voeg een eigen font toe aan jouw project door het in de Assets map van jouw project te plaatsen:



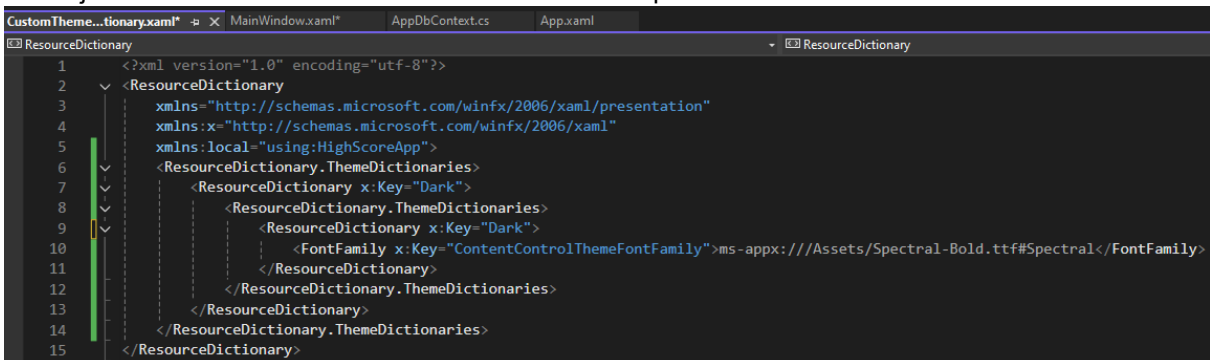
- Voeg deze regel toe aan een Resource Dictionary:

```
<FontFamily x:Key="CustomFont">ms-appx:///Assets/Fonts/BAUHS93.ttf#Bauhaus 93</FontFamily>
```

De Key 'CustomFont' is zelfbedacht, om hem te gebruiken moet je het FontFamily attribuut toevoegen aan een control:

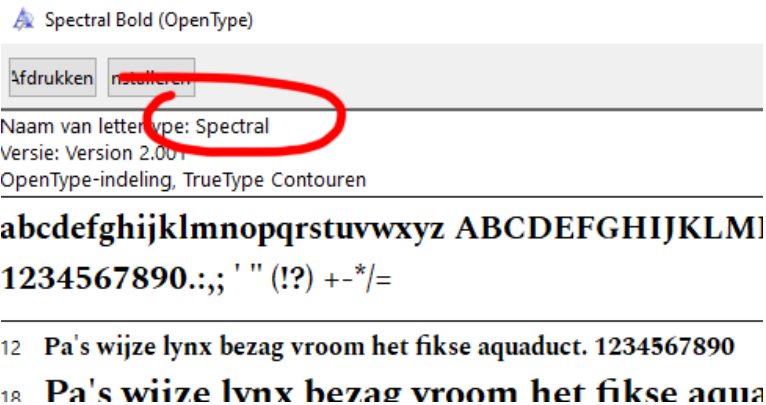
```
<TextBox x:Name="usernameTextBox"
    Header="Username"
    Width="200"
    FontFamily="{StaticResource CustomFont}"
    Margin="5" />
```

- Of als je de font van veel standaard elementen wilt aanpassen:



- Het pad naar het font begint voor fonts in de Assets map altijd: ms-appx:///Assets/

- Schrijf na het pad naar het font een hekje, gevolgd door de naam van het font. Je vind de naam van het font bij .ttf bestanden door het te openen:



`<x:Double>hier_een_getal</x:Double>` voor het kiezen van een getal, zoals de minimale hoogte of breedte van elementen. Bijvoorbeeld:

```
<x:Double x:Key="ContentDialogMinWidth">320</x:Double>
<x:Double x:Key="ContentDialogMaxWidth">548</x:Double>
```

(Standaard waarde uit *Generic.xaml*)

`<Thickness>left_getal,top_getal,right_getal,bottom_getal</Thickness>` voor het kiezen van margins, paddings en border diktes met verschillende hoeveelheden, bijv.:

```
<Thickness x:Key="ComboBoxPopupBorderThemeThickness">2</Thickness>
<Thickness x:Key="ComboBoxItemThemePadding">11,5,11,7</Thickness>
```

(Standaard waarde uit *Generic.xaml*)

In de *Generic.xaml* kun je nog veel meer mogelijkheden vinden én standaard waarden. Gebruik dat bestand om verschillende `x:Key` te vinden en het bijbehorende soort element te ontdekken.

5.17. Een PDF genereren

In veel softwaretoepassingen is het handig om informatie vast te leggen in een document dat makkelijk gedeeld en afgedrukt kan worden. Denk aan facturen, rapporten, certificaten of offertes. Een veelgebruikte standaard hiervoor is het Pdf-formaat.

Voordelen van een PDF ten opzichte van een Word-bestand

- **Consistentie:** Een PDF ziet er op elk apparaat hetzelfde uit. Een Word-document kan er anders uitzien afhankelijk van de versie van Word, de geïnstalleerde lettertypes of instellingen.
- **Niet eenvoudig te wijzigen:** Een PDF is bedoeld om te bekijken en te delen, niet om makkelijk aangepast te worden. Dit voorkomt dat documenten onbedoeld veranderd worden.
- **Geschikt voor afdrucken:** Pdf-bestanden behouden de exacte opmaak en marges, ideaal wanneer je precies wilt printen wat je op je scherm voor je ziet.
- **Breed ondersteund:** Vrijwel elk apparaat kan Pdf-bestanden openen, zonder dat specifieke software nodig is zoals Microsoft Word.

In C# kun je gebruik maken van de open-source bibliotheek **PDFsharp**. PDFsharp maakt het mogelijk om Pdf-bestanden te genereren, bewerken en lezen. Het is een populaire keuze vanwege de eenvoud en de goede integratie met .NET.

Voorbeelden van situaties waarin je PDF zou genereren

- Een webshop genereert een PDF-factuur voor elke bestelling.
- Een schoolapplicatie maakt een PDF-certificaat voor behaalde modules.
- Een boekhoudprogramma exporteert een financieel rapport als PDF.

Voorbeeldcode: Een eenvoudige PDF aanmaken

```
using PdfSharp.Pdf;
using PdfSharp.Drawing;

class Program
{
    static void Main()
    {
        PdfDocument document = new PdfDocument();
        document.Info.Title = "Created with PDFsharp";

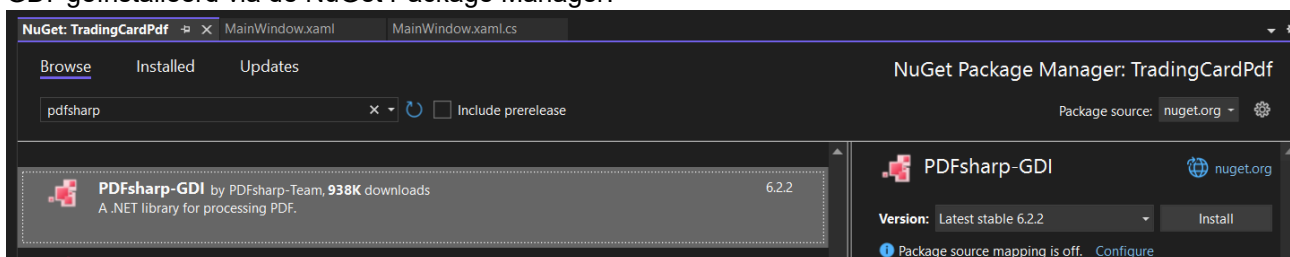
        PdfPage page = document.AddPage();

        XGraphics gfx = XGraphics.FromPdfPage(page);
        XFont font = new XFont("Verdana", 20, XFontStyleEx.Bold);

        gfx.DrawString("Hello, PDF!", font, XBrushes.Black,
            new XRect(0, 0, page.Width, page.Height),
            XStringFormats.Center);

        document.Save("HelloPDF.pdf");
    }
}
```

Om toegang te krijgen tot de PDFSharp namespace in de bovenstaande code is de package 'PDFsharp-GDI' geïnstalleerd via de NuGet Package Manager:



Om een tekst te tonen met twee strepen boven en onderaan de pagina zou deze code gebruikt kunnen worden:

```
// Imports om bovenin de class file toe te voegen
using PdfSharp.Drawing;
using PdfSharp.Pdf;
using PdfSharp.Quality;
using System.IO;

// Methode om binnen de klasse toe te voegen
public void GenerateSimplePdf()
{
    var document = new PdfDocument();
    document.Info.Title = "Hello PDFSharp";

    // Add a single page
    var page = document.AddPage();

    // Create drawing surface
    var gfx = XGraphics.FromPdfPage(page);

    // Top line
    gfx.DrawLine(
        new XPen(XColors.Black, 2),
        40, // x1
        40, // y1
        page.Width.Point - 40, // x2
        40 // y2
    );

    // Bottom line
    gfx.DrawLine(
        new XPen(XColors.Gray, 2),
        40,
        page.Height.Point - 40,
        page.Width.Point - 40,
        page.Height.Point - 40
    );

    var font = new XFont("Arial", 24, XFontStyleEx.Bold);

    gfx.DrawString(
        "Hello PDFSharp!",
        font,
        XBrushes.DarkBlue,
        new XRect(0, 100, page.Width.Point, 50),
        XStringFormats.Center
    );

    var filename = Path.Combine(Path.GetTempPath(), "HelloTest.pdf");
    document.Save(filename);

    PdfFileUtility.ShowDocument(filename);
}
```

Hello PDFSharp!

Onderaan dit script zie je de handige 'PdfFileUtility.ShowDocument'-methode. Die opent de opgeslagen PDF in de standaard PDF reader (zoals je browser, of Adobe Acrobat)

5.18. Opdracht: PDF Genereren Bootcamp

Ga naar de officiële documentatie van PDFsharp en verken de uitleg en voorbeelden. Als je het voorbeeld 'My first PDF file' hebt gevonden, weet je dat je op de goede weg bent.

- Bekijk hoe een Pdf-bestand wordt aangemaakt.
- Welke stappen zie je terug in het voorbeeld?
- Wat betekenen de verschillende methodes die gebruikt worden?

NUGET PACKAGE INSTALLEREN EN EERSTE PDF

Stappenplan:

1. Maak een nieuw WinUI3 project
2. Installeer de 'PDFsharp-GDI' NuGet Package
Deze package werkt alleen voor Windows applicaties, maar heeft als voordeel dat je geen 'GlobalFontSettings.FontResolver' hoeft in te stellen. Alle lettertypen (fonts) die op Windows werken, werken dan in je PDF.
3. Voeg een Button element toe in de XAML met als tekst 'Genereer PDF'
4. Maak een lege 'GenerateSimplePdf'-methode. De methode mag private zijn. De methode gaat alleen wat doen en geen waarde 'returnen'.
5. Voeg een Event Handler toe aan de Button voor wanneer erop geklikt wordt.
6. Roep binnen de Event Handler van de Button de 'GenerateSimplePdf'-methode aan.
7. Vul nu de 'GenerateSimplePdf'-methode met code zoals die op de vorige pagina van dit moduleboek. Zorg dat een PDF gegenereerd wordt met ten minste:
 - Een stuk tekst, zoals jouw naam, of de naam van een karakter uit een game (DrawString)
 - Lijnen, voor decoratie of om een symbool mee te tekenen (DrawLine)
8. Bedenk voor jezelf of je snapt:
 - Welk deel van DrawLine beïnvloed de startpositie van de lijn? En welke de eindpositie?
 - Hoe kun je het lettertype aanpassen van de tekst die met DrawString wordt getoond?
 - En hoe kun je de kleur van de 'kwast' (Brush) aanpassen van de tekst die getekend wordt?
 - Welke code bepaald de kleur van een lijn?
 - Waar wordt het bestand opgeslagen? Bekijk dit eventueel met een breakpoint in de code.

ITEMS IN EEN LIST TONEN

In deze opdracht experimenteer je met het tonen van gegevens uit een List in de PDF.

Stappenplan:

1. Maak een nieuw WinUI3 project en voeg de 'PDFSharp-GDI' NuGet Package weer toe aan het project.

2. Maak een eenvoudige klasse die een factuurregel beschrijft:

```
public class InvoiceItem
{
    public string Description { get; set; }
    public int Quantity { get; set; }
    public decimal UnitPrice { get; set; }

    public decimal Total => Quantity * UnitPrice;
}
```

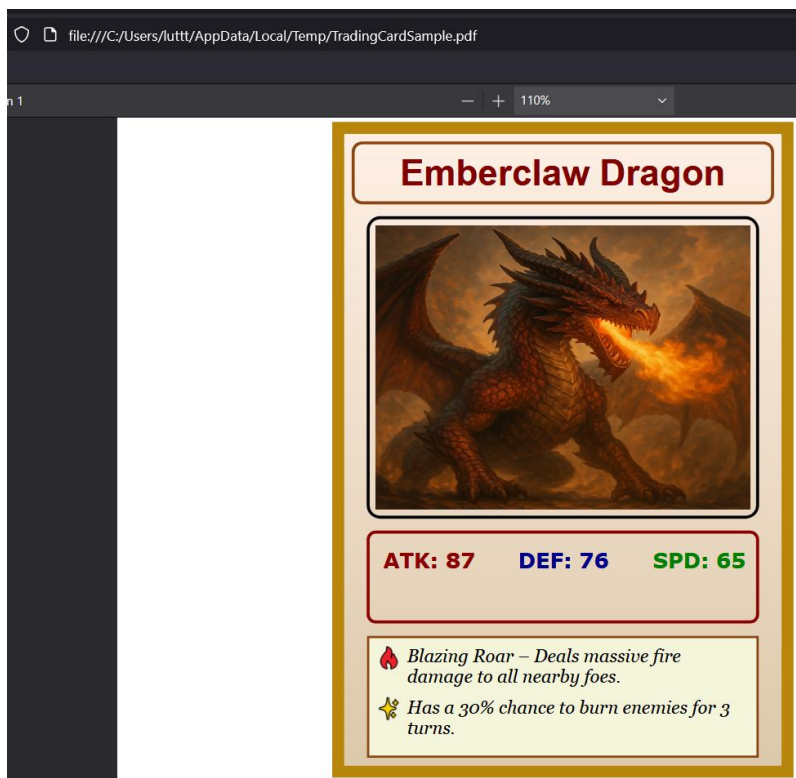
3. Maak een List en vul die met testdata (gewoon in een List, zonder Entity Framework):

```
var items = new List<InvoiceItem>
{
    new InvoiceItem { Description = "Laptop Dell XPS 13", Quantity = 1, UnitPrice = 1299.00m },
    new InvoiceItem { Description = "USB-C Adapter", Quantity = 2, UnitPrice = 24.95m },
    new InvoiceItem { Description = "Draadloze muis Logitech MX Master 3", Quantity = 1, UnitPrice = 89.99m },
    new InvoiceItem { Description = "Monitor LG UltraWide 34\"", Quantity = 2, UnitPrice = 499.00m }
};
```

4. Toon alle gegevens in de PDF met behulp van een loop. Bedenk zelf hoe je de positie aanpast zodat de gegevens netjes onder elkaar getoond worden in de PDF.

EEN COMPLEXE PDF MET AFBEELDINGEN

Je krijgt een stuk voorbeeld-code wat laat zien hoe een trading card (zoals een Pokémon kaart, of Magic The Gathering kaart) gegenereerd kan worden in een PDF. Het dient als demonstratie van hoe je een afbeelding toont in een PDF. De PDF ziet er zo uit in de Firefox browser:



Stappenplan:

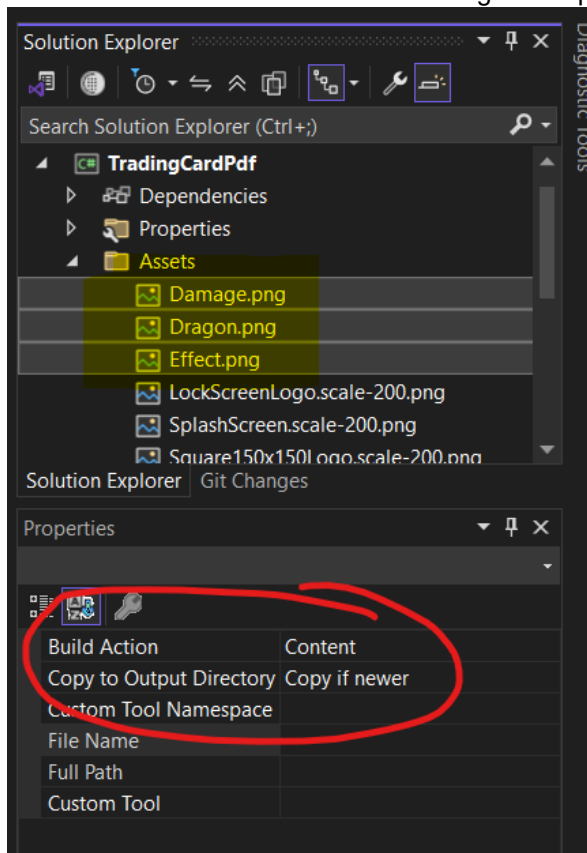
1. Download het project 'H5 Opdracht 5.18 TradingCardPdf Uitwerking.zip' vanuit de Uitwerkingen map op Itslearning.

2. Pak het project uit en start het op in Visual Studio.
3. Bekijk hoe de gegenereerde PDF eruit ziet.
4. Probeer wat je ziet in de PDF te herleiden naar de bijbehorende code.
5. **Let op! Dit moet jij ook doen in jouw project wanneer je afbeeldingen gebruikt:**
 - De afbeeldingen worden in deze code geladen uit de Assets-map, die naast de applicatie staat:

```
var dragonPath = Path.Combine(AppContext.BaseDirectory, "Assets", "Dragon.png");
using var dragonImage = XImage.FromFile(dragonPath);

// Draw inside the portraitRect
gfx.DrawImage(
    dragonImage,
    portraitRect.X + 5,
    portraitRect.Y + 5,
    portraitRect.Width - 10,
    portraitRect.Height - 10
);
```

- Dit werkt omdat in Visual Studio deze eigenschappen zijn veranderd van de afbeeldingen:



Als je dit Properties Window-scherm niet ziet kun je die via 'View' bovenin Visual Studio weer terug in beeld krijgen.

- Dankzij deze eigenschappen worden bij het bouwen van het project de afbeeldingen naar de 'output' directory gekopieerd. Omdat ze dan naast het programma staan, kunnen die gevonden worden via:

```
var dragonPath = Path.Combine(AppContext.BaseDirectory, "Assets", "Dragon.png");
```

EEN VOLLEDIGE FACTUUR GENEREREN

Met deze laatste oefening bereid je jezelf voor op code die je ook in het project Barroc Intens gaat schrijven. Je genereert een factuur met hard-coded gegevens. In Barroc Intens gebruik je de échte gegevens uit de database om de PDF te genereren.

Stappenplan:

- Maak een nieuw WinUI3 project en voeg de PDFSharp-GDI NuGet Package weer toe aan het project.
- Genereer een PDF op basis van deze voorbeeldfactuur. Verzin zelf hard-coded data en plaats die in de factuur:

FACTUUR

BEDRIJFSNAAM
Straat & huisnummer
Postcode & plaats

Bedrijfsnaam
Straat & huisnummer
Postcode & plaats

DATUM:
NUMMER



BESCHRIJVING	AANTAL	TARIEF	BEDRAG
Aanbetaling conform afspraak	1,00	250,00	250,00
Bereft aanbetaling op machine D500 Verwachte leverdatum d.d.			
<i>Subtotaal</i>			250,00
<i>BTW- tarief</i>			21,00%
<i>BTW-bedrag</i>			52,50
TOTAAL			€ 302,50

Gelieve het totaalbedrag binnen 30 dagen te voldoen op rekening 0000000000

BTW nummer
KvK nummer
IBAN nummer

T: 0000000000
E: 0000000000
W: 0000000000

- Bedenk alvast voor Barroc Intens:

- Uit welke modellen gaat de data écht komen?
- Welke eigenschappen komen waar?
- Is er een koppeltabel in jullie database die iets te maken heeft met facturen? En waar komen de gegevens van die koppeltabel op de factuur?

6. Applicatie Implementeren

Bij het ontwikkelen van een C#-applicatie met een database is functionaliteit belangrijk, maar in productie draait het ook om stabiliteit en beveiliging. Dit hoofdstuk behandelt **hoe je een C# applicatie met database professioneel en betrouwbaar in een bedrijf implementeert**.

We bespreken het hosten van de database op een server voor stabiliteit en veiligheid, het gebruik van **migrations** in Entity Framework om de databasestructuur bij te werken zonder dataverlies, en het belang van **back-ups** om dataverlies te voorkomen. Ook komen **beveiliging tegen onbevoegde toegang** en **technieken voor gelijktijdig gebruik van de applicatie zonder conflicten** aan bod.

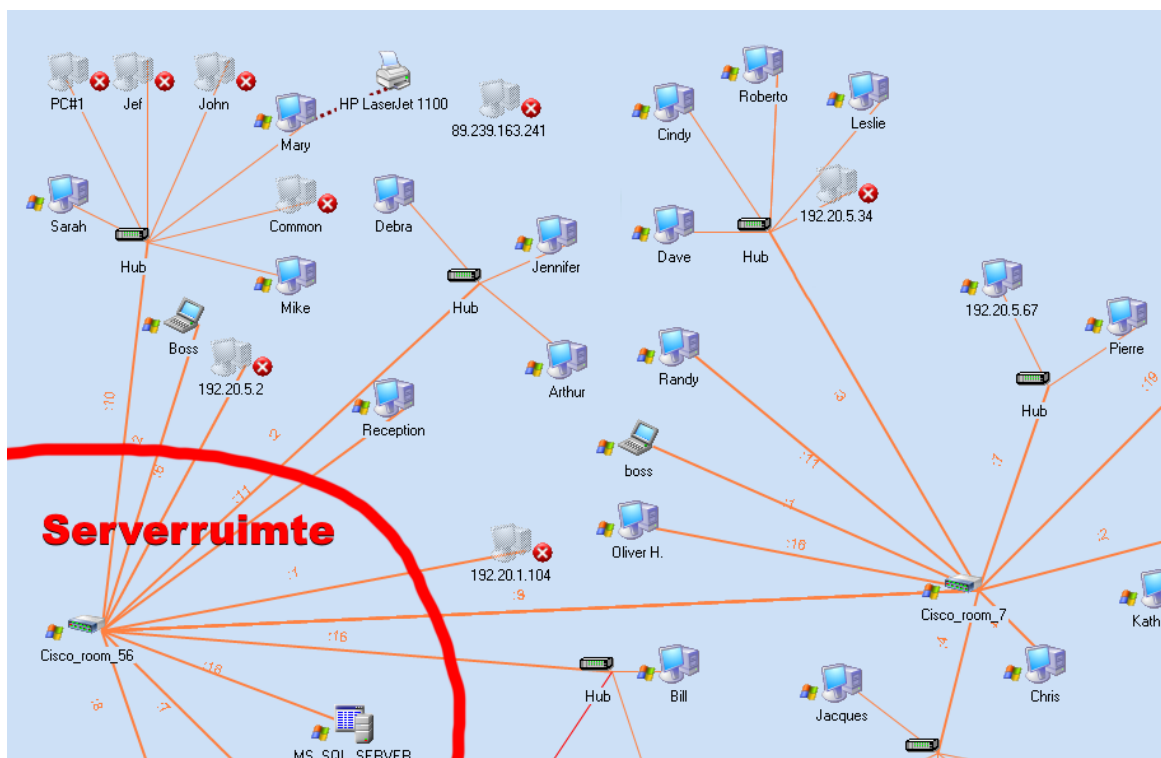
Dit hoofdstuk geeft tools en 'best practices' voor een veilige, betrouwbare en schaalbare databaseomgeving in productie.

6.1. Een C# Applicatie implementeren binnen bedrijven

In een bedrijfsomgeving worden C# applicaties vaak lokaal 'gehost', met database en applicatie binnen het interne netwerk. Dit biedt:

- **Controle:** IT kan de beveiliging en prestaties beheren zonder externe partijen.
- **Snelheid:** Snelle toegang door directe netwerkverbinding.
- **Privacy en Compliance:** Data blijft in het eigen netwerk, eenvoudiger voor naleving van regels als GDPR.

In het geval van een WinUI3 applicatie zou de (MySQL) database op een server kunnen draaien, in een serverruimte binnen het kantoor. De WinUI3 applicatie kan dan op alle computers van medewerkers worden geïnstalleerd. Die verbindt dan lokaal met de database.



Dit is anders dan bij **webapplicaties** (zoals een Laravel applicatie). Webapplicaties worden vaker in de 'cloud' of anderzijds online gehost, wat voordelen biedt zoals:

- **Schaalbaarheid:** Flexibel meer servers online brengen als de applicatie ineens veel verkeer ontvangt.
- **Beschikbaarheid:** Redundantie (extra servers om op terug te vallen) en uptime-garanties. Ook is de applicatie standaard vanaf overal waar een internetverbinding is bereikbaar
- **Veiligheidsuitdagingen:** Minder controle over data, vereist extra beveiligingsmaatregelen.

Later in dit hoofdstuk gaan we kijken hoe we de database beveiligen, zodat we onze gebruikers niet volledige toegang geven tot alle gegevens in de database.

6.2. Database migraties in productie

Tot nu hebben we gewerkt met 'EnsureDeleted' en 'EnsureCreated' om de database op te bouwen. Dit is handig tijdens ontwikkeling, want het geeft ons iedere test een 'schone' database met testgegevens.

In een productieomgeving willen we natuurlijk niet de database verwijderen, iedere keer als de applicatie opstart. Daarom halen we de EnsureDeleted en EnsureCreated weg uit de code, als de applicatie ver af is.

Toch willen we ook in een productieomgeving 'incrementele' wijzigingen aan de databasestructuur kunnen maken. Dat zijn updates waar we bijvoorbeeld een kolom of zelfs hele tabel toevoegen. Om die reden **gebruiken we in productie migraties.**

Migraties maken het mogelijk dat we wijzigingen in het database-schema (ontwerp) opnemen als een reeks programmeerbare stappen. Elke stap, een migratie genaamd, vertegenwoordigt een verandering in het schema, zoals het toevoegen van een nieuwe tabel, het wijzigen van een bestaande tabel of het toevoegen/verwijderen van kolommen.

Hoe kunnen we migraties toepassen in Entity Framework?

De volgende informatie is relevant wanneer jij (bijna) klaar bent met het ontwikkelen van de applicatie. Je wilt dan de EnsureDeleted/Created code niet langer in de applicatie hebben. Wel wil je op de een of andere manier de database kunnen opbouwen. Ook wil je updates kunnen uitvoeren, als je tabellen of kolommen hebt aangepast.

Om migraties in Entity Framework toe te passen, moeten we de volgende stappen volgen:

1. Definieer de Model-klassen, zoals we gewend zijn.
2. Configureer de database context en definieer de DbSet's die de model-klassen representeren, eveneens zoals we tot nu gedaan hebben (met de ApplicationDbContext).
3. Installeer de juiste NuGet package om de Entity Framework migratie commando's voor het project te activeren: **Microsoft.EntityFrameworkCore.Tools**
4. **Genereer de initiële migratie:** Voer het volgende commando uit in de Package Manager Console (onderin Visual Studio, eventueel via Tools > NuGet Package Manager te vinden):

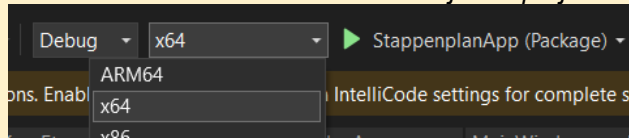
Add-Migration InitialCreate

Dit commando bekijkt de DbSets in de AppDbContext en kijkt of de modellen anders zijn dan wat eerdere migrations hebben vastgelegd. Als er geen migrations zijn maakt het een script dat de database opbouwt volgens de eigenschappen van de modellen (net als EnsureCreated doet).

Krijg je deze foutmelding tijdens het maken van de Migration?

```
PM> Add-Migration InitialCreate
Build started...
Build succeeded.
Failed to load the dll from [C:\Projects\StappenplanApp\StappenplanApp\bin\x86\Debug\net6.0-windows10.0.19041.0\win10-x86\hostpolicy.dll], HRESULT:
0x800700C1
An error occurred while loading required library hostpolicy.dll from [C:\Projects\StappenplanApp\StappenplanApp\bin\x86\Debug\net6.0-
windows10.0.19041.0\win10-x86\]
PM> Add-Migration InitialCreate
```

Stel dan bovenin Visual Studio in dat je het project bouwt voor x64:



5. Nu we het eerste migratiescript hebben aangemaakt gaan we die toepassen op de database. Voer het volgende commando uit:

Update-Database

Het migratiescript voert zich uit en zal alle tabellen en kolommen in de database aanmaken.

6. Wanneer er nu in de toekomst wijzigingen zijn gaan we deze twee stappen opnieuw uitvoeren. We maken een migratiescript (daarin zal Entity Framework enkel de wijzigingen opnemen). Daarna voeren we die uit.

Om dit te oefenen zou je de modellen iets kunnen aanpassen. Hierna voer je dus opnieuw uit:

Add-Migration <MigrationName>

Vervang <MigrationName> door een betekenisvolle naam voor de migratie. Als je bijvoorbeeld een eigenschap 'Username' hebt toegevoegd aan een model 'User' kun je kiezen voor:

AddColumnUsernameToUsers

Hierna pas je de migratie opnieuw toe op de database met:

Update-Database

Bij lokale ontwikkeling zijn de bovenstaande stappen enorm onhandig, vandaar dat we tot nu EnsureDeleted en EnsureCreated hebben gebruikt. Maar in productie brengt dit een voordeel: we kunnen namelijk een migratie toepassen die een kolom toevoegt, zelfs al gebruikers de applicatie al gebruiken.

Let er dan wel op:

- Breng de applicatie tijdelijk offline, zodat de update toegepast kan worden.
- Maak voorafgaand aan de update **altijd** een back-up van de database.
- Controleer of de back-up alle gegevens bevat (controleer de datums).
- Voer de migratie uit, zodat de nieuwe migratiescripts worden uitgevoerd.
- Installeer de nieuwe applicatie op de apparaten van de gebruikers.

Bij schoolprojecten is het netjes dat je ten minste in de laatste sprint zorgt dat het systeem wat je oplevert gebruik maakt van migrations. Zo is de applicatie klaar voor gebruik in productie.

6.3. Opdracht: Even checken

Beantwoord voor jezelf de volgende vragen:

1. Tijdens ontwikkeling is het prima om te gebruiken om de database op te bouwen.
 - a. de EnsureCreated en EnsureDeleted methodes
 - b. migrations
2. In productie gebruiken we altijd om de database op te bouwen.
 - a. de EnsureCreated en EnsureDeleted methodes
 - b. migrations
3. Beschrijf in je eigen woorden waar de volgende commando's voor dienen:

Commando	Beschrijving
Add-Migration	
Update-Database	
Remove-Migration	
Script-Migration	
Get-Migration	

6.4. Database toegang beveiligen binnen een lokaal netwerk

Ook al vertrouwen we de medewerkers binnen het bedrijf met volledige rechten, we moeten die rechten toch beperken. Een veel voorkomend beveiligingsprobleem is namelijk dat een kwaadwillende via een medewerker binnenkomt in een systeem. Daarom willen we de rechten van medewerkers beperken tot enkel wat ze nodig hebben.

Wat we dus NIET doen:

Een MySQL database installeren op een lokale server binnen het bedrijf.
Daar de 'root' gebruiker geen wachtwoord geven.
De connectie string van de WinUI applicatie instellen op
"server=lokaalip;user=root;password=;"

Iedereen kan nu als root inloggen en alle tabellen aanpassen, verwijderen, exporteren, etc. etc.



We willen er alles aan doen om toegang tot de database te beperken. Hieronder vind je enkele strategieën hoe dit kan. **Vraag wel altijd een senior collega om advies wanneer jullie een systeem op deze manier opzetten!**

Wat we dus WEL doen:

- **Beperk gebruikersrechten:** Maak per gebruiker specifieke accounts aan in MySQL, zodat ze alleen toegang hebben tot de tabellen en acties die ze nodig hebben. Geef **geen** administratieve rechten aan iedereen.
Bron: How to create MySQL user with limited privileges? - <https://stackoverflow.com/a/9174908>
- **Sterke wachtwoorden:** Gebruik sterke, unieke wachtwoorden voor elke MySQL-gebruiker. Dit voorkomt dat ongeautoriseerde personen makkelijk toegang krijgen.
- **Beperk 'remote access':**
 - Geef de MySQL gebruikers enkel toestemming om vanaf lokale IP-adressen te verbinden.
Bron: Restricting MySQL connections from localhost to improve security - <https://stackoverflow.com/a/13208652>
 - Zorg dat de MySQL-poort (standaard 3306) alleen bereikbaar is voor de C#-app of servers die er toegang toe moeten hebben. Gebruik een firewall om toegang vanaf ongewenste apparaten te blokkeren.
- **Log monitoring:** Zet MySQL-logging aan en controleer regelmatig wie wat heeft gedaan. Hiermee detecteer je ongebruikelijke toegangspogingen of verdachte activiteiten. Daarnaast is het enorm belangrijk dat na een aanval of lek, teruggevonden kan worden wat er mis is gegaan.

Wanneer iedere medewerker binnen het bedrijf eigen MySQL inloggegevens heeft (met beperkte rechten, zoals alleen lezen, invoeren, aanpassen en verwijderen) kun je op deze manier verbinden via een DbContext:

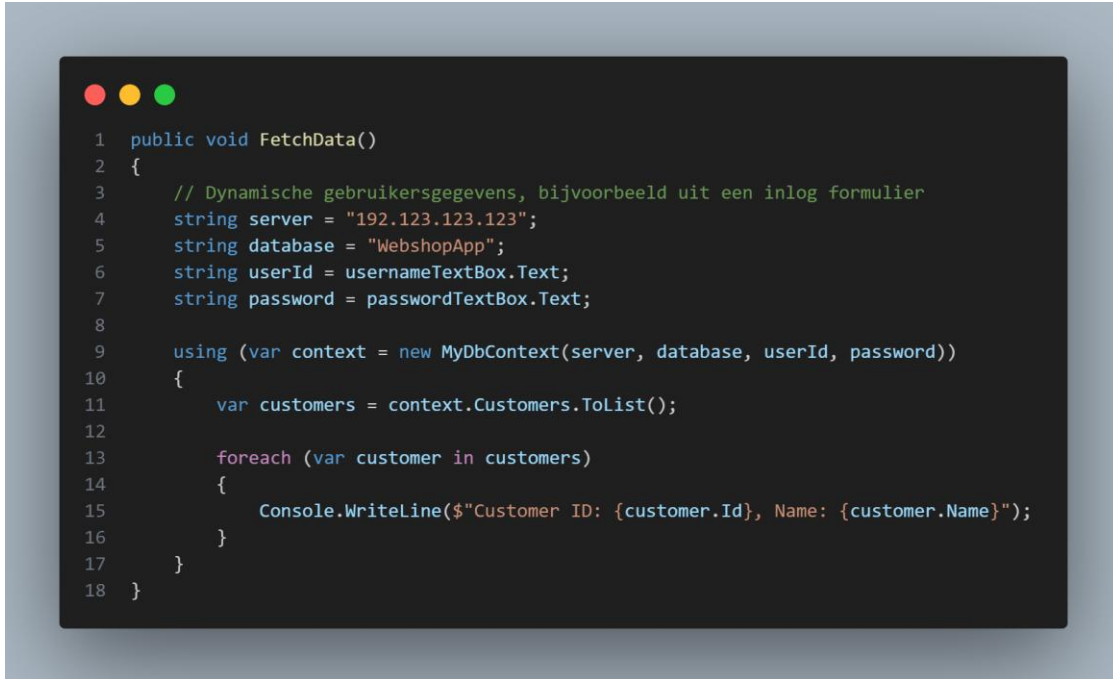
1. Geef jouw AppDbContext een constructor, waarlangs je de gebruikersnaam en het wachtwoord van een gebruiker kunt doorgeven:

```

1  using Microsoft.EntityFrameworkCore;
2
3  public class AppDbContext : DbContext
4  {
5      private readonly string connectionString;
6
7      // Constructor neemt dynamische inloggegevens als parameters
8      public AppDbContext(string server, string database, string userId, string password)
9      {
10         connectionString = $"Server={server};Database={database};User Id={userId};Password={password};";
11     }
12
13     protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
14     {
15         // Gebruik de dynamische verbindingsstring
16         optionsBuilder.UseMySQL(connectionString);
17     }
18
19     // Definieer DbSet's voor je entiteiten
20     public DbSet<Customer> Customers { get; set; }
21     public DbSet<Order> Orders { get; set; }
22 }

```

2. Bij het openen van een databaseverbinding geef je vervolgens de gewenste inloggegevens mee. Bijvoorbeeld:



```
1 public void FetchData()
2 {
3     // Dynamische gebruikersgegevens, bijvoorbeeld uit een inlog formulier
4     string server = "192.123.123.123";
5     string database = "WebshopApp";
6     string userId = usernameTextBox.Text;
7     string password = passwordTextBox.Text;
8
9     using (var context = new MyDbContext(server, database, userId, password))
10    {
11        var customers = context.Customers.ToList();
12
13        foreach (var customer in customers)
14        {
15            Console.WriteLine($"Customer ID: {customer.Id}, Name: {customer.Name}");
16        }
17    }
18 }
```

Op deze manier beperk je de rechten van gebruikers op database niveau. Daarnaast kun je gebruik maken van de 'logging' functies van MySQL, waarmee acties naar logbestanden geschreven kunnen worden. Zo kun je achterhalen wie er wanneer ingelogd heeft.

6.5. Database toegang beveiligen doormiddel van een API

De manier uit de vorige paragraaf is slechts één van de manieren waarop je databasetoegang kunt beperken. Daarnaast kun je ook gebruik maken van een API. Hier gaan we in de volgende module mee aan de slag. Voor nu volgt hier een korte samenvatting.

In plaats van dat een applicatie direct met de database verbindt, kan een API als beveiligde tussenlaag dienen. De API zorgt ervoor dat gebruikers alleen toegang krijgen tot de data en functies die voor hen relevant zijn, zonder directe database-inloggegevens nodig te hebben. Dit verbetert de beveiliging en controle.

Voordelen van een API:

1. **Toegangscontrole:** De API kan specifieke rechten en rollen beheren en biedt alleen de noodzakelijke data aan de gebruiker.
2. **Gevoelige gegevens afschermen:** Gebruikers krijgen nooit directe toegang tot de database of gevoelige gegevens zoals verbindingsstrings en inloggegevens.
3. **Beveiligingsmaatregelen:** Authenticatie (bijv. JWT-tokens) en versleuteling zorgen voor een veilige datastroom tussen de applicatie en de database.

Door de database-toegang via een API te laten verlopen, minimaliseer je de risico's op ongeautoriseerde toegang en verhoog je de controle over de data.

In de volgende module gaan we zelf een Webserver en API bouwen, die je kunt gebruiken voor dit doel.

6.6. 'Concurrency' - wanneer meerdere gebruikers tegelijk de applicatie gebruiken

Stel je de volgende situatie voor: er bestaat een WinUI 3 applicatie die op alle computers van medewerkers is geïnstalleerd. De applicatie verbindt veilig met een MySQL server op het lokale netwerk. De

applicatie wordt gebruikt om aantekeningen vast te leggen over klanten van het bedrijf 'WijLeggenHetVast B.V.':

1. **Marit** opent op haar computer de applicatie. Ze heeft een mail ontvangen van de klant 'ACME Corp.' en wil een aantekening over die mail toevoegen in het systeem.
2. **Marit** navigeert in de applicatie naar de klant en opent deze.
3. **De applicatie** maakt verbinding met de database en laad alle gegevens van ACME Corp. In het aantekeningenveld staat nu:
"Contact gehad met nieuwe klant, op 1-2 terugbellen."
4. **Marit** voegt haar aantekening toe in het aantekeningenveld:
"Contact gehad met nieuwe klant, op 1-2 terugbellen."
Op 23-1 mail ontvangen over bestelling. Per mail informatie gegeven
5. Voordat **Marit** opslaat wordt ze door een collega afgeleid. Ze is even een paar minuten in gesprek.
6. Collega **John** wordt op dit moment gebeld door ACME Corp. In een kort gesprek helpt hij ze verder.
7. **John** opent de applicatie en gaat naar de klant.
8. **De applicatie** maakt verbinding met de database en laad alle gegevens van ACME Corp. In het aantekeningenveld staat nu:
"Contact gehad met nieuwe klant, op 1-2 terugbellen."
9. **John** voegt zijn aantekening toe in het aantekeningenveld:
"Contact gehad met nieuwe klant, op 1-2 terugbellen."
Op 23-1 per telefoon geholpen met vraag over bestelling.
10. **John** slaat de gegevens op. Zijn aantekeningen worden opgeslagen in de database.
11. Kort hierna is **Marit** klaar met het helpen van haar collega. Ze gaat achter haar computer verder en merkt dat ze nog niet opgeslagen heeft. Ze klikt op opslaan.
12. De aantekeningen van **Marit** worden opgeslagen in de database. In de database staat nu:
"Contact gehad met nieuwe klant, op 1-2 terugbellen."
Op 23-1 mail ontvangen over bestelling. Per mail informatie gegeven

Zoals je ziet gaat hier informatie verloren. De aantekeningen van John worden overschreven met de wijzigingen van Marit. Wat hier gebeurt noemen we "Last Write Wins".

Wanneer een applicatie door meerdere gebruikers wordt gebruikt gaat het bovenstaande gebeuren. Het zal voor gebruikers ook niet altijd duidelijk wat er precies gebeurt. Maar ze zullen mogelijk wel aanvoelen 'dat dingen niet werken of niet lijken op te slaan'. **Het is aan de softwareontwikkelaars om hier een oplossing voor te bedenken.**

Entity Framework biedt een oplossing die we hieronder toelichten.

6.7. Entity Framework – 'Concurrency' afhandelen

Entity Framework implementeert 'optimistic concurrency', dit gaat ervanuit dat het niet veel voorkomt dat er conflicten zijn. Als bij het opslaan van een object blijkt dat er tussendoor wijzigingen zijn geweest aan

dat object (zoals in het voorbeeld hierboven) wordt een Exception gegeven, die kan de ontwikkelaar dan afhandelen.

Bij 'pessimistic concurrency' wordt een te bewerken object van tevoren 'op slot' gezet. Zo kan van tevoren bepaald worden dat bewerken niet mogelijk is. Dit is bij Entity Framework niet van toepassing.

Entity Framework gebruikt een 'concurrency token' om te bepalen of een object is aangepast. Je kunt dit zien als een versienummer. Wij als ontwikkelaar moeten hiervoor een eigenschap toevoegen aan onze modellen en die markeren. **Bij het opslaan van een object controleert Entity Framework of de concurrency token is aangepast. Als dat zo is, dan is er een conflict en moet dat worden opgelost. De code ontvangt dan een 'Exception'.**

Bron: <https://learn.microsoft.com/en-us/ef/core/saving/concurrency?tabs=data-annotations>

Laten we een voorbeeld bekijken van 'Application-managed concurrency tokens' uit de documentatie van Microsoft:

1. We voegen aan ons Person model een versie eigenschap toe. Die markeren we met de 'ConcurrencyCheck' data annotation:

```
public class Person
{
    public int PersonId { get; set; }
    public string FirstName { get; set; }

    [ConcurrencyCheck]
    public Guid Version { get; set; }
}
```

2. Hier wordt gebruik gemaakt van een GUID (spreek uit: goe-ied): Een **globally unique identifier** is een pseudowillekeurig getal dat gebruikt wordt in softwaretoepassingen, en dat verondersteld wordt wereldwijd uniek te zijn.
3. Wanneer we een Person object aanpassen stellen we een nieuwe versie in (dit maakt deze aanpak 'application-managed'):

```
var person = context.People.Single(b => b.FirstName == "John");
person.FirstName = "Paul";
person.Version = Guid.NewGuid();
context.SaveChanges();
```

4. Mocht het zo gebeuren, dat er een conflicterende wijziging wordt gemaakt. Dan wordt een 'DbUpdateConcurrencyException' opgegooid. Via een try-catch constructie kunnen we die afhandelen:

```

using var context = new PersonContext();
// Fetch a person from database and change phone number
var person = context.People.Single(p => p.PersonId == 1);
person.PhoneNumber = "555-555-5555";

// Change the person's name in the database to simulate a concurrency conflict
context.Database.ExecuteSqlRaw(
    "UPDATE dbo.People SET FirstName = 'Jane' WHERE PersonId = 1");

var saved = false;
while (!saved)
{
    try
    {
        // Attempt to save changes to the database
        context.SaveChanges();
        saved = true;
    }
    catch (DbUpdateConcurrencyException ex)
    {
        foreach (var entry in ex.Entries)
        {
            if (entry.Entity is Person)
            {
                var proposedValues = entry.CurrentValues;
                var databaseValues = entry.GetDatabaseValues();

                foreach (var property in proposedValues.Properties)
                {
                    var proposedValue = proposedValues[property];
                    var databaseValue = databaseValues[property];

                    // TODO: decide which value should be written to database
                    // proposedValues[property] = <value to be saved>;
                }

                // Refresh original values to bypass next concurrency check
                entry.OriginalValues.SetValues(databaseValues);
            }
            else
            {
                throw new NotSupportedException(
                    "Don't know how to handle concurrency conflicts for "
                    + entry.Metadata.Name);
            }
        }
    }
}

```

In het bovenstaande voorbeeld van Microsoft wordt voorgesteld dat de applicatie een keuze maakt welke gegevens worden opgeslagen. Maar je kunt het ook anders oplossen, bijvoorbeeld:

1. Toon een foutmelding "De gegevens die u probeerde te bewerken zijn tussentijds aangepast. We sturen u terug en laten de nieuwe gegevens zien."
2. Stuur de gebruiker terug naar het formulier.
3. Laat de gegevens van de gebruiker staan (we willen geen gegevens verliezen).

4. Laat duidelijk zien welke gegevens tussendoor aangepast zijn.
5. Geef met een knop de mogelijkheid "Wijzigingen overschrijven (gegevens gaan verloren)"

Of je een van deze manieren gebruikt, of een geheel eigen oplossing, is aan jou als softwareontwikkelaar. Probeer met je collega's een oplossing te vinden die het beste past bij de situatie van jullie opdrachtgever.

6.8. Zelf 'Concurrency' afhandelen

Dat Entity Framework een oplossing biedt is leuk, maar misschien krijg je het niet werkend, of is het niet helemaal geschikt voor jouw situatie. Je bent uiteraard vrij om zelf een manier te bedenken om deze situaties af te handelen. Een andere manier kan bijvoorbeeld zijn:

1. Voeg een 'versienummer' eigenschap toe aan alle modellen, bijv.: "Version"
2. Wanneer een gebruiker een object wil aanpassen haal je alle gegevens op (inclusief het huidige Version nummer)
3. Laat de gebruiker wijzigingen maken.
4. Wanneer de gebruiker klikt op 'opslaan' ga je eerst opnieuw de gegevens ophalen uit de database.
5. Je controleert of het versienummer hetzelfde is als waar de gebruiker mee gewerkt heeft. Zo ja, dan sla je meteen op.
6. Als het versienummer verschilt ga je het probleem met de gebruiker oplossen. Bijvoorbeeld door een foutmelding te tonen, zoals in de vorige paragraaf werd beschreven.

7. Authenticatie & Autorisatie

In dit hoofdstuk gaan we kijken naar de theorie over Authenticatie en Autorisatie. Je krijgt praktische tips hoe je dit op een laagdrempelige manier kunt implementeren in C# met Entity Framework.

Later in de module bouwen we hierop verder en moderniseren we de code.

7.1. Authenticatie

Authenticatie is het proces waarbij het systeem controleert of een gebruiker echt is wie hij zegt te zijn. Dit gebeurt meestal door inloggegevens, zoals een gebruikersnaam en wachtwoord, te verstrekken.

We starten eerst met wat theorie over authenticatie. Daarna krijg je praktische tips hoe je een gebruikerslogin kunt implementeren in C# met Entity Framework.

7.2. Authenticatie en Autorisatie

Authenticatie en autorisatie zijn verschillende concepten. Authenticatie gaat over het controleren van de identiteit van een gebruiker, terwijl autorisatie te maken heeft met het bepalen van de rechten en toegangsbevoegdheden van een geauthentiseerde gebruiker.

7.3. Vormen van authenticatie

Authenticatie kan op verschillende manieren worden uitgevoerd, afhankelijk van de specifieke vereisten en beveiligingsniveaus van een systeem. Hier zijn enkele veelvoorkomende vormen van authenticatie die worden gebruikt in softwareontwikkeling:

1. **Wachtwoordauthenticatie:** Dit is de meest bekende vorm van authenticatie, waarbij een gebruiker een combinatie van een gebruikersnaam en een wachtwoord invoert. Het systeem vergelijkt deze gegevens met opgeslagen referenties om de identiteit van de gebruiker te verifiëren. Het is belangrijk om sterke wachtwoorden te gebruiken en deze veilig op te slaan om ongeautoriseerde toegang te voorkomen.
2. **Biometrische authenticatie:** Deze vorm van authenticatie maakt gebruik van unieke fysieke kenmerken van een persoon, zoals vingerafdrukken, irisscans of gezichtsherkenning, om de identiteit te verifiëren. Biometrische gegevens worden vastgelegd en vergeleken met de opgeslagen gegevens om toegang te verlenen.
3. **Tweefactorauthenticatie (2FA):** Bij tweefactorauthenticatie moet een gebruiker naast een wachtwoord nog een extra verificatiemethode doorlopen. Dit kan bijvoorbeeld een eenmalige verificatiecode zijn die naar een geregistreerd mobiel apparaat wordt gestuurd. Door de combinatie van iets wat de gebruiker weet (wachtwoord) en iets wat de gebruiker bezit (mobiel apparaat), wordt de beveiliging versterkt.
4. **Certificaat gebaseerde authenticatie:** Hierbij wordt gebruik gemaakt van digitale certificaten, die unieke identificatiegegevens bevatten. Een gebruiker heeft een persoonlijk certificaat dat wordt verstrekt door een vertrouwde certificaatautoriteit. Het systeem kan dit certificaat gebruiken om de identiteit van de gebruiker te verifiëren.

7.4. Beveiliging: Wachtwoorden veilig opslaan

Zoals je in eerdere lessen geleerd hebt slaan we wachtwoorden NOOIT als 'plain-text' op. We maken er altijd een 'hash' van met een veilig hash algoritme.

HASHING

Hashing is het omzetten van een string naar een hash, zonder dat die ooit teruggedraaid kan worden. Het is dus anders dan encryptie, want bij encryptie kun je nog 'decrypten'. Bij een hash kan dat niet.

Om een idee te krijgen hoe dat gaat gebruiken we in plaats van wachtwoorden getallen:

1. Alice krijgt zo drie getallen van iemand (**input**). Ze zal die getallen vermenigvuldigen met elkaar (**dit is het hashing algoritme**)
2. Bob geeft Alice de getallen: 91, 33 en 2 (**input**)
3. Alice geeft haar antwoord $91 * 33 * 2 = 6006$ (**dit is de hash**)
4. Stel dat Mallory alleen de hash ziet: 6006, kan hij dan uitvinden welke drie getallen precies de input waren?

Bij hashing kan het gebeuren dat verschillende inputs tot dezelfde hash leiden ($91 * 33 * 2 = 6006$, maar ook $143 * 3 * 14 = 6006$, etc...). Dit heet een 'hash collision', letterlijk vertaald een botsing tussen hashes.

Wij gaan als hashing algoritme SHA512 gebruiken. Dat biedt meer dan genoeg beveiliging, maar naast het wachtwoord gaan we ook een willekeurige 'salt' toevoegen aan het wachtwoord voordat we het geheel 'hashen'. Ook gaan we het 'hashen' een aantal iteraties herhalen. Daarmee wordt de hash weer meerdere keren gehasht. **Waarom? Lees hieronder verder.**

WAAROM EEN SALT EN/OF MEERDERE KEREN HASHEN?

Stel je voor dat we geen Salt gebruiken en maar 1x hashen met sha512. Je kunt deze website gebruiken om een sha512 hash te maken: <https://emn178.github.io/online-tools/sha512.html>

Maak hashes met deze tool voor al deze wachtwoorden:

Wachtwoord	Hash
password	b109f3bbbc244eb82441917ed06d618b9008dd09b3befd1b5e07394c706a8bb980b1d7785e5976ec049b46df5f1326af5a2ea6d103fd07c95385ffab0cacbc86
123456	...
qwerty	...
welkom	...

We zouden met een computerscript een database kunnen bouwen van teksten en hashes. Het script gaat alle mogelijke combinaties van letters en cijfers hashen en slaat het origineel en de hash op. Met die database kunnen we daarna makkelijk opzoeken welke hash bij welk wachtwoord hoort. Daarmee "decrypten" we niet, maar zoeken de hash op in een zogenaamde [Rainbow Table](#).

Of stel je voor dat een hacker inbreekt in een database waar alleen SHA512 gebruikt is. Dit is wat die ziet bij een aantal gebruikers:

Username	PasswordHash
John123	b109f3bbbc244eb82441917ed06d618b9008dd09b3befd1b5e07394c706a8bb980b1d7785e5976ec049b46df5f1326af5a2ea6d103fd07c95385ffab0cacbc86
Mary321	ac1be86492cc783398a863dec35e5f3ebd2a93146b9a5321dd67b094a48cc3d3fd131c8355ba0bee9c5c0575666527364b2f256ef4be74e68ed432a48d48a3e5
Ali	b109f3bbbc244eb82441917ed06d618b9008dd09b3befd1b5e07394c706a8bb980b1d7785e5976ec049b46df5f1326af5a2ea6d103fd07c95385ffab0cacbc86
welkom	fcc26893fb94557bc3b7ce14152da0fe9a32c9627749dc4846138eaa9c14ef61e8790eb02a95fc73407aced41df8fa8bf7748ed3b14aa7c83bedc633f231c176

Ondanks dat de wachtwoorden zijn gehasht, is het meteen duidelijk dat die twee gebruikers hetzelfde wachtwoord hebben, want de hashes zijn hetzelfde.

Voor sterke wachtwoordbeveiliging moeten we dus naast het hashen, ook eerst 'salten'.

Om het door "brute-kracht" raden van hashes moeilijker te maken wordt vóór hashing ook een Salt toegevoegd aan het wachtwoord. Ook wordt meerdere keren gehasht, daarmee wordt het aantal mogelijke hashes zo groot, dat het onrealistisch is om die allemaal vooraf te berekenen en in een rainbow table op te slaan.

*Bij deze een overzicht van het aantal verschillende mogelijkheden voor wachtwoorden van 8 karakters, ervanuitgaande dat SHA512 0,001 milliseconde duurt om uit te rekenen. Het dakje (^) betekent hieronder machtsverheffen ($2^4 == 2*2*2*2$):*

Mogelijke letters en aantal iteraties	Rekensom	Aantal mogelijkheden	Duur
A-Z (26 hoofdletters) en a-z (26 kleine letters)	$(26+26)^8$	53459729000000	~1694 jaar
A-Z en a-z en alle cijfers (10) en !@#\$%^&*()_+ = (14)	$(26+26+10+14)^8$	1113034800000000	~35270 jaar

Als je net als ons hashing algoritme iedere hash nog 10,000x hasht, dan wordt die tijdsduur om alle hashes te berekenen ook 10,000x zo groot. Dus niet 1,694 jaar, maar 16,940,000 jaar. Inclusief alle cijfers en de 14 symbolen niet 35,270 jaar, maar 352,700,000 jaar.

Je ziet dat hashing dus ook deels veilig is door de gigantische tijd die het kost om alle mogelijkheden vooraf te kunnen berekenen.

Het populaire 'bcrypt' hashing algoritme, wat door de PHP password_hash functie wordt gebruikt, slaat die informatie op 'in' de hash:

```
$algoritme$kosten$salt/hash  
$2b$10$b63K/D03WFBktWy552L5XuibmiD5SxCrKg9kHCqOYazWxRjIg14u2
```

De opdeling van de hash is als volgt:

- **Algoritme:** Zal "2a" of "2b" zijn, wat BCrypt betekent
- **Kosten:** Stelt de exponent voor die wordt gebruikt om te bepalen hoeveel iteraties er zijn 2^n
- **Salt:** (16-byte (128-bit)), base64 gecodeerd tot 22 tekens
- **Hash:** (24-byte (192-bit)), base64 gecodeerd tot 31 tekens

Het lijkt misschien onveilig, maar er is geen probleem met het opslaan van de salt in de database. Het biedt toch een extra bescherming. Soms wordt in de code ook nog een extra 'salt' toegevoegd, zodat wanneer de database gehackt is, maar de code niet, er toch nog een onbekende 'salt' is. Dit wordt officieus ook wel de 'pepper' genoemd.

Bronnen:

- <https://blog.logrocket.com/password-hashing-node-js-bcrypt/>
- [https://en.wikipedia.org/wiki/Pepper_\(cryptography\)](https://en.wikipedia.org/wiki/Pepper_(cryptography))

ACHTERHAALDE HASHING ALGORITMES

Het is belangrijk dat je libraries en frameworks gebruikt die je vertrouwt. Vooral bij het omgaan met beveiliging is het verstandig om veel bronnen te raadplegen en te zoeken naar **de veiligste weg** om iets aan te pakken. Zo is het voor PHP niet verstandig om zelf een hashing algoritme te kiezen, maar gebruik je de [password_hash](#) functie die voor je kiest.

Hieronder enkele algoritmes die je sowieso NIET voor beveiliging van gegevens moet gebruiken.

MD2, MD4 and MD5

Quote van StackOverflow:

MD5 for passwords

Using salted md5 for passwords is a bad idea. Not because of MD5's cryptographic weaknesses, **but because it's fast**. This means that an attacker can try [billions](#) of candidate passwords per second on a single GPU.

What you should use are deliberately slow hash constructions, such as scrypt, bcrypt and PBKDF2. Simple salted SHA-2 is not good enough because, like most general purpose hashes, it's fast. Check out [How to securely hash passwords?](#) for details on what you should use.

MD5 for file integrity

Using MD5 for file integrity may or may not be a practical problem, depending on your exact usage scenario.

The attacks against MD5 are collision attacks, not pre-image attacks. This means an attacker can produce two files with the same hash, if he has control over both of them. But he can't match the hash of an existing file he didn't influence.

I don't know if the attacks applies to your application, but personally I'd start migrating even if you think it doesn't. It's far too easy to overlook something. Better safe than sorry.

The best solution in this context is SHA-2 (SHA-256) for now. Once SHA-3 gets standardized it will be a good choice too.

Bron: <https://security.stackexchange.com/a/19908>

SHA-1

In an interview, Yin states that, "Roughly, we exploit the following two weaknesses: One is that the file preprocessing step is not complicated enough; another is that certain math operations in the first 20 rounds have unexpected security problems."

Bron: <https://en.wikipedia.org/wiki/SHA-1#Attacks>

Andere inmiddels defecte algoritmes: https://en.wikipedia.org/wiki/Category:Broken_hash_functions

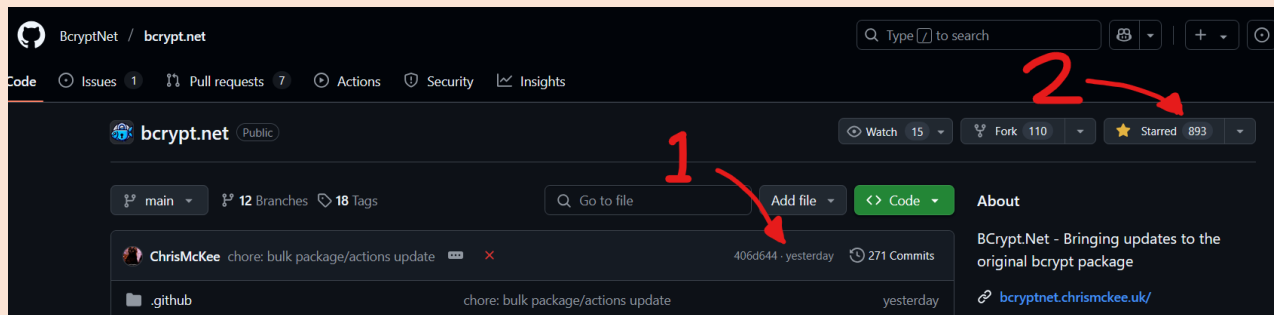
7.5. Hashing in C#

Waar we in PHP een simpele functie als 'password_hash' kunnen gebruiken, bestaat die niet 'out-of-the-box' in C#. Daar moeten we die functie zelf implementeren, óf een package gebruiken als 'BCrypt.net':

NuGet Package: bcrypt.net

<https://github.com/BcryptNet/bcrypt.net>

Voor het veilig hashen van wachtwoorden kunnen we deze **bcrypt.net** NuGet Package gebruiken. **Let bij het gebruiken van packages altijd wel op:**



1. **Wordt dit package onderhouden?**

Als er een recente commit is geweest, geeft dat een signaal dat de package onderhouden wordt. Hopelijk worden fouten erin dus opgelost.

2. **Wordt dit package door meer mensen gebruikt?**

Hoe meer ogen meekijken naar de broncode, hoe zekerder we kunnen zijn dat er geen kwaadwillende code in geplaatst is.

3. **Vertrouw ik de ontwikkelaars?**

Bekijk de hoofd 'contributors' en bepaal of je ze vertrouwt. Gebruiken ze hun echte naam, of werken ze aantoonbaar bij een groot en fatsoenlijk bedrijf?

Overleg in alle gevallen dat je een Package installeert met het ontwikkelteam én eventuele Senior Developers of jullie de package willen gebruiken. Vooral als het gaat om beveiliging wil je zeker weten dat je een degelijke package gebruikt.

Lees in de Package README.md hoe die gebruikt moet worden. Installeer het via NuGet in je project en volg de instructies in de 'Quick Start'. Let daarbij op:

1. Bij het registreren van een gebruiker (ook in de seeder) gebruik je de 'BCrypt.HashPassword'-methode om het wachtwoord te hashen.
2. Sla alleen de hash op in de gebruikerstabel (bij bcrypt bevat de hash bevat ook informatie over de salt en cost)
3. Wanneer een gebruiker inlogt haal je die gebruiker op uit de database.
4. Vergelijk het ingevoerde wachtwoord, met het uit de database opgehaalde hash doormiddel van de 'BCrypt.Verify'-methode.

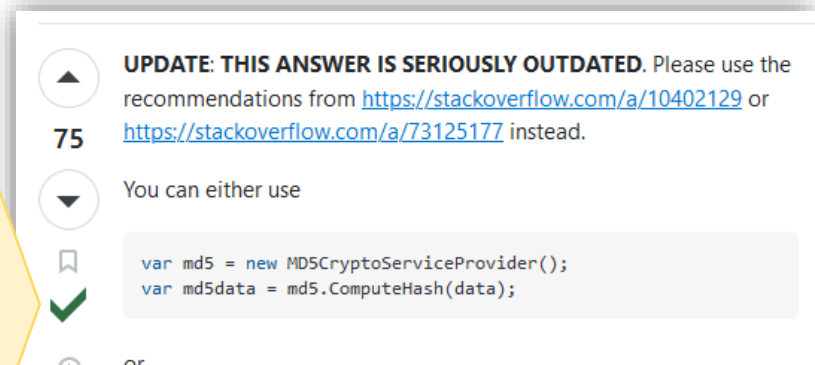
Nu is deze package en code voor je uitgezocht, maar in de toekomst moet je zelf op onderzoek uitgaan om code te schrijven of een juiste package te zoeken. Let dan op het volgende:

- Security is een constant kat-en-muisspel tussen Software Ontwikkelaars en hackers. **Onderzoek ieder project opnieuw of de code die je gebruikt nog veilig is.**

Zo zie je hier een StackOverflow antwoord wat in 2010 misschien prima was, maar inmiddels achterhaald is:

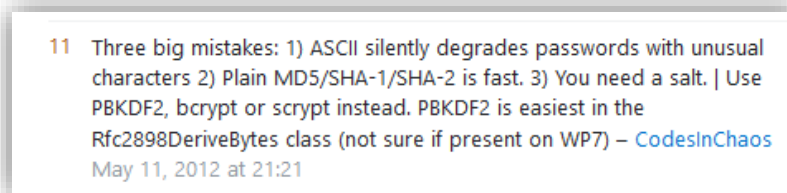
Ondanks dat dit antwoord achterhaald is, staat het nog steeds gemarkeerd als 'geaccepteerd antwoord'.

Dit terwijl het mogelijk onveilige code is.



- **Neem niet zomaar een StackOverflow antwoord over.** Raadpleeg de (Microsoft) documentatie, zodat je de code begrijpt. Soms wordt daar ook aangegeven of er betere manieren zijn om iets aan te pakken.
- **Lees ook het commentaar op StackOverflow.** Soms krijgt een onjuist/onveilig antwoord veel 'upvotes' of wordt deze zelfs geaccepteerd door de vrager. Door het commentaar te lezen kun je beoordelen of je de code kunt vertrouwen.

Voorbeeld van commentaar met extra informatie over een oud antwoord:



- **Let op!** Tijdens je examen maak je een 'proof-of-concept', maar niet qua beveiliging. Zorg dat je een package gebruikt als BCrypt.net, of dat je hashing + salting juist zelf implementeert.

7.6. 'Kijken onder de motorkap': SecureHasher-klasse

De BCrypt.net NuGet package scheelt ons een hoop werk en implementeert hashing op een veilige manier. Maar door de complexe broncode van die package is het wel een beetje een 'black box'. We stoppen er wachtwoorden in en krijgen er veilige hashes uit, maar snappen niet helemaal hoe het werkt. Vandaar dat we deze SecureHasher-klasse hebben. **Neem deze SecureHasher-klasse door zodat je een beeld krijgt hoe de BCrypt.net package ongeveer werkt.**

Op Itslearning kun je een download vinden van deze klasse met daarin meer commentaar en uitleg. Je kunt de klasse naar je project kopiëren.

```
1 using System;
2 using System.Security.Cryptography;
3
4 public static class SecureHasher
5 {
6     private const int saltSize = 16;
7     private const int keySize = 32;
8     private const char segmentDelimiter = '.';
9     private static readonly HashAlgorithmName algorithm = HashAlgorithmName.SHA512;
10
11     public static string Hash(string password, int iterations = 10000)
12     {
13         byte[] salt = RandomNumberGenerator.GetBytes(saltSize);
14         byte[] hash = Rfc2898DeriveBytes.Pbkdf2(
15             password,
16             salt,
17             iterations,
18             algorithm,
19             keySize
20         );
21
22         return string.Join(
23             segmentDelimiter,
24             Convert.ToBase64String(hash),
25             Convert.ToBase64String(salt),
26             iterations,
27             algorithm
28         );
29     }
30
31     public static bool Verify(string inputPassword, string hashedPassword)
32     {
33         string[] segments = hashedPassword.Split(segmentDelimiter);
34
35         byte[] hash = Convert.FromBase64String(segments[0]);
36         byte[] salt = Convert.FromBase64String(segments[1]);
37         int iterations = int.Parse(segments[2]);
38         HashAlgorithmName algorithm = new HashAlgorithmName(segments[3]);
39
40         byte[] inputHash = Rfc2898DeriveBytes.Pbkdf2(
41             inputPassword,
42             salt,
43             iterations,
44             algorithm,
45             hash.Length
46         );
47
48         return CryptographicOperations.FixedTimeEquals(inputHash, hash);
49     }
50 }
```

Dit geeft bijvoorbeeld een hash als deze:

*ymY4kjbcXDRKf+Fhot0yIhyG90F8mqfcgyD9HyE5+Do=:7iAiOs
BmIA/0oXdRNKNCCg==:10000:SHA512*

De hash, salt, iteraties en het algoritme worden in de Hash functie met "." aan elkaar geplakt. In de verify splitsen we op de "." om met die gegevens opnieuw een hash op te bouwen.

De SecureHasher-code is gebaseerd op deze StackOverflow antwoorden:

<https://stackoverflow.com/a/73125177> en <https://stackoverflow.com/a/10402129>

Om een wachtwoord te hashen gebruik je deze klasse als volgt:

```
1 string password = "uns4f3p4ssw0rd";
2 string hashed = SecureHasher.Hash(password);
```

Om een hash te controleren:

```
1 string enteredPassword = "uns4f3p4ssw0rd";
2 bool isPasswordCorrect = SecureHasher.Verify(enteredPassword, hashed);
```

7.7. Opdracht: GameBib (Deel 2)

Deze opdracht bouwt verder op de 'GameBib' opdracht aan het einde van hoofdstuk 5. We gaan daarin nu een gebruikersregistratie en login implementeren.

Instructies:

1. Zoek jouw code erbij van de GameBib opdracht uit hoofdstuk 5. Heb je die opdracht nog niet gemaakt of kun je de code niet meer vinden? Maak die opdracht dan eerst.
2. Voeg een model toe waarmee gebruikersgegevens kunnen worden opgeslagen. Het model moet tenminste deze attributen bevatten:
 - a. Username – Gebruikersnaam waarmee wordt ingelogd
 - b. HashedPassword – Gehashed password wat in database wordt opgeslagen
3. Installeer de BCrypt.net NuGet package, controleer goed dat je deze installeert:
<https://github.com/BcryptNet/bcrypt.net>
4. Maak een registratiescherm waar de gebruiker een account kan aanmaken.
 - a. Zorg daarbij dat het ingevoerd wachtwoord wordt gehashed op de manier die in de BCrypt.net README wordt beschreven.
5. Maak een loginscherm waar de gebruiker kan inloggen.
 - a. Gebruik de door BCrypt.net aangeleverde BCrypt.Verify-methode om het door de gebruiker ingevoerde wachtwoord te controleren met de hash in de database.
Let op: de volgorde van argumenten aan die methode is belangrijk. Lees de 'Quick Start' in de README dus goed.
6. Bewaar (na succesvolle login) de ingelogde gebruiker voor later gebruik binnen de code. Dit is hoe je dat kunt doen:
 - a. Voeg een statische eigenschap toe aan jouw User klasse. Hierin gaan we bij het inloggen de ingelogde gebruiker opslaan:

```
0 references
internal class User
{
    0 references
    public static User LoggedInUser { get; set; }
    0 references
    public int Id { get; set; }
    0 references
    public string Username { get; set; }
    0 references
    public string HashedPassword { get; set; }
```

- b. Vul die statische eigenschap, wanneer de gebruiker de juiste gegevens invoert.
Bijvoorbeeld:

```

using (var db = new ApplicationDbContext())
{
    var user = db.Users.First(u => u.Username == username);

    if (BCrypt.Verify(password, user.HashedPassword))
    {
        // De gebruiker heeft de juiste gebruikersnaam en wachtwoord combinatie ingevoerd
        User.LoggedInUser = user;
        // Hier navigeren we naar de volgende pagina
    }
}

```

- c. Als je hierna wilt weten of iemand ingelogd is controleer je:

```

if (User.LoggedInUser != null)
{
    // ingelogd
}

```

- d. Of als je de naam van de ingelogde gebruiker wilt laten zien:

```

usernameTextBox.Text = User.LoggedInUser.Username;

```

7. Pas de applicatie aan zodat er een many-to-many relatie is tussen Users en Games. Users moeten een game aan hun favorieten kunnen toevoegen.
Zoek de eerdere uitleg over many-to-many relaties op in hoofdstuk 5..

AUTORISATIE EN INLOGGEGEVENS BEWAREN

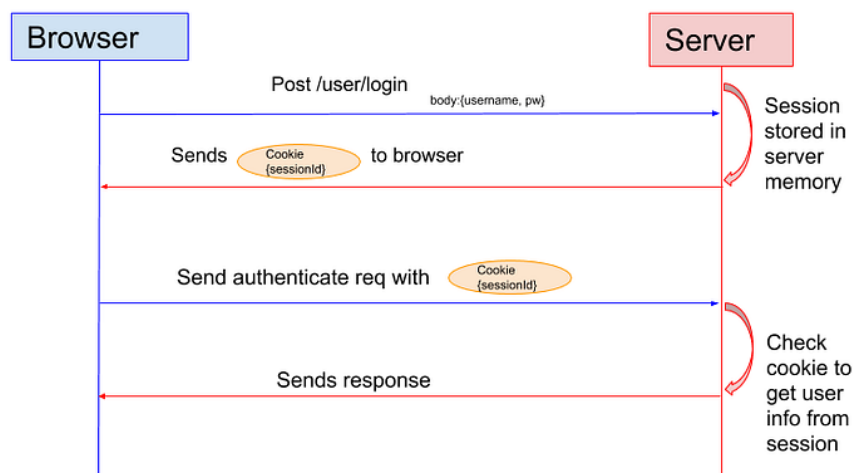
7.8. Sessies

In softwareontwikkeling spelen sessies een belangrijke rol. Een sessie begint wanneer een gebruiker inlogt in een applicatie en eindigt wanneer de gebruiker uitlogt of een tijdje niet actief is. Tijdens een sessie kan de applicatie informatie bijhouden over de gebruiker, zoals voorkeuren en status.

In deze paragrafen zullen we dieper ingaan op sessies en leren hoe ze gebruikt worden om gebruikersinteractie te beheren en de beveiliging te waarborgen in applicaties. We zullen kijken naar hoe sessies werken, het gebruik van session tokens en de beste manieren om sessies veilig te beheren.

7.9. Ingelogd blijven dankzij 'session tokens'

Wanneer een gebruiker succesvol is geauthentiseerd in een applicatie, kan het wenselijk zijn dat de gebruiker ingelogd blijft, zelfs als hij de webpagina of app afsluit. Toch willen we vanwege enorme beveiligingsrisico's niet de gebruikersnaam en het wachtwoord opslaan. Daarom blijven we meestal ingelogd dankzij session tokens.



Sequentiële diagram dat demonstreert hoe session tokens kunnen werken

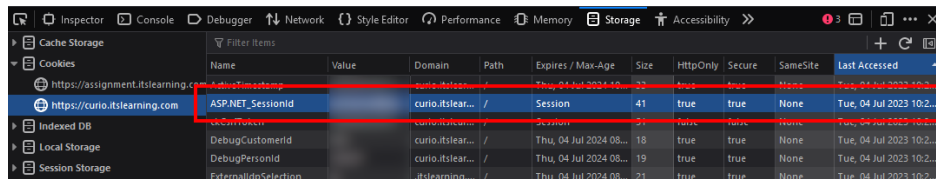
Bron: <https://sherryhsu.medium.com/session-vs-token-based-authentication-11a6c5ac45e4>

Session tokens werken vaak als volgt:

1. **Inloggen en session token genereren:** Wanneer een gebruiker inlogt met geldige inloggegevens, genereert de server een uniek session token dat aan de gebruiker wordt toegewezen. Dit session token wordt opgeslagen op de server en gekoppeld aan de bijbehorende gebruikerssessie.
2. **Opvangen en opslaan van session token:** Nadat het session token is gegenereerd, wordt het teruggestuurd naar de client, meestal via een HTTP-cookie of een ander mechanisme om gegevens op te slaan, zoals de lokale opslag in de browser of app. De client slaat het session token op, zodat het bij toekomstige verzoeken kan worden meegestuurd.
3. **Verificatie met session token:** Bij elk volgende verzoek van de gebruiker naar de server, wordt het session token meegestuurd als bewijs van de geldige gebruikerssessie. De server verifieert het session token om te controleren of de gebruiker nog steeds ingelogd is.
4. **Vernieuwen van session tokens:** Om de beveiliging te verbeteren, kunnen session tokens periodiek vernieuwd worden. Dit betekent dat bij elke interactie tussen de client en de server het oude session token wordt vervangen door een nieuw session token. Dit voorkomt dat kwaadwillende partijen een geldig session token onderscheppen en misbruiken.

5. **Uitloggen en verwijderen van session token:** Wanneer de gebruiker besluit uit te loggen of de sessie wordt beëindigd, wordt het session token verwijderd van de server en de client. Hierdoor kan de gebruiker niet langer toegang krijgen tot beschermde bronnen.

Je kunt session tokens vinden in de cookies van je browser.



Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure	SameSite	Last Accessed
ActiveTimeOutTemp		curio.itslear...	/	Thu, 04 Jul 2024 10:23:10Z	32	true	true	None	Tue, 04 Jul 2023 10:23:10Z
ASP.NET_SessionId		curio.itslear...	/	Session	41	true	true	None	Tue, 04 Jul 2023 10:23:10Z
AuthCookieToken		curio.itslear...	/	Session	31	true	true	None	Tue, 04 Jul 2023 10:23:10Z
DebugCustomerId		curio.itslear...	/	Thu, 04 Jul 2024 08:19:19Z	18	true	true	None	Tue, 04 Jul 2023 10:23:10Z
DebugPersonId		curio.itslear...	/	Thu, 04 Jul 2024 08:19:19Z	19	true	true	None	Tue, 04 Jul 2023 10:23:10Z
EdemalidoSelection		itslearning...	/	Thu, 04 Jul 2024 08:19:19Z	71	true	true	None	Tue, 04 Jul 2023 10:23:10Z

Voorbeeld van Itslearning session token opgeslagen in browser cookies.

Omdat session tokens dienen als vervanging voor jouw gebruikersnaam en wachtwoord is het enorm belangrijk dat je ze geheim houdt. *Doe je laptop daarom altijd op slot (Windows + L) als je wegloopt, gebruik een veilige pincode/wachtwoord/vingerafdruk en open geen programma's/bestanden die je niet vertrouwt, anders loop je risico dat iemand je cookies (en dus al je accounts) steelt.*

7.10. 'Session tokens' in C#

Laten we kijken naar een voorbeeld implementatie van session tokens in C#. We gaan het voornamelijk gebruiken om een inlogsessie te onthouden, vandaar dat we het hier een RememberToken noemen.

1. We voegen aan onze User klasse een RememberToken eigenschap toe:

```
11 references
internal class User
{
    4 references
    public static User LoggedInUser { get; set; }

    2 references
    public int Id { get; set; }

    3 references
    public string Username { get; set; }

    3 references
    public string HashedPassword { get; set; }

    0 references
    public string RememberToken { get; set; }
}
```

In de volgende stap gaan we bij het inloggen de RememberToken vullen en opslaan. We slaan het zowel in de database op als in een 'cookie' bestand. Dan kunnen we de volgende keer dat de applicatie opstart automatisch inloggen op basis van die token.

2. Een token is het een willekeurige reeks tekens. Als je op internet zoekt kom je veel oplossingen tegen die de 'Random'-klasse gebruiken. Maar dat is niet 'cryptografisch random', wat wil zeggen dat de waardes die de Random-klasse bedenkt te zijn voorspellen. Om die reden gebruiken we System.Security.Cryptography.RandomNumberGenerator. We hebben dat hier in een statische methode in User geplaatst:

```

public static string GenerateRememberToken()
{
    string characters = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789";
    int length = 10;
    char[] randomChars = new char[length];

    for (int i = 0; i < length; i++)
    {
        randomChars[i] = characters[RandomNumberGenerator.GetInt32(0, characters.Length)];
    }

    return new string(randomChars);
}

```

3. Nu we de token kunnen genereren gaan we dat doen na succesvol inloggen, daarbij slaan we de token op **in de database** en **in een tekstbestand**:

```

string username = usernameTextBox.Text;
string password = passwordTextBox.Text;

using (var connection = new AppDbContext())
{
    User user = connection.Users
        .Include(u => u.QuestUsers)
        .ThenInclude(qu => qu.Quest)
        .FirstOrDefault(u => u.Username == username);

    if (user != null && SecureHasher.Verify(password, user.HashedPassword))
    {
        // Genereer een token en sla die op bij de gebruiker
        user.RememberToken = User.GenerateRememberToken();

        // Sla de remember token ook op in een 'cookie' bestand
        StorageFolder storageFolder = ApplicationData.Current.LocalFolder;
        StorageFile cookieFile = await storageFolder.CreateFileAsync(
            "remember_token.txt",
            CreationCollisionOption.ReplaceExisting);

        await FileIO.WriteTextAsync(cookieFile, user.RememberToken);

        connection.SaveChanges();

        User.LoggedInUser = user;

        loginButton.Content = "Logout";
        usernameTextBox.IsEnabled = false;
        passwordTextBox.IsEnabled = false;
        passwordTextBox.Text = "";

        questsListView.ItemsSource = user.QuestUsers;
    }
}

```

4. Nu is de laatste stap, om bij het opstarten te bekijken:
- Is er een remember_token.txt bestand?
 - Kunnen we een gebruiker vinden met de token in dat bestand?
 - Zo ja, log die gebruiker dan in.

Dat kan er zo uitzien:

```
public MainWindow()
{
    this.InitializeComponent();

    TryLoadUser();
}

1 reference
async void TryLoadUser()
{
    using (var connection = new AppDbContext())
    {
        // Uitgecomment, omdat anders onze RememberToken verdwijnt bij het opnieuw aanmaken van de database
        //connection.Database.EnsureDeleted();
        //connection.Database.EnsureCreated();

        StorageFolder storageFolder = ApplicationData.Current.LocalFolder;

        try
        {
            var cookieFile = await storageFolder.GetFilesAsync("remember_token.txt");
            string rememberToken = await FileIO.ReadTextAsync(cookieFile);

            User user = connection.Users
                .Include(u => u.QuestUsers)
                .ThenInclude(qu => qu.Quest)
                .FirstOrDefault(u => u.RememberToken == rememberToken);

            if (user != null)
            {
                User.LoggedInUser = user;

                loginButton.Content = "Logout";
                usernameTextBox.IsEnabled = false;
                passwordTextBox.IsEnabled = false;

                questsListView.ItemsSource = user.QuestUsers;
            }
        } catch (FileNotFoundException) {
            // Geen cookie bestand gevonden, dus we hoeven niets te doen
        }
    }
}
```

7.11. Opdracht: GameBib (Deel 3)

Deze opdracht bouwt verder op de 'GameBib (Deel 2)' opdracht eerder in dit hoofdstuk. Daar hebben we al een loginsysteem gemaakt met gebruikersregistratie. Maar nu moet de gebruiker steeds opnieuw inloggen. We gaan daarom nu toevoegen dat de gebruiker ingelogd blijft, zelfs als de applicatie na sluiten opnieuw wordt opgestart. Dit gaan we implementeren door bij het inloggen een session token te genereren en die bij de gebruiker op de pc én in de database op te slaan.

Deze opdracht is extra uitdagend. Naast de verhoogde complexiteit gaan we je in deze opdracht zelf op onderzoek sturen. Lees de documentatie grondig en werk (en test) in kleine stappen. Veel succes!

Instructies:

1. Zoek jouw code erbij van de 'GameBib (Deel 2)' opdracht eerder in dit hoofdstuk. Heb je die opdracht nog niet gemaakt of kun je de code niet meer vinden? Maak die opdracht dan eerst.
2. Voeg aan de User model een plek toe om de session token op te slaan.
3. Genereer bij het inloggen een session/remember token, let daarbij op het volgende:
 - Een session token is een string met willekeurige tekens.
 - Gebruik geen oplossingen met de 'Random'-klasse, maar kies een 'Cryptographically Secure' oplossing. De resultaten van de Random-klasse zijn voorspelbaar en dus onveilig. Lees hier meer over de juiste oplossing: <https://jonathancrozier.com/blog/how-to-generate-a-cryptographically-secure-random-string-in-dot-net-with-c-sharp>

4. Sla de session token bij de User op in de database (via Entity Framework)
5. Sla de session token ook op in een bestand bij de gebruiker. Let op dat WinUI applicaties niet zomaar overal mogen lezen en opslaan. Zoek hier meer informatie: <https://learn.microsoft.com/en-us/windows/apps/design/app-settings/store-and-retrieve-app-data>
6. Controleer bij het opstarten van de applicatie of de session token is opgeslagen op de computer. Zo ja? Haal dan de gebruiker op uit de database, aan de hand van die session token. Is er geen token, of is die niet te vinden in de database? Toon dan het inlogscherf.

7.12. Verschillen met de praktijk

We vinden het belangrijk om te vermelden dat hoe we tot nu toe hebben gewerkt niet helemaal is hoe applicaties in de praktijk werken. Tot nu stonden de database en applicatie op hetzelfde apparaat (die van ons als ontwikkelaar). Dat is makkelijk tijdens ontwikkelen en het maakt de uitleg voor nu makkelijker. Later in deze module maken we een stap naar een externe database. Dat is ook vaak hoe het in de praktijk zal werken.

Ter illustratie enkele voorbeelden:

Restaurant applicatie voor restaurant met maar één locatie:

Serverinrichting:

De server is een computer met de laatste Windows updates. Er is een MySQL database op geïnstalleerd met behulp van Laragon. De server staat achter de keuken in een gekoeld server-hok

Client applicaties:

- De tablets van de bediening
- Kassa computers

Verbinding:

Via het lokale netwerk (over dezelfde router). De applicaties geven als hostname niet 'localhost', maar het lokale IP van de server (bijv.: 195.0.0.25)

Boekhoudt systeem voor meerdere bedrijven:

Serverinrichting:

Er draait een API console applicatie op een Windows server in een 'server-rack' in een datacentrum. De API is gemaakt in C# en is een ASP.NET MVC applicatie.

Client applicatie:

De computers van boekhouders bij verschillende bedrijven

Verbinding:

Over het internet. De applicatie krijgt toegang tot de gegevens via de API. Ieder bedrijf krijgt een eigen API-sleutel, zodat ze alleen toegang krijgen tot hun eigen gegevens.

We gaan in latere hoofdstukken leren hoe we via een API toegang kunnen verschaffen tot gegevens. Dat is tevens de aanbevolen manier van werken.

7.13. Autorisatie

Autorisatie verwijst naar het proces van het beheren van gebruikersrechten en het controleren van toegangsrechten tot bepaalde functies, gegevens of bronnen in een applicatie. Het doel van autorisatie is ervoor te zorgen dat alleen geautoriseerde gebruikers bepaalde acties kunnen uitvoeren of toegang hebben tot specifieke bronnen.

In de meest simpele vorm kun je autorisatie als volgt implementeren:

1. Bedenk de verschillende rollen, bijvoorbeeld:

Rol	Nummer	Mogelijkheden
Administrator	99	Kan alle gegevens inzien
Gebruiker	0	Kan alleen eigen gegevens inzien

2. Voeg een attribuut toe aan de User waarin we het nummer van de rol van de gebruiker plaatsen (bijv.: RoleId) . Zorg dat standaard de minst gevaarlijke rol (0, Gebruiker) wordt toegekend.
3. Controleer bij het uitvoeren van acties wat de rol is van de gebruiker.
4. Plaats de rollen in statische klasse om duidelijker te maken dat 99 de administrator is en 0 de gebruiker:

```
1 public static class Roles
2 {
3     public const int Admin = 99;
4     public const int User = 0;
5 }
```

We gebruiken hier 'const', omdat deze getallen nooit veranderen gedurende de uitvoer van het programma. C# kan hierdoor iets efficiënter werken.

Het controleren van de rol kan dan bijvoorbeeld zo:

```
1 if (user.RoleId == Roles.Admin)
2 {
3     Console.WriteLine("You are an admin!");
4 }
```

Wist je dat...

Het netjes is om (zoals in de code hierboven) getallen of teksten met een bijzondere betekenis, op te slaan in een constante eigenschap in een klasse of in een Enumeration.

Wanneer we dit niet doen kan de betekenis van een getal onduidelijk (en daarom onjuist) zijn:

```
1 if (user.RoleId == 99)
2 {
3     Console.WriteLine("You are an admin!");
4 }
```

Wat 99 hierboven betekent is niet meteen duidelijk. Dit wordt een 'magic number' genoemd.

Aanvullende bron: [https://en.wikipedia.org/wiki/Magic_number_\(programming\)](https://en.wikipedia.org/wiki/Magic_number_(programming))

Zo'n statische klasse met rollen erin is handig om 'magic numbers' te voorkomen. In de meeste talen is ook het concept 'enumeration' beschikbaar om precies dit op te lossen. Dezelfde code als hierboven, ziet er dan zo uit:

```
public enum Roles
{
    User = 0,
    Admin = 99,
}
```

De rest van de code, zoals de vergelijking, blijft hetzelfde.

7.14. Opdracht: GameBib (Deel 4: Rollen implementeren)

Deze opdracht bouwt verder op de 'GameBib (Deel 3)' opdracht eerder in dit hoofdstuk. Daar hebben we al een loginsysteem gemaakt met gebruikersregistratie en de login met behulp van een session token opgeslagen. Als kers op de taart gaan we nu autorisatie toevoegen. We gaan het zo maken dat een administrator de gegevens van alle gebruikers mag zien, maar gebruikers enkel hun eigen gegevens.

Instructies:

1. Zoek jouw code erbij van de eerdere GameBib opdracht (deel 3) eerder in dit hoofdstuk. Heb je die opdracht nog niet gemaakt of kun je de code niet meer vinden? Maak die opdracht dan eerst.
2. Pas het User model aan, zodat deze ook een attribuut heeft voor de rol.
3. Maak een statische klasse óf enumeration waarin je de rol nummers bijhoudt. (*zie theorie hierboven*)
4. Pas de functionaliteiten aan zodat de gegevens van alle gebruikers afgeschermd zijn. Gebruikers mogen alleen hun eigen gegevens inzien zoals bijvoorbeeld de username en favorieten.
5. Wanneer de gebruiker de administrator rol heeft mag deze alle gegevens zien. Maak hiervoor een scherm waar de admin alle gebruikers kan zien.

Inleveropdracht 7.15: GameBib Compleet

Je hebt in verschillende opdrachten de GameBib opdracht uit het feedbackmoment van hoofdstuk 5 uitgebreid. Controleer of de GameBib applicatie compleet is aan de hand van deze checklist. Lever de code ervan dan in op Itslearning.

Checklist:

Naast de eisen van het GameBib CRUD feedbackmoment in hoofdstuk 5:

- De gebruiker kan een account registreren.
- De gebruiker kan inloggen
- Gebruikers kunnen een game als favoriet markeren.
- De inlogsessie blijft bewaard nadat de applicatie is afgesloten.
- Gebruikers kunnen alleen hun eigen gegevens zien.
- Administratoren hebben inzage in alle gegevens van gebruikers.

(Zie voor meer informatie de theorie voorafgaand aan deze inleveropdracht)

Inleveren:

- Lever de gehele broncode in als .zip op Itslearning.
- Maak screenshots van alle schermen in de applicatie en lever naast de .zip in op Itslearning.
- Maak screenshots van de C# code van alle schermen en lever naast de .zip in op Itslearning

Veel succes met de opdracht. Game on!

8. Een API consumeren

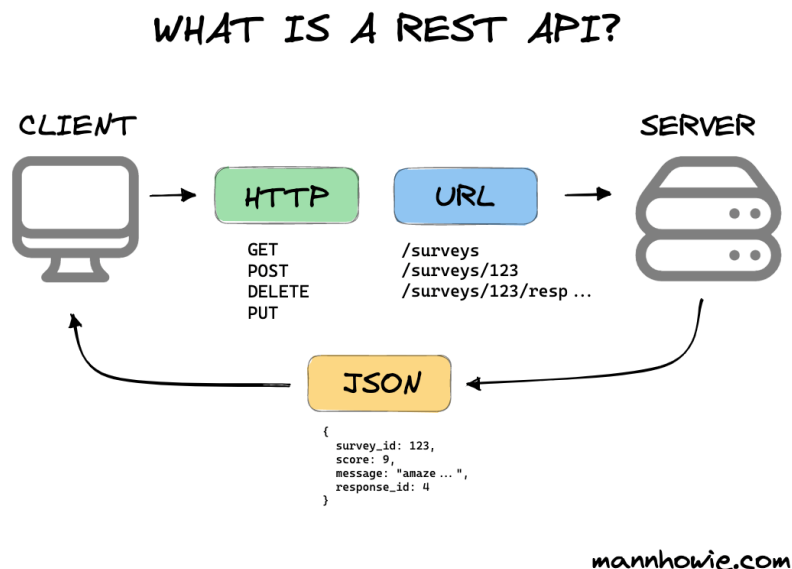
8.1. Wat is een API?

Een API, of **Application Programming Interface**, is een set van protocollen, routines en tools die softwareapplicaties in staat stellen met elkaar te communiceren.

In eenvoudigere woorden dient een API als een tussenpersoon tussen verschillende softwaresystemen, waardoor ze gegevens en instructies kunnen uitwisselen. Het definieert de regels en standaarden voor hoe applicaties met elkaar kunnen communiceren en specificeert de soorten verzoeken en antwoorden die zijn toegestaan.

De API is vaak actief op de **server** en de applicatie die de API aanroept noemen we de **client**. Een client kan een webpagina die *in Javascript* via *fetch* de API aanroept. Het kan ook een C# applicatie zijn die gebruik maakt van de HttpClient-klasse om de API aan te roepen.

Het "aanroepen van een API" is simpelweg het laden van een bijzondere webpagina. Wat de webpagina bijzonder maakt is dat deze geen HTML en CSS bevat, maar 'JSON' (of XML). Vrijwel alle programmeertalen en systemen kunnen JSON begrijpen en omzetten naar objecten in code. Daarmee kunnen we code laten reageren op gegevens die in JSON genoteerd zijn.



Bron: <https://mannhowie.com/rest-api>

In het bovenstaande plaatje zien we dat de **client** via de methodes GET, POST, DELETE of PUT een verzoek kan doen naar een bepaalde URL. Die URL is een weblocatie op een **server**. Er zijn verschillende URL's die verschillende acties uitvoeren we noemen dat ook wel **API routes** of **endpoints**.

Bijvoorbeeld:

Methode	Route / Endpoint	Beschrijving
GET	/surveys	Geeft alle enquêtes terug
GET	/surveys/123	Geeft de enquête terug met survey_id '123'
POST	/surveys	Voegt een enquête toe aan de server. De inhoud van het POST verzoek moet een JSON-object zijn met daarin de gegevens van de enquête.
DELETE	/surveys/123	Verwijdert de enquête met survey_id '123'

Een ander voorbeeld: stel dat je een weer-app ontwikkelt die informatie over de actuele weersomstandigheden nodig heeft. In plaats van je eigen meteorologische meetinstrumenten op

verschillende locaties in Nederland te installeren, kun je de API van een weerprovider gebruiken om de benodigde gegevens in realtime op te halen.

Door een API te gebruiken, kun je tijd en middelen besparen door de functionaliteit van bestaande systemen te benutten, en het kan ook de algehele prestaties en schaalbaarheid van je applicatie verbeteren.

8.2. Moderne systemen

Waar we in vorige hoofdstukken direct vanaf de client applicatie hebben verbonden met onze MySQL database, is dat niet gewoonlijk bij moderne systemen. In plaats van rechtstreeks verbinding te maken met de database, maakt de client applicatie een verzoek naar de API. De API fungeert vervolgens als een brug tussen de client applicatie en de database, waarbij het verzoek wordt ontvangen, verwerkt en doorgestuurd naar de database.

API's bieden verschillende voordelen voor moderne systemen. Ze zorgen voor een betere scheiding van verantwoordelijkheden, waarbij de database-logica wordt afgeschermd van de client applicatie. Dit bevordert de modulariteit en flexibiliteit van het systeem, waardoor onderdelen gemakkelijker kunnen worden gewijzigd of vervangen zonder de andere componenten te beïnvloeden.

Bovendien bieden API's een gestandaardiseerde manier van communicatie, waardoor verschillende applicaties kunnen samenwerken ongeacht de technologieën die ze gebruiken. Ze stellen ontwikkelaars in staat om op een consistente en voorspelbare manier gegevens op te halen, bij te werken of te verwijderen uit de database.

Daarnaast kunnen API's ook beveiligingsvoordelen bieden. Door een API als tussenlaag te gebruiken, kan toegangscontrole en autorisatie worden toegepast op basis van de gebruiker of de client applicatie. Dit helpt om ongeautoriseerde toegang tot de database te voorkomen en de gegevensintegriteit te waarborgen.

8.3. Voorbeelden

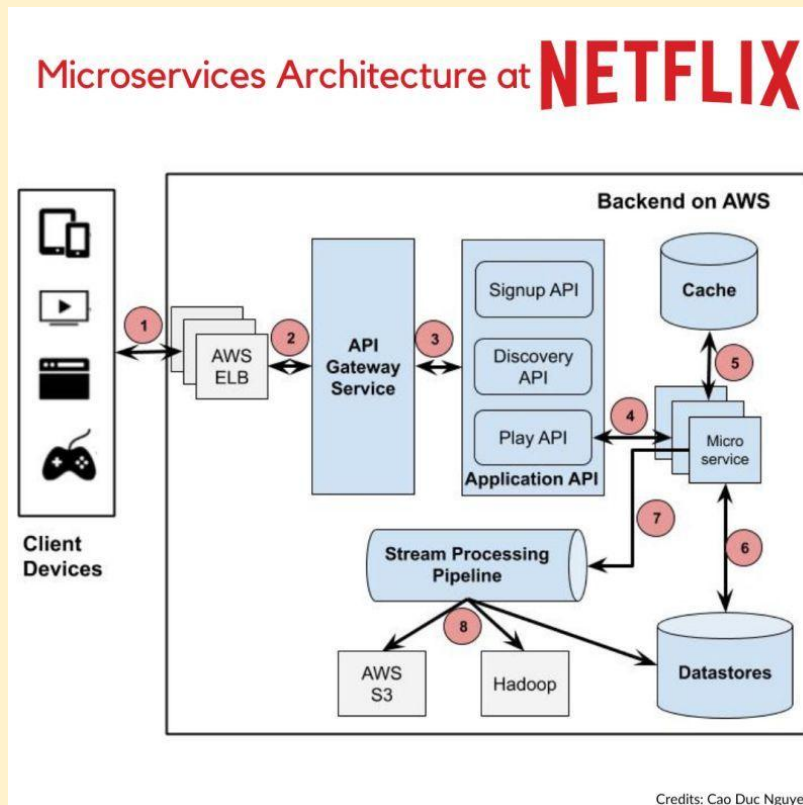
Netflix, Instagram en YouTube zijn populaire voorbeelden van moderne systemen die API's gebruiken om de communicatie tussen hun client applicaties en databases mogelijk te maken:

- **Netflix** maakt in de client applicatie, zoals de Netflix-website of mobiele app, gebruik van API's om gegevens op te halen, zoals gebruikersprofielen, aanbevelingen, film- en tv-showinformatie, enzovoort. Deze API's fungeren als een tussenlaag die de client applicatie verbindt met de backend-systemen van Netflix, waar de databases zich bevinden. Op deze manier kan de client applicatie de benodigde gegevens ophalen en presenteren aan de gebruikers.
- **Instagram**, maakt ook gebruik van API's om de interactie tussen de client applicatie (de Instagram-website of mobiele app) en de backend-databases mogelijk te maken. De API's van Instagram stellen de client applicatie in staat om functies uit te voeren zoals het ophalen en plaatsen van foto's, het volgen van andere gebruikers, het beheren van profielinformatie en het weergeven van nieuwsfeeds. Deze API's fungeren als een brug tussen de client applicatie en de databases van Instagram, waardoor gebruikersgegevens en -activiteiten efficiënt kunnen worden verwerkt en opgeslagen.
- **YouTube** maakt ook uitgebreid gebruik van API's. De API's van YouTube stellen ontwikkelaars in staat om functionaliteiten te integreren in hun eigen applicaties, zoals het zoeken naar video's, het uploaden van video's, het beheren van afspeellijsten, het ophalen van kanaalinformatie, het reageren op video's, enzovoort. Deze API's bieden een gestandaardiseerde manier voor ontwikkelaars om toegang te krijgen tot de enorme hoeveelheid video-inhoud en gebruikersgegevens die in de databases van YouTube worden opgeslagen.

Bonus: API's en microservices

Een bijzonder voorbeeld is Netflix, omdat dat gebruik maakt van 'microservices'. Netflix heeft voor ieder onderdeel van hun systeem een 'microservice' ontwikkeld, wat simpelweg **een kleine opzichzelfstaande API is die maar een paar bij elkaar horende functies uitvoert**.

Het registreren van een gebruikersaccount bij Netflix is een microservice. Wanneer het enorm druk is op de website worden automatisch meer kopieën van die microservice gestart, waardoor de servers niet overbelast raken. Ook maken deze microservices het makkelijk om te ontwikkelen, omdat ieder team bij Netflix aan hun eigen API kan werken, zonder daarbij code van andere teams in de weg te zitten. Netflix brengt via een 'API Gateway' de client applicaties naar de juiste dienst op de achtergrond.



Bron: <https://smartstudios.io/blog/serving-200-million-online-subscribers-the-netflix-way/>

Ondanks dat wij niet verder duiken in het concept van microservices is het waardevol om ten minste bekend te zijn met de term 'microservice'.



Wanneer je meer wilt weten over dit onderwerp kun je hier een interessante video vinden over de Netflix microservices, gegeven door een oud-medewerker van Netflix: <https://www.youtube.com/watch?v=CZ3wluvmHeM>

8.4. Een API aanroepen

We hebben nu een idee over wat een API is en waarom we het zouden gebruiken. Wanneer een client applicatie een API gebruikt noemen we dat ook wel 'consumeren' (*Engels: to consume*). Zoals eerder gezegd is het aanroepen van een API simpelweg het laden van een bijzondere webpagina. Laten we eens kijken naar een voorbeeld: **PokéAPI** (<https://pokeapi.co/>).

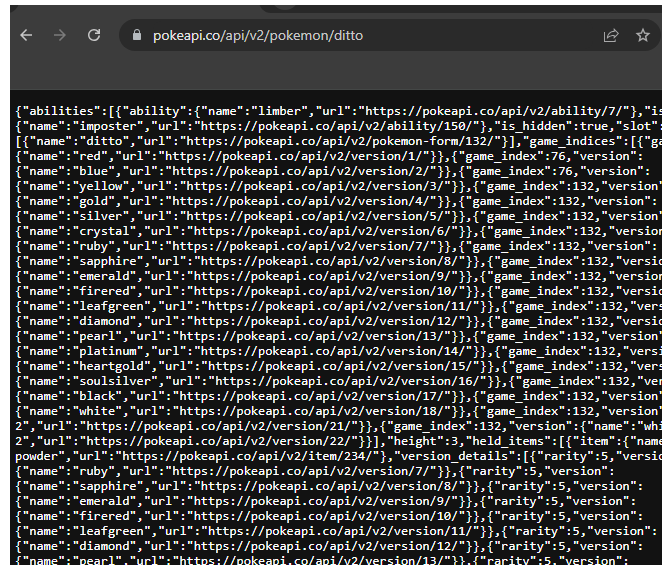
In onze lessen gebruiken we graag PokéAPI omdat het een gratis toegankelijke API is, waar geen bijzonderheden zijn qua authenticatie. Waar je bij de meeste



API's moet registreren kunnen we hier direct bij allerlei gegevens uit de Pokémon games.

Neem bijvoorbeeld deze API route: <https://pokeapi.co/api/v2/pokemon/ditto>

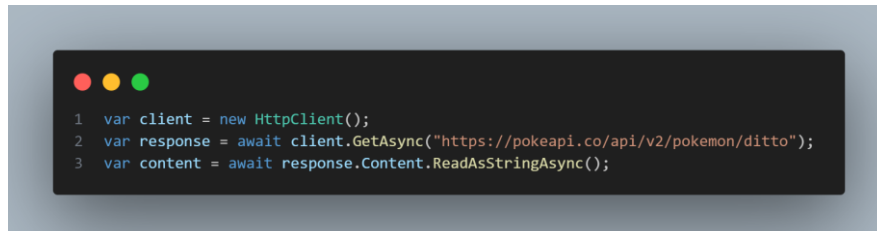
Wanneer je deze in de browser bezoekt zie je de volgende JSON:



```
{
  "abilities": [
    {
      "ability": {
        "name": "limber",
        "url": "https://pokeapi.co/api/v2/ability/7/"
      },
      "is_hidden": false,
      "slot": 1
    }
  ],
  "name": "ditto",
  "url": "https://pokeapi.co/api/v2/pokemon-form/132/",
  "game_indices": [
    {
      "game_index": 132,
      "version": "red"
    },
    {
      "game_index": 132,
      "version": "blue"
    },
    {
      "game_index": 132,
      "version": "yellow"
    },
    {
      "game_index": 132,
      "version": "gold"
    },
    {
      "game_index": 132,
      "version": "silver"
    },
    {
      "game_index": 132,
      "version": "crystal"
    },
    {
      "game_index": 132,
      "version": "ruby"
    },
    {
      "game_index": 132,
      "version": "sapphire"
    },
    {
      "game_index": 132,
      "version": "emerald"
    },
    {
      "game_index": 132,
      "version": "firered"
    },
    {
      "game_index": 132,
      "version": "leafgreen"
    },
    {
      "game_index": 132,
      "version": "diamond"
    },
    {
      "game_index": 132,
      "version": "pearl"
    },
    {
      "game_index": 132,
      "version": "platinum"
    },
    {
      "game_index": 132,
      "version": "heartgold"
    },
    {
      "game_index": 132,
      "version": "soulsilver"
    },
    {
      "game_index": 132,
      "version": "black"
    },
    {
      "game_index": 132,
      "version": "white"
    },
    {
      "game_index": 132,
      "version": "2"
    },
    {
      "game_index": 132,
      "version": "2"
    }
  ],
  "height": 13,
  "held_items": [
    {
      "item": {
        "name": "powder",
        "url": "https://pokeapi.co/api/v2/item/234/"
      },
      "rarity": 5,
      "version": "ruby"
    },
    {
      "item": {
        "name": "sapphire",
        "url": "https://pokeapi.co/api/v2/item/8/"
      },
      "rarity": 5,
      "version": "sapphire"
    },
    {
      "item": {
        "name": "emerald",
        "url": "https://pokeapi.co/api/v2/item/9/"
      },
      "rarity": 5,
      "version": "emerald"
    },
    {
      "item": {
        "name": "firered",
        "url": "https://pokeapi.co/api/v2/item/10/"
      },
      "rarity": 5,
      "version": "firered"
    },
    {
      "item": {
        "name": "leafgreen",
        "url": "https://pokeapi.co/api/v2/item/11/"
      },
      "rarity": 5,
      "version": "leafgreen"
    },
    {
      "item": {
        "name": "diamond",
        "url": "https://pokeapi.co/api/v2/item/12/"
      },
      "rarity": 5,
      "version": "diamond"
    },
    {
      "item": {
        "name": "pearl",
        "url": "https://pokeapi.co/api/v2/item/13/"
      },
      "rarity": 5,
      "version": "pearl"
    }
  ]
}
```

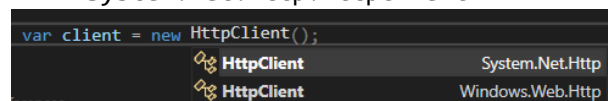
Sommige browsers stijlen JSON met extra opmaak en functies. Open de broncode van de pagina om te zien wat de API daadwerkelijk teruggeeft (alleen tekst)

Om een API te consumeren in C# gebruiken we de HttpClient-klasse. Dat geeft ons een soort onzichtbare browser waarmee we web verzoeken kunnen doen:



```
1 var client = new HttpClient();
2 var response = await client.GetAsync("https://pokeapi.co/api/v2/pokemon/ditto");
3 var content = await response.Content.ReadAsStringAsync();
```

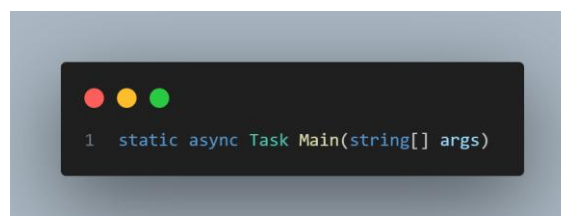
Er zijn twee versies, wij gebruiken 'System.Net.Http.HttpClient':



```
var client = new HttpClient();
```

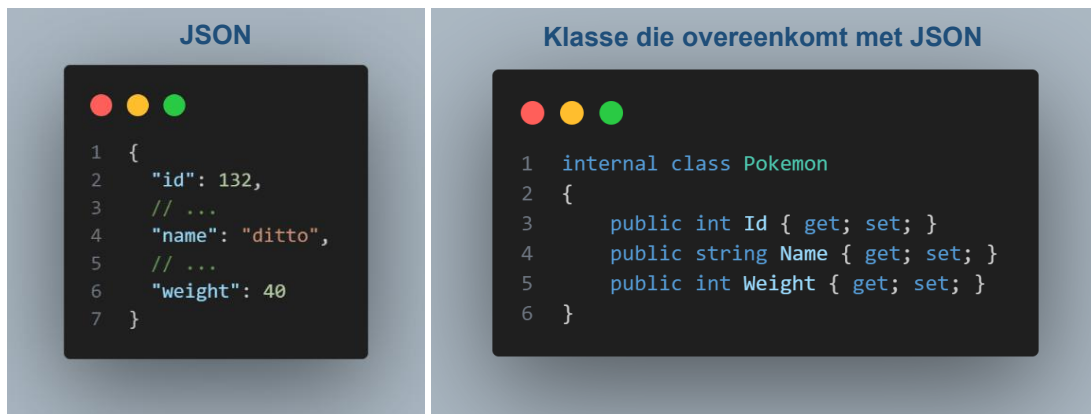
HttpClient (System.Net.Http)
HttpClient (Windows.Web.Http)

Omdat het doen van een web verzoek lang kan duren zijn enkele methodes hier **asynchroon (Async)**. Om die reden moet de methode waarin dit gebruikt wordt ook asynchroon zijn. Bij een Console App ziet een asynchrone 'Main' signature er zo uit:

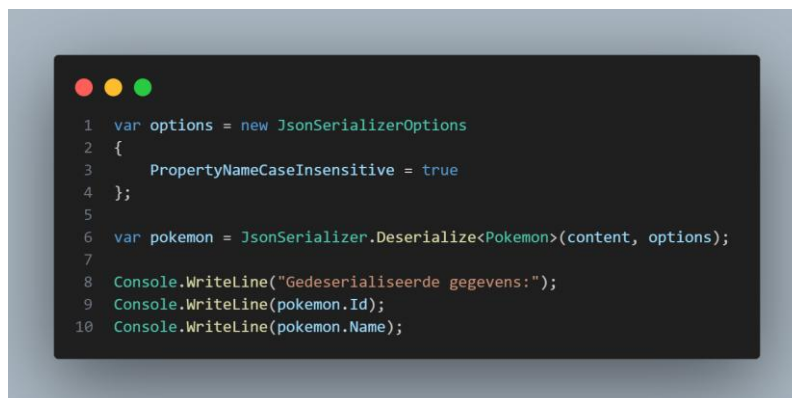


```
1 static async Task Main(string[] args)
```

Na de bovenstaande code staat in de 'content' variabele de JSON van de Pokémon genaamd "ditto". Dat JSON-object bevat allerlei informatie die we mogelijk willen ophalen, zoals bijvoorbeeld de id, naam en het gewicht van de Pokémon. Om die gegevens op te halen moeten we in C# een klasse maken die overeenkomt met de JSON:



Vervolgens kunnen we de JSON ‘deserialiseren’ naar een C# object. Met deserialiseren bedoelen we: van tekst naar een object. Serialiseren is het tegenovergestelde: van een object naar tekst. Deserialiseren gaat als volgt:



De functie om te (de)serialiseren werkt dankzij `using System.Text.Json`;

We geven aan de generieke methode **JsonSerializer.Deserialize<T>** het type van de klasse op de plek T. Zo weet de serializer welke eigenschappen er in de JSON verwacht kunnen worden. Omdat we de C# conventies voor eigenschappen willen aanhouden (PascalCase), maar in de JSON “id”, “name” en “weight” zonder hoofdletters zijn geschreven, moeten we ook een optie instellen om hoofdletterongevoeligheid aan te zetten:

```

var options = new JsonSerializerOptions
{
    PropertyNameCaseInsensitive = true
};

```

8.5. JSON en deserialiseren

JSON staat voor Javascript Object Notation en het is een gestructureerd datatransmissieformaat dat veel wordt gebruikt voor het uitwisselen van gegevens tussen een server en een client. Het is leesbaar voor zowel mensen als computers.

JSON is opgebouwd uit twee hoofdonderdelen: eigenschappen en waarden. Het gebruikt een structuur die lijkt op een dictionary.

Een JSON-object begint met een opening accolade '{' en eindigt met een sluitende accolade '}'. Tussen de accolades bevinden zich eigenschap-waarde-paren, gescheiden door een dubbele punt ':'.

Een eigenschap heeft een string naam en kan een waarde van verschillende typen bevatten, zoals een tekenreeks, een getal, een boolean, een array of zelfs een ander JSON-object.

Hier is een voorbeeld van een JSON-object:



```

1  {
2    "name": "John Doe",
3    "age": 25,
4    "married": false,
5    "hobbies": ["football", "reading", "traveling"],
6    "address": {
7      "street": "Main St",
8      "city": "New York",
9      "state": "NY"
10   }
11 }

```

In dit voorbeeld zijn er 5 eigenschappen:

- “name” heeft een string als waarde
- “age” heeft een getal als waarde
- De waarde van “married” is van het type een boolean
- “hobbies” heeft een array als waarde. Ieder item in deze array is van het type “string”
- “address” heeft een JSON-object als waarde, met 3 eigenschappen:
 - “street” heeft een string als waarde
 - Etc...

Zoals we zien herkennen we arrays in JSON aan blokhaken. Daarnaast is het mogelijk om doormiddel van ‘nesting’ JSON-object in JSON-objecten te hebben. Het is ook mogelijk om arrays in arrays, of arrays in JSON-objecten te plaatsen.

Wanneer je je een JSON-array als “hobbies” wilt deserialiseren geef je de eigenschap het type “List<T>”. In het geval van “hobbies” zou het best passende type voor T logischerwijs string zijn.

Wanneer een eigenschap weer een object is, moet daar een andere klasse voor gemaakt worden. Om de bovenstaande JSON te deserialiseren zouden de volgende klassen nodig zijn:



```

1  internal class Person
2  {
3      public string Name { get; set; }
4      public int Age { get; set; }
5      public bool Married { get; set; }
6      public List<string> Hobbies { get; set; }
7      public Address Address { get; set; }
8  }
9
10 internal class Address
11 {
12     public string Street { get; set; }
13     public string City { get; set; }
14     public string State { get; set; }
15 }

```

Om (weer met hoofdletterongevoeligheid) de JSON te deserialiseren gebruik je:

```

1 var personJson = "{\"name\":\"John Doe\",\"age\":25,\"married\":false}";
2
3 var options = new JsonSerializerOptions
4 {
5     PropertyNameCaseInsensitive = true
6 };
7
8 var person = JsonSerializer.Deserialize<Person>(personJson, options);
9
10 Console.WriteLine($"{person.Name} woont in {person.Address.City}");

```

Soms is de JSON die je wilt deserialiseren een array. Dan kun je direct aan de **JsonSerializer.Deserialize<T>** een List met gepast type geven. Voor een array met getallen gebruik je bijvoorbeeld:

```

1 var arrayJson = "[3, 4, 5]";
2 var numbers = JsonSerializer.Deserialize<List<int>>(arrayJson);
3
4 Console.WriteLine(numbers[1]); // 4

```

Als je geen van de methodes in List nodig hebt (zoals toevoegen en zoeken van items). Dan werkt in plaats van een List een array ook als type:

```

1 var numbers = JsonSerializer.Deserialize<int[]>(arrayJson);

```

8.6. Een test API

Wanneer je een client wilt bouwen, maar nog geen API hebt zijn er verschillende manieren om een tijdelijke nep-API op te zetten. Zo'n nep-API geeft simpelweg JSON als antwoord, zonder dat het echt aan een database is gekoppeld. We noemen dat 'mocking'. Dit zijn enkele manieren om een nep-API op te zetten:

1. Je kunt een simpel PHP-script schrijven om JSON terug te geven:

```

1 <?php
2
3 echo '['
4     . '{"id":1,"name":"Curio","postalCode":"1234AB"},'
5     . '{"id":2,"name":"Hizmet","postalCode":"4814AA"}'
6     . ']' ;

```

2. Je kunt gebruik maken van een online tool om mock-data terug te geven:

- a. <https://mocki.io/fake-json-api>
- b. <https://jsonplaceholder.typicode.com/>
- c. <https://mockend.com/>
- d. <https://mockapi.io/>

Als we optie 1 gebruiken hierboven en die via Laragon als 'test.php' beschikbaar stellen, dan krijgen we een nep-API route 'http://localhost/test.php':

```
[{"id":1,"name":"Curio","postalCode":"1234AB"}, {"id":2,"name":"Hizmet","postalCode":"4814AA"}]
```

Die route kunnen we in C# gebruiken om onze client te testen.

8.7. Opdracht: Gegevens ophalen en tonen

Laten we het ophalen van JSON-gegevens eens oefenen. We gaan nog geen echte API gebruiken, maar oefenen met test-gegevens die we aanbieden via een nep-API doormiddel van 'mocking'.

Instructies:

7. Bekijk nogmaals de bovenstaande opties over 'mocking'. Zet een nep-API op via een van die manieren. Bied de volgende JSON-testgegevens aan:

```
[
  { "id": 1, "name": "Curio", "postalCode": "1234AB" },
  { "id": 2, "name": "Hizmet", "postalCode": "4814AA" }
]
```

8. Start een nieuw Console Applicatie (.NET 7.0) project
9. Maak de benodigde klassen om de bovenstaande JSON te deserialiseren.
10. Haal met behulp van de HttpClient-klasse de JSON-testgegevens op.
11. Deserialiseer de JSON-gegevens naar C# objecten.
12. Laat de gegevens op deze manier in de console zien:

```
[#1 Curio]
1234AB

[#2 Hizmet]
4814AA
```

Gebruik een foreach loop om door alle bedrijven in de List/array te lopen.

Inleveropdracht 8.8: Een API-Client (waarin de gebruiker kan zoeken) (tijdsduur: +/- 60 minuten)

Ontwikkel een WinUI applicatie die gegevens ophaalt van een API (bijvoorbeeld een mock-API of een echte publieke API, zoals pokeapi.co). Implementeer functionaliteiten waarmee de gebruiker kan zoeken naar specifieke items en de resultaten kan filteren op basis van bepaalde eigenschappen. De resultaten moeten overzichtelijk in de console worden weergegeven.

Deze opdracht bouwt ook verder op wat je in de hoofdstukken 3 t/m 5 al geleerd hebt. Zo kun je bijvoorbeeld de LINQ-methode 'Where' gebruiken, om door alle, uit de API opgehaalde, items te zoeken.

Vereisten:

- Maak dit als WinUI applicatie
- Zorg dat ten minste 2 routes van de gekozen API worden geconsumeerd om functionaliteiten te ontwikkelen. Bijvoorbeeld bij PokeAPI:
 - Alle Pokémon: <https://pokeapi.co/api/v2/pokemon?limit=2000>
 - Informatie over een specifieke Pokémon: <https://pokeapi.co/api/v2/pokemon/zapdos>
- In de code bouw je modellen die overeenkomen met de JSON-structuur van de API. Deserialiseer de JSON via deze modellen met **JsonSerializer.Deserialize<T>**.
Bijv.: var allPokemon = JsonSerializer.Deserialize<List<Pokemon>>(jsonFromApi);
- Je kunt gegevens die je hebt opgehaald in de ItemsSource zetten van een ListView. Dat werkt zoals je gewend bent.
- Gebruik de ItemClick event op de ListView om de informatie over een specifieke Pokémon op te halen en te tonen.
- Zorg dat de gebruiker kan zoeken naar items, bijvoorbeeld op de naam van de Pokémon.
 - Gebruik 'Where' om door de opgehaalde items te zoeken, bijvoorbeeld:
`var search = "pika";
var foundPokemon = allPokemon.Where(p => p.Name.Contains(search));`
- Zorg dat de gebruiker meer details kan zien over een item, bijvoorbeeld van een Pokémon: hoogte, gewicht en sprites.
Zie hier hoe je een URL van een afbeelding kunt gebruiken om de afbeelding in WinUI te tonen:
<https://stackoverflow.com/a/11267358>
- **Bonus:** Laat de gebruiker filteren op een vaste set eigenschappen, bijvoorbeeld het type van de Pokémon

Inleveren:

- Maak screenshots van de code die verzoeken doet aan de API.
- Maak een .zip-bestand van de gehele broncode van de API.
- Lever de screenshots en .zip in op Itslearning.

Veel succes met de opdracht!

9. Eigen webserver en API bouwen

Om een API te kunnen aanbieden moeten we een webserver opzetten. Die gaat op verzoeken reageren met JSON-antwoorden.

In Blok B heb jij bij WDV-III en DBS-I geleerd over front-end, back-end en de request/response cyclus. Laten we de theorie daaruit nog eens kort herhalen.

9.1. Front-end / backend (lagen van een webapp)

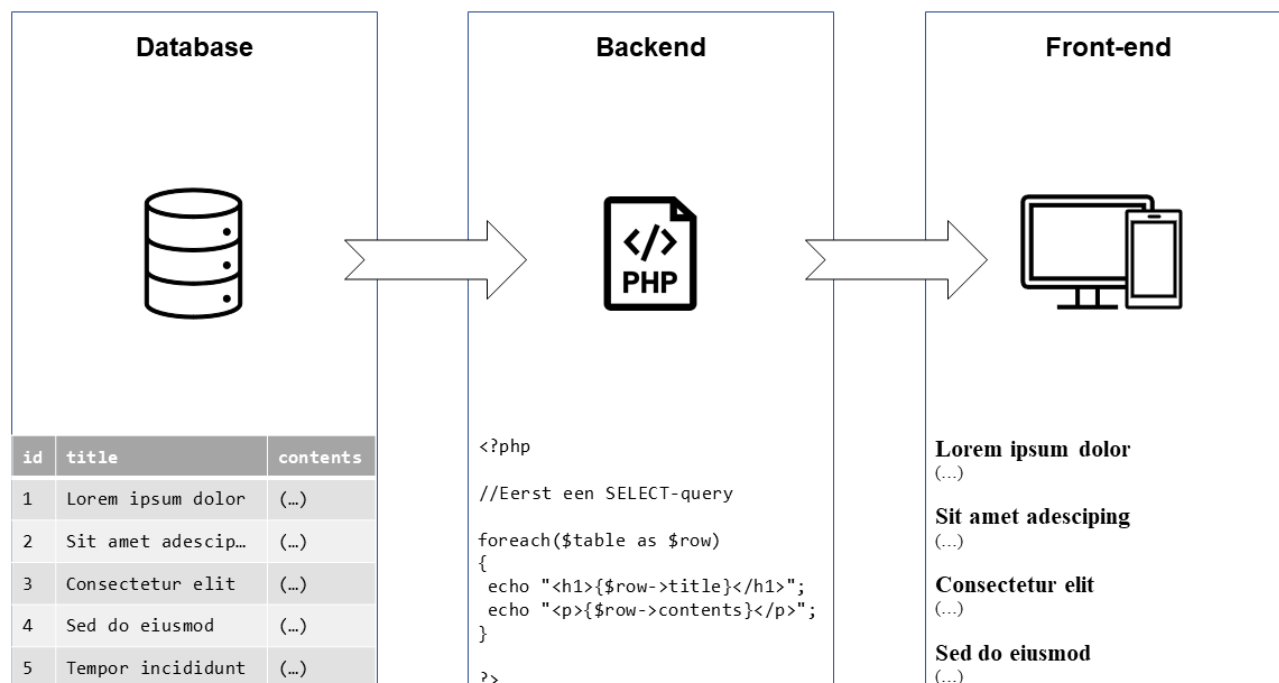
Het bouwen van een webapp werkt iets anders dan een lokale Windowsapplicatie met bijvoorbeeld Python of C#. In die laatste programmeertalen staat je héle programma op de computer van de gebruiker. Alle code (of de gecompileerde versie daarvan) werkt dus eigenlijk lokaal.

Bij een webapp is dat anders; het back-end gedeelte (bijv. de PHP code of het C# server programma) staat alleen op de server. De gebruiker krijgt alleen de front-end op zijn computer, de HTML en CSS dus, of in het geval van een API: de JSON.

Kernbegrip: Lagen van een webapp (front-end / backend / database)

Een applicatie bestaat vaak uit drie “lagen”. Bij een webapp zijn die lagen ook echt van elkaar gescheiden (dat is bij Windows-apps bijvoorbeeld niet zo):

- **Database:** tabellen met gegevens.
- **Backend:** Het “brein” van de applicatie. De backend haalt informatie uit de database (bijvoorbeeld alle nieuwsberichten) en maakt daar HTML-pagina's of JSON-antwoorden van. De backend zorgt ook voor het opslaan van nieuwe gegevens. De back-end is de baas, de front-end doet alleen verzoeken (die door de back-end gecontroleert worden)
- **Front-end:** wat de gebruiker ziet in de browser, HTML en CSS die voor een deel gegenereerd zijn door het server programma. Of een JSON-antwoord in het geval van een API.



Je back-end programma genereert dus eigenlijk een stuk van de HTML, wanneer een gebruiker je webapp bezoekt. De server (backend) zet de hele pagina in elkaar en stuurt die naar de gebruiker toe, die op zijn eigen computer de front-end kan bekijken.

Daarnaast regelt de backend ook het invoegen, aanpassen en verwijderen van items. Dat gebeurt natuurlijk altijd op aanvraag van een gebruiker. Ook het bouwen van een HTML-pagina gebeurt pas wanneer een gebruiker je site bezoekt. Er gaat altijd een “request” aan vooraf.

9.2. Request / response (hoe het internet werkt)

Als je een website bezoekt doet je browser dus eigenlijk een *request* naar de server waar de website staat. Die server antwoordt door de webpagina op te sturen; we noemen dat een *response*. Het gebruiken van een website levert een hele reeks van die *requests* en *responses* op. Iedere link die je aanklikt en ieder formulier dat je verstuurt zorgen voor een nieuw verzoek naar de server.

Kernbegrip 5: Request/response-cycle

De **request/response-cycle** verklaart wat er op de achtergrond gebeurt als je een website bezoekt. De browser doet een verzoek (*request*) naar de server, waarop de server een antwoord stuurt (*response*).

Iedere actie op een website leidt tot een nieuwe *request* (denk aan; link aanklikken, formulier versturen).

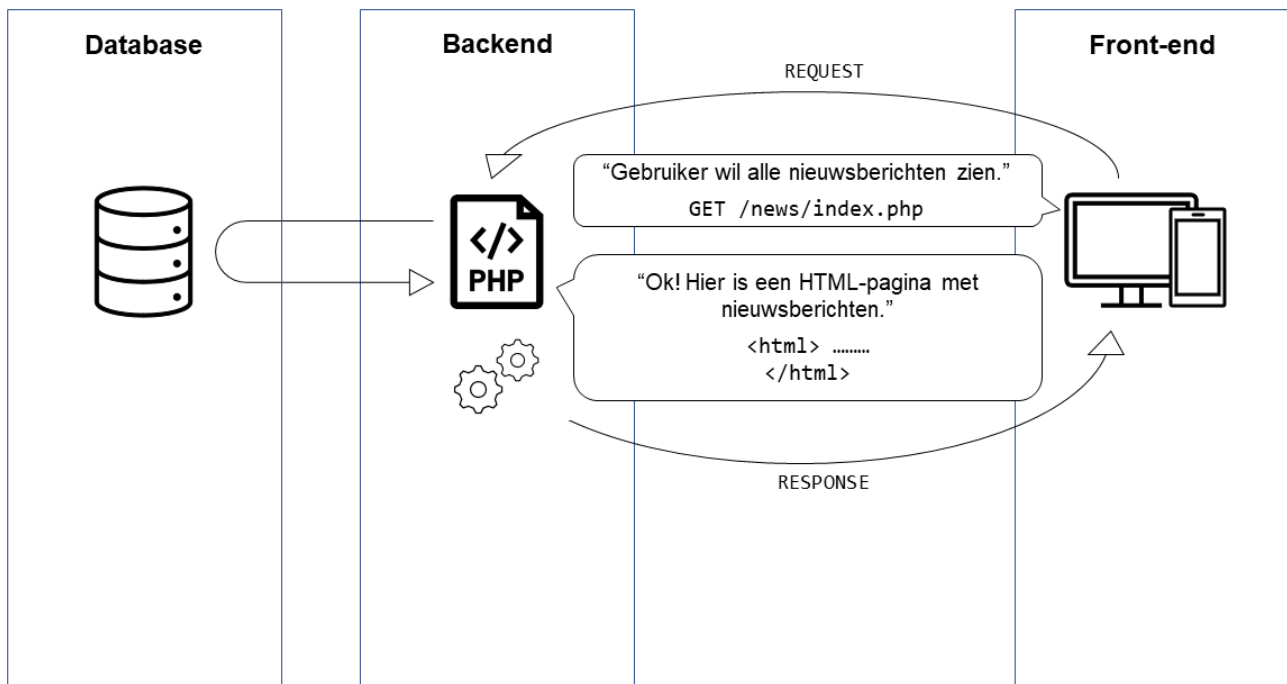
- **Request:** een verzoek of opdracht aan de server. Er zijn twee soorten *requests*:
 - GET: het verzoek om een pagina te tonen. Bijvoorbeeld GET /users/index.php zal een overzicht van alle gebruikers opvragen.
 - POST: het versturen van een formulier, inclusief gegevens. Bijvoorbeeld POST /backend/loginController.php ['user'=>'example', 'password' => '****'] zal de *request* zijn die je browser doet nadat je op “login” drukt onderaan een formulier.
- **Response:** na een verzoek gaat de server aan de slag. Bijvoorbeeld om alle gebruikers uit de database te halen en in een HTML-lijst te zetten, of om de gebruiker in te loggen. Daarna stuurt de server een antwoord terug. Die *response* kan een stuk HTML zijn, maar ook een *redirect* naar een andere pagina.

Na een *request* volgt altijd een *response*, daarom noemen we het een cyclus. Dit is de basis van het **HTTP-protocol**. Je moet nog weten dat de server je niet onthoudt; wanneer je een tweede *request* doet heeft de server geen idee dat jij dezelfde persoon bent als van het eerste verzoek. We zeggen ook wel dat HTTP een “**stateless**” protocol is (technieken die wel onthouden wie je bent, noemen we *stateful*).

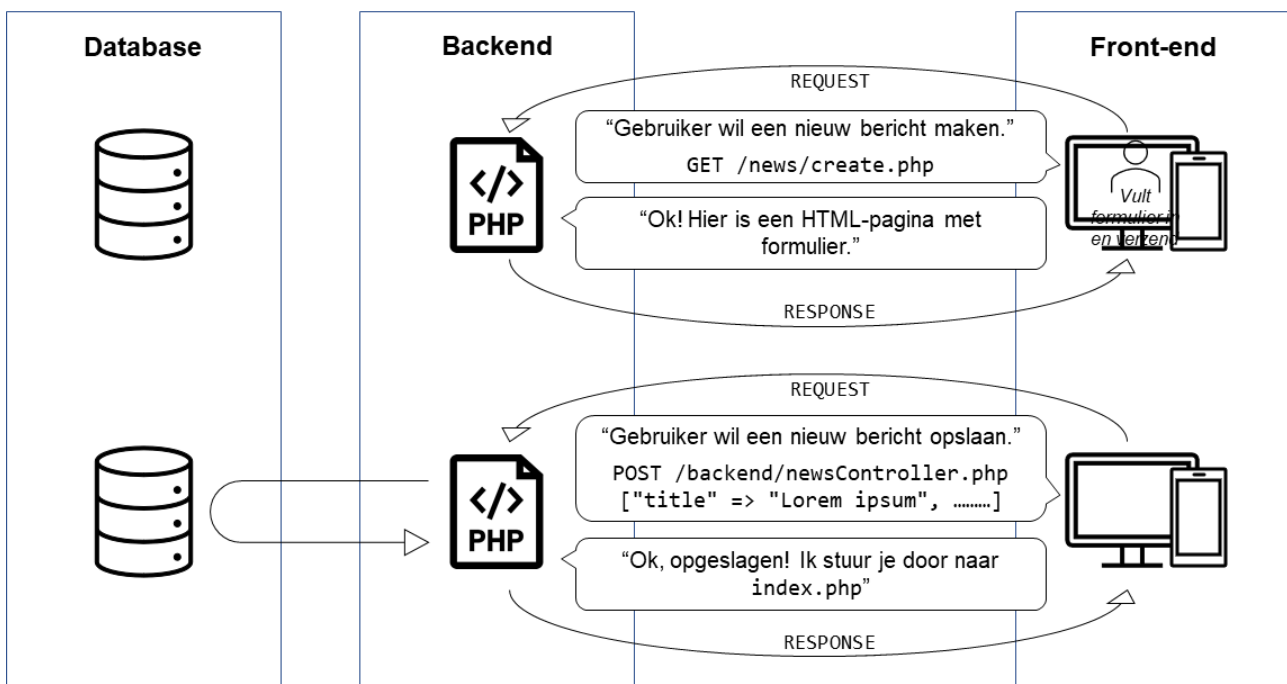
Deze kennisclip legt het principe nog eens uit met animaties erbij: <https://youtu.be/bFMkDEPDkDo>.

Op onderstaande afbeelding zie je de cyclus duidelijk terug:

- De computer vraagt om een pagina te zien (*request*).
- De server doet wat slimme dingen (query naar de database, PHP-code uitvoeren).
- De server stuurt als antwoord een stuk HTML terug (*response*).



In deze afbeelding zie je hoe er ook vaak twee cycli op elkaar volgen: eerst vraagt de computer om de pagina met een formulier, daarna vult de gebruiker het formulier in en verstuurt dat. Het versturen van het formulier is de tweede cyclus die je ziet.



9.3. Bestaande webserversoftware

Voordat we zelf een webserver gaan bouwen is het handig om te kijken welke bestaande oplossingen er zijn:

- **Apache HTTP Server**
 - Oudste en meest gebruikte webserver.
 - Open-sourceproject beheerd door de Apache Software Foundation.
 - Flexibel, krachtig en ondersteunt verschillende modules.

- Ondersteunt diverse programmeringstalen zoals PHP, Python, en Perl.
 - Draait op verschillende besturingssystemen (Unix/Linux, MacOS, Windows).
 - Ideaal voor zowel statische als dynamische webpagina's.
 - Vaak gebruikt in combinatie met MySQL en PHP als de LAMP-stack (Linux, Apache, MySQL, PHP).
 - Dit is de webserver die gebruikt wordt in ontwikkeltools zoals Laragon of XAMPP.
- **NGINX**
 - Populaire open-source webserver, reverse proxy, load balancer en HTTP cache.
 - Gericht op hoge prestaties, stabiliteit en laag geheugengebruik.
 - Efficiënt beheer van gelijktijdige verbindingen, geschikt voor drukke websites.
 - Ontworpen om de C10K-problematiek aan te pakken (beheer van tienduizenden gelijktijdige verbindingen).
 - Ondersteunt diverse platforms, waaronder Linux, MacOS, en Windows.
 - Vaak gebruikt als front-end proxy voor Apache-servers om efficiëntie te verbeteren.
- **Microsoft Internet Information Services (IIS)**
 - Internet gebaseerde diensten voor servers op Microsoft Windows.
 - Flexibele, beveiligde en beheerbare webserver.
 - Ondersteunt ASP.NET en andere talen zoals PHP.
 - Geïntegreerd in Windows Server, beschikbaar voor sommige niet-server Windows-versies.
 - Biedt een grafische gebruikersinterface voor websitebeheer.

Bij alle drie die webserver zijn er overeenkomsten hoe ze in de basis werken. Laten we eens kijken vanuit hoe Apache het noemt, en hoe dat werkt bij de andere webserver:

- **Virtual Hosts:**
 - *Apache HTTP Server:* In Apache worden Virtual Hosts gebruikt om meerdere domeinen of websites op één server te hosten. Dit wordt meestal bereikt met de VirtualHost-configuratieblokken in de Apache-configuratiebestanden, waarin specifieke domeinen en de bijbehorende documentroot en instellingen worden gespecificeerd.

```

<VirtualHost *:80>
# The ServerName directive sets the request scheme, hostname and port that
# the server uses to identify itself. This is used when creating
# redirection URLs. In the context of virtual hosts, the ServerName
# specifies what hostname must appear in the request's Host: header to
# match this virtual host. For the default virtual host (this file) this
# value is not decisive as it is used as a last resort host regardless.
# However, you must set it for any further virtual host explicitly.
#ServerName www.example.com

ServerAdmin webmaster@ostechnix1.lan
ServerName ostechnix1.lan
ServerAlias www.ostechnix1.lan
DocumentRoot /var/www/html/ostechnix1.lan/public_html

# Available loglevels: trace8, ..., trace1, debug, info, notice, warn,
# error, crit, alert, emerg.
# It is also possible to configure the loglevel for particular
# modules, e.g.
#LogLevel info ssl:warn

ErrorLog ${APACHE_LOG_DIR}/error.log
CustomLog ${APACHE_LOG_DIR}/access.log combined

# For most configuration files from conf-available/, which are
# enabled or disabled at a global level, it is possible to
# include a line for only one particular virtual host. For example the
# following line enables the CGI configuration for this host only
# after it has been globally disabled with "a2disconf".
#Include conf-available/serve-cgi-bin.conf
</VirtualHost>

```

- **NGINX:** In NGINX wordt het concept van Virtual Hosts bereikt met behulp van de server-blokken in de configuratiebestanden. Elk server-blok definieert de configuratie voor een specifieke website of domein.
- **IIS:** In IIS wordt het concept van Virtual Hosts ook wel "Site" genoemd. Elke website wordt als een afzonderlijke site geconfigureerd, met zijn eigen bindings naar poorten en IP-adressen. Dit stelt IIS in staat om meerdere websites op dezelfde server te hosten.

- **Route Afhandeling:**

- **Apache HTTP Server:** Apache maakt gebruik van het mod_rewrite-module om route afhandeling uit te voeren. Hiermee kunnen ontwikkelaars URL-rewriting en routingregels definiëren om verzoeken om te leiden op basis van bepaalde patronen of voorwaarden.
- **NGINX:** NGINX maakt gebruik van location-blokken in de configuratie om route afhandeling te specificeren. Hiermee kunnen ontwikkelaars URL-patronen matchen en de bijbehorende acties, zoals omleidingen of proxy-passing, definiëren.

```

root /var/www/html;

# Add index.php to the list if you are using PHP
index index.html index.htm index.nginx-debian.html;

server_name _;

location / {
    # First attempt to serve request as file, then
    # as directory, then fall back to displaying a 404.
    try_files $uri $uri/ =404;
}

location /images/ {
    root /data;
}

```

- **IIS:** In IIS kunnen ontwikkelaars route afhandeling bereiken door gebruik te maken van de URL-rewrite module. Dit biedt mogelijkheden voor URL-rewriting en route-omleidingen op basis van aangepaste regels.

9.4. Wat je nodig hebt om een eigen webserver te schrijven in C#

Allereerst moet je je ervan bewust zijn dat we onze webserver als Console App gaan bouwen. Dit doen we om onze applicatie zo efficiënt mogelijk te houden. We hoeven geen mooie interface voor de webserver zelf. De administrator die de webserver aan zet, zal simpelweg instellen dat de webserver Console App bij het opstarten van de server opstart.

Dankzij de ingebouwde netwerkfunctionaliteiten in Windows is het redelijk toegankelijk om een eigen webserver te schrijven in C#. We kunnen de **System.Net.HttpListener** gebruiken:

<https://learn.microsoft.com/en-us/dotnet/api/system.net.httplistener?view=net-7.0>

Die **HttpListener** gaat zoals de naam al beschrijft: luisteren naar HTTP verzoeken. We stellen een adres en poort in waarop deze luistert en starten de listener:

```
using var listener = new HttpListener();
listener.Prefixes.Add("http://localhost:8080/");
listener.Start();
```

Vanaf dan kunnen we opvragen of er een HTTP verzoek is binnengekomen op de poort waar we naar luisteren:

```
HttpListenerContext context = listener.GetContext();
HttpListenerRequest request = context.Request;
HttpListenerResponse response = context.Response;
```

De **.GetContext()** methode is bijzonder hier, want de applicatie blijft in de **GetContext**-methode wachten totdat er een verzoek is binnengekomen. Als je een breakpoint zet na die regel merk je dat die pas wordt bereikt wanneer een verzoek binnenkomt. *Dat werkt net zoals Console.ReadLine() die wacht totdat de gebruiker iets getypt heeft en op enter drukt.*

Wanneer er een verzoek is binnengekomen kunnen we uit de **Request** en **Response** van de **HttpListenerContext** informatie halen over het verzoek en het antwoord opbouwen en versturen:

Een verzoek beantwoorden:

```
// Zet de string om naar rauwe bytes
byte[] buffer = System.Text.Encoding.UTF8.GetBytes(responseString);

// We moeten in het antwoordpakket aangeven hoe lang ons antwoord is
response.ContentLength64 = buffer.Length;

// Schrijf je antwoord naar de 'OutputStream' in het antwoordpakket. Dit is een
// soort tunnel waardoor je het antwoord kunt schrijven naar de client.
Stream output = response.OutputStream;
output.Write(buffer, 0, buffer.Length);

// Klaar met schrijven? Sluit dan de tunnel.
output.Close();
```

In de **HttpListenerContext Response** zit een **"OutputStream"**. In C# kom je de term **Stream** (in het Nederlands 'stroom' zoals in waterstroom) tegen op deze plekken:

- Schrijven of lezen naar bestanden
- Schrijven of lezen van/naar computers over een netwerk (zoals het internet)

Je kunt een Stream zien als een tunnel naar een eindadres. Je moet eerst door de tunnel roepen hoe lang het bericht is (`response.ContentLength64`) en vervolgens het bericht byte-voor-byte erdoor schrijven. We kunnen tekst niet zomaar sturen, maar moeten het dus omzetten naar bytes.

Als de ontvanger aan de andere kant van de Stream tunnel de gegevens gaat lezen weet die dankzij de `ContentLength64` hoe lang het bericht is. Zodra alle bytes binnen zijn kan die dan het bericht weer opbouwen tot het originele type. Als het een string was bijvoorbeeld tot een string met:

```
string textAgain = System.Text.Encoding.UTF8.GetString(buffer);
```

Extra: Tekst op een computer via 'Encoding':

De onderstaande video legt uit hoe computers tekst representeren door gebruik te maken van tekstencodings. **Unicode** is een standaard die tekens een unieke code (codepoint) toewijst, zodat dezelfde tekens wereldwijd consistent worden weergegeven. Encodings zoals UTF-8 vertalen deze codepoints naar een reeks bytes die computers begrijpen. UTF-8 is efficiënt omdat het ASCII-tekens compact opslaat (1 byte), terwijl meer complexe tekens meer bytes krijgen. De ASCII codering was een van de eerste 'encodings' in de westerse wereld. In ASCII is de codepoint voor de hoofdletter 'A' nummer 65, 'B' is 66 ... Kleine letter 'a' is 97, 'b' is 98 en zo zijn er nog veel meer codepoints voor andere tekens:

```
cook@pop-os:~$ ascii -d
```

0 NUL	16 DLE	32	48 0	64 @	80 P	96 `	112 p
1 SOH	17 DC1	33 !	49 1	65 A	81 Q	97 a	113 q
2 STX	18 DC2	34 "	50 2	66 B	82 R	98 b	114 r
3 ETX	19 DC3	35 #	51 3	67 C	83 S	99 c	115 s
4 EOT	20 DC4	36 \$	52 4	68 D	84 T	100 d	116 t
5 ENQ	21 NAK	37 %	53 5	69 E	85 U	101 e	117 u
6 ACK	22 SYN	38 &	54 6	70 F	86 V	102 f	118 v
7 BEL	23 ETB	39 '	55 7	71 G	87 W	103 g	119 w
8 BS	24 CAN	40 (	56 8	72 H	88 X	104 h	120 x
9 HT	25 EM	41)	57 9	73 I	89 Y	105 i	121 y
10 LF	26 SUB	42 *	58 :	74 J	90 Z	106 j	122 z
11 VT	27 ESC	43 +	59 ;	75 K	91 [	107 k	123 {
12 FF	28 FS	44 ,	60 <	76 L	92 \	108 l	124
13 CR	29 GS	45 -	61 =	77 M	93]	109 m	125 }
14 SO	30 RS	46 .	62 >	78 N	94 ^	110 n	126 ~
15 SI	31 US	47 /	63 ?	79 O	95 _	111 o	127 DEL

De complete ASCII-set, inclusief 'extended ASCII codes' bevat in totaal 256 karakters. Dat past precies in **1 byte**:

$$\begin{array}{cccccccc} 128 & 64 & 32 & 16 & 8 & 4 & 2 & 1 \\ \hline 1 & + & 1 & + & 1 & + & 1 & + & 1 & + & 1 & + & 1 & + & 0 & = & 255 \end{array}$$

Ook emoji's zijn op deze manier gecodeerd. Zie hier de encoding voor het hart emoji (❤️):

Unicode character	Oct	Dec	Hex	HTML
Results 1 - 2 of 2				
❤️ heavy black heart	023544	10084	0x2764	❤
🖤 variation selector-16	0177017	65039	0xFE0F	️

Bron: <https://unicodelookup.com/##E2%9D%A4%EF%B8%8F/1>

Het proces omvat omzettingen tussen symbolen, codepoints en binaire data, essentieel voor compatibiliteit in digitale communicatie.



<https://www.youtube.com/watch?v=MijmeoH9LT4>

9.5. Opdracht: 'Hello World' webserver

In deze opdracht beginnen we klein: **we maken een eigen webserver (als C# Console App)** die enkel reageert met 'Hello World':

1. Start een C# Console App project
2. Maak in de Main-methode eerst de `HttpListener` aan, die gaat luisteren op poort 8080:

```
1 HttpListener listener = new HttpListener();
2 listener.Prefixes.Add("http://localhost:8080/");
3 listener.Start();
4 Console.WriteLine("Listening...");
```

3. Nu gaan we de `HttpListener` laten wachten op een binnenkomend verzoek:

```
1 HttpListenerContext context = listener.GetContext();
```

De applicatie 'bevriest' op deze regel, totdat iemand naar 'http://localhost:8080/' gaat in de browser.

4. Na de `GetContext`-regel gaan we een antwoord formuleren. In dit voorbeeld gaan we simpelweg 'Hello World' in wat basis HTML antwoorden:

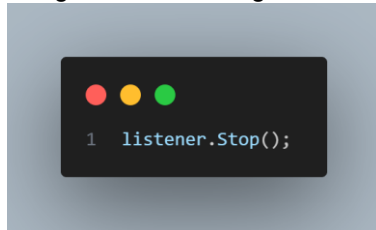
```
1 HttpListenerResponse response = context.Response;
2
3 string responseString = "<html><body>Hello World</body></html>";
4 byte[] buffer = System.Text.Encoding.UTF8.GetBytes(responseString);
5 response.ContentLength64 = buffer.Length;
6
7 System.IO.Stream output = response.OutputStream;
8 output.Write(buffer, 0, buffer.Length);
9 output.Close();
```

Opvallend hier is:

- We moeten het antwoord als 'bytes' sturen. Vandaar de 'byte[]' (byte array) buffer.
- Om de string naar bytes om te zetten gebruiken we UTF8-encoding
- We moeten de lengte aangeven van het antwoord wat we gaan geven
- Via de 'response.OutputStream' schrijven we de bytes 'over het internet' (in ons geval blijft het lokaal)
- Na het schrijven sluiten we de stream met 'Close'

5. Dit is het enige wat onze applicatie gaat doen. Maar aangezien de applicatie hierna klaar is, willen we wel de `HttpListener` laten stoppen. Anders wordt poort 8080 bezet gehouden door een applicatie die al niet meer actief is.

Voeg deze laatste regel toe aan het einde van de `Program.Main`:



6. Start de applicatie en ga in je browser naar `localhost:8080`, je ziet 'Hello World'. Op de achtergrond eindigt daarna de applicatie, kijk maar eens terug in de console.

9.6. Routes in een webapplicatie

Routing in een webserver houdt in dat verschillende URL's worden gekoppeld aan bepaalde acties of antwoorden. Een webserver kan de URL-structuur van een verzoek gebruiken om te bepalen welke actie of pagina aan de gebruiker moet worden teruggestuurd. Bijvoorbeeld, bij het bezoeken van `"/hello"` kan de server een groet tonen, en bij `"/goodbye"` een afscheid.

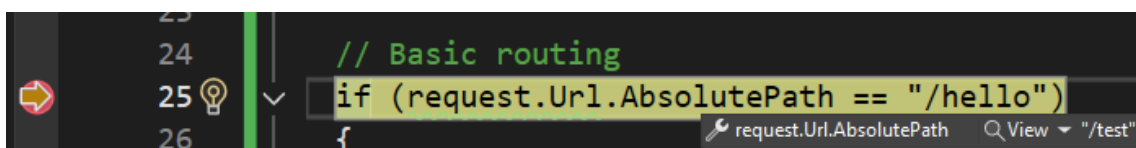
Een belangrijk onderdeel van deze logica is het herkennen van de URL-paden, wat in C# kan worden gedaan met **`HttpRequest`**:

```
HttpContext context = listener.GetContext();
HttpRequest request = context.Request;
```

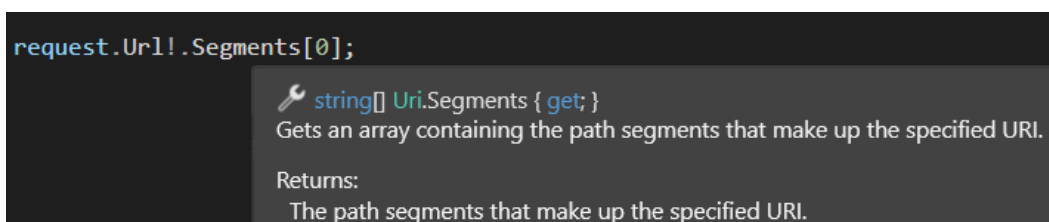
De URL-paden worden eenvoudig geëxtraheerd via **`request.Url.AbsolutePath`** of via **`request.Url.Segments`**, waarna je logica toepast om verschillende routes af te handelen.

`request.Url.AbsolutePath`:

Bij het navigeren naar `'http://localhost:8080/test'` is de `AbsolutePath`: `/test`



`request.Url.Segments`:



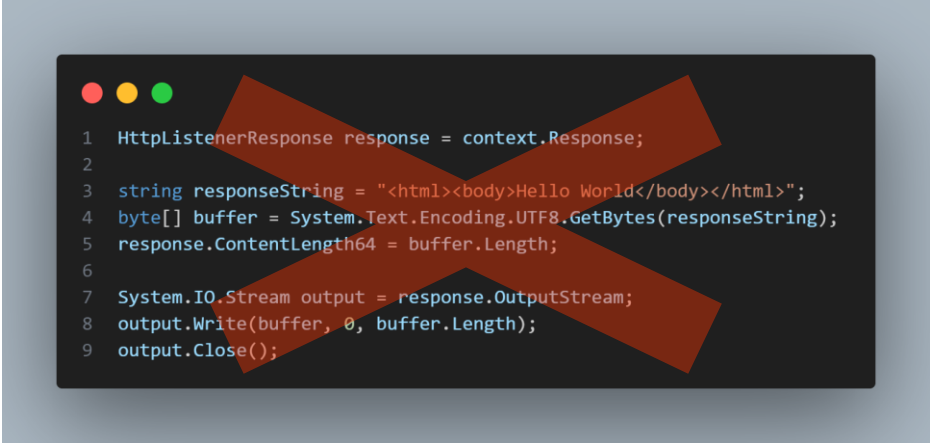
`request.Url.Segments` bevat bij een verzoek naar `http://localhost:8080/user/1`:

- `[0] = /`
- `[1] = /user/`
- `[2] = 1`

9.7. Opdracht: Meerdere routes in onze webserver

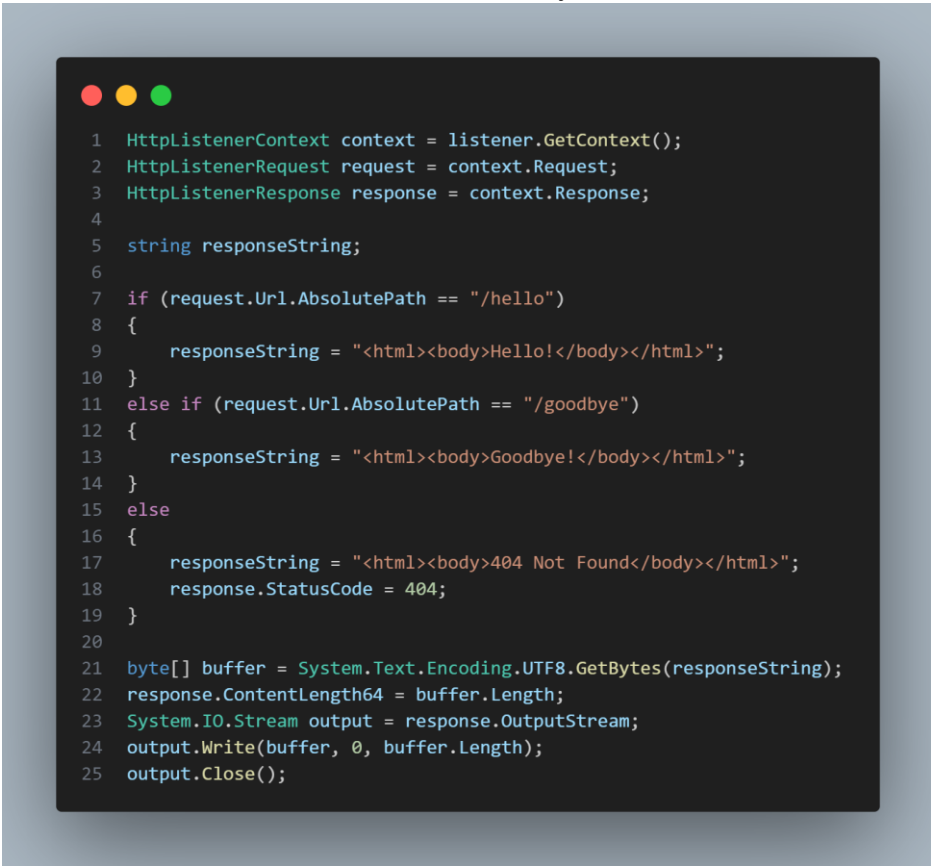
We breiden de vorige opdracht uit. In plaats van enkel 'Hello World' te tonen gaan we meer routes aanmaken.

1. Verwijder uit de vorige opdracht deze code:



```
1 HttpListenerResponse response = context.Response;
2
3 string responseString = "<html><body>Hello World</body></html>";
4 byte[] buffer = System.Text.Encoding.UTF8.GetBytes(responseString);
5 response.ContentLength64 = buffer.Length;
6
7 System.IO.Stream output = response.OutputStream;
8 output.Write(buffer, 0, buffer.Length);
9 output.Close();
```

2. Maak nu een aantal if-statements die, afhankelijk van de URL, andere HTML laat zien:



```
1 HttpListenerContext context = listener.GetContext();
2 HttpListenerRequest request = context.Request;
3 HttpListenerResponse response = context.Response;
4
5 string responseString;
6
7 if (request.Url.AbsolutePath == "/hello")
8 {
9     responseString = "<html><body>Hello!</body></html>";
10 }
11 else if (request.Url.AbsolutePath == "/goodbye")
12 {
13     responseString = "<html><body>Goodbye!</body></html>";
14 }
15 else
16 {
17     responseString = "<html><body>404 Not Found</body></html>";
18     response.StatusCode = 404;
19 }
20
21 byte[] buffer = System.Text.Encoding.UTF8.GetBytes(responseString);
22 response.ContentLength64 = buffer.Length;
23 System.IO.Stream output = response.OutputStream;
24 output.Write(buffer, 0, buffer.Length);
25 output.Close();
```

Merk op dat de 'else' een 404 pagina laat zien en ook de StatusCode instelt op 404. Alhoewel het instellen van de status-code niet verplicht is, is het wel netjes. Bij het maken van een API wordt de status-code vaker gebruikt, dan kan de app die de API consumeert makkelijk controleren of het verzoek succesvol was (statuscode 200).

3. Na die code komt wederom de 'listener.Stop();' om de HttpListener af te sluiten wanneer het programma stopt.
4. Start het programma en open <http://localhost:8080/hello> in de browser. Je ziet het antwoord, en op de achtergrond sluit het programma af, omdat het klaar is.

5. Dat het programma na één verzoek stopt is niet bepaald handig voor een webserver. We willen dat de applicatie na ieder verzoek opnieuw wacht op het volgende verzoek. Zo kan het zo veel mogelijk bezoekers bedienen.

Zet alle code vanaf 'HttpListenerContext context = listener.GetContext();' t/m 'output.Close();' in een while-loop die blijft herhalen.

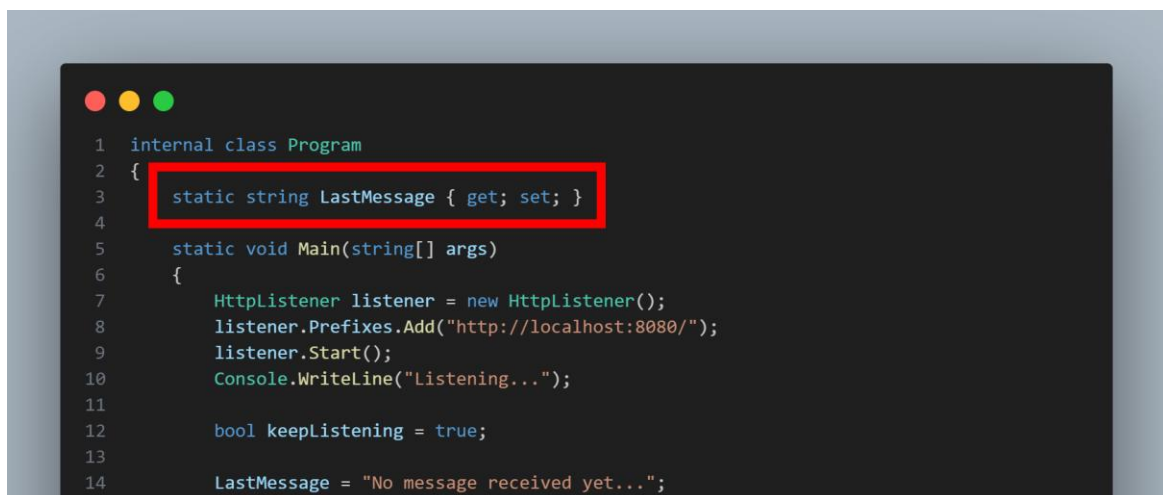
Na deze stappen kun je een basale webapplicatie maken. Die kan verschillende HTML-pagina's teruggeven, afhankelijk van de opgevraagde URL.

9.8. Gegevens bewaren

Weet je nog wat het betekende dat PHP 'stateless' is?

Waar PHP stateless is, is een Console App in C# stateful. Eigenschappen die we in de Program toevoegen, blijven voor alle verzoeken beschikbaar. Neem het volgende voorbeeld:

1. We maken een statische eigenschap van het type string in de Program. Hier gaan we dadelijk een bericht in plaatsen:



```
1 internal class Program
2 {
3     static string LastMessage { get; set; }
4
5     static void Main(string[] args)
6     {
7         HttpListener listener = new HttpListener();
8         listener.Prefixes.Add("http://localhost:8080/");
9         listener.Start();
10        Console.WriteLine("Listening...");
11
12        bool keepListening = true;
13
14        LastMessage = "No message received yet...";
```

2. Als iemand de homepage bezoekt (/) laten we het bericht zien wat in die eigenschap is gezet:



```
16 while (keepListening)
17 {
18     HttpListenerContext context = listener.GetContext();
19     HttpListenerRequest request = context.Request;
20     HttpListenerResponse response = context.Response;
21
22     string responseString;
23
24     if (request.Url.AbsolutePath == "/")
25     {
26         responseString = $"<html><body>{LastMessage}</body></html>";
27     }
```

3. Als iemand navigeert naar: <http://localhost:8080/submit?message=Hallo%20wereld> wordt dat bericht in de eigenschap geplaatst.



```
28 else if (request.Url.AbsolutePath.StartsWith("/submit"))
29 {
30     // Get the message from the query parameter message
31     string message = request.QueryString["message"];
32     LastMessage = message;
33     responseString = $"<html><body>Message received: {message}</body></html>";
34 }
```

4. Als je daarna terug naar de homepage gaat is dat bericht, zelfs tussen de verschillende verzoeken, bewaard gebleven.
5. Wanneer het programma afsluit zijn alle waardes natuurlijk wel verloren.

We noemen dit ook wel 'In-Memory' opslag.

9.9. Query Parameters

Nieuw in het bovenstaande voorbeeld is **het meegeven en ophalen van gegevens via de URL**. Dit noemen we **Query Parameters of Query String**:

Een verzoek aan <http://localhost:8080/submit?message=test> resulteert in de waarde "test" van de message query parameter. Je kunt in de code op deze manier de waarde van die parameter ophalen:

```
string message = request.QueryString["message"];
```

De eerste query parameter komt na de URL met een **vraagteken (?)**. Alle volgende query parameters volgen na een 'ampersand'-teken: **&**

Bijvoorbeeld: <http://localhost:8080/submit?message=Hallo%20Wereld&author=Tim>

Je ziet dat spaties niet zomaar in de URL komen, maar omgezet worden naar **%20**. Dit noemen we '**URL encoding**'. Als je op internet zoekt naar 'URL encode online' krijg je veel sites die je kunnen helpen om speciale tekens om te zetten. Zo wordt een spatie **%20** en een dubbele punt wordt **%3A**.

Stel dat je dus het '&' teken wilt meesturen, maar wilt voorkomen dat het wordt gezien als query parameter, dan moet je het dus URL-encoden naar **%26**. Zie hier hoe we bijvoorbeeld deze waardes meesturen:

- | | | |
|----------------------------|---|---|
| • Hallo Wereld & universum | → | <i>Hallo%20Wereld%20%26%20universum</i> |
| • 13:37 | → | <i>13%3A37</i> |

Om zo deze URL op te bouwen:

<http://localhost:8080/submit?message=Hallo%20Wereld%20%26%20universum&time=13%3A37>

9.10. Opdracht: ChatMeister Web Editie

Weet je nog dat we in de vorige lessen een proof-of-concept chat applicatie hebben gemaakt? De ChatMeister opdrachten gingen daarover. In deze en de volgende opdracht gaan we de eerste stappen zetten om van dat proof-of-concept een werkende chat-applicatie te maken.

De casus

We beginnen klein: laten we een web-versie maken waarin gebruikers:

- Berichten kunnen sturen naar een chatroom via de route:
/send-message?message=Hier%20het%20bericht&room=1&author=Tim
Met de query parameters:

- message:** het bericht
- room:** het id van de chatroom waar het bericht bij hoort
- author:** de naam van de persoon van wie het bericht is

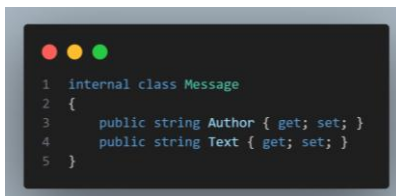
- Berichten kunnen ophalen van een chatroom via de route:
/get-messages?room=1

Met de query parameter:

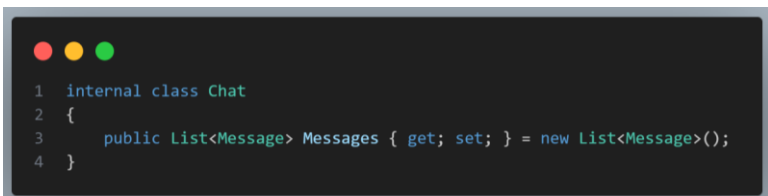
- room:** het id van de chatroom waarvan we de berichten willen ophalen

De opdracht

- Maak een nieuw C# Console App project
- Start zoals in de vorige opdrachten de HttpListener en laat die luisteren op <http://localhost:8080/>
- Gebruik een while loop om op verzoeken te wachten (`listener.GetContext()`)
- Geef voor nu altijd 'Hello World' als antwoord.
- Test de applicatie voordat je verder gaat.
- Nu gaan we de chat applicatie bouwen. Begin met het aanmaken van twee classes waarin we onze Chat en Message data gaan opslaan:

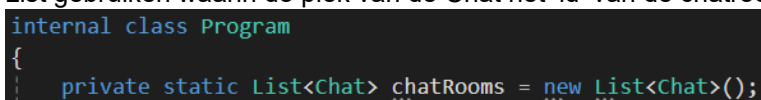


```
1 internal class Message
2 {
3     public string Author { get; set; }
4     public string Text { get; set; }
5 }
```



```
1 internal class Chat
2 {
3     public List<Message> Messages { get; set; } = new List<Message>();
4 }
```

- Maak een statische eigenschap in de Program waarin we Chat objecten gaan opslaan. We gaan een List gebruiken waarin de plek van de Chat het 'id' van de chatroom is:



```
internal class Program
{
    private static List<Chat> chatRooms = new List<Chat>();
}
```

- De route om berichten te sturen krijg je bij deze:

```
1 if (request.Url.AbsolutePath == "/send-message")
2 {
3     int chatroom = int.Parse(request.QueryString["room"]);
4     string message = request.QueryString["message"];
5     string author = request.QueryString["author"];
6     Chat foundChat = null;
7
8     if (chatroom < chatRooms.Count)
9     {
10         foundChat = chatRooms[chatroom];
11     }
12     else
13     {
14         foundChat = new Chat();
15         chatRooms.Add(foundChat);
16     }
17
18     foundChat.Messages.Add(new Message
19     {
20         Author = author,
21         Text = message
22     });
23
24     responseString = "<html><body>Message Sent!</body></html>";
25 }
```

Bekijk de code en probeer te begrijpen waarom keuzes gemaakt zijn. Vraag je bijvoorbeeld af waarom `int.Parse` nodig is. En bedenk hoe een chat gevonden of aangemaakt wordt.

9. Bedenk zelf de code die nodig is om de `/get-messages` route te maken. Je krijgt deze tips:
 - a. Via de `'room'` query parameter wordt een index meegegeven, gebruik die om de Chat uit de `chatRooms` List op te halen.
 - b. Om berichten uit een Chat naar JSON-string te veranderen gebruik je:
`string json = JsonSerializer.Serialize(messages);`
 - c. Geef een 404 als er om een index wordt gevraagd die niet bestaat.
 - d. Laat ook de auteur van het bericht zien, bijv.: "Janiek: Hallo wereld"
10. Bezoek wanneer je beide routes hebt gemaakt eerst: <http://localhost:8080/send-message?message=hallo%20wereld&author=Janiek&room=0>
Hiermee maak je een bericht aan. Omdat je room 0 meegeeft wordt die de eerste keer aangemaakt. Het bericht wordt toegevoegd aan de berichten.
11. Bezoek na het aanmaken van het bericht deze route: <http://localhost:8080/get-messages?room=0>.
Je ziet de door jouw ingestuurde berichten.

Na deze stappen heb je een basale web-chatapplicatie. Dat is best tof!

Bij deze een paar opmerkingen en toekomstige verbeterpunten achteraf:

- Je kunt nu berichten uit naam van iemand anders sturen.
- **Deze applicatie is onveilig en onder andere kwetsbaar voor XSS (Cross-Site-Scripting):** Een hacker kan HTML in berichten schrijven, die dan uitgevoerd wordt in de browser van iemand die het leest.

Een hacker kan bijvoorbeeld javascript uitvoeren bij iemand die de chatroom bekijkt, door dit bericht in te sturen: <http://localhost:8080/send-message?message=hoi%20daar&author=%3Cscript%3Ealert%28%27Hello%20World%27%29%3C%2Fscript%3E&room=0>

De in dit voorbeeld geïnjecteerde code '`<script>alert("Hello World")</script>`' is redelijk onschuldig. Maar een hacker kan dit ook gebruiken om cookies voor deze website te stelen, dat is veel schadelijker.

- **De chatroom id moet bij het aanmaken van een bericht wel geldig zijn:** als de List van Chats leeg is, dan moet room=0 meegegeven worden. Als er al 2 chats zijn dan mag de room parameter maximaal 2 zijn, ieder andere waarde zal een foutmelding geven.

De bovenstaande punten (en meer) zouden in een verder uitgewerkte versie verbeterd moeten worden. Dat hoef je nu niet te doen.

10. WinUI app verbinden met eigen API

In dit hoofdstuk gaan we de ChatMeister WinUI-app verbinden met onze eigen API. We gaan zien hoe we in één Visual Studio solution twee/drie projecten tegelijk ontwikkelen.

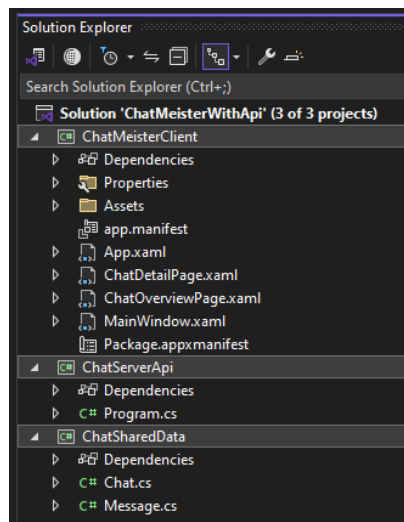
Ondanks dat de API straks op een server wordt gezet, en de WinUI-app bij gebruikers op de computer geïnstalleerd wordt, is het slim dit in één Visual Studio Solution te ontwikkelen.

10.1. Slim ontwikkelen: 1 solution, meerdere projecten

We hebben in het vorige hoofdstuk al een API gebouwd voor ons Chat-systeem. In dit hoofdstuk bouwen we daarvoor een client.

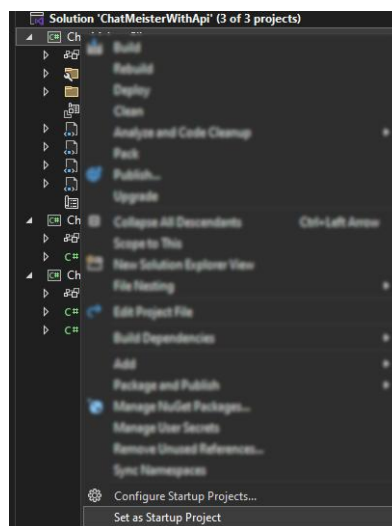
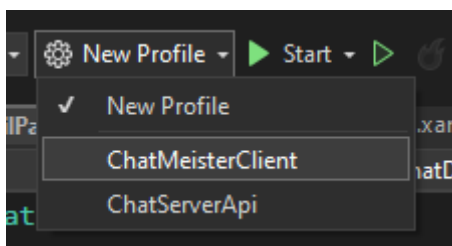
Als we slim software willen ontwikkelen, kunnen we er zo voor kiezen om 1 Visual Studio Solution te maken, en daar zowel onze API, als Client-app, als losse projecten in te zetten. Dit doen we, zodat we tegelijkertijd de API en consumerende WinUI-app kunnen bouwen, zonder Visual Studio meerdere keren te openen (wat traag is).

Het combineren van meerdere projecten in één solution helpt ook met versiebeheer. We kunnen alle bij elkaar horende projecten in één Git repository zetten.



Meerdere projecten tegelijk opstarten

Tot nu hadden we vaak maar één project in een solution. Als we dan op de 'Start/debug'-knop drukte om de applicatie te starten, dan startte het enige project wat we hadden. Bij meerdere projecten kunnen we kiezen welke we willen starten. Dat kan door de drop-down naast de 'Start'-knop te gebruiken, of met de rechtermuis op een project het als 'Startup Project' in te stellen:

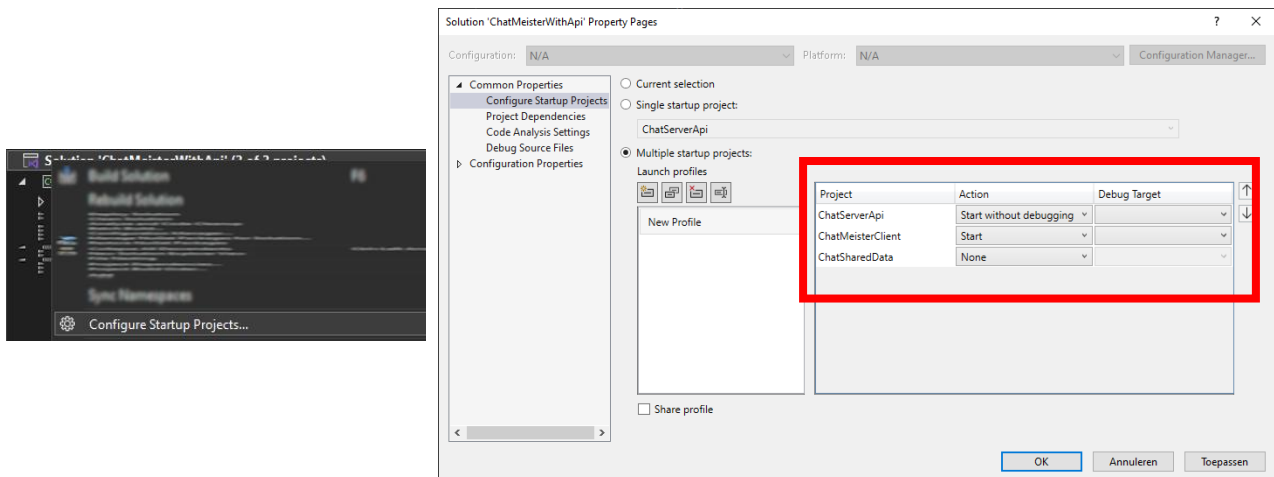


Visual Studio kan meerdere projecten starten, maar slechts één project kan in 'debug mode' gestart worden. De 'debug mode' zorgt ervoor dat de code stopt bij 'breakpoints', dat we er stap-voor-stap doorheen kunnen lopen. Het zorgt er ook voor dat bij een 'Exception', we naar de plek in de code gestuurd worden waar die gebeurde.

Stel je bent het API aan het ontwikkelen, dan wil je die in debug modus starten, door het als 'Startup Project' in te stellen. Daarna kun je de WinUI-app starten zonder debug-modus, om de API daarlangs te testen (door de .exe direct op te starten in de bin map):

ChatMeisterWithCustomApiStartpunt > ChatMeisterClient > bin > x64 > Debug > net8.0-windows10.0.19041.0 > win-x64			
Naam	Gewijzigd op	Type	Grootte
ChatMeisterClient.deps.json	6-12-2024 15:53	JSON Source File	
ChatMeisterClient.dll	6-12-2024 15:53	Toevoegingspakket	
ChatMeisterClient.exe	6-12-2024 15:53	Toevoegingspakket	

Om dit proces makkelijker te maken, kun je in Visual Studio instellen welke projecten in debug-mode gestart moeten worden, en welke 'gewoon' moeten opstarten. Je kunt ook de volgorde van opstarten aanpassen. Vervolgens kun je met één druk op de 'Start'-knop dit allemaal laten starten. Je stelt het opstarten van meerdere projecten in via de 'Configure Startup Projects...' configuratie bij het klikken met op jouw solution met de rechter-muisknop:



Let wel op: wanneer meerdere projecten gestart zijn, zal de 'Stop'-knop in Visual Studio alleen het project in debug mode stoppen. Je moet zelf andere projecten sluiten.

Deze instellingen worden in het startpunt bij een komende opdracht meegeleverd.

⚠ **Reminder! De API en Client staan alleen tijdens ontwikkeling op hetzelfde apparaat.**

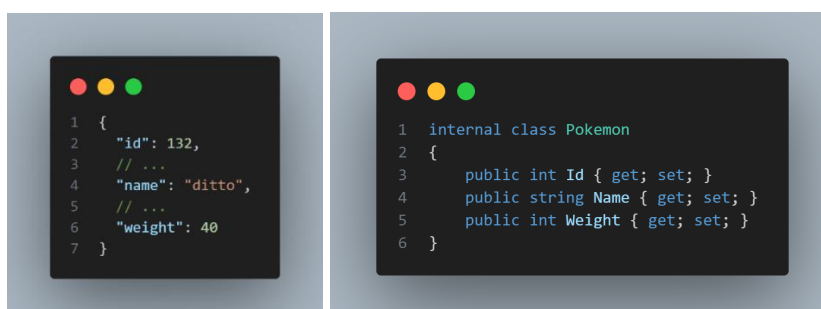
Probeer je in te beelden dat de API en Client niet op hetzelfde apparaat staan. Dat is tijdens het ontwikkelen wel (zo staan ze zelfs in dezelfde solution), maar in de praktijk staat de API op een server in een datacenter en is de Client een applicatie op de computer van een gebruiker.

De API dient als 'poortwachter' voor de database. De database is alleen toegankelijk voor de API. De client kan alleen met de API communiceren via HTTP verzoeken (REST API).

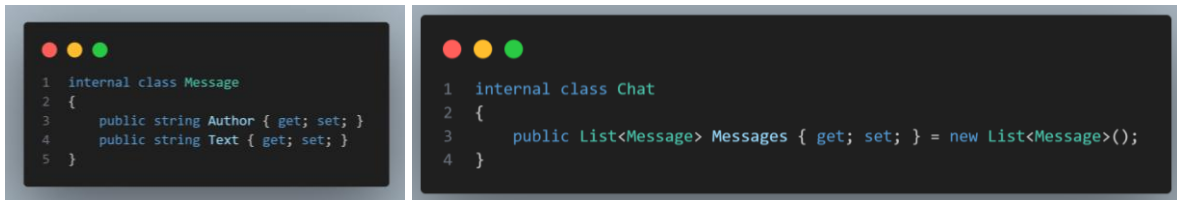
Dit betekent dat je moet zorgen dat de Client *geen* verbinding maakt met de database, maar in plaats daarvan verzoeken stuurt naar de API als HTTP-verzoeken.

📁 **10.2. Chat en Message klassen delen tussen projecten**

In hoofdstuk 8 zagen we dat we voor het consumeren van een API, de JSON moeten nabouwen als klasse. We zagen dit bij het 'PokeAPI' voorbeeld:



Tegelijkertijd hebben we bij de API diezelfde klassen nodig. Zo hebben we bij de ChatMeister API een Chat en Message klasse nodig om berichten en chatrooms bij te houden:



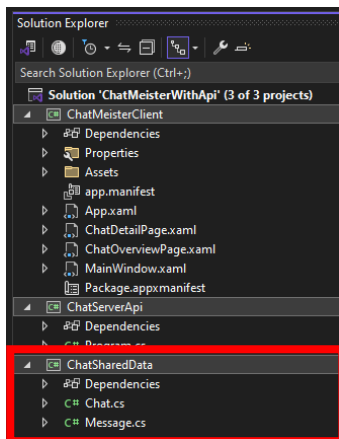
De API zal objecten van die types vervolgens omzetten naar JSON en als antwoord teruggeven.

```
string json = JsonSerializer.Serialize(messages);
```

Nu kennen we in softwareontwikkeling het concept 'DRY: Don't Repeat Yourself'. We doen ons best om geen code te hoeven herhalen. **Om die reden gaan we code delen tussen de Client en API-projecten.**

We delen code tussen twee projecten, door een derde project te maken van het type 'Class Library'. Daarin plaatsen we gemeenschappelijke classes (alleen de modellen in ons geval). Vervolgens voegen we dat derde project als 'Project Reference' toe in de andere projecten.

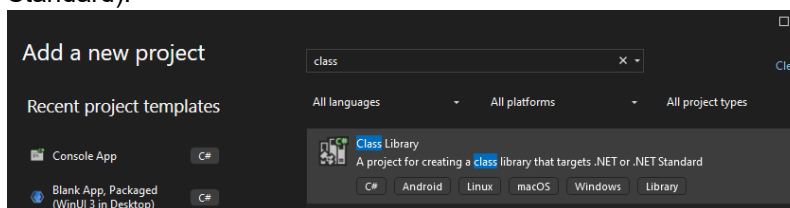
Het resultaat in een Solution met 3 projecten:



Stap-voor-stap een Class Library (.dll) maken en toevoegen aan andere projecten

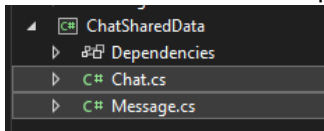
In de opdracht die volgt staat een 'ChatSharedData' Class Library project. Hierin zitten de klassen die zowel in de Client als in de API nodig zijn. Dit is hoe die is aangemaakt en gekoppeld in de Client en API projecten:

1. Rechtermuisklik op de Solution > Add > New Project...
2. Zoeken naar "class", kies 'Class Library' (met C# icoon) en een beschrijving over .NET (of .NET Standard):

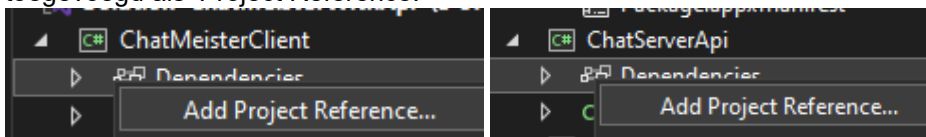


3. In het scherm erna wordt een naam voor het project gekozen, zoals 'ChatSharedData'.

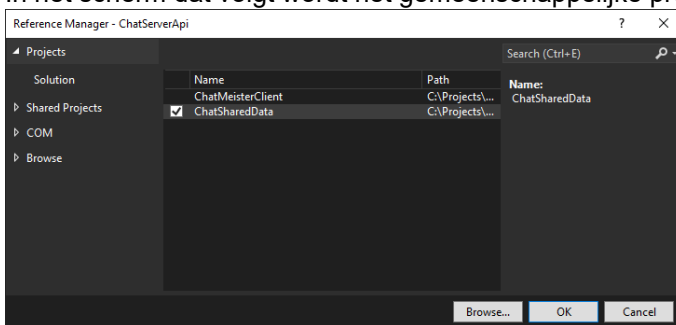
4. Er wordt een versie van .NET gekozen die compatibel is met de projecten die het gaan gebruiken. Vaak precies dezelfde .NET versie (bijv.: .NET 8.0)
5. Na het aanmaken van het project, staat er standaard alleen een class in 'Class1.cs'. Die verwijder je vaak.
6. Een Class Library heeft geen Program en geen Main. Dit omdat het geen executable wordt, maar als .dll onderdeel wordt van een andere applicatie.
7. Na het aanmaken van het project, worden de klassen in dat ChatSharedData project aangemaakt:



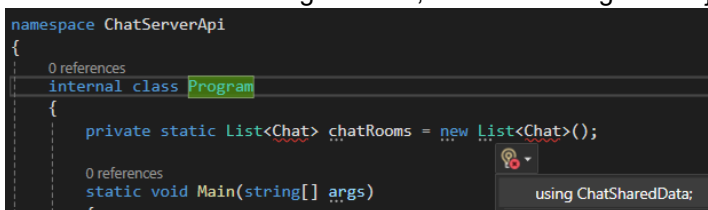
8. Bij de projecten die gebruik maken van dat gemeenschappelijke 'ChatSharedData' project wordt het toegevoegd als 'Project Reference':



9. In het scherm dat volgt wordt het gemeenschappelijke project aangevinkt.



10. Na deze stap zijn **alle 'public' classes, methodes en eigenschappen** van het gemeenschappelijke project beschikbaar in de projecten die het als 'Project Reference' hebben toegevoegd.
11. Als we een klasse willen gebruiken, moeten we nog wel de juiste 'using' toevoegen:



Deze aanpak scheelt herhalende code, want zowel onze Client als API hebben dezelfde model klassen nodig.

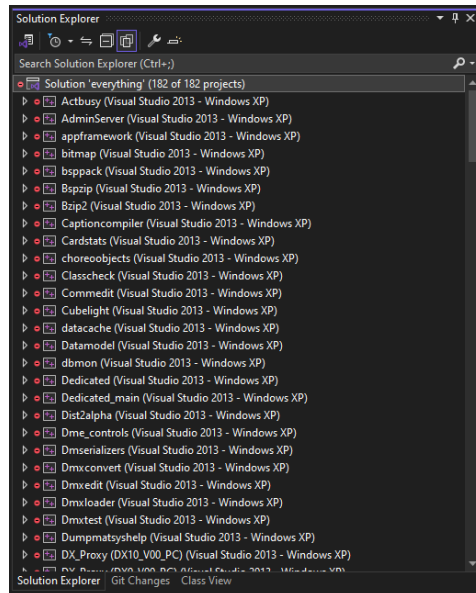
Hoe wordt zo'n class library gecompileerd? In de executable, of naar iets los daarvan?

Als je de solution bouwt worden de Client en API-projecten los van elkaar gecompileerd. Ieder krijgen ze een executable (.exe). De Client .exe start een WinUI3 applicatie en de API .exe start de webserver.

Wanneer je naar de '.exe' gaat van ieder project (bijv.: <projectmap>/bin/x64/.../ChatServerApi.exe) dan zie je in dezelfde map als die .exe een '.dll' bestand met de naam van het gedeelde project. Die '.dll' (dynamic link library) bevat onze gedeelde klassen. Zowel het Client als API-project krijgen deze '.dll'.

Zijn meerdere projecten en class libraries er alleen om herhalende code te voorkomen?

Als je kijkt in de installatiemap van een van je favoriete games (of veel andere Software) zie je ook een hoop van die '.dll'-bestanden. Dit is omdat dat soort games gemaakt worden door enorm grote teams met mensen. Om slim de taken te verdelen en de code overzichtelijk te houden worden games daarom opgesplitst in afzonderlijke delen. Ieder deel is een project in de Solution van de game. Ieder project kan door een ander team ontwikkeld worden. Zo zie je hieronder (ongeveer) de structuur van Counter-Strike 2:



Een applicatie over meerdere projecten verdelen kan dus ook helpen om structuur en overzicht aan te brengen in software broncode.

Let op: 3 projecten, 1 solution, twee verschillende locaties waar dit gebruikt gaat worden.

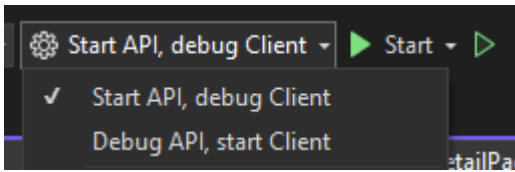
Ondanks dat we de Client en API-projecten samen ontwikkelen, in een enkele solution, gaan ze op andere plekken gebruikt worden. Daarbij gaan andere doelgroepen ermee aan de slag:

- **De client-applicatie:**
 - Voor eindgebruikers.
 - Wordt geïnstalleerd op hun pc, bijvoorbeeld die op hun bureau in hun kantoor.
 - Heeft een netwerk/internetverbinding nodig om te kunnen communiceren met de API.
- **De API-webserver:**
 - Voor IT-beheerders van het bedrijf waarvoor de software is ontwikkeld.
 - Wordt geïnstalleerd op een server. Die kan in de serverruimte van het kantoor staan, of aan de andere kant van de wereld op een server in een datacenter.
 - Heeft een netwerking/internetverbinding nodig om verzoeken te kunnen ontvangen van clients.

Het enige moment dat de Client en API dus op dezelfde computer staan zal tijdens het ontwikkelen van de projecten zijn.

10.3. Opdracht: ChatMeisterWithApi Solution

In deze opdracht krijg je een startpunt met daarin 1 solution, met 3 projecten. Met deze opdracht ervaar je hoe het is om te werken met 3 projecten in 1 solution. Er zijn al twee 'Launch Profiles' gemaakt:



Zo kan de ontwikkelaar kiezen:

- De API eerst gestart moet worden, daarna het Client project in debug-mode.

Of:

- De API eerst gestart moet worden in debug-mode, daarna het Client project.

Het derde gedeelde project (ChatSharedData) wordt niet gestart. Omdat het met 'Add Project Reference' aan de Client en API is toegevoegd, zijn de klassen erin beschikbaar in die beide projecten.

Omdat deze opdracht voornamelijk gaat over het ervaren hoe het is om met 3 projecten in 1 solution te werken, blijft de code die je zelf moet schrijven erg beperkt.

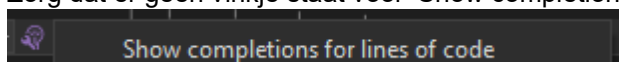
Lees na deze lijst met functionaliteiten het stappenplan, voordat je aan de slag gaat:

- De WinUI applicatie en API starten beide zonder problemen.
- Het tonen van alle Chats in het overzichtsscherm werkt, je kunt op een Chat klikken om naar de ChatDetailPage te gaan.
- De API werkt zonder database. Alle chats en berichten staan in een eigenschap van de Program in API. De opslag is wat we noemen 'In-memory' (in het geheugen). *Als de API sluit gaan alle chats en berichten dus verloren.*
- Het toevoegen van berichten werkt, maar berichten zijn nog niet zichtbaar in het detail scherm. Berichten kunnen ook nog niet opgehaald worden uit de API.
- Deze functies missen dus:
 - In de API (**ChatServerApi**) is de `/get-messages` route nog niet geïmplementeerd.
 - In de Client (**ChatMeisterClient**) is de `LoadChatMessages`-methode nog niet geïmplementeerd.

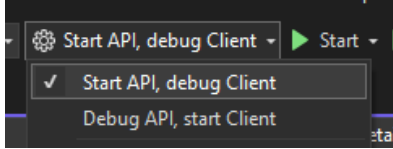
Stappenplan:

1. Deze opdracht is niet effectief als je een AI de code laat schrijven. Je moet dit zelf kunnen uitvinden, door terug te kijken in dit moduleboek en/of de bestaande code in het aangeleverde startpunt te begrijpen. Zet daarom suggesties van bijvoorbeeld GitHub Copilot uit in Visual Studio:

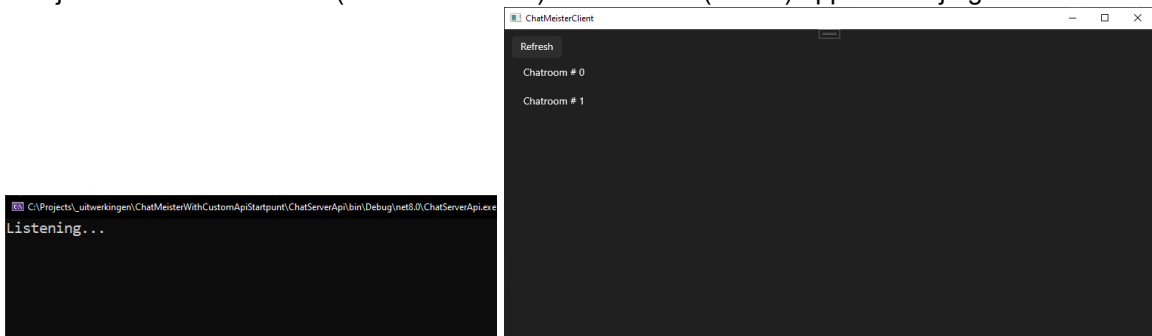
- Klik linksonder in het code-venster van Visual Studio op dit paarse icoon: 
- Zorg dat er geen vinkje staat voor 'Show completions for lines of code':



- Download het startpunt van Itslearning en pak het uit naar een plek op jouw computer:
S_NATIVE: CSD-IV > Lesmateriaal > H3 ChatMeesterWithApi Solution - Startpunt.zip
- Open de solution en zorg dat het volledig is geladen. Wacht tot alle NuGet packages zijn opgehaald (zorg dat je een internetverbinding hebt).
- Start de API en Client door te zorgen dat een vinkje staat voor de 'Launch Profile' genaamd 'Start API, debug Client' en klik daarna op de gewoonlijke 'Start'-knop:



- Bekijk hoe nu zowel de API (Console scherm) en de Client (WinUI) applicatie zijn gestart:



- Controleer welke functies al werken en welke niet (zie de lijst met functionaliteiten voor deze opdracht).
- Ga in je browser naar: <http://localhost:8080/get-messages> en bekijk de uitvoer. Als het goed is wordt een lege JSON-array gegeven als antwoord.



- Klik op de rode stop knop om te stoppen met debuggen:
- Merk op dat de WinUI applicatie nu wel gesloten is, omdat die in debug-mode werd gestart door de 'Launch Profile'. Op de achtergrond staat de API nog wel open, die moet je dus handmatig sluiten met de sluit-knop (x) rechtsboven op het console scherm.

Nu ga je eerst de API-code voor de '/get-messages' route implementeren:

- Open de Program.cs van het API project.
- Zoek deze plek, waar de '/get-messages' geïmplementeerd moet worden:

```
else if (path == "/get-messages")
{
    // TODO: Haal hier de 'room' parameter op uit de query parameter
    // TODO: Toon van de Chat met die id (zie chatRooms field) de berichten
    // TODO: Toon de berichten als JSON, zodat de client het kan deserialiseren
    // TODO: Haal dit weg en zorg dat `responseString` de JSON van berichten bevat:
    responseString = "[";
}
else if (path == "/get-rooms")
```

- Zorg de responseString een JSON wordt van alle berichten in de gekozen Chat. De Chat die gekozen wordt, wordt via de query parameter 'room' meegegeven (voor de eerste chat bijvoorbeeld ?room=0) .
Gebruik voor schrijven van deze code de voorbeelden die al in het startpunt staan en de theorie die voorafging aan deze opdracht.

13. Test na het schrijven van code eerst in de browser of de uitvoer is wat je verwacht. Als het goed is zie je de volgende JSON:

```
[{"Author": "Alice", "Text": "Hello"}, {"Author": "Bob", "Text": "Hi"}]
```

In woorden: Een **array** met daarin **meerdere (Message) objecten**, die ieder zowel 'Author' als 'Text' eigenschappen hebben

Nu dat de API de mogelijkheid geeft om berichten van een Chat op te halen gaan we de Client aanpassen:

14. Open de ChatDetailPage.xaml.cs in het Client (ChatMeisterClient) project.
15. Dit scherm wordt geopend wanneer je in het overzicht scherm een Chat aanklikt.
16. Zoek deze plek, waar de LoadChatMessages-methode nog niet is geïmplementeerd:

```
// Load the messages of the chat from the API
2 references
private async void LoadChatMessages()
{
    // TODO: Haal met een HttpClient hier de berichten op voor de chat.
    // TODO: Stuur de Chat id mee via de 'room' query parameter
    var currentChatId = CurrentChat.Id;
    // TODO: Zet de JSON-array van berichten om naar een lijst van Message.
    // TODO: Zet die lijst in chatMessagesListView.ItemsSource
}
```

17. Zorg dat een verzoek wordt gedaan naar de '/get-messages' route van de API.
18. Deserialiseer de JSON naar een lijst van Message objecten.
19. Toon die berichten in de chatMessagesListView
20. Je bent klaar wanneer je:
- Bij het openen van een Chat, direct alle berichten uit die Chat ziet
 - Bij het toevoegen van een bericht, je dat bericht meteen ziet verschijnen (*hier zou je niks anders voor hoeven te doen, anders dan de LoadChatMessages implementeren*)

Met deze opdracht heb je ervaren hoe het is om te werken met 1 solution met daarin 3 projecten. Daarbij heb je nu een compleet voorbeeld van een Client en API voor een chat applicatie.

10.4. Van In-Memory opslag naar een MySQL database

In het startpunt van de vorige opdracht is ervoor gezorgd dat in de API de Chats enkel in het geheugen worden opgeslagen. Dit wordt 'In-Memory' opslag genoemd.

```
5 namespace ChatServerApi
6 {
7     0 references
8     internal class Program
9     {
10         private static List<Chat> chatRooms = new List<Chat>();
11
12         0 references
13         static void Main(string[] args)
14         {
15             HttpListener listener = new HttpListener();
16             listener.Prefixes.Add("http://localhost:8080/");
17             listener.Start();
18             Console.WriteLine("Listening...");
19
20             AddTestData();
21
22             bool keepListening = true;
```

Een groot nadeel hiervan is dat gegevens verloren gaan wanneer de API applicatie afsluit. Daarom ga je in de volgende opdracht aan de slag om een MySQL database te gebruiken.

Voor het toevoegen van MySQL database opslag hoef je gelukkig niets nieuws te leren. Je gebruikt zoals je in hoofdstuk 3 hebt geleerd de 'Pomelo.EntityFrameworkCore.MySql' NuGet package. **Deze hoef je enkel te installeren voor het API project (ChatServerApi).** De Client gaat de gegevens via die API krijgen en maakt zelf geen verbinding met de database. Sterker nog, we veranderen geheel niks aan de Client applicatie, want die werkt al zoals we willen.

10.5. Opdracht: Implementeren MySQL in API Project

Je gaat jouw code uit de vorige opdracht (de ChatMeisterWithApi Solution) aanpassen, zodat er MySQL gebruikt wordt voor de opslag van Chats en Messages, in plaats van In-Memory opslag.

Bij deze opdracht lopen we ook tegen een aantal bugs aan die we gaan oplossen. Zo weet je in de toekomst hopelijk dat je bij deze opdracht kunt terugkijken om te zien hoe je die bugs ook alweer had opgelost.

Stappenplan

1. Kopieer de broncode die bij de vorige opdracht (de ChatMeisterWithApi Solution) hebt geschreven
2. Open deze nieuwe broncode in Visual Studio en maak de volgende aanpassingen **alleen aan het ChatServerApi project**.
3. Voeg het Pomelo MySQL NuGet package, waar je al bekend mee bent, toe aan **alleen het API project**.
4. Zorg dat beide modellen (Chat en Message) in het gedeelde project een Id eigenschap hebben. Zo kan Entity Framework er tabellen voor aanmaken met een Primary Key.
5. Zorg dat het Message model een eigenschap 'ChatId' krijgt én de bijbehorende navigation property:



```
1 public class Message
2 {
3     public int Id { get; set; }
4     public int ChatId { get; set; }
5     public string Author { get; set; }
6     public string Text { get; set; }
7
8     public Chat Chat { get; set; }
9 }
```

6. Voeg een ApplicationDbContext toe in het API project:
 - a) Maak bovenin de ApplicationDbContext DbSet eigenschappen aan voor de Chat en Message modellen. *Let hierbij op dat je de juiste 'using' toevoegt, zodat deze klassen in het ChatSharedData project worden gevonden.*
 - b) De OnConfiguring maak je aan zoals je gewend bent
 - c) In de OnModelCreating maak je zoals je gewend bent de seeder aan. Vul het met testgegevens zoals de gegevens die nu in de AddTestData-methode staan.
Let op: Entity Framework staat niet toe dat een Id '0' is. Dus zorg dat de eerste Chat Id '1'

heeft:

```
1  protected override void OnModelCreating(ModelBuilder modelBuilder)
2  {
3      base.OnModelCreating(modelBuilder);
4
5      modelBuilder.Entity<Chat>().HasData(
6          new Chat { Id = 1 },
7          new Chat { Id = 2 }
8      );
9
10     modelBuilder.Entity<Message>().HasData(
11         new Message { Id = 1, ChatId = 1, Author = "Alice", Text = "Hello" },
12         new Message { Id = 2, ChatId = 1, Author = "Bob", Text = "Hi" },
13         new Message { Id = 3, ChatId = 2, Author = "Alice", Text = "How are you?" },
14         new Message { Id = 4, ChatId = 2, Author = "Charlie", Text = "Fine, thanks" }
15     );
16 }
```

- d) Verwijder de AddTestData-methode die eerder In-Memory test-data aanmaakte:

```
// Adds some test data to the chatRooms list for testing purposes.
1 reference
private static void AddTestData()
```

7. Verwijder het 'chatRooms' veld bovenin de Program, waar In-Memory Chats werden bijgehouden:

```
private static List<Chat> chatRooms = new List<Chat>();
```

8. Vervang de plekken waar het verwijderde 'chatRooms' veld nog gebruikt wordt met 'db.Chats'. Zodat de DbSet<Chat> gebruikt wordt uit de verbinding met de database.

9. Start aan het begin van de Program een verbinding met de database, door een AppDbContext instantie aan te maken, bijv.:

```
private static AppDbContext db;

0 references
static void Main(string[] args)
{
    db = new AppDbContext();
    db.Database.EnsureCreated();
}
```

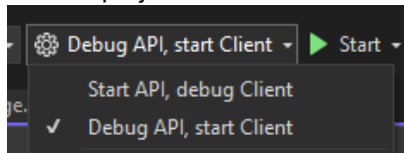
Op deze manier is de db variabele beschikbaar in de gehele Program. Ook binnen de SendMessage en CreateChat methodes.

10. Zorg met 'EnsureCreated' dat de databasestructuur ook wordt aangemaakt.
11. Als je in de vorige stap geen 'using' gebruikt om de AppDbContext automatisch af te laten sluiten, zorg dan dat vóór het sluiten van de applicatie de verbinding met de database netjes wordt afgesloten:

```
Console.WriteLine("Shutting down...");
db.Dispose();
```

'Dispose' is de methode die een 'using' automatisch zou opruimen aan het einde van diens scope.

12. Test dit project met het 'Launch Profile' dat de API start in debug-mode:



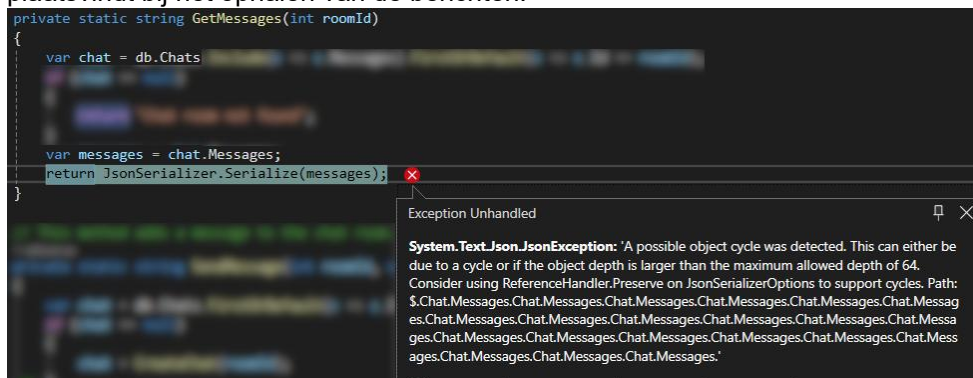
Op deze manier wordt je bij Exceptions naar de juiste plek in de code gewezen.

13. Als het goed is krijg je geen foutmeldingen en starten beide projecten.
14. **Probleem #1:** in de ChatDetailPage zien we geen berichten. Zoek uit hoe dit kan gebeuren. Gebruik breakpoint om uit te vinden wat er mis gaat. Wissel tussen beide 'Launch Profiles', zodat je de ene keer breakpoints kunt zetten in de Client en de andere keer in de API.
15. Tips om dit probleem op te lossen:

- Loop stap-voor-stap door de relevante code met behulp van breakpoints.
- Bedenk:
 - Krijgt de Client wel antwoord van de API?
 - Als de Client geen berichten krijgt van /get-messages, waarom niet?
 - Lukt het de API om de berichten uit de database op te halen? Waarom niet?
 - Wat haalt de API wel op? Welke tabel? Zitten de berichten ook in die tabel?
 - Wat moet je doen om ook de berichten op te halen?

Kom je er na de tips en flink debuggen (ten minste 30 minuten) niet zelfstandig aan uit? Dan heeft de docent (of een klasgenoot) de oplossing voor je.

16. **Probleem #2:** Na het oplossen van de bovenstaande bug is er nog een laatste Exception die plaatsvindt bij het ophalen van de berichten:



17. Lees de Exception foutmelding goed. Wat valt je op aan wat er achter 'Path:' staat in de foutmelding? `$.Chat.Messages.Chat.Messages...`
18. Deze foutmelding geeft aan dat er een oneindige verwijzing heen en weer is tussen Chat en Message. Dit komt vanwege de Navigation Properties die we daar hebben. Denk na over dit probleem en of je de oorzaak begrijpt.
19. Om dit probleem op te lossen zijn er twee manieren:

- Gebruik het '[JsonIgnore]' attribuut op de Chat eigenschap. Op die manier heeft Chat wel een verwijzing naar meerdere Message objecten (List<Message>), maar verwijst Message

bij het serialiseren naar JSON niet terug naar de Chat.

```
1 using System.Text.Json.Serialization;
2
3 namespace ChatSharedData
4 {
5     public class Message
6     {
7         public int Id { get; set; }
8         public int ChatId { get; set; }
9         public string Author { get; set; }
10        public string Text { get; set; }
11
12        [JsonIgnore]
13        public Chat Chat { get; set; }
14    }
15 }
16
```

- b) Gebruik een **DTO: Data Transfer Object** die niet de relaties teruggeven, maar alleen die gegevens die we in de API resultaten willen laten zien. DTO's zijn klassen die lijken op de volledige klasse. Zo is het enige verschil tussen het Message model en de Message DTO dat er geen Navigation Property is bij het DTO:

```
1 public class MessageDto
2 {
3     public int Id { get; set; }
4     public int ChatId { get; set; }
5     public string Author { get; set; }
6     public string Text { get; set; }
7 }
```

(Deze DTO klasse kun je in hetzelfde bestand zetten als waar de Message staat, of in een eigen bestand in hetzelfde project)

Wanneer we het API antwoord opbouwen, geven we niet de Message objecten terug, maar kopiëren we de inhoud van iedere Message naar een nieuw MessageDto object:

```
var messages = chat.Messages.Select(m => new MessageDto
{
    Id = m.Id,
    ChatId = m.ChatId,
    Text = m.Text,
    Author = m.Author,
});

return JsonSerializer.Serialize(messages);
```

Op een van deze manieren voorkomen we de oneindige heen-en-weer verwijzing die de Exception veroorzaakte.

20. Test de applicatie. Als het goed is worden de berichten nu netjes geladen bij het openen van een Chat.
21. Probeer een bericht toe te voegen. Je ziet het bericht meteen verschijnen.

22. Herstart de applicatie en je zult zien dat de toegevoegde berichten niet zijn opgeslagen. Voeg nog 'db.SaveChanges()' toe aan de SendMessage-methode:

```
// This method adds a message to the chat room.
1 reference
private static string SendMessage(int roomId, string message, string author)
{
    var chat = db.Chats.FirstOrDefault(c => c.Id == roomId);
    if (chat == null)
    {
        chat = CreateChat(roomId);
    }
    chat.Messages.Add(new Message { Author = author, Text = message });
    db.SaveChanges();
    return "Message sent";
}
```

23. Test dat nu het tonen, toevoegen en opslaan van berichten naar behoren werkt.

Na deze laatste wijziging is de omzet van In-Memory naar opslag in een MySQL database compleet.

10.6. Opdracht: Basis Relaties en API Endpoints

Om te wennen aan het werken met een many-to-many relatie en API-routes, maak je een eenvoudige applicatie waarin boeken en auteurs worden gekoppeld. Je bouwt een basis-API en test het ophalen van gegevens.

Instructies:

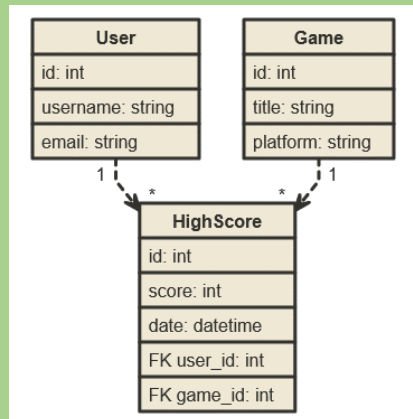
1. Maak de benodigde modellen aan voor een boekenbibliotheek:
 - **Author** (Id, Name)
 - **Book** (Id, Title)
 - **BookAuthor** (Id, BookId, AuthorId)
2. Maak database seeds in je AppDbContext voor enkele auteurs en boeken.
3. Maak routes in de API voor:
 - **/books**: Geeft een JSON-array terug met alle boeken inclusief hun auteurs.
 - **/authors**: Geeft een JSON-array terug met alle auteurs inclusief de boeken die zij hebben geschreven.
 - **/book/{id}**: Geeft een JSON-object terug met een specifiek boek en de bijbehorende auteurs.
 - **/author/{id}**: Geeft een JSON-object terug met een specifieke auteur en zijn/haar boeken.
4. Test de API in de browser
 - Haal een lijst met boeken op
 - Haal een lijst met auteurs op
 - Vraag details op van een specifiek boek of auteur

Extra uitdaging (bonus)

- Maak een eenvoudige WinUI applicatie waarin je een boek-ID kunt invoeren en de details van dat boek worden opgehaald en getoond.
- Test de API met een tool als Postman (<https://www.postman.com/>) of Insomnia (<https://insomnia.rest/>)

Inleveropdracht 10.7: Games Leaderboard (tijdsduur: +/- 150 minuten)

Maak een spelletjes leaderboard waar highscores van verschillende spelers kunnen worden opgeslagen en opgevraagd voor verschillende games.



(Je mag ook zelf een onderwerp kiezen waarin eenzelfde many-to-many relatie bestaat zoals hierboven. Vervang in dat geval hieronder de modellen en klassenamen met wat voor jou van toepassing is)

Instructies:

1. Maak de benodigde modellen aan voor het door jouw gekozen onderwerp, *bijv.: User, Game, HighScore*
2. Maak database seeds in je ApplicationDbContext voor bijvoorbeeld de *User, Game en HighScore* modellen
3. Maak routes in de API voor:
 - a. **/highscores:** Geeft een JSON-array terug met alle HighScores (gebruik Entity Framework om alle HighScores op te halen, zoals je geleerd hebt in de hoofdstukken 3 t/m 5)
 - b. **/highscore/{id}:** Geeft een JSON-object terug met daarin alleen informatie over het HighScore object met het meegegeven ID
4. Maak een bijbehorende WinUI applicatie die de API aanroept om informatie op te halen en te tonen.
5. **Zorg dat ten minste dit lukt:** de gebruiker kan in een invoerveld in de WinUI applicatie een ID van een HighScore ingeven. Via een verzoek aan de Show-action API worden de details van die highscore opgehaald en getoond in de WinUI app. *Gebruik hiervoor de kennis uit hoofdstuk 8.*
6. **Extra uitdaging (bonus):** Laat een overzicht zien van alle HighScores (opgehaald via de Index-action). Bij het klikken op een HighScore worden de details getoond. *Gebruik een ListView. Als je de opgehaalde data deserialiseert als List kan je die in de ItemsSource instellen.*

Inleveren:

- Maak screenshots van de code die de /highscores en /highscore/{id} routes werkend maakt.
- Maak screenshots van andere code waar je trots op bent, of die volgens jou laat zien dat je de lesstof beheerst.
- Lever de screenshots in op Itslearning.

Veel succes met de opdracht!

11. Een API beveiligen

API's zijn de digitale voordeuren van applicaties, en je wilt niet dat zomaar iedereen binnenwandelt. Daarom worden ze beveiligd met verschillende technieken, zoals:

- **Authenticatie & Autorisatie** – Wie ben je en wat mag je?
- **Rate Limiting** – Niet te vaak aanbellen!
- **Encryptie** – Alles in een beveiligde envelop stoppen.

11.1. API Keys: Jouw geheime paspoort

Een **API Key** is een unieke sleutel die een gebruiker of applicatie identificeert. Zie het als een VIP-polsbandje voor een exclusieve club: zonder toegangscode kom je niet binnen.

Hoe werkt het?

1. Je vraagt een API Key aan bij de dienst (bijv. Google Maps API).
2. Bij elke API-aanvraag stuur je deze key mee in de 'header' of via een (query) parameter.
3. De server checkt of de key geldig is en wat je mag doen.

Als je zelf een server hebt, dan ben jij degene die de API key uitgeeft en controleert.

Beveiligingstips

-  **De API Key niet in je code hardcoden!** Gebruik omgevingsvariabelen of een los config bestand (zie hoofdstuk 4.16)
-  **Beperk rechten.** Geef een sleutel alleen toegang tot wat nodig is.
-  **Stel limieten in.** Zo voorkom je misbruik.

11.2. HTTP Client en API Keys: Praktisch Voorbeeld

Een **HTTP Client** wordt gebruikt om HTTP-verzoeken te sturen en te ontvangen. Dit is handig bij het communiceren met API's.

Wat is een HTTP Header?

Een **header** bevat extra informatie over het verzoek, zoals authenticatiegegevens. Denk aan een pakketje: de **body** is de inhoud, en de **header** is het etiket met afzender, bestemming en verzendinstructies.

API Key in de Header meesturen (C# - HttpClient)

```

using System;
using System.Net.Http;
using System.Threading.Tasks;

class Program
{
    private static readonly HttpClient client = new HttpClient();

    static async Task Main()
    {
        string apiUrl = "https://api.example.com/data";
        string apiKey = "your-api-key-here";

        // API Key toevoegen in de headers
        client.DefaultRequestHeaders.Add("Authorization", $"Bearer {apiKey}");

        HttpResponseMessage response = await client.GetAsync(apiUrl);

        if (response.IsSuccessStatusCode)
        {
            string responseData = await response.Content.ReadAsStringAsync();
            Console.WriteLine("Response data: " + responseData);
        }
        else
        {
            Console.WriteLine($"Error: {response.StatusCode}");
        }
    }
}

```

Wat gebeurt hier?

1. **HttpClient wordt aangemaakt** – Dit object verstuurt het verzoek.
2. **API Key wordt in de header gezet** – Dit gebeurt met Authorization: Bearer <api-key>. Sommige API's vragen om x-api-key in plaats van Authorization.
3. **GET-verzoek wordt verzonden** – De client roept de API aan met GetAsync().
4. **Response wordt verwerkt** – Als het goed gaat, wordt de data getoond.

Waarom de API Key in de Header?

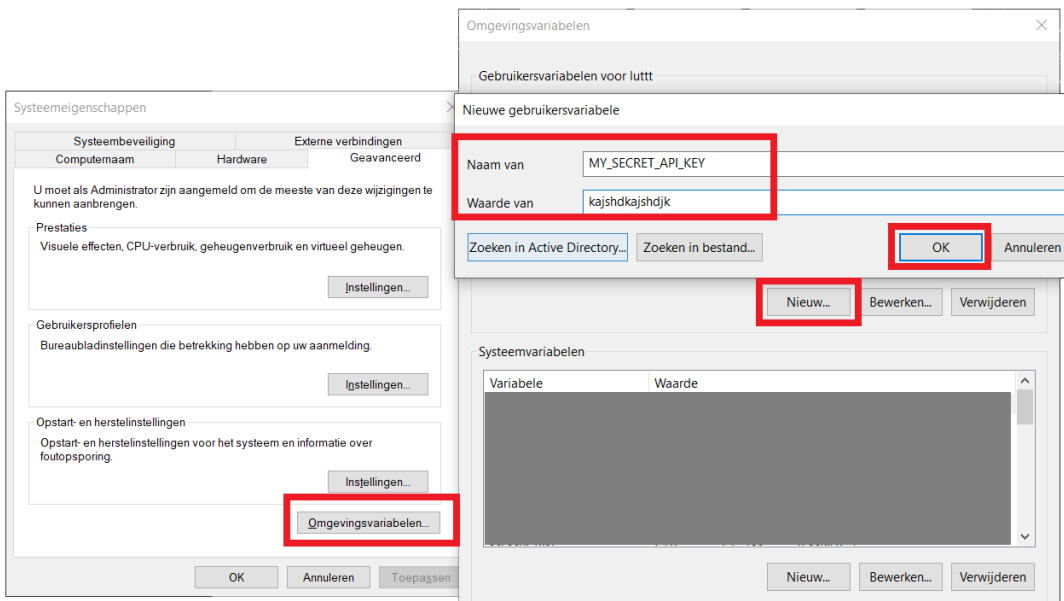
- **Veiliger dan in de URL** (waar hij zichtbaar kan zijn in serverlogs).
- **Beter gescheiden van de request body**.
- **Veel API's vereisen dit als standaard**.

Wil je extra beveiliging? Lees dan verder hoe je **omgevingsvariabelen** gebruikt om de API Key niet in de code op te slaan.

11.3. De API sleutel veilig houden: omgevingsvariabelen

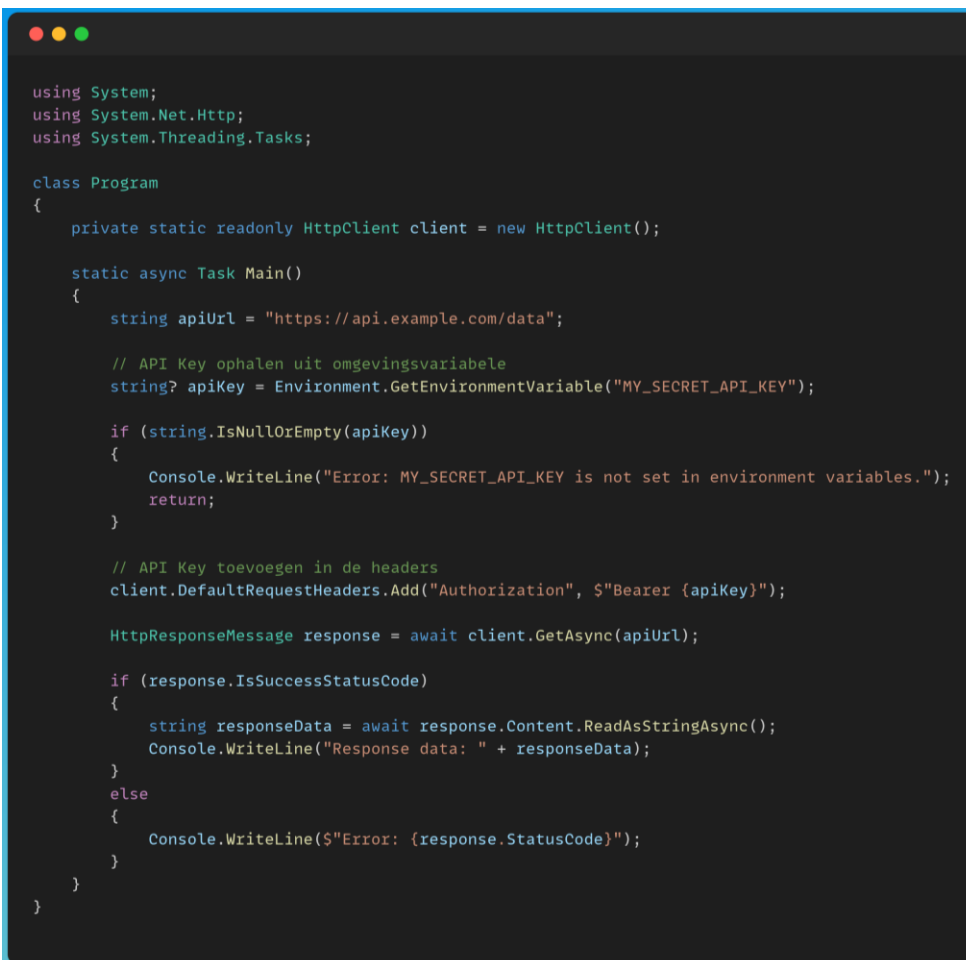
Het opslaan van API Keys in de code is onveilig. Een betere aanpak is om **omgevingsvariabelen** (**Environment Variables**) te gebruiken. Dit voorkomt dat gevoelige informatie per ongeluk in versiebeheer belandt.

De omgevingsvariabelen van een systeem bevatten ‘variabelen’ met waardes die door programma’s uitgelezen kunnen worden. Je kunt ze handmatig instellen door in het startmenu te zoeken naar “omgevingsvariabelen” of “environment” en ze via de systeemeigenschappen aan te maken:



Misschien ken je dit scherm al van de 'PATH' omgevingsvariabele. Die wordt gebruikt door de Command Prompt, om te weten in welke map gezocht moet worden naar programma's. Maar je kunt hier dus ook zelf variabelen aanmaken.

Nu kunnen we de omgevingsvariabele gebruiken in een **HttpClient-verzoek**.



Alternatieve oplossing: config bestand

In hoofdstuk 4.16 heb je gelezen hoe je de `ConnectionString` voor de databaseverbinding uit een los config-bestand kunt laden. Hier kun je ook API-keys in plaatsen. Doordat dit config-bestand niet in Git wordt opgeslagen voorkomt je dat geheime API sleutels uitlekken op een (publieke) GitHub repository.

11.4. API Key Authenticatie aan de Serverkant met `HttpListener` in C#

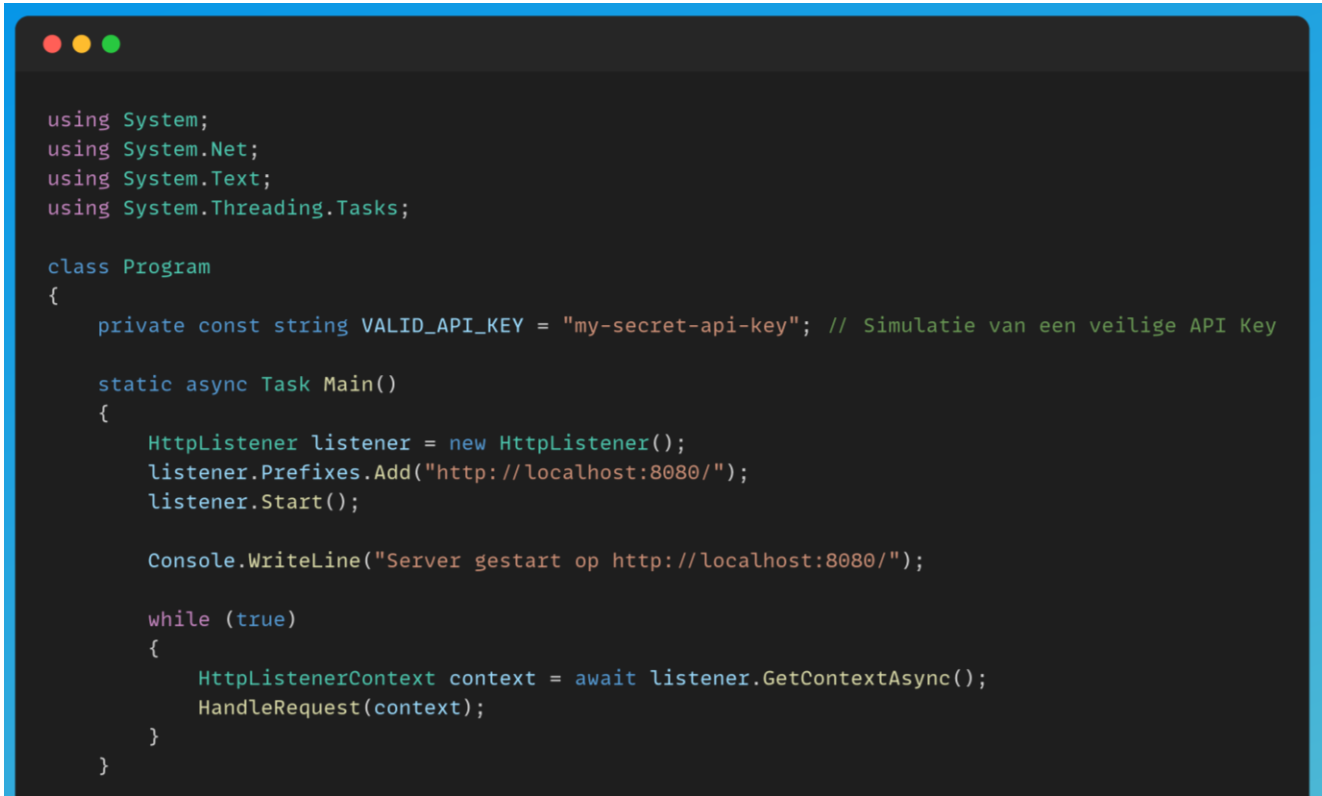
Nu je weet hoe je een API Key **meegeeft** in een request, laten we kijken hoe een server deze key **controleert**. Dit doen we met `HttpListener`, een eenvoudige ingebouwde webserver die we eerder hebben besproken.

Een API-server controleert elke inkomende aanvraag op een geldige API Key in de HTTP-header. Als de key ontbreekt of onjuist is, wordt de toegang geweigerd.

Wat we gaan bouwen in dit voorbeeld:

- Een `HttpListener` die luistert op een poort (bijv. 8080).
- Controleert of de API Key correct is via de Authorization header.
- Stuurt een **200 OK** bij een geldige key en **403 Forbidden** bij een ongeldige key.

In dit voorbeeld maken we weer een Console App, waarbij we beginnen met het starten van de `HttpListener`:



```
using System;
using System.Net;
using System.Text;
using System.Threading.Tasks;

class Program
{
    private const string VALID_API_KEY = "my-secret-api-key"; // Simulatie van een veilige API Key

    static async Task Main()
    {
        HttpListener listener = new HttpListener();
        listener.Prefixes.Add("http://localhost:8080/");
        listener.Start();

        Console.WriteLine("Server gestart op http://localhost:8080/");

        while (true)
        {
            HttpListenerContext context = await listener.GetContextAsync();
            HandleRequest(context);
        }
    }
}
```

De rest van de logica handelen we af met een `HandleRequest` methode, die we als volgt inrichten:

```

static void HandleRequest(HttpListenerContext context)
{
    HttpListenerRequest request = context.Request;
    HttpListenerResponse response = context.Response;

    // Haal de API Key uit de headers
    string? apiKey = request.Headers["Authorization"];

    if (apiKey == $"Bearer {VALID_API_KEY}") // Vergelijken met onze key
    {
        string responseString = "✅ Toegang verleend!";
        byte[] buffer = Encoding.UTF8.GetBytes(responseString);
        response.StatusCode = (int)HttpStatusCode.OK;
        response.OutputStream.Write(buffer, 0, buffer.Length);
    }
    else
    {
        string responseString = "❌ Toegang geweigerd. Ongeldige API Key.";
        byte[] buffer = Encoding.UTF8.GetBytes(responseString);
        response.StatusCode = (int)HttpStatusCode.Forbidden;
        response.OutputStream.Write(buffer, 0, buffer.Length);
    }

    response.OutputStream.Close();
}
}

```

Wat gebeurt hier?

1. **HttpListener start op poort 8080** en wacht op inkomende verzoeken.
2. **Wanneer een verzoek binnenkomt:**
 - De server controleert of er een Authorization header is.
 - Als de API Key correct is, wordt **200 OK** teruggestuurd.
 - Als de key ontbreekt of fout is, krijgt de client een **403 Forbidden**.
3. **De client ontvangt een tekstresponse** met "Toegang verleend" of "Toegang geweigerd".

Testen van de API Server

Nu we in de header een sleutel mee moeten sturen, zal testen in de browser moeilijk gaan. Om te testen heb je deze twee opties:

1. Gebruik **PowerShell**, een uitgebreide terminal met code functionaliteiten:
 - a) Zoek PowerShell in het Windows startmenu
 - b) Zorg dat de server van het bovenstaande script aan staat
 - c) Voer dit uit in PowerShell:


```
Invoke-RestMethod -Uri "http://localhost:8080/" -Headers @{"Authorization"="Bearer my-secret-api-key"}
```


2. Gebruik **CURL** (wordt geïnstalleerd met Git for Windows):
 - a) Open een Git Bash terminal
 - b) Zorg dat de server van het bovenstaande script aan staat
 - c) Voer dit uit in Git Bash:

```
curl -H "Authorization: Bearer my-secret-api-key" http://localhost:8080/
```

Wanneer je wel de juiste sleutel meestuurt zou je dit moeten zien:  **"Toegang verleend!"** en in andere gevallen:  **"Toegang geweigerd. Ongeldige API Key."**

11.5. Opdracht: Gebruik omgevingsvariabelen in de server-code

In het vorige voorbeeld staat de API sleutel hard-coded in de server-code.

Neem de code uit het voorbeeld over en pas het aan zodat de API sleutel uit de omgevingsvariabelen wordt gehaald.

11.6. Opdracht: veilig verbinden met de API in de client-code

Maak een client applicatie die met de API uit het vorige voorbeeld verbindt. Zorg dat de API sleutel veilig opgeslagen wordt, dus:

- Of in de omgevingsvariabelen
- Of in een config bestand (zie hoofdstuk 4.16)

Zorg dat de client applicatie simpelweg toont: er is verbinding.

Hiermee oefen je:

- Hoe je een API sleutel veilig opslaat
- Hoe je een API sleutel meegeeft via de Headers van een verzoek.

11.7. Sleutels Genereren in een Server (C#)

Tot nu toe hebben we in het server voorbeeld gezien dat de server slechts 1 API sleutel had. Dat is niet veilig, want als die lekt weten we niet wie er gebruik van maakt. Daarom geven API's vaak meerdere sleutels uit, gebonden aan een gebruiker. Zo kan per gebruiker worden gevolgd hoeveel ze de API gebruiken.

Wanneer een gebruiker toegang nodig heeft tot een API, moet de server een **unieke API Key** genereren. Dit gebeurt meestal bij registratie of aanvraag.

Een goede API Key:

- Is **uniek** (geen duplicaten!)
- Is **lang en willekeurig** (moeilijk te raden)
- Wordt **veilig opgeslagen** (bijvoorbeeld gehashed in de database)

In het volgende voorbeeld genereren we een **API Key** en slaan deze op in een database met Entity Framework. Je weet al hoe je dit met EF kunt doen, dus we herhalen alleen kort hoe opslag werkt.

1. API Key Genereren (Server-Side C#)

Hier is een methode om een veilige API Key te genereren:

```

using System;
using System.Security.Cryptography;
using System.Text;

public static class ApiKeyGenerator
{
    public static string GenerateApiKey()
    {
        using var rng = new RNGCryptoServiceProvider();
        byte[] apiKeyBytes = new byte[32];
        rng.GetBytes(apiKeyBytes);
        return Convert.ToBase64String(apiKeyBytes);
    }
}

```

Om te zorgen dat een écht random reeks aan tekens wordt bedacht, wordt een klasse uit de 'System.Security.Cryptography' library gebruikt.

Let op dat de 'Random'-klasse niet veilig is, omdat waardes voorspeld kunnen worden door hackers.

2. API Key Opslaan in de Database (Entity Framework)

Korte herhaling:

- a) Maak een Entity Model voor gebruikers met een API Key.
- b) Maak een EF Core Context om gegevens op te slaan (AppDbContext)

```

using System;

public class User
{
    public int Id { get; set; }
    public string Username { get; set; };
    public string ApiKey { get; set; };
}

```

3. Gebruiker Maken + API Key Genereren

Maak bij het registreren een API sleutel aan voor de gebruiker, bijvoorbeeld:

```

string apiKey = ApiKeyGenerator.GenerateApiKey();

var newUser = new User { Username = username, ApiKey = apiKey };
db.Users.Add(newUser);
db.SaveChanges();

```

⚠ Reminder! De API en Client staan alleen tijdens ontwikkeling op hetzelfde apparaat.

Probeer je in te beelden dat de API en Client niet op hetzelfde apparaat staan. Dat is tijdens het ontwikkelen wel (zo staan ze zelfs in dezelfde solution), maar in de praktijk staat de API op een server in een datacenter en is de Client een applicatie op de computer van een gebruiker.

De API dient als 'poortwachter' voor de database. De database is alleen toegankelijk voor de API. De client kan alleen met de API communiceren via HTTP verzoeken (REST API).

Dit betekent dat je moet zorgen dat de Client *geen* verbinding maakt met de database, maar in plaats daarvan verzoeken stuurt naar de API als HTTP-verzoeken.

11.8. Opdracht: ChatMeister met API-keys

Pak de laatste gemaakte ChatMeister applicatie van Hoofdstuk 10 in dit moduleboek erbij. Laten we deze chat applicatie verder uitbouwen tot een realistisch prototype van een chat applicatie.

Opdrachten:

- (server + client) Bouw een registratiefunctie voor de chat applicatie
- (server) Neem de API-sleutel generator klasse over uit de voorgaande uitleg.
- (server) Zorg dat gebruikers bij het registreren ook een API sleutel krijgen. Sla deze op bij de rest van de User gegevens.
- Bouw een inlogfunctie in de client en server applicaties:
 - (client) Zorg in de client applicatie dat gebruikers kunnen inloggen
 - (server) Zorg dat bij juist invoeren van de inloggegevens, de API-sleutel als antwoord wordt gegeven.
 - (client) Sla de API-sleutel die de server als antwoord geeft lokaal op, zodat deze meegegeven kan worden in volgende verzoeken.
- (server) Zorg dat alle chat routes (ophalen berichten, toevoegen berichten) controleren of een geldige API sleutel is meegegeven via de Headers van het verzoek.
- (server) Controleer wanneer een gebruiker een verzoek stuurt om een bericht te plaatsen of de API sleutel die ze meegeven van hun is. Zorg er op deze manier voor dat gebruikers geen berichten op naam van een andere gebruiker kunnen plaatsen.

11.9. Afronding modules CSD

De volgende inleveropdracht is een feedbackmoment waarmee je deze CSD-IV module afrond. Daarmee ben je ook klaar met de lessen van CSD en daarmee meer dan klaar voor je examen. Goed bezig geweest!

Op jouw proeve van bekwaamheid bouw je in een groep een complexe CRUD applicatie voor een opdrachtgever. Het dient een gedegen prototype te zijn, maar hoeft niet per se een API met API beveiliging te bevatten. Als je tijdens de proeve van bekwaamheid met je groep tijd hebt én zeker weet dat het lukt om een server/client structuur aan te brengen, dan mag je dat uiteraard doen om extra indruk te maken.

Veel tijd over? Wil je je graag verdiepen in ASP.NET? Jouw docent kan je een 'Bonus'-moduleboek geven die je naar eigen inzicht kunt doornemen. Die bevat veel theorie en opdrachten die in voorgaande jaren is gebruikt, maar nu is vervangen (door alle theorie die je hebt doorgenomen deze module).

In de volgende (en tevens laatste) inleveropdracht laat je zien dat je deze belangrijke basis in jouw vakgebied beheerst. **Succes en veel plezier!**

Inleveropdracht 11.10: API Bouwen en Consumeren (tijdsduur: +/- 180 minuten)

Bouw een BingeTracker-applicatie **met client-app én server** waarmee gebruikers films en series kunnen volgen op hun favoriete streamingplatforms. De applicatie moet CRUD-functionaliteiten bieden. Via jouw eigen webserver bied je op een veilige manier toegang tot de database, dit wordt de server. De client wordt een WinUI 3-applicatie.

De API moet de vereiste eindpunten blootleggen, en de clienttoepassing moet de API gebruiken om een gebruiksvriendelijke interface te bieden voor het beheren en beoordelen van films en series.

Vereisten:

- Ontwerp en implementeer een server met de volgende functionaliteiten:
 - (8 punten) Eindpunten voor CRUD-bewerkingen op films en series.
 - (2 punten) Er is een **many-to-many** relatie tussen een film/serie en genre. *'Lord of the Rings' kan bijvoorbeeld zowel Fantasy, Action en Adventure hebben als genre.*
 - (1 punt) Elke film en serie moet eigenschappen hebben, zoals titel, platform (streamingplatform), genre, releasejaar en beoordeling.
 - (4 punten) De API moet de gegevens opslaan in een MySQL database. *Hiervoor gebruik je zoals gewoonlijk Entity Framework (met de Pomelo NuGet package)*
- Implementeer een WinUI 3-clienttoepassing die de API gebruikt en de volgende functies biedt:
 - (2 punten) Toon een lijst met films en series die zijn opgehaald via de API.
 - (2 punten) Laat de gebruiker nieuwe films en series toevoegen met relevante details.
 - (2 punten) Stel de gebruiker in staat om bestaande film- en seriedetails bij te werken.
 - (2 punten) Bied de mogelijkheid om films en series te verwijderen.
 - (2 punten) Laat de gebruiker films en series beoordelen met behulp van een beoordelingssysteem, bijvoorbeeld van 1 tot 5 sterren.
 - (2 punten) Implementeer **ten minste 2** filter- of sorteropties voor films en series op basis van verschillende criteria, zoals naam, genre, releasejaar of beoordeling.
- (2 punten) De API moet alle CRUD-bewerkingen afhandelen en reageren met de juiste statuscodes en foutmeldingen.

Opmerkingen:

- De toepassing vereist geen authenticatie of gebruikersbeheer. Ga uit van één enkele gebruiker. Als **BONUS** mag je wel meerdere gebruikers met authenticatie implementeren.

Inleveren:

- Maak screenshots van alle controller actions.
- Maak een .zip-bestand van de gehele broncode van de API.
- Lever de screenshots en .zip in op Itslearning.

Veel succes met de opdracht!

 **Let op:** Het inleveren van deze opdracht geldt als feedback-moment. Je ontvangt studiepunten wanneer je deze succesvol afrondt. Houdt na het inleveren Itslearning in de gaten en verwerk eventuele feedback die je van de docent krijgt.

*Er zijn in totaal **29 punten** te behalen bij deze opdracht. Met een cesuur van 70% moet je ten minste 20,5 punten scoren voor een voldoende (5,6).*

IV. Voor de docent

Enkele punten van aandacht:

1. Kijk in de documentatie van Microsoft goed of de studenten de juiste versie van .NET bekijken. Op het moment van schrijven kiezen we .NET 8.0.
2. WinUI 3 is pas sinds kort stabiel en heette tijdens ontwikkeling 'Project Reunion'. Die naam kom je mogelijk nog tegen in de documentatie of op Stack Overflow.
3. Vanaf hoofdstuk 5 wordt een groter beroep gedaan op de zelfredzaamheid van studenten. Houdt er rekening mee dat ze tijdig om hulp vragen. Coach hier op de volgende zaken:
 - Effectief gebruik zoekmachines
 - Kleine prototypes: soms is het lastig om een functie meteen in een bestaande broncode toe te voegen. Dan kan het helpen om een versimpeld prototype te maken (bijvoorbeeld in een Console App.) om een functie in het klein te onderzoeken.
 - Analytisch vermogen: als ontwikkelaars hebben we niet alle antwoorden. Laat studenten zelf structuren en systemen bedenken. Er is niet één weg. De wegen die we in dit moduleboek bewandelen zijn slechts één van vele mogelijke oplossingen. Vaak zijn dit wel de meest voor de hand liggende en minst complexe wegen, maar niet altijd de meest juiste.

in de laatste hoofdstukken van deze module ligt de aandacht op de volgende doelen:

- Nieuwe kennis en vaardigheden eigen maken op het gebied van API's.
- Met een API de data in onze applicatie ook op afstand beschikbaar maken.
- Comfortabel worden met het ontwikkelen van complexe CRUD-applicaties ter voorbereiding op de proeve van bekwaamheid.

Tijdens de lessen is het vooral de bedoeling dat het lesmateriaal wordt afgestemd op de leerbehoefte. Als een student behoefte heeft aan herhaling, geef die dan oefeningen om te herhalen. Is iemand al verder en zoekt die verdere uitdaging? Geef die het 'Bonus' moduleboek, waarin extra opdrachten en theorie staan (die in vorige jaren gegeven zijn).

Extra: Uitgebreider Stappenplan ChatMeister 2.0

Zoekt de student na het lezen van de ChatMeister 2.0 opdracht, en het samen uitzoeken in de lessen, extra uitleg over hoe de ChatMeister applicatie functioneel werkt? Dan kun je een uitgebreid stappenplan aanbieden wat uitleg geeft over hoe de interactie is tussen de client-app en API.

Er staat niet veel code in dat stappenplan, maar het geeft een beeld van de volgorde van schermen en functionaliteiten.

Dit bestand heet: **Opdracht CSD-IV ChatMeister 2.0 - Uitgebreid.pdf** en staat op de gewoonlijke plek (naast de .docx van dit moduleboek)

Oplossing voor probleem #1 bij opdracht H3: 'Implementeren MySQL in API Project'

```
private static string GetMessages(int roomId)
{
    var chat = db.Chats.Include(c => c.Messages).FirstOrDefault(c => c.Id == roomId);
    if (chat == null)
    {
        return "Chat";
    }
    var messages = chat.Messages;
}
```

using Microsoft.EntityFrameworkCore; ▶

Fix with Copilot

CS1061 'DbSet<Chat>' does not contain a definition for 'Include' and no accessible extension method 'Include' accepting a first argument of type 'DbSet<Chat>' could be found (are you missing a using directive or...)

